



1ST EDITION

Microservices Design Patterns in .NET

Making sense of microservices design and
architecture using .NET Core

TREVOIR WILLIAMS



Microservices Design Patterns in .NET

Making sense of microservices design and architecture
using .NET Core

Trevor Williams



BIRMINGHAM—MUMBAI

Microservices Design Patterns in .NET

Copyright © 2022 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Gebin George

Publishing Product Manager: Pooja Yadav

Senior Editor: Rounak Kulkarni

Technical Editor: Maran Fernandes

Copy Editor: Safis Editing

Project Coordinator: Deeksha Thakkar

Proofreader: Safis Editing

Indexer: Manju Arasan

Production Designer: Vijay Kamble

Developer Relations Marketing Executive: Sonakshi Bubbar

Business Development Executive: Kriti Sharma

First published: January 2023

Production reference: 1161222

Published by Packt Publishing Ltd.

Livery Place

35 Livery Street

Birmingham

B3 2PB, UK.

ISBN 978-1-80461-030-5

www.packt.com

I dedicate this publication to my loving, hard-working, and irreplaceable parents, whose tremendous sacrifices gave me the best possible start in life. They remain a steady source of inspiration, and I try to make them proud of everything I do. I thank my caring, understanding wife, whose unwavering support nudged me through this tiring endeavor. I also thank my students, who constantly challenge me to remain on top of the latest technologies and concepts and always push me forward in this field.

– Trevor Williams

Contributors

About the author

Trevor Williams is a passionate software and system engineer with a strong drive to share the best of his knowledge with students around the globe. He holds a master's degree in computer science, majoring in software development. He has also received multiple Microsoft Azure certifications.

His work experience includes software engineering and consulting, database development, cloud system administration, and lecturing. He enjoys teaching IT and development skills and guides students in gaining the latest knowledge with practical application in the modern industry.

About the reviewer

Marius Iulian Mihailescu is an associate lecturer at Spiru Haret University and co-founder/chief research officer at Dapyx Solution, Bucharest, Romania. His work is focused on applied/theoretical cryptography and information security, dealing with the identification process of threats and vulnerabilities using artificial intelligence and machine learning. He is currently also working as an IT project manager at the Institute for Computers, Romania, where he is responsible for the development process of several projects.

His research interests are extended to areas such as computer vision, biometrics, machine learning, artificial intelligence, and deep learning. The goal of his research is focused on understanding the real requirements of complex systems (cloud computing, big data, IoT, etc.) and their applications in different domains (such as computer forensics, behavioral psychology, and financial derivatives), and to provide real and practical solutions for guaranteeing the **Confidentiality, Integrity, and Availability (CIA)** of the processes.

Marius completed his Ph.D. studies in computer science on improving the security techniques for guaranteeing the confidentiality and integrity of the biometrics data at the Faculty of Mathematics and Computer Science at the University of Bucharest, Romania. During this time, he was also a part-time research associate for the ATHOS (Automated System of Authentication through Biometric Signature) project from S.C. Softwin S.R.L., where he worked on improving the load balancing mechanisms in a parallel computing system.

Table of Contents

Preface

xv

Part 1: Understanding Microservices and Design Patterns

1

Introduction to Microservices – the Big Picture 3

A deep dive into microservices and its key elements	3	Development speed	10
Building a monolith	4	Improved data storage	10
Building microservices	7	Monitoring	10
		Deployment	11
Assessing the business need for microservices	8	Determining the feasibility of implementing microservices	12
Scalability	9	Microservices and .NET Core	12
Availability	9	Summary	13

2

Working with the Aggregator Pattern 15

Technical requirements	15	Aggregates and aggregate roots	19
Exploring DDD and its significance	16	Relationships in aggregates	20
Exploring the pros and cons of DDD	18	Handling relationships that span aggregates	21
DDD and microservices	18	Aggregates versus entities	21
The purpose and use of aggregate patterns	19	Entities and why we need them	21
		Practical uses of entities in code	22

A rich domain model versus an anemic domain model	23	Understanding and using value objects	26
		Summary	28

3

Synchronous Communication between Microservices 29

Technical requirements	29	communication	32
Use cases for synchronous communication	30	Implementing gRPC synchronous communication	35
Challenges of microservice communication	30	HTTP versus gRPC communication	37
Implementing synchronous communication	32	Disadvantages of synchronous communication between microservices	38
Implementing HTTP synchronous		Summary	39

4

Asynchronous Communication between Microservices 41

Technical requirements	41	Configuring a message bus (RabbitMQ or Azure Service Bus)	47
Functioning with asynchronous communication	41	Implementing RabbitMQ in an ASP.NET Core web API	48
HTTP asynchronous communication	43	Implementing Azure Service Bus in an ASP.NET Core API	51
Understanding Pub-Sub communication	43	Disadvantages of asynchronous communication between microservices	55
Understanding message queues	43	Summary	56
Understanding message bus systems	44		
Understanding eventual consistency	46		

5

Working with the CQRS Pattern 57

Technical requirements	57	Creating a command handler	63
Why use CQRS for microservices development?	58	Invoking a command	64
Benefits of the CQRS pattern	59	Implementing a query	66
Disadvantages of the CQRS pattern	60	Creating a query model	66
Using the Mediator pattern with CQRS in .NET	60	Creating a query handler	67
Implementing a command	62	Invoking a query	68
Creating a command model	62	Summary	69

6

Applying Event Sourcing Patterns 71

Technical requirements	72	Exploring domain events in our application	75
What are events?	72	Creating an event store	81
Key attributes of events	72	How to store events	81
What can event sourcing patterns do for me?	73	Implementing event sourcing using a relational database	82
Pros of event sourcing	73	Implementing event sourcing using a non-relational database	83
Cons of event sourcing	74	Reading state with CQRS	85
What are domain events?	75	Summary	86
Domain events and event sourcing	75		

Part 2: Database and Storage Design Patterns

7

Handling Data for Each Microservice with the Database per Service Pattern 89

Technical requirements	89	Non-relational databases	97
How to make use of the Database-Per-Service pattern	90	Choosing a database technology	98
A single database technology per service	90	Choosing an ORM	100
Using different database technologies per service	91	Choosing a database development technique	103
Disadvantages of this pattern	92	Implementing the repository pattern	105
Developing a database	93	Summary	112
Relational databases	94		

8

Implement Transactions across Microservices Using the Saga Pattern 113

Technical requirements	113	Pros and cons	118
Exploring the Saga pattern	114	Understanding and implementing orchestration	119
Issues and considerations	115	Rolling back on failure	120
Understanding and implementing choreography	116	Pros and cons	121
Rolling back on failure	117	Summary	121

Part 3: Resiliency, Security, and Infrastructure Patterns

9

Building Resilient Microservices	125
Technical requirements	126
The importance of service resiliency	126
Possible failure scenarios and how to handle them	126
Implementing resiliency with caching and message brokers	127
Using a message broker	127
Using a caching layer	128
Using Redis Cache	128
Implementing retry and circuit breaker policies	131
Retry policy with Polly	132
Circuit breaker policy with Polly	133
Summary	134

10

Performing Health Checks on Your Services	137
Technical requirements	137
Health checks and microservices	138
The liveness health check	140
Readiness health checks	140
Implementing ASP.NET Core health checks	141
Adding liveness health checks	141
Adding readiness health checks	147
Configuring health probes in orchestrators	148
Summary	150

11

Implementing the API and BFF Gateway Patterns	151
Technical requirements	152
What is the API gateway pattern?	152
Advantages of an API gateway	153
Disadvantages of an API gateway	155
Implementing the API gateway pattern	155
Thick API gateways	156
Implementing an API gateway using Azure API Management	156
Implementing an API gateway using Ocelot	160
Adding cache management	164
Adding rate limiting	165

Adding response aggregation	167	BFF pattern using Ocelot	171
Backend for Frontend pattern	170	Summary	174

12

Securing Microservices with Bearer Tokens 177

Technical requirements	178	microservices	191
Bearer tokens for securing communications	178	Configuring IdentityServer	192
Understanding bearer tokens	179	Securing an API using IdentityServer	202
Implementing bearer token security	182	Securing the Ocelot API gateway with IdentityServer	206
Securing API with bearer tokens	182	Additional API security considerations	207
Generating and issuing bearer tokens	186	Summary	207
Using IdentityServer4 to secure			

13

Microservice Container Hosting 209

Technical requirements	209	Understanding docker-compose and images	221
Using containers in microservices development	210	Adding docker-compose to a project	222
What can containers do for me?	210	Publishing containers to a container registry	227
Understanding Docker	212	Public versus private container registries	227
Understanding container images	212	Creating and uploading custom images	228
Pros and cons of containers	213	Summary	230
Authoring a Dockerfile	214		
Launching a containerized application	218		
Using native .NET container support	220		

14

Implementing Centralized Logging for Microservices 231

Technical requirements	232	Integrating with Seq	242
Logging and its importance	232	Distributed log tracing	244
Choosing what to log	232	Enhanced logging for distributed tracing	245
Using the .NET logging API	234	Summary	247
Adding Serilog	240		
Log aggregation and its uses	241		

15

Wrapping It All Up 249

Aggregator pattern	249	Using the saga pattern across services	254
Synchronous and asynchronous communication	250	Resilient microservices	255
CQRS	250	Importance of health checks	256
Event sourcing patterns	251	API Gateways and backend for frontend	257
Database per service pattern	252	Bearer token security	257
Relational databases	252	Containers and microservices	259
Non-relational databases	253	Centralized logging	261
One database for all services	253	Summary	263
One database per service	253		

Index 265

Other Books You May Enjoy 276

Preface

Hello there! We are here to explore design and development patterns that we can leverage when building a microservice application with .NET. Microservice architecture involves separating a potentially complex application into smaller, more maintainable services that must work together. Essentially, we take one big application and break it down into smaller parts.

This approach introduces a fresh crop of complexities in the application's design since these now separate services need to collaborate to deliver a unified experience to the end user. As such, we need to understand the various drawbacks of this architectural approach and strategize on how we can address the various concerns.

We will focus on using .NET, given that Microsoft has a proven track record of releasing and supporting top-notch tooling and support for the most recent technologies that allow us to develop cutting-edge solutions. Some of the reasons we focus on development with .NET are as follows:

- **Well maintained:** Microsoft is constantly pushing the boundaries and introducing new ways to accomplish old things and new ways to implement solutions and increase productivity. It is a well-maintained, supported, and documented ecosystem.
- **Performance:** .NET increases its performance with each new release. Microservices must be as performant and responsive as possible to ensure that the end user's experience is as clean as possible.
- **Cross-platform:** .NET Core is cross-platform and can be deployed on virtually any technology stack. This reduces the limitations surrounding deploying and supporting services written using .NET technology.
- **Support for various technologies:** Each problem has a technology that helps us to implement a solution. .NET has support for many technologies, making it a great candidate for universal development needs.

We want to ensure that we understand the possibilities and strategies needed while developing an application based on microservices architecture. We will review the theory behind each problem and then explore the potential solutions and technologies that help us to implement the best possible solution to our challenges while developing a solution.

Who this book is for

This book is designed for .NET developers who wish to demystify the various moving parts of microservices architecture. To get the most out of the book, you should ideally fit into one of these categories:

- **Team leads:** Leaders who need to understand the various moving parts and the theory behind design decisions that need to be made
- **Senior developers:** Developers who need to appreciate how to guide the development efforts and implement complex blocks of code
- **Intermediate .NET developers:** .NET developers who have a working knowledge of the .NET ecosystem and want to dig deeper into developing more complex solutions

The overall content of this book will assist you in understanding the dynamics of microservices application design and assist you in reaching the next level of development.

What this book covers

Chapter 1, Introduction to Microservices – the Big Picture, looks at microservice architecture at a high level and seeks to understand some of the early problems we might encounter and explores design patterns that address them.

Chapter 2, Working with the Aggregator Pattern, explores how domain-driven design and the aggregate pattern lay the foundation for scoping requirements and the foundation of microservice design.

Chapter 3, Synchronous Communication between Microservices, explores how we make microservices communicate synchronously and the potential drawbacks of this method.

Chapter 4, Asynchronous Communication between Microservices, looks at asynchronous communication between services, which allows us to send data and move along, regardless of the availability or potential long runtime of the other microservices being called.

Chapter 5, Working with the CQRS Pattern, explores the CQRS pattern and why it is useful in microservices development.

Chapter 6, Applying Event Sourcing Patterns, discusses the intricacies of event sourcing and how we can implement this to ensure that our data between services stays in sync.

Chapter 7, Handling Data for Each Microservice with Database per Service Pattern, covers the best practices surrounding implementing different databases in different services.

Chapter 8, Implement Transactions across Microservices Using the Saga Pattern, explores the Saga pattern and how it helps us implement transactions across our microservices.

Chapter 9, Building Resilient Microservices, reviews implementing retry and exit strategy logic for more resilient communication between services.

Chapter 10, Performing Health Checks on Your Services, reviews how we can implement health checks in our ASP.NET Core APIs and why they are essential.

Chapter 11, Implementing API and BFF Gateway Patterns, dives into implementing API gateways, the backed for frontend pattern, and how they help us to create a robust microservices application.

Chapter 12, Securing Microservices with Bearer Tokens, reviews how bearer tokens can secure communications with each service.

Chapter 13, Microservice Container Hosting, explores containerization and how we can leverage containers to efficiently host our microservices.

Chapter 14, Implementing Centralized Logging for Microservices, explores the steps and best practices for aggregating logs from several services into one viewing area.

Chapter 15, Wrapping It All Up, discusses the key points from each chapter and highlights how each plays a role in developing a microservices application.

To get the most out of this book

You will need knowledge of API development using ASP.NET Core and basic database development knowledge. This book is best suited for intermediate-level developers looking to improve their understanding of development design patterns.

Software/hardware covered in the book	Operating system requirements
ASP.NET Core 6/7	Windows, macOS, or Linux
C# 9/10	Windows, macOS, or Linux
Docker	Windows, macOS, or Linux

Most examples are shown using NuGet package manager commands and Visual Studio 2022. These commands can be translated into dotnet CLI commands, which can be used on any operating system and with alternative IDEs to Visual Studio.

Download the example code files

You can download the example code files for this book from GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET>. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots and diagrams used in this book. You can download it here: <https://packt.link/dD3Jv>.

Conventions used

There are several text conventions used throughout this book.

Code in text: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, commands, and keywords. Here is an example: “As it stands, our `CreateAppointmentHandler` will handle everything that is needed in the situation.”

A block of code is set as follows:

```
public class AppointmentCreated : IDomainEvent
{
    public Appointment { get; set; }
    public DateTime ActionDate { get; private set; }
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: “Select **System info** from the **Administration** panel.”

Tips or important notes

Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at customer-care@packtpub.com and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit www.packtpub.com/support/errata and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit authors.packtpub.com.

Share Your Thoughts

Once you've read *Microservices Design Patterns in .NET*, we'd love to hear your thoughts! Please click [here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/9781804610305>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1:

Understanding Microservices and Design Patterns

This part focuses on the fundamentals of microservices design patterns. You will learn about the many moving parts and scoping techniques needed for deciding the overall approach to design a microservices application. By the end of this part, you will realize the intricacies surrounding this development pattern and understand some of the pros and cons of this approach.

This part has the following chapters:

- *Chapter 1, Introduction to Microservices – The Big Picture*
- *Chapter 2, Working with the Aggregator Pattern*
- *Chapter 3, Synchronous Communication between Microservices*
- *Chapter 4, Asynchronous Communication between Microservices*
- *Chapter 5, Working with the CQRS Pattern*
- *Chapter 6, Applying Event Sourcing Patterns*

Introduction to Microservices – the Big Picture

Microservices are being featured in every avenue of software development. Microservices are a software development style that has been touted to increase development speed and efficiency while improving software scalability and delivery. This development technique is not unique to any stack and has become extremely popular in Java, .NET, and JavaScript (Node JS) development. While the use of microservices is seen as a pattern, there are several subpatterns that are employed to ensure that the code base is as effective as possible.

This chapter is the first of this 15-chapter book, which will cover design patterns used in microservices. We will be focusing on implementing them using the .NET Core development stack, and you will learn how code can be written and deployed. You will learn about design and coding patterns, third-party tools and environments, and best practices for handling certain scenarios in application development with microservices.

In this chapter, we are going to cover the following topics:

- A deep dive into microservices and its key elements
- Assessing the business need for microservices
- Determining the feasibility of implementing microservices

A deep dive into microservices and its key elements

Traditionally, in software development, applications have developed as a single unit or *monolith*. All of the components are tightly coupled, and a change to one component threatens to have rippling effects throughout the code base and functionality. This makes long-term maintenance a major concern and can hinder developers from rolling out updates quickly.

Microservices will have you assess that monolith, break it into smaller and more perceivable applications. Each application will relate to a subsection of the larger project, which is called *domain*. We will then develop and maintain the code base per application as independent units. Typically, microservices are developed as APIs and may or may not interact with each other to complete operations being carried by users through a unifying user interface. Typically, the microservice architecture comprises a suite of small independent services, which communicate via **HTTP (REST APIs)** or **gRPC (Google Remote Procedure Call)**. The general notion is that each microservice is autonomous, has a limited scope, and aids in a collectively loosely coupled application.

Building a monolith

Let's imagine that we need to build a health facility management web application. We need to manage customer information, book appointments, generate invoices, and deliver test results to customers. If we were to itemize all the steps needed to build such an application, key development and scoping activities would include the following:

1. Model the application, and scope the requirements for our customer onboarding, user profiles, and basic documents.
2. Scope the requirements surrounding the process of booking an appointment with a particular doctor. Doctors have schedules and specialties, so we have to present the booking slots accordingly.
3. Create a process flow for when a match is found between a customer and a doctor. Once a match is found, we need to do the following:
 - I. Book the doctor's calendar slot
 - II. Generate an invoice
 - III. Potentially collect a payment for the visit
 - IV. Send email notifications to the customer, doctor, and other relevant personnel
4. Model a database (probably relational) to store all this information.
5. Create user interfaces for each screen that both customers and the medical staff will use.

All of this is developed as one application, with one frontend talking to one backend, one database, and one deployment environment. Additionally, we might throw in a few third-party API integrations for payment and email services. This can be load balanced and hosted across multiple servers to mitigate against downtime and increase responsiveness:

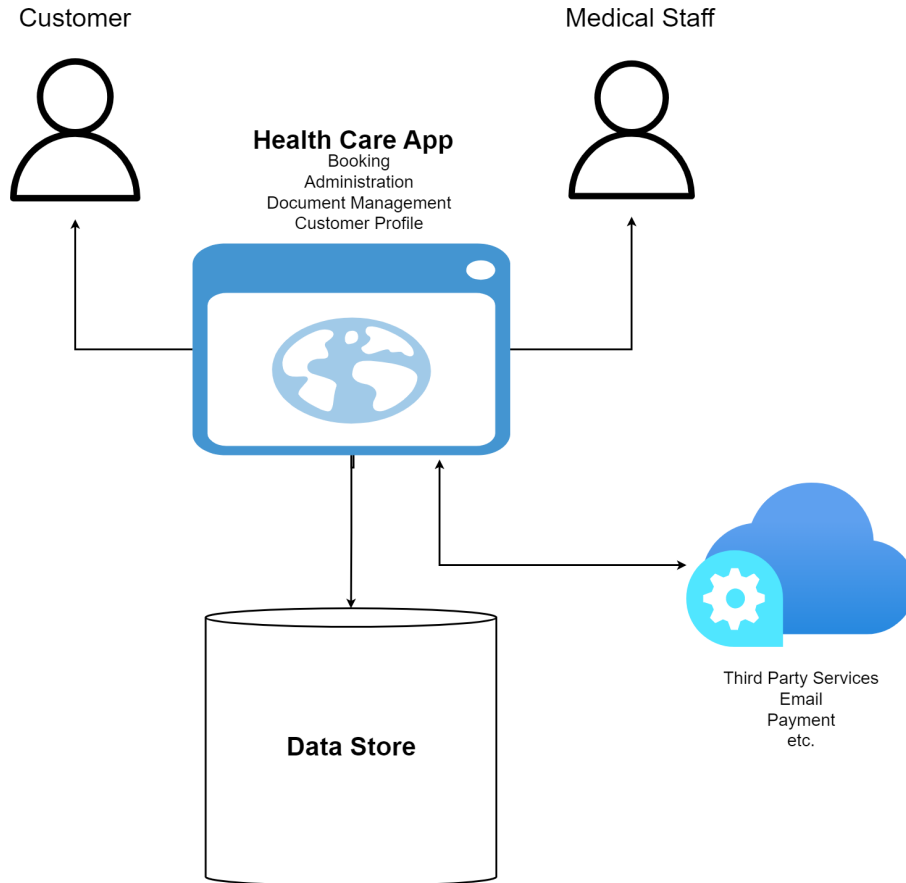


Figure 1.1 – Application building

However, this monolithic architecture introduces a few challenges:

- Attempts to extend functionality might have ripple effects through multiple modules and introduce new database and security needs.

Potential solution: Perform thorough unit and integration testing.

- The development team runs the risk of becoming very dependent on a particular stack, making it more difficult to keep the code base modern.

Potential solution: Implement proper versioning strategies and increment them as the technology changes.

- As the code base expands, it becomes more difficult to account for all of the moving parts.

Potential solution: Use clean architectural methods to keep the code base loosely coupled and modular.

The reality is that we can overcome some of these challenges with certain architectural decisions. This all-in-one architecture has been the de facto standard, and frankly, it works. This project architecture is simple, easy enough to scope and develop, and is supported by most, if not all, development stacks and databases. We have been building them for so long that perhaps we have become oblivious to the real challenges that prevail as we try to extend and maintain them in the long term.

Figure 1.2 shows the typical architecture of a monolithic application:

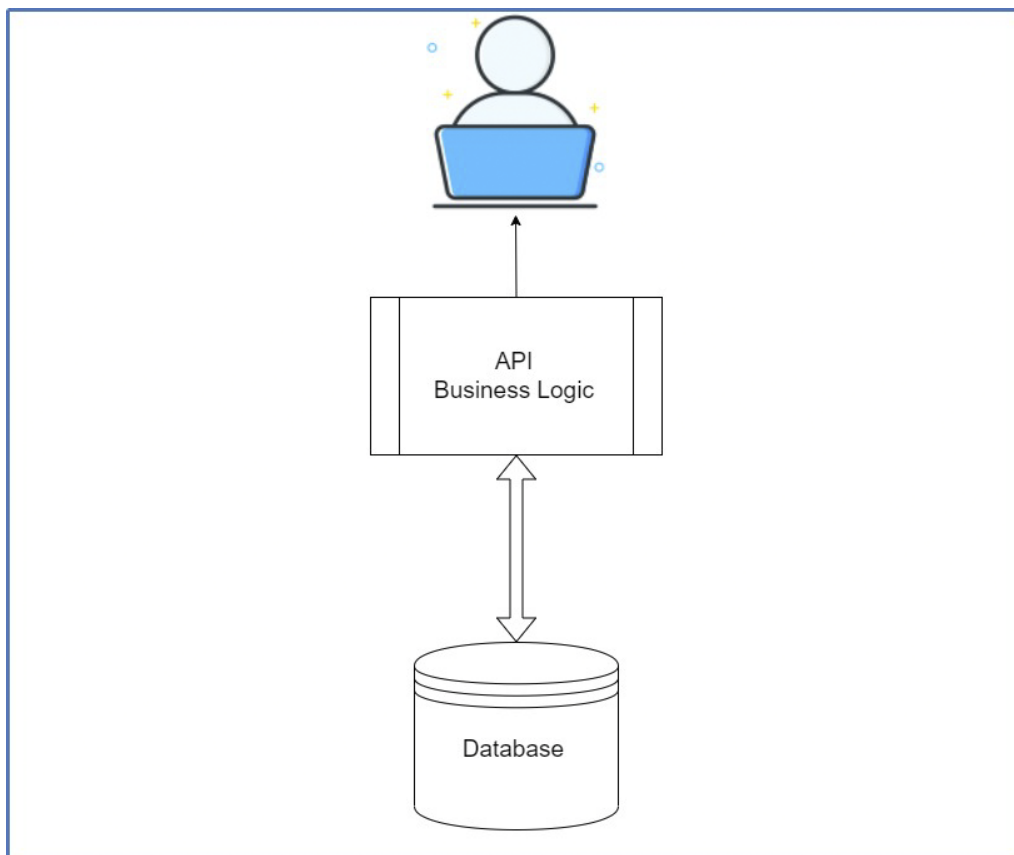


Figure 1.2 – One user interface is served by an API or code library with business logic and is serviced by one database

Now that we have explored the monolithic approach and its potential flaws, let us review a similar application built using microservices.

Building microservices

Now, let us take the same application and conceptualize how it could be architected using microservices. During the design phase, we seek to identify the specific functionalities for each tranche of the application. This is where we identify our domains and subdomains; then, we begin to scope standalone services for each. For example, one domain could be customer management. This service will solely handle the user account and demographic information. Additionally, we could scope bookings and appointments, document management, and finally, payments. This then brings another issue to the foreground: we have dependencies between these three subdomains when we need service independence instead. Using *domain-driven design*, we then scope out where there are dependencies and identify where we might need to duplicate certain entities. For instance, a customer needs representation in the booking and appointments database as well as payments. This duplication is required if we are using separate databases per service (which is strongly encouraged).

The microservices require us to properly scope the flow of operations that involve multiple services playing a part. For instance, when making a booking, we need to do the following:

6. Retrieve the customer making the booking.
7. Ensure that the preferred time slot is available.
8. If available, generate an invoice.
9. Collect the payment.
10. Confirm the appointment.

That process alone has some back-and-forth processing between the services. Properly orchestrating these *service conversations* is very critical to having a seamless system and adequately replacing a monolithic approach. Therefore, we introduce various design patterns and approaches to implementing our code and infrastructure. Even though we break potentially complex operations and workflows into smaller and more perceivable chunks, we end up in the same position where the application needs to carry out a specific operation and carry out the original requirements as a whole.

Figure 1.3 shows the typical architecture of a microservices application:

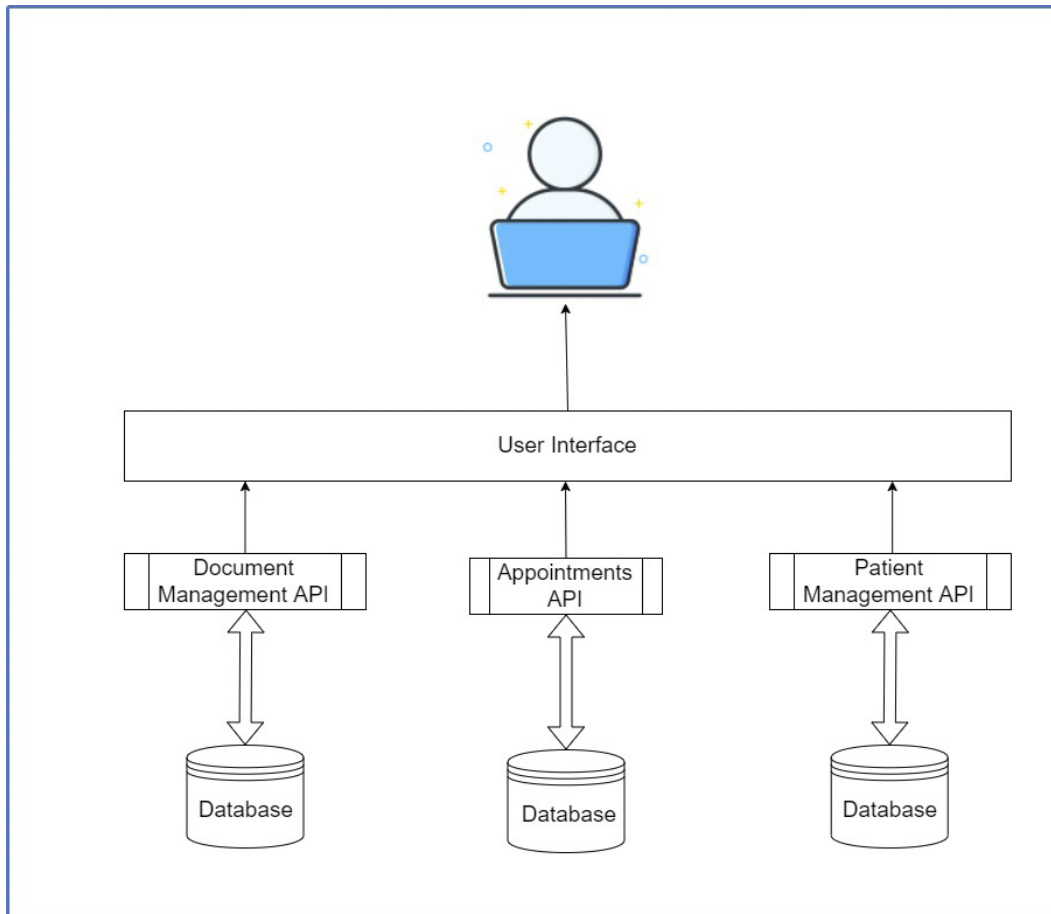


Figure 1.3 – Each microservice is standalone and unifies in a single user interface for user interactions

Now that you are familiar with the differences between the monolithic and microservices approaches, we can explore the pros and cons of using the microservices design pattern.

Assessing the business need for microservices

As we have seen so far, microservices are not easy to author, and they come with many cross-cutting concerns and challenges. It is always important to ask yourself *Why?* and *Do I really need it?* before implementing any design patterns.

At a high level, some benefits of this approach are listed as follows:

- Scalability
- Availability
- Development speed
- Improved data storage
- Monitoring
- Deployment

In the following sections, we will dive into the details of each.

Scalability

In the monolithic approach, you scale all or nothing. In microservices, it is easier to scale individual parts of the application and address specific performance gaps as they arise. If a vaccine becomes widely available and customers are encouraged to book an appointment online, then we are sure to experience a large load during the first few weeks. Our customer microservice might not be too affected by that, but we will need to scale our booking and appointments and payments services.

We can scale horizontally, which means that we can allocate more CPU and RAM when the load increases. Alternatively, we can scale vertically by spawning more instances of the service to be load balanced. The better method is relative to the service's needs. Using the right hosting platforms and infrastructure allows us to automate this process.

Availability

Availability means the probability of a system being operational at a given time. This metric goes hand in hand with the ability to scale, but it also addresses the reliability of the underlying code base and hosting platform. The code base plays a big part in that, so we want to avoid, as much as possible, a single point of failure. A single point of failure affects the entire system if it fails at any point. For example, we will be exploring the *gateway pattern*, where we will aggregate all services behind one point of entry. For our distributed services to remain available, this gateway must be always online.

This can be achieved by having vertical instances that balance the load and distribute the expected responsiveness of the gateway and, by extension, the underlying services.

Development speed

Given that the application has been broken into domains, developers can focus their efforts on ensuring that their set of features is being developed efficiently. This also contributes to how quickly features can be added, tested, and deployed. It will now become a practical approach to have one team per subdomain. Additionally, it becomes much easier to scope the requirements for a domain and focus on fewer functional requirements for a piece of work. Each team can now be independent and own the service from development to deployment.

This allows *Agile* and *DevOps* methodologies to be easier to implement, and it is easier to scope resource requirements per team. Of course, we have seen that services will still need to communicate, so we will still have to orchestrate the integration between the teams. So, while each team is independent, they will still need to make their code and documentation available and easy enough to access. Version control also becomes important since the services will be updated over time, but this must be a managed process.

Improved data storage

Our monolithic application uses one database for the entire application. There are situations where you might end up using one database for multiple microservices, but this is generally discouraged, and a *database-per-service* approach is preferred. Services must be autonomous and independently developed, deployed, and scaled. This is best accomplished if each service has its own data storage. It makes even more sense when you consider that the type of data being stored might influence the type of data storage that is used. Each service might require a different type of data store, ranging from relational database storage such as **Microsoft SQL Server** to document-based database storage such as **Azure Cosmos DB**. We want to ensure that changes to a data store will only affect the associated microservice.

Of course, this will bring its own challenges where data will need to be synchronized across the services. In the monolith, we could wrap all steps inside one transaction, which might lead to performance issues for potentially long-running processes. With microservices, we face the challenge of orchestrating distributed transactions, which also introduces performance risks and threatens the immediate consistency of our data. At this point, we must turn to the concept of *eventual consistency*. This means that a service publishes an event when its data changes and subscribing services use that event as a signal to update their own data. This approach is made possible through event-sourcing patterns. We accept the risk that, for a period, data might be inconsistent across subdomains. Message queue systems such as **Kafka**, **RabbitMQ**, and **Azure Service Bus** are generally used to accomplish this.

Monitoring

One of the most important aspects of a distributed system is monitoring. This allows us to proactively ensure uptime and mitigate against failures. We need to be able to view the health of our service instances. We also begin to think about how we can centralize logs and performance metrics in a unified

manner, sparing us the task of going to each environment manually. Tools such as **Kibana**, **Grafana**, or **Splunk** allow us to create a rich dashboard and visualize all sorts of information about our services.

One very important bit of information is a *health check*. Sometimes, a microservice instance can be running but is failing to handle requests. For example, it might have run out of database connections. With health checks, we can see a quick snapshot of the service's health and have that data point returned to the dashboard.

Logging is also a crucial tool for monitoring and troubleshooting. Typically, each microservice would write its own logs to files in its environment. From these logs, we can see information about errors, warnings, information, and debug messages. However, this is not efficient for a distributed system. At this point, we use a log aggregator. This gives us a central area to search and analyze the logs from the dashboards. There are a few log aggregators you can choose from such as **LogStash**, **Splunk**, or **PaperTrail**.

Deployment

Each microservice needs to be independently deployable and scalable. This includes all the security, data storage, and additional assets that our services use. They must all live on physical or virtual servers, whether on-premises or in the cloud. Ideally, each physical server will have its own memory, network, processing, and storage. A virtual infrastructure might have the same physical server with the appropriate resource allocations per service. Here, the idea is that each microservice instance is isolated from the other and will not compete for resources.

Now, each microservice will need its own set of packages and supporting libraries. This then becomes another challenge when provisioning different machines (physical or virtual) and their operating systems. We then seek to simplify this by packaging each microservice as a container image and deploying it as a container. The container will then encapsulate the details of the technology used to build a service and provide all the CPU, memory, and microservice dependencies needed for operation. This makes the microservice easy to move between testing and production environments and provides environment consistency.

Docker is the go-to container management system and works hand in hand with container orchestration services. Orchestration becomes necessary to run multiple containers across multiple machines. We need to start the correct containers at the correct time, handle storage considerations, and address potential container failures. All of these tasks are not practical to handle manually, so we enlist the services of **Kubernetes**, **Docker Swarm**, and **Marathon** to automate these tasks. It is best to have all the deployment steps automated and be as cost-effective as possible.

Then, we look to implement an integrated pipeline that can handle the continuous delivery of our services, with as minimal effort as possible, while maintaining the highest level of consistency possible.

We have explored quite a bit in this section. We reviewed why we might consider using a microservices approach in our development efforts. Also, we investigated some of the most used technologies for this approach. Now, let us turn our attention to justifying our use of microservices.

Determining the feasibility of implementing microservices

As we explore the microservices approach, we see where it does address certain things, while introducing a few more concerns. The microservices approach is certainly not a savior for your architectural challenges, and it introduces quite a few complexities. These concerns and complexities are generally addressed using design patterns, and using these patterns can save time and energy.

Throughout this book, we will explore the most common problems we face and look at the design pattern concepts that help us to address these concerns. These patterns can be categorized as follows.

Let us explore what each pattern entails:

- **Integration patterns:** We have already discussed that microservices will need to communicate. Integration patterns serve to bring consistency to how we accomplish this. Integration patterns govern the technology and techniques that we use to accomplish cross-service communications.
- **Database and storage design patterns:** We know that we are in for a challenge when it comes to managing data across our distributed services. Giving each service its own database seems easy until we need to ensure that data is kept consistent across the different data stores. There are certain patterns that are pivotal to us maintaining a level of confidence in what we see after each operation.
- **Resiliency, security, and infrastructure patterns:** These patterns seek to bring calm and comfort to a brewing storm. With all the moving parts that we have identified, it is important to ensure that as many things as possible are automated and consistent in the deployment. Additionally, we want to ensure that security is adequately balanced between the system needs and a good user experience. These patterns help us to ensure that our systems are always performing at peak efficiency.

Next, let us discuss using .NET Core as our development stack for microservices.

Microservices and .NET Core

This book addresses implementing microservices and design patterns using .NET Core. We have mentioned that this architectural style is platform agnostic and has been implemented using several frameworks. Comparably, however, ASP.NET Core makes microservices development very easy and offers many benefits, including cloud integrations, rapid development, and cross-platform support:

- **Excellent tooling:** The SDK required for .NET development can be installed on any operating system. This is further complemented by their lightweight and open source development tool, **Visual Studio Code**. You can easily create an API project by running `dotnet new webapi` on your computer. If you prefer the fully powered Visual Studio IDE, then you might be limited to Windows and macOS. You will have all the tools you need to be successful regardless of the operating system.

- **Stability:** At the time of writing this book, the latest stable version is *.NET 7*, with standard term support. The .NET development team is always pushing the envelope and ensuring that reverse compatibility is maintained with each major version release. This makes updating to the next version much less difficult, and you need not worry about too many breaking changes all at once.
- **Containerization and scaling:** ASP.NET Core applications can easily be mounted on a *Docker* container, and while this is not necessarily new, we can all appreciate a guaranteed render speed and quality. We can also leverage Kubernetes and easily scale our microservices using all the features of K8s.

.NET development has come a long way, and these are exciting times to push the boundaries of what we can build, using their tools and services.

Summary

By now, I hope you have a better idea of what microservices are, why you may or may not end up using this architectural style, and the importance of using design patterns. In each of the chapters of this book, we will explore how to use design patterns to develop a *solid* and reliable system based on microservices, using .NET Core and various supporting technologies.

We will remain realistic and explore the pros and cons of each of our design decisions and explore how various technologies play integral parts in helping us to tie it all together.

In this chapter, we explored the differences between designing a monolith and microservices, assessed the feasibility of building microservices, and explored why .NET Core is an excellent choice for building microservices

In the next chapter, we will look at implementing the **Aggregator Pattern** in our microservices application.

2

Working with the Aggregator Pattern

In the previous chapter, we looked at some of the key elements that make up microservices. We will turn our attention to more practical applications of these concepts, starting with the **aggregator pattern** and **domain-driven design**. These combine to set a premise for scoping an application being built on microservices design principles.

The aggregator pattern and domain-driven design go hand in hand. For now, we will refer to **domain-driven design** as **DDD**. So, a DDD aggregate is a group of domain objects, combined as a single unit. In practicality, we might have a customer record different from its documents, but it is prudent of us to display all of these bits of data as a single point of data, an aggregate.

After reading this chapter, we will be able to do the following:

- Understand DDD
- Understand how to derive domains in a system process
- Understand the importance of aggregates and the aggregate pattern
- Distinguish between aggregates and entities
- Understand value objects and their role in the design process

Technical requirements

The code references used in this chapter can be found in the project repository, which is hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch02>

Ensure that you have at least one of the following software installed on your machine to be able to execute this code (use the links to download and install):

- Visual Studio 2022: <https://visualstudio.microsoft.com/vs/>
- Visual Studio Code: <https://code.visualstudio.com/download>

Exploring DDD and its significance

DDD is a software design approach that encourages us as developers to assess processes and subprocesses and decipher all the atomic elements therein. **Atomic** means that one process might have many moving parts, and while they all combine to give one output, they have their own routines to carry out. Each subprocess can be seen as self-governing and can further be attributed to a *domain*. This motivates us to break up a monolith into independent microservices that do their own thing against their own data. That is a domain.

Before we go much further, let's take some time to explore certain keywords and their definitions:

- **Models:** These are abstractions that define aspects of a domain and are used to solve domain problems. We organize information about the target domain into smaller pieces and call them models. A model is a central point of reference in our design and development process. These models can then be grouped into logical modules and dealt with one at a time. A domain contains too much information for just one model and sometimes, parts of the information can just be omitted. For instance, our healthcare management system does need to capture customer information, but we do not need to know their eye and hair colors. This might be a simple example, but it might get far more complicated than that in a real scenario. Sifting through the relevant parts of the body of knowledge will require close collaboration with developers, domain experts, and fellow designers.
- **Ubiquitous language:** This is a language that is unique to a domain model and is used by team members within the context of activities related to the specific domain. We have already established that models for a domain need to be developed through collaboration with domain experts. Given the difference in skillsets and perceptions, there will be communication barriers. Developers tend to think and speak about concepts relative to programming. They generally think and talk in terms of inheritance, polymorphism, and so on. Unfortunately, domain experts don't usually know or care for any of that. Domain experts will use their own jargon and terms that developers will not understand. This gap in communication does not bode well for any team.
- **Bounded context:** This defines boundaries in a system or subsystem that informs the work that a particular team will carry out and the focal point of their efforts, within their domain. A *bounded context* is a logical boundary of a domain, where terms and rules apply. All terms, definitions, and concepts inside this boundary form the *ubiquitous language*. Establishing bounded contexts is a core step in DDD, and it is strategically used in scoping large models, in

large teams. In DDD, we subdivide our larger models into bounded contexts and then we scope the relationships that exist in between. Context mapping in DDD can be confusing without real-world examples. Let's use our healthcare management system as a sample implementation for two scenarios that use bounded context maps and learn to analyze the relationships between the maps. Say we have the context of document management and patient appointments. Both have unrelated and related concepts. Documents only exist in a document management system but will have a reference to patients. Context mapping is a common strategy used in DDD to depict the relationships that exist between bounded contexts.

Figure 2.1 shows a relationship between two domains:

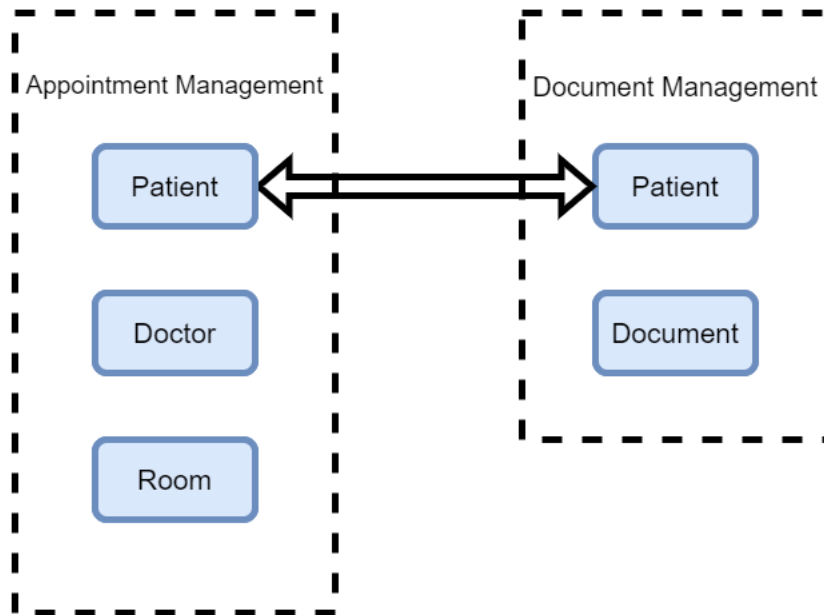


Figure 2.1 – Each domain is standalone, but sometimes data overlaps. Both appointment and document management need patient data

Let's take a step back from microservice design patterns and assess what it takes to just build software. A domain is a category of business rules and operations. If your software is to be used in a bank, then the domain is banking; if it is used in a hospital, then the domain is healthcare.

So, the software we develop must complement the domain in order to solve the overall problem. The core concepts and elements of the domain must be present in the software's design and models.

Exploring the pros and cons of DDD

DDD is a big commitment. It promotes focus on smaller, individual pieces of a domain, and the resulting software is more flexible. It breaks our application into smaller chunks and allows us to more easily modify application parts and components, with fewer side effects. The application's code tends to be well organized and highly testable, and the business logic for the domain is isolated to that particular code base. Even if you don't use DDD end to end for a project, the principles can be beneficial to your application implementation activities. DDD is best used for breaking down complex business logic. It is not suitable for applications with simple requirements and business logic for creating, adding, updating, and deleting data. DDD is time-consuming and requires expert-level domain knowledge. So, bear in mind that non-technical resource persons will be required and have to be available throughout the duration of a project.

Now that we understand DDD at a high level, let's explore how the concepts that it promotes tie into microservice design patterns.

DDD and microservices

Implementations of applications that have been scoped using DDD are best implemented through microservices. By now, we can appreciate that a microservice architecture promotes dividing one large concept for an application into self-contained and independent services. So, to use the concept of boundaries and context, each microservice serves its own bounded context. Each one will have its own models, language, business rules, and technology stack.

This is not a catch-all fix though, since perfect alignment between a microservice and a bounded context might not always be true, but some applications, including ours, are perfect candidates for microservices and DDD.

We can consider a number of scenarios where we can isolate certain services that are not the most obvious ones and wouldn't have been originally scoped as bounded contexts. Take, for example, email and alert systems. It is easy enough to place that logic and functionality in the web application, such that when an appointment is submitted, we send confirmations to our patients and alerts to the staff. This is reasonable, but we could also create separate message queue-based services that serve the sole purpose of delivering these messages. That way, the web application has even less responsibility, and we run less risk of inadvertently modifying UI logic when addressing an email or alert maintenance concern.

Ultimately, because DDD suggests that we separate contexts into standalone tranches, a microservice architecture is the perfect development pattern to support this ambition. Bear in mind that DDD serves as an initial guideline to carve out business rules that can stand on their own, and each microservice will be developed to support that set of business rules, as well as implement supporting services in the most efficient and loosely coupled manner.

Now that we can appreciate how DDD and microservices go hand in hand, let's begin looking into the aggregator pattern and how we can begin assessing the different models and data that need to be captured.

The purpose and use of aggregate patterns

The *aggregate pattern* is a specific software design pattern within DDD. It promotes the collection of related entities and aggregates them into a unit.

Aggregates make it easier to define ownership of elements in large systems. Without them, we run the risk of sprawling and trying to do too much. After we have identified the different contexts in the domain, we can then begin to extract the exact data we need from potentially multiple contexts and sources and model them.

Aggregates and aggregate roots

An aggregate comprises one or more entities and value object models that, in one way or another, interact. This interaction then encourages us to treat them as a unit for data changes. We also want to ensure that, at all times, there is consistency in the aggregate before making changes. In our concept of a healthcare management system, we have already scoped that we have a patient, who more than likely also has an address. A set of changes to a patient record and their address should be treated as a single transaction. We also want to consider that aggregates have roots or a parent object for all other aggregate members.

Aggregates make it easier to enforce certain rules for data and validation across multiple objects. So, in our example so far, a patient can have multiple addresses but needs to have at least one to be in a valid state. These kinds of constraints are easier to apply across the board from a higher level of the root. It is also easier to ensure that data changes follow **ACID (Atomicity, Consistency, Isolation, and Durability)** principles. We will explore these more in a later chapter.

The aggregate roots also help us to maintain invariants. Invariants are non-negotiable conditions that ensure that a system is consistent. A good metric to use in determining what should be an aggregate root is to consider whether deleting a record should trigger a cascade deletion of other objects in the hierarchy. In essence, our aggregate root represents a cluster of associated objects, treated as a unit for data-related changes.

Figure 2.2 shows a relationship between a root and a non-root entity:

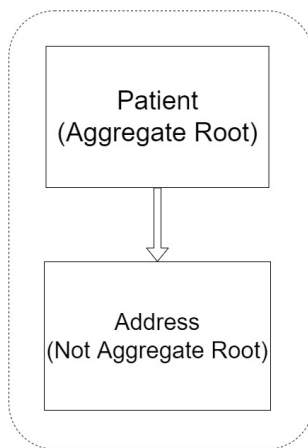


Figure 2.2 – Patient is our aggregate root, and using Address as an example, we have other entities related to the root object

As you can see, the **Patient** entity plays the role of the aggregate root and has a relationship with the **Address** entity.

It is always good to use diagrams to visualize how different entities and objects relate. This will assist in giving a broader understanding during the scoping exercise, as we assess the role that each model will play in the domain and how our data strategies will come to life.

Now, let's turn our sights on exploring relationships a bit more. We need to look at other parts of the system and determine what should be a child, a parent, or simply a sibling.

Relationships in aggregates

Considering that aggregates are clusters of related objects, it is important for us to fully appreciate the relationships between these objects. Generally speaking, we consider relationships to be two-way associations – that is, object A is related to object B, and vice versa. For example, a patient has an appointment, and we need them to have an appointment. This way of thinking might contradict the tenets of DDD though, in the sense that we are trying to simplify things, and a two-way relationship might add complexity to the mission at hand.

In DDD, we want to promote the notion that one-way relationships will suffice. If we go for two-way relationships, which might very well happen, we need to ensure that the added complexity is justifiable. A relationship allows one object to traverse the details of the other. This means, for the patient, we should be able to see all the details of the related appointment, but we do not need to see all the details of the patient when going in the other direction. A simple ID reference to the patient can suffice. If we introduce a full bidirectional relationship, then we create a direct dependency between appointments

and patients, which isn't necessarily true. A good measure to use in defining our models is to ask, *"Can I define this object without needing the other?"*

A good guideline to use to govern our decisions is that our aggregates should always flow in a single direction from the root to its dependents, and never the other way around.

We have looked at relationships that are obvious and tightly knit, but what happens when relationships are more widely spread? Let's discuss how we handle relationships that traverse aggregates.

Handling relationships that span aggregates

We know that aggregates are boundaries between logical groupings in our applications. We enforce these segregations by restricting direct references to objects in the aggregate if they are not the root. In our example of the patient and the address, we can safely make the patient record reference the address, making our address an entity or value object.

The key thing to note in this association is that the only way to get the correct address for a patient is by searching the patient record. The address won't be referenced anywhere else. A patient, however, can be referenced from other aggregates, such as from an appointment record or a document. So, it is important to understand when a bit of data can be referenced directly, or not, and we can use this to guide what aggregates can be made central to our application's design.

Think about designing our data objects for a database access library such as Entity Framework, where we have to consider the implications of all **Create, Read, Update, and Delete (CRUD)** operations against our data. Based on the general design pattern of our entity classes, we would place navigation objects inside of both entities in the relationship, but this could lead to cascading issues if not managed properly. This is a key design decision to make, as it is sometimes better to remove the navigation object, sacrificing some of the magic of Entity Framework, and retain greater control and predictability of how our models will interact. By retaining only the foreign key ID reference, we can better enforce one way for aggregates to relate to non-root entities. Now that we have an understanding of aggregates, aggregate roots, and how we formulate them, let's explore entities and compare how they differ from aggregates.

Aggregates versus entities

As discussed, aggregates are conceptually composed of entities and value objects that relate to each other in some way. We need to understand fully what an entity is and the role that it plays in our development process.

Entities and why we need them

Decisions made in DDD are driven by behavior, but behaviors require objects. These objects are referred to as *entities*. An entity is a representation of data in your system, something that you need to be able to retrieve, track changes on, and store. Entities also typically have an identity key, most

commonly an auto-incrementing integer or a GUID value. In code, you would want to create a base entity type that allows you to set the desired key type relative to the derived type. Here is an example of a `BaseEntity` class in C#:

```
public abstract class BaseEntity<TId>
{
    public TId Id { get; set; }
}
```

The `BaseEntity` class will take a generic type parameter, which allows us the flexibility of setting the ID type as needed. We also ensure that we set the class type as `abstract` to prevent independent instantiation of `BaseEntity`.

It is one thing to have a mental map of the entities and data persistence strategy that you intend to implement. But modeling and coding are oftentimes different activities when the buck stops. Given the unique demands of DDD, there are patterns and techniques that can be employed to ensure that certain technical attributes are implemented in our DDD-styled entities. It is important to establish the central entity for your system and then design all other parts around that one. For instance, it could be said that appointment booking is the most central operation of our system, so all other entities are just to be referenced. In another scope, patient record management could be seen as the most integral part, so we would want to focus on making that aspect as robust as possible.

These scenarios show that the decisions that you make need to be relative to the mission at hand. One size does not fit all, and your design considerations need to be what is best for your overall context. Beyond that, we need to ensure that we have a good grasp of the operations to be carried out before we can start implementing domain events, repositories, factories, and any other business logic-related elements.

Now, let's look at the concrete uses of entities in our DDD-styled system. We need to understand how entities should be employed and their actual uses in our system.

Practical uses of entities in code

An entity is primarily defined by its identity and is important to the domain model. It is very important that we design and implement entities carefully.

An entity represents data that might need to traverse multiple microservices and, as a result, needs to have an identity value that can uniquely identify it in any system. We generally use sequential integers for our ID values in a relational database, but given this constraint, we cannot rely on that sequential value being used in multiple databases. For this reason, we usually employ the use of a **GUID**, which is a generally randomly generated block of an alphanumeric string. It is not sequential, but it is easier to count on it being consistent, since we set it in code rather than rely on a database to set it.

The same identity can be modeled across multiple microservices. In a scenario where an identity value is shared across microservices, this doesn't necessarily suggest that the same attributes and behaviors will be the same in each microservice or bounded context. For instance, the patient entity in the Patient Management microservice might contain all of the key attributes and behaviors of the patient we would have scoped, but the same entity in the appointment booking microservice might only need minimal data and behaviors, as needed by the appointment booking process. The entity's contents will always be relative to the requirements of the microservices or bounded context.

Domain entities generally implement behaviors in the form of methods, as well as data attributes. In DDD, domain entities need to implement behaviors and logic that are only useful for the specific domain or entity. In the case of our `patient` class, there must be validation-related tasks and operations implemented as methods. The methods will handle invariants and rules of the entity so that they are not spread across the application layer.

At this point, we have started to see that our entity models might not just be classes with properties but might also implement behaviors. Next, we take a look at anemic and rich domain models and how we implement them.

A rich domain model versus an anemic domain model

At this point, it is good to appreciate the difference between an *anemic model* and a *rich model*. A rich model is more behavioral in nature and fits the description of what we have described – that is, a model that implements methods for tasks relative to the model within the domain. An anemic model is more data-centric and tends to only implement properties. Anemic models are usually implemented as child entities, where there isn't any special logic. The logic is implemented in the aggregate root, or the business logic layers. Anemic domain models are implemented using procedural style programming. This means that the model has no behaviors and only exposes properties for the data points that it will be storing. We then tend to put all our behavior in `service` objects in the business layer and run the risk of ending up with spaghetti code, thus losing the advantages that a domain model provides.

Let's take a look at a simpler or anemic entity model:

```
public class Patient : BaseEntity<int>
{
    public Patient(string name, string sex, int?
        primaryDoctorId = null)
    {
        Name = name;
        Sex = sex;
        PrimaryDoctorId = primaryDoctorId;
    }
    public Patient(int id)
```



```
    {
        Id = id;
    }
    private Patient() // required for EF
    {
    }
    public string Name { get; private set; }
    public string Sex { get; private set; }
    public int? PrimaryDoctorId { get; private set; }
    public void UpdateName(string name)
    {
        Name = name;
    }
    public override string ToString()
    {
        return Name.ToString();
    }
}
```

It is a good idea to enforce encapsulation in your class by requiring values to be set upon instantiation of the object. At the end of the day, your decision on how rich or anemic your model is depends on the use or general operations of the microservice. Anemic models might be perfect for more simple CRUD services, where DDD might be a stretch for what you need to design the system. They are more simply used to model our persistence models, since we only use the models for data storage and CRUD purposes. In the following code block, we will look at an example of the `Appointment` class being implemented as a rich domain model, including logic to handle certain key operations on the data.

The example has been broken into smaller chunks of code to highlight the different general components of a rich data model:

```
public class Appointment : BaseEntity<Guid>
{
    public Appointment(Guid id,
        int appointmentTypeId,
        Guid scheduleId,
        int doctorId,
        int patientId,
        int roomId,
        DateTime start,
```

```
        DateTime end,  
        string title,  
        DateTime? dateTimeConfirmed = null)  
{  
    Id = id;  
    AppointmentTypeId = appointmentTypeId;  
    ScheduleId = scheduleId;  
    DoctorId = doctorId;  
    PatientId = patientId;  
    RoomId = roomId;  
    Start = start;  
    End = end;  
    Title = title;  
    DateTimeConfirmed = dateTimeConfirmed;  
}
```

At a minimum, we need to ensure that we use constructors to enforce object creation rules. We list the values that are needed at a minimum and do the assignments upon creation. It is also common practice to include validation checks and/or default values at this stage:

```
    public void UpdateRoom(int newRoomId)  
    {  
        if (newRoomId == RoomId) return;  
        RoomId = newRoomId;  
    }  
    public void UpdateStartTime(DateTime newStartTime,)  
    {  
        if (newStartTime == Start) return;  
        Start = newStartTime;  
    }  
    public void Confirm(DateTime dateConfirmed)  
    {  
        if (DateTimeConfirmed.HasValue) return;  
        DateTimeConfirmed = dateConfirmed;  
    }  
}
```

We also have some examples of behaviors that we implement in the method. Traditionally, you would want to implement these in the business logic layer or a repository, but for a rich data model, we equip it with the methods it needs to morph its own data as needed. We can also implement our own validation rules in these methods.

Now that we understand what an entity is, the rules surrounding how they are created, and have general guidelines on how they can be implemented, we can now explore value objects and how they differ from entities in our domain model.

Understanding and using value objects

We have observed the main attributes that entity objects should be identified by, which are continuity and identity, and not necessarily their values. This brings us to ask the question, *what do we call objects that are indeed defined by their values?* These are value objects. They too have their place in the domain model, as they are used to measure and quantify parts of the domain. They do not boast identity keys in the same way that entities do, but their keys are formed through the composition of the values of all their properties, hence the name *value objects*.

Given that the data they store is so important in defining their identity and uniqueness in our system, it is of the utmost importance that these objects never change once created and are immutable. It is also important to understand the differences between entity models and value objects.

Figure 2.3 shows a comparison between entities and value objects:

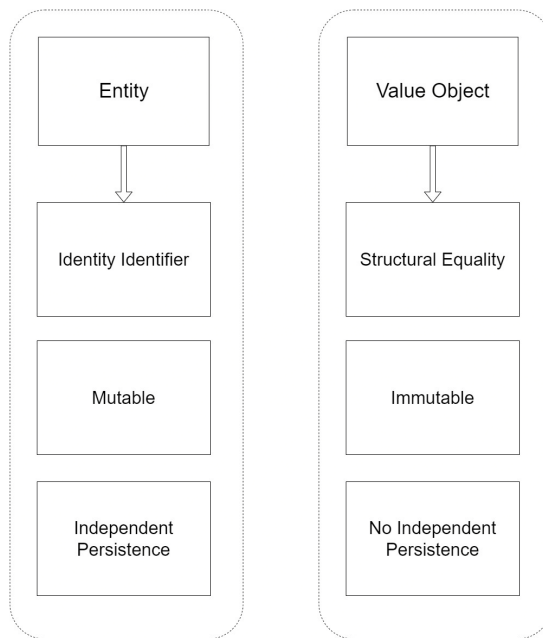


Figure 2.3 – Value objects are fundamentally different from domain entities, and it is important to appreciate these differences

Immutability means that the object's properties should never change once these objects have been created. Instead, another instance should be created with the new intended values when necessary. If these objects need to be compared, we can do so by comparing all the values. This has become easier and a bit more practical since the introduction of `record` types in C# 10. Records are different from class and structs in that `record` types are based on value-based equality for comparisons. Two `record` objects are considered equal if the record definitions are identical and the values in both records are equal for every field.

Value objects are allowed to have methods and behaviors, but their scope should be limited. Methods should only compute and never change the state of the value object, or values therein, and note that it is immutable. Just remember, if new values are required, we should create a new object for that purpose.

Let's delve a bit deeper into the basics of value objects. The truth is that we use them all the time in our development tasks, probably without noticing. A common example of these would be string objects. A string in .NET and most other languages is a collection of characters, or a `char` array. This character collection gives the string a value or a specific meaning. If we change one value of the character array, or reorder them, then we change the whole meaning of the string. In .NET, it is relatively easy to augment these values through string manipulation methods that allow us to change the letter cases or extract a part of the string. In doing these operations, we don't actually change the value of the string, but we actually end up getting a whole new object with the new values. As we said, immutable objects do not change in value, but a new object must be created if a change is desired.

When scoping value objects for your system, it is important to assess all the information that is needed from the start to make them airtight. A good example of making sure the information is correct would be weight. It is easy enough to store data on a patient and state that they weigh 50. But 50 by itself is useless, considering how many possible units of measurement there are. So, in practicality, this value has no meaning without the unit. Fifty lbs (pounds) is an entirely different measurement from 50 kg. We would also need to ensure that the `class` or `record` type being used to store this information places restrictions on which value can be updated at one time. For instance, changing the numeric value is fine, as a person may have gained or lost weight, but allowing the same flexibility to update the unit by itself can have a deeper impact on what the numeric value really means in terms of the weight change. It would be a good idea to ensure that the unit can only be set when simultaneously setting the number value of the weight. You can also consider appointment scheduling. We should never entertain the acceptance of a start date and time without an accompanying end date and time. If we set this appointment start and end date time combination in a `record` type, then it will make it much easier to carry out equality checks for clashes, and we don't need to clutter our code with overloads and excess logic to ensure that the appointment times are acceptable for the system.

The most important goal here, once again, is to ensure that the state of the value object is not changed after it is created. So, whenever you choose to use a `record` or a `class` type, the values should be set through the constructor at the time of object creation, and all validation and invariant checks need to be in the constructor as well. Values should also generally be set to be read-only types in order to guard against modifications beyond that. Do remember though that, with a `class` type, you will need to ensure that you include appropriate logic for equality comparison, whereas a `record` type comes with that built in, since it is based on value-based equality semantics.

We have looked into value objects and what makes them so much different from entities. We have also reviewed the best ways to implement them in C# code, to ensure their unique characteristics.

Summary

In this chapter, we explored quite a few things. We sought to understand the fundamentals of DDD and what makes it so different from a regular software design approach. We then broke down the elements of DDD into what the data objects and expectations would be. Finally, we reviewed value objects and explored under what circumstances we would formulate them, and the best ways to implement them in C#.

In our next chapter, we will explore the chain of responsibility pattern and how synchronous communication is best implemented between our microservices.

Synchronous Communication between Microservices

In the previous chapter, we learned about aggregator patterns and how they help us scope our storage considerations for our microservices. Now, we will focus on how our services communicate with each other during the application's runtime.

We have already established that microservices should be autonomous and should handle all operations relating to tranches of the domain operations that are to be completed. Even though they are autonomous by design, the reality is that some operations require input from multiple services before an end result can be produced.

At that point, we need to consider facilitating communication, where one service will make a call to another, wait on a response, and then take some action based on that said response.

After reading this chapter, we will be able to do the following:

- Understand why microservices need to communicate
- Understand synchronous communication with HTTP and gRPC
- Understand the disadvantages of microservice communication

Technical requirements

The code references used in this chapter can be found in the project repository, which is hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch03>.

Use cases for synchronous communication

Considering everything that we have covered so far regarding service independence and isolation, you are probably wondering why we need to cover this topic. The reality is that each service covers a specific tranche of our application's procedures and operations. Some operations have multiple steps and parts that need to be completed by different services, and for this reason, it is important to properly scope which service might be needed, when it will be needed, and how to best implement communication between the services.

Interservice communication needs to be efficient. Given that we are talking about a number of small services interacting to complete an activity, we need to ensure that the implementation is also robust, fault-tolerant, and generally effective.

Figure 3.1 gives an overview of synchronous communication between microservices:

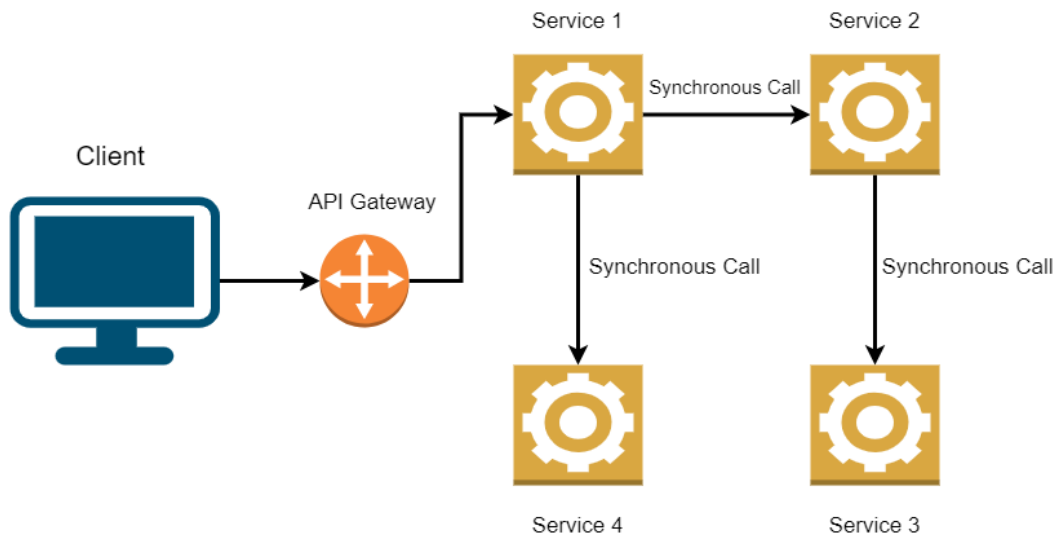


Figure 3.1 – One request might require several follow-up calls to additional services

Now that we understand why services need to communicate, let us discuss the different challenges that surround interservice communication.

Challenges of microservice communication

At this point, we need to accept that we are building a far more complex and distributed system than a monolith would permit. This comes with its own challenges when navigating the general request-response cycle of a web service call, the appropriate protocols to be used, and how we handle failures or long-running processes. Generally speaking, we have two broad categories of communication

in *synchronous* and *asynchronous* communication. Beyond that, we need to scope the nature of the operation and make a call accordingly. If the operation requires an immediate response, then we use synchronous techniques, and for long-running processes that don't necessarily need a response immediately, we make it asynchronous.

As mentioned before, we need to ensure that our interservice operations boast of the following:

- **Performance:** Performance is always something we have in the back of our minds while developing a solution. Individually, we need each service to be as performant as possible, but this requirement extends to communication scenarios, too. We need to ensure that when one service calls another, the call is done using the most efficient method possible. Since we are predominantly using *REST APIs*, *HTTP* communication will be the go-to method. Additionally, we can consider using *gPRC*, which allows us to call a REST API with the benefit of higher throughput and less latency.
- **Resilient:** We need to ensure that our service calls are done via durable channels. Remember that hardware can fail, or there can be a network outage at the same time as the service call is being executed. So, we need to consider two patterns that will make our services resilient, *Retry* and *Circuit Breaker*:
 - **Retry pattern:** Transient failures are common and temporary failures can derail an operation's completion. However, they tend to go away by themselves, and we would prefer to retry the operation a few times, as opposed to failing the application's operation completely. Using this pattern, we retry our service call a few times, based on a configuration, and should we not have any success, trigger a timeout. For operations that augment the data, we will need to be a bit more careful since the request might get sent and a transient failure might prevent a response from being sent. This doesn't mean that the operation wasn't actually completed, and retrying might lead to unwanted outcomes.
 - **Circuit breaker pattern:** This pattern is used to limit the number of times that we try to make the service call. Multiple calls might fail because of how long a transient failure takes to resolve itself, or the number of requests going to the service might cause a bottleneck in the available system resources and allocations. So, with this pattern, we could configure it to limit the amount of time we spend trying to call one service.
- **Traced and monitored:** We have established that a single operation can span multiple services. This brings another challenge in monitoring and tracing activities through all the services, from one originating point. At this point, we need to ensure that we are using an appropriate tool that can handle distributed logging and aggregate them all into a central place for easier perusal and issue tracking.

Now that we have a clearer picture of why we need to communicate and what challenges we might face, we will look at practical situations for synchronous communication.

Implementing synchronous communication

Synchronous communication means that we make a direct call from one service to another and wait for a response. Given all the fail-safes and retry policies that we could implement, we still evaluate the success of the call based on us receiving a response to our call.

In the context of our hospital management system, a simple query from the frontend will need to be done synchronously. If we need to see all the doctors in the system to present a list to the user, then we need to invoke a direct call to the doctors' API microservice, which fetches the records from the database and returns the data with, more than likely, a 200 response that depicts success. Of course, this needs to happen as quickly and efficiently as possible, as we want to reduce the amount of time the user spends waiting on the results to be returned.

There are several methods that we can use to make an API call, and HTTP is the most popular. Support for HTTP calls exists in most languages and frameworks, with C# and .NET not being exceptions. Some of the advantages of HTTP come through standardized approaches to reporting; the ability to cache responses or use proxies; standard request and response structures; and standards for response payloads. The payload of an HTTP request is, generally, in JSON. While other formats can be used, JSON has become a de facto standard for HTTP payloads given its universal, flexible, and easy-to-use structure for data representation. RESTful API services that adhere to HTTP standards will represent information available in the form of resources. In our hospital management system, a resource can be a *Doctor* or *Patient*, and these resources can be interacted with using standard HTTP verbs such as GET, POST, PUT, or DELETE.

Now that we can visualize how synchronous HTTP communication happens in a microservice, let us take a look at some coding techniques that we can use in .NET to facilitate communication.

Implementing HTTP synchronous communication

In this section, we will be looking at some code examples of HTTP communication for when we want to call an API in a .NET application.

The current standard for HTTP-based API communication is **REST (Representational State Transfer)**. RESTful APIs expose a set of methods that allow us to access underlying functionality via standard HTTP calls. Typically, a call or *request* consists of a URL or endpoint, a verb or method, and some data.

- **URL or endpoint:** The URL of the request is the address or the API and the resource that you are trying to interact with.
- **Verb or method:** The most commonly used verbs are GET (to retrieve data), POST (to create a record), PUT (to update data), and DELETE (to delete data).
- **Data:** Data accompanies a call, when necessary, in order to have a complete request. For instance, a POST request to the booking microservice would need to have the details of the booking that needs to be created.

The next major part of this conversation comes in the form of a response or HTTP status code. HTTP defines standard status codes that we use to interpret the success or failure of our request. The categories of status codes are listed as follows:

- **1xx (Informational):** This communicates protocol-level information.
- **2xx (Success):** This indicates that the request was accepted, and no errors occurred during processing.
- **3xx (Redirection):** This indicates that an alternative route needs to be taken in order to complete the original request.
- **4xx (Client Error):** This is the general range for errors that arise from the request, such as poorly formed data (400) or a bad address (404).
- **5xx (Server Error):** These are errors that indicate that the server failed to complete the task for some unforeseen reason. When constructing our services, it is important that we properly document how requests should be formed, as well as ensure that our responses are in keeping with the actual outcomes.

In **C#**, we have access to a library called *HttpClient*, which provides methods that map to each HTTP verb, allowing us to bootstrap our calls and pass in the data that we require for the operation. It is also a good idea to have a list or directory of endpoints to be used. This can be accomplished through a combination of listing addresses in `appsettings.json`, and then we can have a class of constants, where we define the behaviors or resources for the service.

Our `appsettings.json` file would be decorated with the following block, which allows us to access the service address values from anywhere in our app:

```
"ApiEndpoints": {  
  "DoctorsApi": "DOCTORS_API_ENDPOINT",  
  "PatientsApi": "PATIENTS_API_ENDPOINT",  
  "DocumentsApi": "DOCUMENTS_API_ENDPOINT"  
}
```

The values per service configuration will be relative to the published address of the corresponding web service. This could be a localhost address for development, a published address on a server, or a container instance. We will use that base address along with our endpoint, which we can define in our static class.

In order to have consistency in our code, we can implement baseline code for making and handling HTTP requests and responses. By making the code generic, we pass in our expected class type, the URL, and whatever additional data might be needed. This code looks something like this:

```
public class HttpRepository<T> : IHttpRepository<T> where T  
: class
```

```
{
    private readonly HttpClient _client;

    public HttpRepository(HttpClient client)
    {
        _client = client;
    }

    public async Task Create(string url, T obj)
    {
        await _client.PostAsJsonAsync(url, obj);
    }

    public async Task Delete(string url, int id)
    {
        await _client.DeleteAsync($"{url}/{id}");
    }

    public async Task<T> Get(string url, int id)
    {
        return await _client.GetFromJsonAsync<T>($"{url}/{id}");
    }

    public async Task<T> GetDetails(string url, int id)
    {
        return await _client.GetFromJsonAsync<T>($"{url}/{id}/details");
    }

    public async Task<List<T>> GetAll(string url)
    {
        return await _client.
            GetFromJsonAsync<List<T>>($"{url}");
    }

    public async Task Update(string url, T obj, int id)
```

```
{  
    await _client.PutAsJsonAsync($"{url}/{id}", obj);  
}  
}
```

The `HttpClient` class can be injected into any class and used on the fly, in any ASP.NET Core application. We create a generic HTTP API client factory class as a wrapper around the `HttpClient` class in order to standardize all RESTful API calls that will originate from the application or microservice. Any microservice that will need to facilitate RESTful communication with another service, can implement this code and use it accordingly.

Now that we have an idea of how we handle calls through HTTP methods, we can review how we set up gRPC communication between our services.

Implementing gRPC synchronous communication

At this point, we should be comfortable with REST and the HTTP methods of communicating with and between our microservices. Now, we will pivot into exploring gRPC in a bit more detail.

RPC is short for **Remote Procedure Call**, and it allows us to call another service in a manner that resembles making a method call in code. For this reason, using gRPC in a distributed system works well. Communications can happen much quicker, and the entire framework is lightweight and performant from the jump. This is not to say that we should wholly swap all REST methods for gRPC. We know by now that we simply choose the best tool for our context and make it work accordingly, but it is good to know that gRPC is best used for scenarios where efficiency is paramount.

Indeed, gRPC is fully supported by ASP.NET Core, and this makes it a great candidate for use in our .NET Core-based microservices solution. Given its contract-based nature, it naturally enforces certain standards and expectations that we try to emulate when creating our own REST API service classes with interfaces. It starts with a file called a **proto**, which is the contract file. This contract outlines the properties and behaviors that are available and is exposed by the server (or broadcasting microservice).

The code snippets are as follows (parts have been omitted for brevity):

// Protos/document-search-service.proto

```
syntax = "proto3";  
option csharp_namespace = "HealthCare.Documents.Api.Protos";  
package DocumentSearch;  
service DocumentService {  
    rpc GetAll (Empty) returns (DocumentList);  
}
```

```
rpc Get (DocumentId) returns (Document);
}

message Empty{}

message Document {
    string patientId = 1;
    string name = 2;
}

message DocumentList{
    repeated Document documents = 1;
}

message DocumentId {
    string Id = 1;
}
```

This proto class defines some methods that we want to allow for the document management service. We have defined a method to retrieve all documents, and another that will retrieve a document based on the provided ID value. Now that we have our proto defined, we will need to implement our methods in an actual service class:

```
public class DocumentsService : DocumentService.
    DocumentServiceBase
    {
        public override Task<Document> Get(DocumentId request,
            ServerCallContext context)
        {
            return base.Get(request, context);
        }

        public override Task<DocumentList> GetAll(Empty
            request, ServerCallContext context)
        {
            return base.GetAll(request, context);
        }
    }
```

In each method, we can carry out the actions needed to complete the operation, which, in this context, will be our database query and potential data transformation.

Next, we need to ensure that the calling service has a representation of the contract and that it knows how to make the calls. The following is a sample of how we would connect to the gRPC service at its address, create a client, and make a request for information:

```
// The port number must match the port of the gRPC server.
using var channel = GrpcChannel.ForAddress("ADDRESS_OF_
    SERVICE");
var client = new DocumentService.DocumentService
    Client(channel);
var document = await client.Get(
    new DocumentId { Id = "DOCUMENT_ID" });
```

Now that we have seen some code examples of gRPC, let us look at a head-to-head comparison of HTTP REST and gRPC.

HTTP versus gRPC communication

We have seen examples of how we can interact with our services via the HTTP or RESTful methods and the gRPC protocol. Now, we need to have a clearer picture of when we would choose one method over the other.

The benefits of using REST include the following:

- **Uniformity:** REST provides a uniform and standard interface for exposing functionality to subscribers.
- **Client-server independence:** There is clear independence between the client and the server applications. The client only interacts with URIs that have been exposed or are needed for functionality. The server is oblivious to which clients might be subscribing.
- **Stateless:** The server does not retain information about the requests being made. It just gets a request and produces a response.
- **Cacheable:** API resources can be cached to allow for faster storage and retrieval of information per request.

Note that gRPC does have its merits as to why it is being touted as a viable alternative to REST communication. Some of these merits include the following:

- **Protocol buffers:** Protocol buffers (or protobufs for short) serialize and deserialize data as binary, leading to higher data transmission speeds and smaller message sizes, given the much higher compression rate.

- **HTTP2:** HTTP2, unlike HTTP 1.1, supports the expected request-response flow, as well as bidirectional communication. So, if a service receives multiple requests from multiple clients, it can achieve multiplexing by serving many requests and responses simultaneously.

You can see the obvious and not-so-obvious advantages of using either method for web service creation and communication. Some developers have deemed gRPC the future, given its lighter weight and more efficient nature. However, REST APIs remain far more popular, are easier to implement, and have more third-party tool support for code generation and documentation. Most microservice architecture-based projects are built using REST APIs, and quite frankly, unless you have specific requirements that lead to a gRPC implementation, it might be a risk to adopt gRPC at a larger scale at this stage.

Given that we have explored so much about synchronous communication and the most common methods that are used to facilitate it, let us look at some of the disadvantages that surround having our microservices talk to each other.

Disadvantages of synchronous communication between microservices

While it is the go-to method for service-to-service communication, it might not always be the best option at that moment. Most cases might even prove that it is not the best idea to begin with.

Do remember that our users will be waiting on the result of a service-to-service call to manifest itself to them on the user interface. That means, for however long this communication is occurring, we have a user or users sitting and waiting on the interface to continue loading and furnish a result.

From an architectural point of view, we are violating one of the key principles of microservice design, which is having services that stand on their own, without knowing much, or preferably, anything, about each other. By having two services speak, there is knowledge about another service and implementation details being defined, which have very little to do with the service's core functionality. Also, this introduces an undesirable level of tight coupling between services, which increases exponentially for each service that needs to speak to another service. Now a change to one service can have undesired functionality and maintenance effects on the others.

Finally, if we end up with a chain of service calls, this will make it more difficult to track and catch any errors that occur along the calls. Imagine that we implement a form of *Chain of Responsibility* with our service calls where one service calls another, and the result is used to call another, and so on. If we have three service calls happening back-to-back and the first one fails, we will get back an error and won't be able to determine at which point this error occurred. Another issue could be that we had successful calls, and the first error breaks the chain, thus wasting the usefulness of what has transpired in the chain before that.

It is always good to understand the pros and cons of the techniques that we employ. I do agree that synchronous communication is sometimes necessary, but we must also be aware of the additional development effort, in both the short and long term, that will be needed as a result of its employment. At this point, we begin to think of alternatives such as asynchronous communication and event-driven programming.

Summary

In this chapter, we explored quite a few things. We sought to understand what synchronous communication between web resources is, the protocols that are most commonly used, and the potential pros and cons of these techniques. We looked, in detail, at how HTTP communication occurs and can be implemented using C# and compared that with gRPC techniques. Additionally, we compared the two to ensure that we know when the best time would be to use either.

In the next chapter, we will explore asynchronous communication between services, the best practices, and what problems could be solved through this service-to-service communication method.

Asynchronous Communication between Microservices

We have just reviewed synchronous communication between microservices and the pros and cons of that method. Now, we will take a look at its opposite counterpart, asynchronous communication.

Synchronous communication is needed at times and, based on the operation being carried out, can be unavoidable. It does introduce potentially long wait times as well as potential break points in certain operations. At this point, it is important to properly assess the operation and decide whether immediate feedback from an additional service is required to continue. Asynchronous communication means that we send data to the next service but do not wait for a response. The user will be under the impression that the operation was completed, but the actual work is being done in the background.

From that review, it is obvious that this method of communication cannot always be used but is necessary to implement certain flows and operations efficiently in our application.

After reading this chapter, we will be able to do the following:

- Understand what asynchronous communication is and when we should use it
- Implement Pub-Sub communication
- Learn how to configure a message bus (**RabbitMQ** or **Azure Service Bus**)

Technical requirements

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch04>

Functioning with asynchronous communication

Let us revisit some vital definitions and concepts before moving forward.

There are basically two messaging patterns that we will employ between microservices:

- **Synchronous communication:** We covered this pattern in the previous chapter, where one service calls another directly and waits for a response
- **Asynchronous communication:** In this pattern, we use messages to communicate without waiting for a response

There are times when one microservice needs another to complete an operation, but at the moment, it doesn't need to know the outcome of the task. Let us think of sending confirmation emails after an appointment has been successfully booked by our health care management system. Imagine the user waiting on the user interface to complete its loading operation and show the confirmation. In between them clicking **Submit** and seeing the confirmation, the booking service needs to complete the following operations:

1. Create an appointment record
2. Send an email to the doctor booked for the appointment
3. Send an email to the patient who booked the appointment
4. Create a calendar entry for the system

Despite our best efforts, the booking service attempting to complete these operations will take some time and might lead to a less than pleasing user experience. We could also argue that the responsibility of sending emails should not be native to the booking service. In the same way, calendar management should stand on its own. So, we could refactor the booking service's tasks as follows:

1. Create an appointment record
2. Synchronously call the email service to send an email to the doctor booked for the appointment
3. Synchronously call the email service to send an email to the patient who booked the appointment
4. Synchronously call the calendar management service to create a calendar entry for the system

Now, we have refactored the booking service to do fewer operations and offload the intricacies of non-appointment booking operations to other services. This is a good refactor, but we have retained and potentially amplified the main flaw in this design. We are still going to wait on the completion of one potentially time-consuming operation before we move on to the next operation, which runs the same risk. At this point, we can consider how important waiting on a response to these actions really is, relative to us entering the booking record in our database, which is the most important operation relative to allowing the user to know the outcome of the booking process.

In this setting, we can make use of an asynchronous communication pattern to ensure that the major operation gets completed and other operations, such as sending an email and entering a calendar entry, can happen eventually, without compromising our user experience.

At a very basic level, we can still implement asynchronous communication using HTTP patterns. Let us discuss how effective this can be.

HTTP asynchronous communication

In seeming contrast to what we have explored so far, we can actually implement asynchronous communication via HTTP. If we assess how HTTP communication works, we form an assessment of success or failure based on the HTTP response we receive. Synchronously, we would expect the booking service to call the email service and then wait on an HTTP 200 OK successful response code before it tries to move to the next command. Synchronously, we would actually try to send the email at the moment, and based on the success or failure of that operation, we would form our response.

Asynchronously, we would let the email service respond with an HTTP 202 ACCEPTED response, which indicates that the service has accepted the task and will carry it out eventually. This way, the booking service can continue its operation based on that promise and spend less time on the operation. In the background, the email service will carry out the task when it gets around to it.

While this does alleviate some of the pressure that the booking service carries, there are other patterns, such as the Pub-Sub pattern, that can be implemented to make this process smoother. Let us review this pattern.

Understanding Pub-Sub communication

The Pub-Sub pattern has gained a fair amount of popularity and acclaim and is widely used in distributed systems. *Pub* is short for *Publisher* and *Sub* is short for *Subscriber*. Essentially, this pattern revolves around publishing data, contextually called messages, to an intermediary messaging system, which can be described as resilient and reliable, and then having subscribing applications monitor this intermediary system. Once a message is detected, the subscribing application will conduct its processing as necessary.

Understanding message queues

Before we explore the Pub-Sub method, we need to understand the basics of messaging systems and how they work. The first model we will look at is called a *message queue*. A message queue is usually implemented as a bridge between two systems, a publisher and a consumer. When a publisher places messages in the queue, a consumer processes the information in the message as soon as it becomes available. Queues enforce a **first-in-first-out (FIFO)** delivery method, so the order of processing can always be guaranteed. This is usually implemented in a one-to-one communication scenario, so a specific queue is provisioned for each subscribing application. Payment systems tend to use this pattern heavily, where the actual sequence of submitted payments matters and they need to ensure the resilience of the instructions. If you think about it, payment systems generally have a very low failure rate. Most of the time when we submit a payment request, we can rest assured that it will be completed successfully at some point in the future.

Figure 4.1 shows a publisher interacting with several message queues.

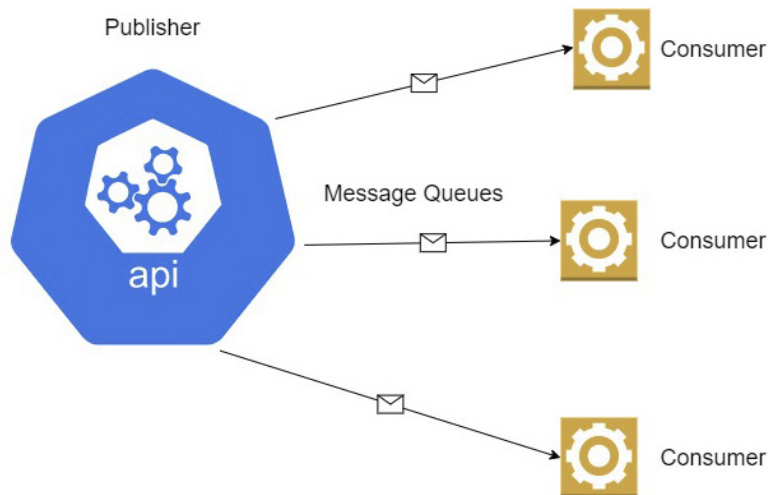


Figure 4.1 – Each message queue ensures that each application gets the exact data it needs and nothing more

The unfortunate thing here is that one bad message can spell trouble for the other messages waiting in the queue, and so we have to consider this potential downside. We also have to consider that we have our first chink in the armor. Messages are also discarded after they are read, so special provisions need to be made for messages that were not processed successfully.

While message queues do have their uses and bring a certain level of reliability to our system design, in a distributed system, they can be a bit inefficient and introduce some cons that we might not be prepared to live with. In that event, we turn our attention to a more distributed messaging setup such as a message bus. We will discuss this next.

Understanding message bus systems

A *message bus*, *event bus*, or *service bus* provides interfaces, where one published message can be processed by multiple or competing subscribers. This is advantageous in scenarios where we need to publish the same data to multiple applications or services. This way, we do not need to connect to multiple queues to send a message, but we can have one connection, complete one *send* action, and not worry about the rest.

Figure 4.2 shows a publisher interacting with a message, which has several consumers.

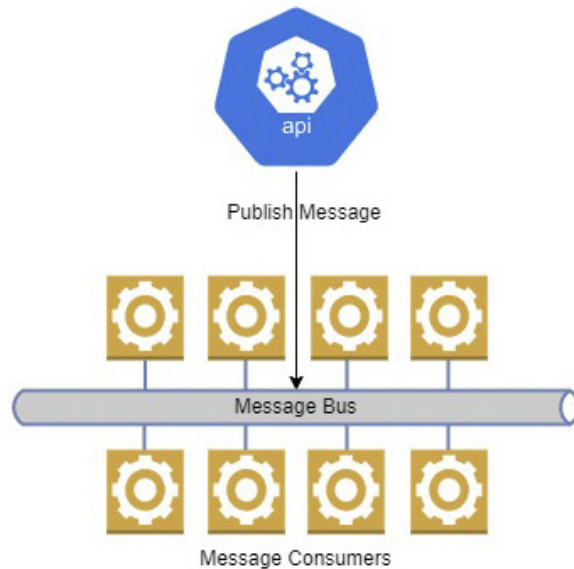


Figure 4.2 – This message bus has several consumers or subscribers listening for messages

Going back to our scenario where booking an appointment has several operational concerns, we can use a message bus to distribute data to the relevant services and allow them to complete their operations in their own time. Instead of making direct calls to the other APIs, our appointment booking API will create a message and place it on the message bus. The email and calendar services are subscribed to the message bus and process the message accordingly.

This pattern has several advantages where decoupling and application scalability are concerned. This aids in making the microservices even more independent from each other and reduces limitations associated with adding more services in the future. It also adds stability to the overall interactions of our services, since the message bus acts as a storage intermediary for the data needed to complete an operation. If a consuming service is unavailable, the message will be retained, and the pending messages will be processed once normalcy is restored. If all services are running and messages are backing up, then we can scale the number of instances of the services to reduce the message backlog more quickly.

There are several types of messages that you may encounter, but we will focus on two for this chapter. They are **command** and **event** messages. Command messages essentially request some action is performed. So, our message to the calendar services sends an instruction for a calendar entry to be created. Given the nature of these commands, we can take advantage of this asynchronous pattern and have the messages get picked up eventually. That way, even large numbers of messages will be processed.

Event messages simply announce that some action took place. Since these messages are generated after an action, they are worded in the past tense and can be sent to several microservices. In this case, the message to the email service can be seen as an event and our email service will relay that information accordingly. This type of message generally has just enough information to let the consuming services know what action was completed.

Command messages are generally aimed at microservices that need to post or modify data. Until a message is consumed and actioned, the expected data will not be available for some amount of time. This is one of the key downsides to the asynchronous messaging model, and it is termed **eventual consistency**. We need to explore this in more detail and discover the best approaches to take.

Understanding eventual consistency

One of the biggest challenges we face in microservices design is data management and developing a strategy for keeping data in sync, which sometimes means that we need to have multiple copies of the same data in several microservice databases. **Eventual consistency** is a notion in distributed computing systems that accepts that data will be out of sync for a period of time. This type of constraint is only acceptable in distributed systems and fault-tolerant applications.

It is easy enough to manage one dataset in one database, as is the case with a monolithic application, since data will always be up to date for any other part of the application to be able to access. A single database approach to our application gives us the guarantee of ACID transactions, which we discussed earlier, but we still are met with the challenge of **concurrency** management. Concurrency refers to the fact that we might have multiple versions of the same data available at different points. This challenge is easier to manage in a single database application but presents a unique challenge in a distributed system.

It is a reasonable assumption that when data changes in one microservice, data will change in another and lead to some inconsistency between the data stores for a period. The **CAP theorem** introduces the notion that we cannot guarantee all three of the main attributes of a distributed system: **consistency**, **availability**, and **partition tolerance**:

- **Consistency:** This means that every read operation against a data store will yield the current and most up-to-date version of the data, or an error will be raised if the system cannot guarantee that it is the latest.
- **Availability:** This means that data will always be returned for a read operation, even if this is not the latest guaranteed version.
- **Partition tolerance:** This means that systems will operate even when there might be transient errors that would stop a system under normal circumstances. Imagine we have minor connectivity issues between our services and/or messaging systems. Data updates will be delayed, but this principle will suggest that we need to choose between **availability** and **consistency**.

Choosing consistency or availability is a very important decision moving forward. Given that microservices need to be generally always available, we must be careful with our decision and how strictly we impose our constraints around a system's consistency policy – eventual or strong.

There are scenarios where strong consistency is not required, since all the work that was performed by an operation is completed or rolled back. These updates are either lost (if rolled back) or will propagate to the other microservices in their own time, without having any detrimental effects on the overall operations of the application. If this model is selected, we can gauge our user experience through sensitization and letting them know that updates are not always immediate across the different screens and modules.

Using the Pub-Sub model is the number one way to implement this kind of event-driven communication between services, where they all communicate via a messaging bus. With the completion of each operation, each microservice will publish an event message to the message bus, and other services will pick this up and process it eventually.

To implement eventual consistency, we usually use event-driven communication using a publish-subscribe model. When data is updated in one microservice, you can publish a message to a central messaging bus, and other microservices that have copies of the data can receive a notification by subscribing to the bus. Because the calls are asynchronous, the individual microservices can continue to serve requests using the copy of the data that they already have, and the system needs to tolerate that while it might not be consistent right away, meaning that the data may not all be in sync immediately, eventually it will be consistent across the microservices. Of course, implementing eventual consistency can be more complex than just firing and forgetting messages to a message bus. Later in this book, we'll look at ways we can mitigate the risks.

Until now, we have explored the concepts of using event-driven or Pub-Sub patterns to facilitate asynchronous communication. All of these concepts have been based on the idea that we have a messaging system persisting and distributing messages between services and operations. Now, we need to explore some of our options in the forms of RabbitMQ and Azure Service Bus.

Configuring a message bus (RabbitMQ or Azure Service Bus)

After waxing poetic about messaging buses and queues, we can finally discuss two excellent options for facilitating message-based service communication. They are **RabbitMQ** and **Azure Service Bus**.

These are by no means the only options, nor are they the best, but they are popular and can get the job done. Alternatives that you may encounter include **Apache Kafka**, which is famed for its high performance and low latency, or **Redis Cache**, which can double as a simple key-value caching store but also as a message broker. Ultimately, the tool you use is relative to what you need and what the tool offers your context.

Let us explore how we can integrate with RabbitMQ in a .NET Core application.

Implementing RabbitMQ in an ASP.NET Core web API

RabbitMQ is the most deployed and used open source message broker, at least at the time of writing. It supports multiple operating systems, has a ready-to-go container image, and is a reliable intermediary messaging system that is supported by several programming languages. It also provides a management UI that allows us to review messages and overall system performance as part of monitoring measures. If you plan to deploy a messaging system on-premises, then RabbitMQ is an excellent option.

RabbitMQ supports sending messages in two main ways – **queues** and **exchanges**. We already have an appreciation for what queues are, and exchanges support the message bus paradigm.

Let us take a look at what it takes to configure RabbitMQ on a Windows computer and what supporting C# code is needed to publish and consume. Let us start by including the `MassTransit.RabbitMQ` RabbitMQ package via NuGet. In our `Program.cs` file, we need to ensure that we configure `MassTransit` to use RabbitMQ by adding the following lines:

```
builder.Services.AddMassTransit(x =>
{
    x.UsingRabbitMq();
});
```

This creates an injectable service that can be accessed in any other part of our code. We need to inject `IPublishEndpoint` into our code, and this will allow us to submit a message to the RabbitMQ exchange:

```
[ApiController]
[Route("api/[controller]")]
public class AppointmentsController : ControllerBase
{
    private readonly IPublishEndpoint _publishEndpoint;
    private readonly IAppointmentRepository
        _appointmentRepository;
    public AppointmentsController (IAppointmentRepository
        appointmentRepository, IPublishEndpoint
        publishEndpoint)
    {
        _publishEndpoint = publishEndpoint;
        _appointmentRepository = appointmentRepository;
    }
    [HttpPost]
    public async Task<IActionResult> CreateAppointment
```

```
(AppointmentDto appointment)
{

    var appointment = new Appointment()
    {
        CustomerId = AppointmentDto.CustomerId,
        DoctorId = AppointmentDto.DoctorId,
        Date = AppointmentDto.Date
    });
    await _appointmentRepository.Create(appointment);
    var appointmentMessage = new AppointmentMessage()
    {
        Id = appointment.Id
        CustomerId = appointment.CustomerId,
        DoctorId = appointment.DoctorId,
        Date = appointment.Date
    });
    await _publishEndpoint.Publish(appointmentMessage);
    return Ok();
}
```

After creating an appointment record, we can share the record's details on the exchange. The different subscribers will pick up this message and process what they need. Consumers are generally created as windows or background worker services that are always on and running. The following example shows how a consumer's code might look:

```
public class AppointmentCreatedConsumer :
    IConsumer<AppointmentMessage>
{
    public async Task Consume(ConsumeContext<Appointment
        Message> context)
    {
        // Code to extract the message from the context and
        // complete processing - like forming email, etc...
        var jsonMessage =
            JsonConvert.SerializeObject(context.Message);
    }
}
```

```
        Console.WriteLine($"AppointmentCreated message:
                           {jsonMessage}");
    }
}
```

Our consumers will be able to receive any message of the `AppointmentMessage` type and use the information as they need to. Note that the data type for the message exchange is consistent between the producer and the consumer. So, it would be prudent of us to have a `CommonModels` project that sits in between and serves up these common data types.

If we implement this consumer in a console application, then we need to register a **Bus Factory** and subscribe to the expected event endpoint. In this case, the endpoint that will be generated in RabbitMQ based on the type being published is `appointment-created-event`. To create a console application that will listen until we terminate the instance, we need code that looks like this:

```
var busControl = Bus.Factory.CreateUsingRabbitMq(cfg =>
{
    cfg.ReceiveEndpoint("appointment-created-event", e =>
    {
        e.Consumer<AppointmentCreatedConsumer>();
    });
});

await busControl.StartAsync(new CancellationToken());

try
{
    Console.WriteLine("Press enter to exit");

    await Task.Run(() => Console.ReadLine());
}
finally
{
    await busControl.StopAsync();
}
```

Now that we have seen in a nutshell what it takes to communicate with a RabbitMQ exchange, let us review what is needed to communicate with the cloud-based Azure Service Bus.

Implementing Azure Service Bus in an ASP.NET Core API

Azure Service Bus is an excellent choice for cloud-based microservices. It is a fully managed enterprise message broker that supports queues as well as Pub-Sub topics. Given Microsoft Azure's robust availability guarantees, this service supports load balancing, and we can be assured of secure and coordinated message transfers if we choose this option. Similar to RabbitMQ, Azure Service Bus has support for **queues** and **topics**. Topics are the direct equivalent of **exchanges**, where we can have multiple services subscribed and waiting on new messages. Let us reuse the concept we just explored with RabbitMQ and review the code needed to publish a message on a topic and see what consumers would look like. We are going to focus on the code in this section and assume that you have already created the following:

- An Azure Service Bus resource
- An Azure Service Bus topic
- An Azure Service Bus subscription to the topic

These elements all need to exist, and we will retrieve a connection string to Azure Service Bus through the Azure portal. To get started with the code, add the `Azure.Messaging.ServiceBus` NuGet package to the producer and consumer projects.

In our publisher, we can create a service wrapper that can be injected into parts of the code that will publish messages. We will have something like this:

```
public interface IMessagePublisher {
    Task PublisherAsync<T> (T data);
}

public class MessagePublisher: IMessagePublisher {

    public async Task PublishMessage<T> (T data, string
        topicName) {
        await using var client = new ServiceBusClient
            (configuration["AzureServiceBusConnection"]);
        ServiceBusSender sender = client.CreateSender
            (topicName);
        var jsonMessage =
            JsonConvert.SerializeObject(data);
        ServiceBusMessage finalMessage = new
            ServiceBusMessage(Encoding.UTF8.GetBytes
                (jsonMessage))
```

```
    {  
        CorrelationId = Guid.NewGuid().ToString()  
    };  
  
    await sender.SendMessageAsync(finalMessage);  
  
    await client.DisposeAsync();  
}
```

In this code, we declare an interface, `IMessagePublisher.cs`, and implement it through `MessagePublisher.cs`. When a message comes in, we create `ServiceBusMessage` and submit it to the specified topic.

We need to ensure that we register this service so that it can be injected into other parts of our code:

```
services.AddScoped<IMessagePublisher, MessagePublisher>();
```

Now, let us look at the same controller and how it would publish a message to Azure Service Bus instead of RabbitMQ:

```
[ApiController]  
[Route("api/[controller]")]  
public class AppointmentsController : ControllerBase  
{  
    private readonly IMessageBus _messageBus;  
    private readonly IAppointmentRepository  
        _appointmentRepository;  
    public AppointmentsController (IAppointmentRepository  
        appointmentRepository, IMessageBus messageBus)  
    {  
        _appointmentRepository = appointmentRepository;  
        _messageBus = messageBus;  
    }  
  
    [HttpPost]  
    public async Task<IActionResult> CreateAppointment  
        (AppointmentDto appointment)  
    {  
        var appointment = new Appointment()  
    }
```

```
{
    CustomerId = AppointmentDto.CustomerId,
    DoctorId = AppointmentDto.DoctorId,
    Date = AppointmentDto.Date
});
await _appointmentRepository.Create(appointment);

var appointmentMessage = new AppointmentMessage()
{
    Id = appointment.Id
    CustomerId = appointment.CustomerId,
    DoctorId = appointment.DoctorId,
    Date = appointment.Date
});
await _messageBus.PublishMessage(appointmentMessage,
    "appointments");
return Ok();
}
}
```

Now that we know how we can set up the publisher code, let us review what we need for a consumer. This code could be used by a background worker or Windows service to continuously monitor for new messages:

```
public interface IAzureServiceBusConsumer
{
    Task Start();
    Task Stop();
}
```

We start with defining an interface that outlines `Start` and `Stop` methods. This interface will be implemented by a consumer service class, which will connect to Azure Service Bus and begin executing code that listens to Service Bus for new messages and consumes them accordingly:

```
public class AzureServiceBusConsumer :
    IAzureServiceBusConsumer
{

```

```
private readonly ServiceBusProcessor
    appointmentProcessor;
private readonly string appointmentSubscription;
private readonly IConfiguration _configuration;
public AzureServiceBusConsumer(IConfiguration
    configuration)
{
    _configuration = configuration;
    string appointmentSubscription =
        _configuration.GetValue<string>
            ("AppointmentProcessSubscription")
    var client = new ServiceBusClient
        (serviceBusConnectionString);

    appointmentProcessor = client.CreateProcessor
        ("appointments", appointmentSubscription);
}

public async Task Start()
{
    appointmentProcessor.ProcessMessageAsync +=
        ProcessAppointment;
    appointmentProcessor.ProcessErrorAsync +=
        ErrorHandler;
    await appointmentProcessor
        .StartProcessingAsync();
}

public async Task Stop()
{
    await appointmentProcessor
        .StopProcessingAsync();
    await appointmentProcessor.DisposeAsync();
}

Task ErrorHandler(ProcessErrorEventArgs args)
```

```
{
    Console.WriteLine(args.Exception.ToString());
    return Task.CompletedTask;
}

private async Task ProcessAppointment
    (ProcessMessageEventArgs args)
{
    // Code to extract the message from the args,
    parse to a concrete type, and complete
    processing - like
forming email, etc...
    await args.CompleteMessageAsync(args.Message);
}
}
```

From these two examples of how we interact with message bus systems, we can see that they are conceptually very similar. Similar considerations and techniques would be employed for any of the other message bus systems supported by .NET Core libraries.

Surely, there are trade-offs when we implement this type of message-based communication. We now have an additional system and potential point of failure, so we must consider the additional infrastructural requirements. We also see that the code required adds more complexity to our code base. Let us dive into some of the disadvantages of this approach.

Disadvantages of asynchronous communication between microservices

As with any system or programming method, there are always advantages and disadvantages that come with the territory. We have already explored why having an asynchronous messaging pattern is a good idea for operations that might be long-running. We need to ensure that the end user doesn't spend too much time waiting on an entire operation to be completed. Messaging systems are an excellent way to shorten the perceived time it takes to complete an operation and allows services to operate as efficiently as possible on their own. They also aid in decoupling systems, allowing for greater scalability and introducing a certain level of stability to a system where data transfer and processing are concerned.

Now, disadvantages creep in when we analyze the real level of complexity that this pattern can introduce. Far more coordination needs to be considered when designing how our services interact with others, what data they need to share, and what events need to be posted when operations are completed. In the synchronous model, we will be more sure of tasks getting completed down the line, since we

cannot move forward without a favorable response from the next service along the chain. In the asynchronous model using queues and buses, we have to rely on the consuming service(s) posting an event regarding the state of completion. We also have to ensure that there are no repeated calls and, in some cases, need to make a concerted effort to ensure that messages are processed in a specific order.

This brings us to another disadvantage in the form of data consistency. Remember that the initial response from the message bus suggests that the operation is successful, but this just means that data was successfully submitted to the bus. After this, our consuming services still need to follow through and complete their operations. If one or more of these services fails to process and potentially submits the data to a data store, then we'll end up with inconsistency in our data. This is something to be mindful of, as it can lead to detrimental side effects and user attrition.

Summary

In this chapter, we explored a few things. We did a blow-by-blow comparison of how a process would be handled through synchronous API communication versus asynchronous communication. We then expanded our general knowledge of how messaging systems can be leveraged to support an asynchronous communication model for our services. In all of this, we discussed challenges that we can face with data consistency between operations and how we can gauge acceptable metrics for this unavoidable factor. In the latter parts, we reviewed two popular messaging systems and then discussed some of the outright disadvantages that we have to contend with in this paradigm.

In our next chapter, we will explore the **Command-Query Responsibility Segregation (CQRS)** pattern and how it helps us to write cleaner code in our services.

Working with the CQRS Pattern

We now know that microservices require a bit of foresight during the planning phase, and we need to ensure that we employ the best patterns and technology to support our decisions. In this chapter, we will be exploring another pattern that has gained much acclaim in helping us to write clean and maintainable code. This is the **Command Query Responsibility Segregation or Separation (CQRS)** pattern, which is an extension of the **Command-Query Separation (CQS)** pattern.

This pattern allows us to cleanly separate our query operations from our command operations. In essence, a query asks for data, and the command should modify data in one way or another by the end of the operation.

As programmers, we tend to employ **Create, Read, Update, and Delete (CRUD)** in our applications. Considering that every application's core functionality is to support CRUD operations, this is understandable. But the more intricate the application gets, the more we need to consider the business logic surrounding each of these operations, relative to the problem domain we are addressing.

At that point, we begin to use words such as *behavior* and *scenarios*. We begin to consider structuring our code in a manner that allows us to isolate behaviors and easily determine whether this behavior is simply a request for data or will augment data by the end of the operation.

After reading this chapter, you will achieve the following:

- Understand the benefits of the CQRS pattern and why it is used for microservices development
- Know how to implement commands in the CQRS pattern
- Know how to implement queries in the CQRS pattern

Technical requirements

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch05>.

Why use CQRS for microservices development?

CQS was introduced as a pattern that would help developers separate code that does read operations from code that does write operations. The shortcoming with it was that it didn't account for establishing specific models for each operation. CQRS built on this and introduced the concept of having specific models, tailored for the operation to be carried out.

Figure 5.1 shows a typical CQRS architecture:

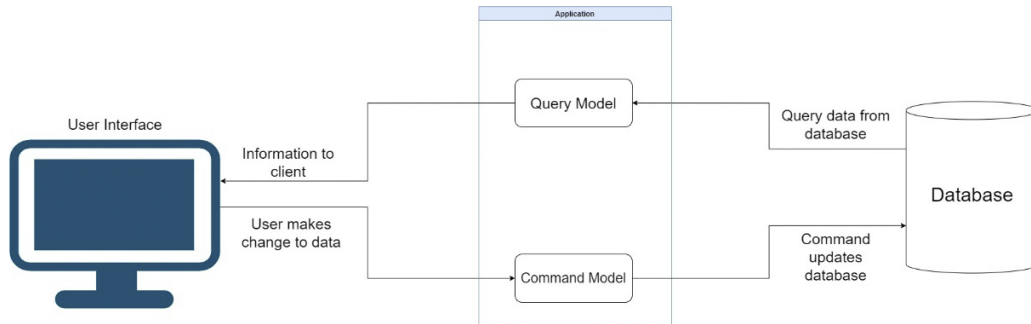


Figure 5.1 – The application will interact with models for read operations and models for write operations, known as commands

If you look a bit more closely, it can be argued that CQS only accounts for one data store, meaning we are doing read/write operations against the same database. CQRS would suggest that you have separate data stores, having potentially a standard relational database for your write operations and conducting read operations for a separate store, such as a document database or data warehouse. The implementation of multiple data stores is not always an option, nor is it a must.

CQRS has gained much acclaim since its introduction in development and is touted as a very important staple in microservice design. The truth is, it can be used in standard applications, so it is not unique to microservices. It also adds a new level of complexity to the development effort as it introduces the need for more specific classes and code to be written, which can lead to project bloat.

It is said, “...when we have a hammer, everything looks like a nail...” and this remains true in the context of when we hear of a new pattern and feel the need to use it everywhere. I suggest that you apply caution and careful consideration before using this pattern and ensure that its value in the application is justified.

For bigger applications, CQRS is recommended to help us structure our code and more cleanly handle potentially complex business logic and moving parts. Though it is complex, it does have benefits. Let us review the benefits of implementing the CQRS pattern in our applications.

Benefits of the CQRS pattern

CQRS is about splitting a single model into two variations, one for reads and one for writes. The end goal, however, is a bigger spoke in the wheel. The first benefit of this approach is scalability.

It is important to assess the read/write workload of the system you are building. Read operations are arguably more intensive than write operations, given that one read may require copious amounts of data from several tables and each request might have its own requirements around what the data needs to look like. One school of thought encourages that we employ a data store that is dedicated to and optimized for reading operations. This allows us to scale read operations separately from write operations.

An example of a dedicated read store could be a *data warehouse*, where data is constantly being transformed by some form of data transformation pipeline, from the write data store, which is probably a normalized *relational database*. Another commonly used technology is NoSQL databases such as **MongoDB** or **Cosmos DB**. The data constructs provided by the data warehouse or NoSQL databases represent a denormalized, read-only version of the data from the relational data store.

Figure 5.2 shows a CQRS architecture with two databases:

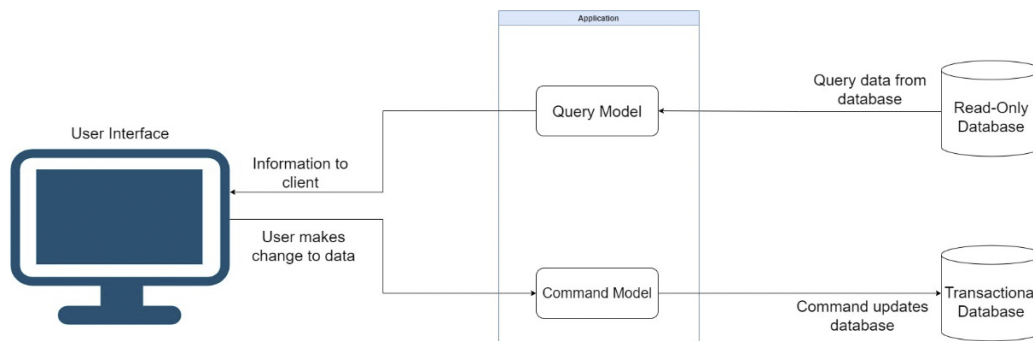


Figure 5.2 – The query model represents read operation-optimized representations of the data from the transactional database

The second benefit is performance. Though it seems to go hand in hand with scalability, there are different dynamics that we consider. Using separate data stores isn't always a viable option, so there are other techniques that we can employ to optimize our operations that wouldn't be possible with a unified data model. We can apply caching, for instance, to queries that do read operations. We can also employ database-specific features and fine-tuned raw SQL statements for our requests in contrast to **object-relational mapping (ORM)** code on the side of the command.

Another benefit that we can reap from this pattern is simplicity. This sounds contradictory given that we mentioned complications in the earlier parts of this chapter, but it depends on the lens that you use to assess the rewards you will reap in the long run. Commands and queries have different needs,

and it is not reasonable to use one data model to suit both sets of needs. CQRS forces us to consider creating specific data models for each query or command, which leads to more maintainable code. Each new data model is responsible for a specific operation, and modification therein will have little to no impact on other aspects of our program.

We see here that there are a few benefits to using CQRS in our projects. But where there are pros, there are cons. Let us review some of the downsides to employing this design pattern.

Disadvantages of the CQRS pattern

Every pattern that we consider must be investigated thoroughly for the value that it adds, and the potential drawbacks. We always want to make sure that the benefits outweigh the disadvantages and that we won't live to regret these major design decisions. CQRS is not ideal for applications that will do simple CRUD operations. It is behavior- or scenario-driven, as we have stated, so be sure that you can justify the use cases before you take the plunge. This pattern will lead to a perceived duplication of code since it promotes the concept of **separation of concerns (SoC)** and encourages that commands and queries have dedicated models.

On a personal note, I have seen development leads start off with all the best intentions and implement CQRS in projects that, in truth, didn't require it. The projects turned out to be overcomplicated, leaving all the newer developers completely confused as to what went where.

As we have seen, there is a case for multiple data storage areas when we use CQRS. Arguably, that is the most complete implementation of it. Once we introduce multiple data stores, we introduce data consistency problems and need to implement *Event Sourcing* techniques and include **service-level agreements (SLAs)** to let our users know of the potential gap between our read/write operations. We also must consider the additional costs in terms of infrastructure and general operation when we have multiple databases. With multiple databases comes multiple potential points of failure and the need for additional monitoring and fail-safe techniques to ensure that the system runs as smoothly as possible, even in the face of outages.

CQRS can add value to a project when it is applied sensibly. When your project has complex business logic, or a distinct need for separating data stores and read/write operations, then CQRS will shine as an appropriate architectural choice.

This pattern is not unique to any programming language, and .NET has excellent support for the pattern. Now let us review implementing CQRS using the Mediator pattern with the help of a few tools and libraries.

Using the Mediator pattern with CQRS in .NET

Before we get into how we implement CQRS, we should discuss a supporting pattern called the *Mediator* pattern. The Mediator pattern involves us defining an object that embodies how objects interact with each other. So, we can avoid two or more objects having direct dependencies on each

other and, instead, use a mediator in between that will orchestrate the dependencies and route requests to appropriate *handlers*. A handler will define all the details of the operation to be carried out based on the scenario or task to which the model is associated.

Figure 5.3 shows how the Mediator pattern works:

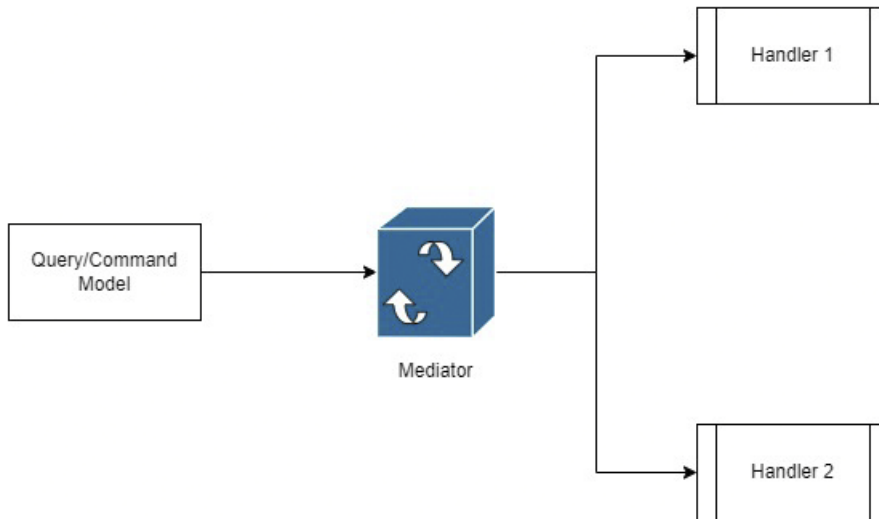


Figure 5.3 – The query/command model is registered in the mediator engine, which then selects the appropriate handler for an operation

The Mediator pattern becomes useful relative to the implementation of the CQRS pattern since we need to promote loose coupling between the code being defined for each task. It allows us to maintain *loose coupling* and promotes more testability and scalability.

In .NET Core, we have an excellent third-party package called `MediatR` that helps us implement this pattern with relative ease. `MediatR` assumes the role of an *in-process* mediator where it manages how classes interact with each other during the same process. One limitation here is that it might not be the best package if we wanted to separate commands and queries across different systems. Notwithstanding that drawback, it helps us to author CQRS-based systems with relative ease, efficiency, and reliability. We can develop strongly typed code that will ensure that we do not mismatch models and handlers, and we can even construct pipelines to govern the entire behavior of a request as it flows through the complete cycle.

In a .NET Core application, we can set up MediatR using the following steps:

1. Install the `MediatR.Extensions.Microsoft.DependencyInjection` NuGet package
2. Modify our `Program.cs` file with the following line: `builder.Services.AddMediatR(typeof(Program));`

Once we have done this, we can proceed to inject our `IMediator` service into our controllers or endpoints for further use. We will also need to implement two files that will directly relate to each other as the command/query model and the corresponding handler.

Now that we have explored the foundations of setting up a CQRS implementation using the Mediator pattern, let us review the steps required to implement a command.

Implementing a command

As we remember, a command is expected to carry out actions that will augment the data in the data store, otherwise called a write operation. Given that we are using the Mediator pattern to govern how we carry out these operations, we will need a command model and a handler.

Creating a command model

Our model is relatively simple to implement. It tends to be a standard class or record, but with MediatR present, we will implement a new type called `IRequest`. `IRequest` will associate this model class with an associated handler, which we will be looking into in a bit.

In the example of making an appointment in our system, we can relatively easily implement a `CreateAppointmentCommand.cs` file like this:

```
public record CreateAppointmentCommand (int
    AppointmentTypeId, Guid DoctorId, Guid PatientId, Guid
    RoomId, DateTime Start, DateTime End, string Title):
    IRequest<string>;
```

We use a record type in this example, but it could just as easily be a class definition if that is more comfortable for you. Notice that we inherit from `IRequest<string>`, which outlines to MediatR the following:

- This command should be associated with a handler with a corresponding return type.
- The handler that is associated with this command is expected to return a string value. This will be the appointment Id value.

Commands don't always need to return a type. If you don't expect a return type, you may simply inherit from `IRequest`.

Now that we have an idea of how the command model needs to look, let us implement the corresponding handler.

Creating a command handler

Our handler is where the logic for carrying out the task usually sits. In some implementations and in the absence of a rich data model, you can carry out all the validations and additional tasks required to ensure that the task is handled properly. This way, we can better isolate the business logic that is expected when a particular command is sent for execution.

Our handler for our command to create an appointment will be called `CreateAppointmentHandler.cs` and can be implemented like this:

```
public class CreateAppointmentHandler :
    IRequestHandler<CreateAppointmentCommand, string>
{
    private readonly IAppointmentRepository _repo;
    public CreateAppointmentHandler(IAppointmentRepository
        repo /* Any Other Dependencies */)
    {
        _repo = repo
        /* Any Other Dependencies */
    };
    public async Task<string> Handle
        (CreateAppointmentCommand request, CancellationToken
            cancellationToken)
    {
        // Handle Pre-checks and Validations Here
        var newAppointment = new Appointment
        (
            Guid.NewGuid(),
            request.AppointmentTypeId,
            request.DoctorId,
            request.PatientId,
            request.RoomId,
            request.Start,
```



```
        request.End,  
        request.Title  
    );  
    await _repo.Add(newAppointment);  
  
    //Perform post creation hand-off to services bus.  
  
    return newAppointment.Id.ToString();  
}  
}
```

There are several things to take note of here:

- The handler inherits from `IRequestHandler<>`. In the type brackets, we outline the specific *command model type* that the handler is being implemented for and the *expected return type*.
- A handler is implemented like any other class and can have dependencies injected in as needed.
- After completing our operations, we must return a value with a data type that matches the one outlined by `IRequest<>` from the command model.
- `IRequestHandler` implements a method called `Handle` that defaults to the outlined return type of `IRequest<>` attached to the command model. It will automatically generate a parameter called `request`, which will be of the command model's data type.

If we did not require a return value, we would have to return `Unit.Value`. This is the default void representation of `MediatR`.

Now that we have some boilerplate code for implementing a command model and its corresponding handler, let us look at making an actual call to the handler from a controller.

Invoking a command

We have our controller defined for appointment booking operations and it needs to have the dependency for our `IMediator` service present to begin orchestrating our calls. We need to inject our dependency like this:

```
private readonly IMediator _mediator;  
public AppointmentsController(IMediator mediator) =>  
    _mediator = mediator;
```

Once we have this dependency present, we can begin making our calls. What makes this solution so clean is that we do not need to worry about the business logic and specific handlers for anything. We

need to only create a command-model object with the appropriate data and then send it using our mediator, which will then call the appropriate handler. The code for our POST method is shown here:

```
[HttpPost]
public async Task<ActionResult> Post([FromBody]
    CreateAppointmentCommand createAppointmentCommand)
{
    await _mediator.Send(createAppointmentCommand);
    return StatusCode(201);
}
```

There are several ways to implement this section. Some alternatives include the following:

- Using *models* or **Data Transfer Objects** (or **DTOs** for short) to accept data from the API endpoint. This means that we will then transfer the data points from the incoming model object to our command model before sending, as illustrated here:

```
[HttpPost]
public async Task<ActionResult> Post([FromBody]
    AppointmentDto appointment)
{
    var createAppointmentCommand = new
        CreateAppointmentCommand { /* Assign all values
            here*/}
    await _mediator.Send(createAppointmentCommand);
    return StatusCode(201);
}
```

- Instead of defining all the data properties in the command model, we use the DTO as a property of the command model so that we can pass it along with the mediator request, as follows:

```
// New Command Model with DTO property
public record CreateAppointmentCommand
    (AppointmentDto Appointment) : IRequest<string>;

// New Post method
[HttpPost]
public async Task<ActionResult> Post([FromBody]
    AppointmentDto appointment)
{
```

```
var createAppointmentCommand = new
    CreateAppointmentCommand { Appointment =
        appointment; }
await _mediator.Send(createAppointmentCommand);
return StatusCode(201);
}
```

You can observe that each method of authoring the code amounts to the same thing, and that is we need to compose the appropriate model type with the appropriate values before sending it off to our mediator. The mediator would have already figured out which model matches which handler and will proceed to carry out its operation.

Commands and queries generally follow the same implementation. In the next section, we will look at implementing a query model/handler pair using our mediator and CQRS patterns.

Implementing a query

A query is expected to search for data and return a result. This search might be complicated, or it might be simple enough. The fact, though, is that we implement this pattern as an easy way to segregate the query logic from the originator of the request (such as the controller) and from the command logic. This type of separation increases a team's ability to maintain either side of the application without stepping on each other's toes, so to speak. We will similarly use the Mediator pattern to govern how we carry out these operations, and we will need a query model and a handler.

Creating a query model

Our model is simple enough as we can leave it empty or include properties that will play a part in the process to be carried out in the handler. We inherit `IRequest<>`, which defines a return type. I would go out on a limb and say that a return type is necessary, considering that this is a query.

Let us look at two examples of queries, one that will retrieve all the appointments in the database and another that will retrieve only by ID. We can define `GetAppointmentsQuery.cs` and `GetAppointmentByIdQuery.cs` like this:

```
public record GetAppointmentsQuery(): IRequest
    <List<Appointment>>;
public record GetAppointmentByIdQuery(string Id):
    IRequest<AppointmentDetailsDto>;
```

Just take note of the fact that either query model is specific to what it needs to represent. The `GetAppointmentsQuery` model doesn't need any properties since it is just going to be used as an outline for the handler to make an association. `GetAppointmentByIdQuery` has an `Id`

property for the obvious reason that the ID is going to be needed for the handler to correctly execute the task at hand. The difference in return types is also a crucial point to note as that sets the tone for what the handler will be able to return.

We need to ensure that we craft our query models specifically for the type of data that we are expecting to retrieve from the matching handler. Now, let us look at implementing these handlers.

Creating a query handler

Our query handlers will execute the expected queries and return the data as defined by `IRequest<>`. As previously outlined, the ideal usage of this pattern will see us using a separate data store where the data being queried is already optimized for return. This would make our query operation efficient and reduce the need for data transformation and sanitization.

In our example, we are not that fortunate to have a separate data store, so we will use the same repository to query the transactional data store. Our handler for our query to get all appointments will be called `GetAppointmentsHandler.cs` and can be implemented like this:

```
public class GetAppointmentsHandler : IRequestHandler
    <GetAppointmentsQuery, List<Appointment>>
    {
        private readonly IAppointmentRepository _repo;
        public GetAppointmentsHandler
            (IAppointmentRepository repo) => _repo = repo;

        public async Task<List<Appointment>>
            Handle(GetAppointmentsQuery request,
                CancellationToken cancellationToken)
        {
            return await _repo.GetAll();
        }
    }
```

This handler's definition is simple as we simply retrieve a list of appointments. When we need to get an appointment by ID, it would imply that we need the details of the appointment. This will call for a more complex query that might involve joins or, better yet, require synchronous API calls to get the details of related records. This is where the database design's strong points or flaws will come into play. If we were guided by our **Domain-Driven Design (DDD)** principles, then we would have been sure to include some additional information about records from other databases in this one. Without getting

into too many details about that point, we need to ensure that we retrieve enough data to populate a model of type `AppointmentDetailsDto` for return, like so:

```
public class GetAppointmentByIdHandler :
    IRequestHandler<GetAppointmentByIdQuery,
        AppointmentDetailsDto>
{
    private readonly IAppointmentRepository _repo;
    public GetAppointmentByIdHandler
        (IAppointmentRepository repo)
    {
        _repo = repo;
    }

    public async Task<AppointmentDetailsDto>
        Handle(GetAppointmentByIdQuery request,
            CancellationToken cancellationToken)
    {
        // Carry out all query operations and convert
        the result to the expected return type
        return new AppointmentDetailsDto{ /* Fill model
            with appropriate values */ };
    }
}
```

After we have gathered the data needed for this particular scenario, we construct our return object and send it back to the original sender.

Now, let us look at what the controller actions look like as they seek to complete the queries.

Invoking a query

Using the same controller, we can assume that the `IMediator` dependency has already been injected and execute the following code:

```
[HttpGet]
public async Task<ActionResult<Appointment>> Get()
{
    var appointments = await _mediator.Send(new
```

```
        GetAppointmentsQuery());  
        return Ok(appointments);  
    }  
  
    [HttpGet("{id}")]  
    public async Task<ActionResult<AppointmentDetailsDto>>  
        Get(string id)  
    {  
        var appointment = await _mediator.Send(new  
            GetAppointmentByIdQuery(id));  
        return Ok(appointment);  
    }  
}
```

Our actions look similar. Each one is defined with the parameters it needs from a request. They then call the `mediator` object and new objects of the expected query model type. The mediator will orchestrate the call and route the request to the appropriate handler.

Summary

There are several benefits that can be reaped from this approach to development. We have already outlined that project bloat is a part of the territory, but the level of consistency and structure that can be enforced and guaranteed is perhaps worth the additional effort.

In this chapter, we explored the CQRS pattern and how we can employ it in our microservice application. We took time to assess the problems that we need to address, to bring real context to why we added this new level of complexity. We then looked at how we restructure our code to facilitate our handlers and request/command objects. We also looked at some design decisions that we can make in terms of our data stores for reading and writing operations.

In our next chapter, we will explore Event Sourcing patterns that will tie into our CQRS pattern and help us to keep our data relevant throughout our application.

Applying Event Sourcing Patterns

In the previous chapter, we explored a prolific pattern in CQRS. This pattern encourages us to create a clear separation between code and data sources that govern read and write operations. With this kind of separation, we risk having our data out of sync in between operations, which introduces the need for additional techniques to ensure data consistency.

Even without CQRS, we must contend with the typical microservices pattern where each service is expected to have its own data store. Recall that there will be situations where data needs to be shared between services. There needs to be some mechanism that will adequately transport data between services so that they will remain in sync.

Event sourcing is touted as a solution to this issue, where a new data store is introduced that keeps track of all the command operations as they happen. The records in this data store are considered events and contain enough information for the system to track what happens with each command operation. These records are called events and they act as an intermediary store for event-driven or asynchronous services architecture. They can also act as an audit log as they will store all the necessary details for replaying changes being made against the domain.

In this chapter, we will explore the event sourcing pattern and justify its use as a solution to our potentially out-of-sync databases.

After reading this chapter, you will be able to do the following:

- Understand what events are and what event sourcing can do for you
- Apply event sourcing patterns in your application code
- Use the CQRS pattern to create events and read states in between events
- Create an event store using a relational or non-relational database

Technical requirements

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch06>.

What are events?

An event, within the context of software development, refers to something that happens because of an action being completed. Events are then used to carry out actions in the background, such as the following:

- Storing data for analytics purposes
- Notification of completed actions
- Database auditing

Key attributes of events

Events can be used to build the foundation of any application's core functionality. While the concept can be suitable for many situations, it is important for us to understand some key attributes of events and properly scope the need for their introduction, as well as uphold certain standards in our implementations:

- **Immutability:** This word refers to the unchangeable nature of an object. Within the context of an event, once something has happened, it becomes a fact. That means we cannot change it or the outcome in the real world. We extend this same feature to our events and ensure that they cannot be changed after they are generated.
- **Single occurrence:** Each event is unique. Once it has been generated, it cannot be repeated. Even if the same thing happens later, it should be recognized as a new event.
- **Historical:** An event should always represent a point in time. This way, we can trace what happened and when in the past. This discipline is also displayed in the way that we name our events, where we use the past tense to describe the event.

Events at their best do not contain any behavior or business logic. They generally only serve as a point-in-time data collection unit and help us to track what is happening at different points in our application.

Now that we have a good idea of what events are and why they are used at a high level, let us focus on more practical uses of events and event sourcing patterns.

What can event sourcing patterns do for me?

Applications built with microservices architecture are structured to have a set of loosely coupled and independent services. Using the **database-per-service pattern**, we further segregate each service by giving it an individual data store. This now presents a unique challenge to keep the data in sync between services. It becomes more difficult given that we need to compromise on our ACID principles. We can recall that the acronym **ACID** stands for **atomicity, consistency, isolation, and durability**. We are most concerned about the principle of atomicity in this context. We cannot guarantee that all our write operations will be completed as a unit. The atomic principle dictates that all data operations should complete or fail as a unit. Given the allowance for different technologies to be used for the data stores, we cannot absolutely guarantee that.

Considering all these factors, we turn to a new pattern called event sourcing, which allows us to persist messages that keep track of all the activities occurring against data in each service. This pattern is especially useful for asynchronous communication between services where we can keep track of all changes in the form of **events**. These events can act as the following:

- **Persistent events:** Events contain enough detail to inform and recreate domain objects
- **Audit log:** Events are generated with each change, so they can double as audits
- **Entity state identification:** We can use events to view the point-in-time state of an entity on demand

The idea of tracking changes against entities is called **replaying**. We can replay events in two steps:

1. Grab all or partial events stored for a given aggregate.
2. Iterate through all events and extract the relevant information to freshen up the instance of the aggregate.

Event sourcing is essentially all about querying records in some way using an **aggregate ID** and **timestamp**. The aggregate ID represents the unique identifier column, or primary key value, for the original record for which the event was raised. The timestamp represents the point in time that the event was raised. The queries required for this look similar for relational and non-relational event stores. The event replay operation requires that we iterate through all the events, grab information, and then change the state of the target aggregate. In addition to the aggregate ID and timestamp, we will also have all the information needed to fill in the bits of data needed for the aggregate.

Now, let us review some of the benefits of using events in our systems.

Pros of event sourcing

We have been looking into the idea of tracking the history of operations that happen against our data, particularly our aggregates. We have encountered the concepts of events and replays. Now let us look at what event replays are, how they may benefit us, and what other benefits exist from using events.

Event replays and the way that we conduct our updates depend on whether the aggregate is a domain class or not. If the aggregate relies on domain services for manipulation, we need to be clear that replays are not about repeating or redoing commands. A command, based on our understanding of CQRS, changes the state and data in the database. This also has the potential of being a long-running operation with event-data-generating side effects, which we may not want. A replay is about looking at data and performing logic to extract information. On the other hand, event replays copy the effects of events and apply them to fresh instances of the aggregate. Altogether, stored events may be processed differently relative to the application employing the technique.

Events are bits of data that are stored at a lower level than the plain state. This means that we can reuse them to build any projection of the data that we need. Ad hoc projects of the data can be used for the read data store in a CQRS project structure, data analytics, business intelligence, and even artificial intelligence and simulations. Contextually, if we have a stream of events and can extract a specific subset, then we can replay them and perform ad hoc calculations and processes to generate custom and potentially new information. Events are constant and will always be the same now and later. This is an additional benefit in that we can always be sure that we will be able to count on the data for consistency.

As in life, for every set of benefits, there is a lingering set of downsides. Let us explore some of the general concerns around event sourcing.

Cons of event sourcing

In exploring event sourcing, we must bear in mind that we need to introduce an additional data store and additional services that might impede the application's performance. Let us review some concerns.

Performance is always important in an application. So, when introducing a new pattern or set of processes, it is prudent of us to ensure that the performance impact is minimal. What happens when we need to process too many events to rebuild data? This can quickly become an intensive operation based on the number of logged events, which is only going to grow since the event store will be an **append-only** data store.

To address this, we take snapshots of the aggregate state and business entities that have recently been amended. We can then use these snapshots as a stored version of the record and use them as a recent version of the data, sparing the need to iterate through potentially many events. This operation is best complimented by having a **read-only** data store to pair with our CQRS pattern. The snapshot will be used for read operations going forward.

Now that we have looked at some of the more serious implications of this pattern and techniques that can be used to reduce the impact it might have on our application, let us review how event sourcing and domain events relate to each other so that we can strengthen our foundational knowledge.

What are domain events?

Earlier in this book, we discussed the use of DDD as a design pattern that helps us to scope the different services that might be required as we develop our microservices application. Events can be employed in the implementation of this pattern to help us to model expected outcomes within our bounded contexts. Events are scoped based on the ubiquitous language that has been established within the bounded context and is informed by decisions within the domain.

Within the domain, aggregates are responsible for creating domain events and our domain events are usually raised based on the outcome of some user action, or command. It is important to note that domain events are not raised based on actions such as the following:

- Button clicks, mouse moves, page scroll events, or simple application exceptions. Events should be based on the established ubiquitous language of the bounded context.
- Events from other systems or outside of the current context. It is important to properly establish the boundaries between each domain context.
- Simple user requests to the system. A user request at this point is a command. The event is raised based on the outcome of the command.

Now let us get a better understanding of why domain events are integral to implementing event sourcing patterns.

Domain events and event sourcing

Event sourcing is implemented to provide a single point of reference for the history of what has happened within a bounded context. Simply put, event sourcing uses domain events to store the states that an aggregate has gone through. We have already seen that event sourcing will have us store the record ID, a timestamp, and details that help us to understand what the data looked like at that moment. Properly implementing **domain events** within a bounded context will lay a foundation for a good implementation of event sourcing and so proper scoping and implementation are important.

Implementing domain events in code can be done relatively simply using the **MediatR** library, which was so integral in our CQRS pattern implementation. In the next section, we will look at adjusting our application to implement domain events.

Exploring domain events in our application

Now, let us consider introducing domain events to our appointment booking system. As far as we can see, we have several activities that need to be completed when an appointment is booked in our system. We might also need to extend the capabilities of our system to support the idea that changes might be needed to the original appointment and should be tracked.

Let us use the email dispatching activity. This needs to happen when an appointment is accepted into the system and saved. As it stands, our `CreateAppointmentHandler` will handle everything that is needed in the situation. We then run into the challenge of separating concerns since we probably don't want our handler to be responsible for too many actions. We would do well to separate our email dispatch operation into its own handler.

Using MediatR, we can introduce a new type of handler called `INotificationHandler<T>`. This new base type allows us to define handlers relative to data types modeled from events that inherit from another MediatR base type, called `INotification`. These event types should be named according to the action that it is created to facilitate and will be used as the generic parameter in our `INotificationHandler<T>`. Our `INotificationHandler<T>` base type will be inherited by a handler or handlers that will carry out any specific actions relative to the additional actions required.

In code, we would want to start with some fundamental base types that will help us to define our concrete event types. The first would be `IDomainEvent`, which will serve as a base type for all our domain events that will follow. Its definition looks something like this:

```
public interface IDomainEvent : INotification
{
}
```

Our interface inherits from MediatR's built-in `INotification` interface so that any derived data type will automatically also be a notification type. This `IDomainEvent` interface also helps us to enforce any mandatory data that must be present with any event object, such as the date and time of the action.

Now that we have our base types, let us define our derived event class for when an appointment gets created. We want to ensure that we name our event type in a manner that accurately depicts the action that raised the event. So, we will call this event type `AppointmentCreated`. We simply inherit from our `IDomainEvent` interface and then define additional fields that correspond with the data that is needed for the event to adequately carry out additional work:

```
public class AppointmentCreated : IDomainEvent
{
    public Appointment { get; set; }
    public DateTime ActionDate { get; private set; }

    public AppointmentCreated(Appointment appointment,
        DateTime dateCreated)
    {
        Appointment = appointment;
        ActionDate = dateCreated;
    }
}
```

```

    }

    public AppointmentCreated(Appointment appointment)
        : this(appointment, DateTime.Now)
    {
    }
}

```

In our `AppointmentCreated` derived event type, we have defined a property for our appointment and a constructor that makes sure that an object of `Appointment` is present at the time of creation. In this case, it is up to you to decide how much or little information you would require for the event to effectively be handled. For instance, some types of events might only need the appointment's ID value. Be very sure to scope this properly however, and send as much information as is needed. You do not want to send only the ID and then need to query for additional details and risk potentially many event handlers trying to fetch details from just an ID value.

Now let us look at defining handlers for our event type. Note that I said *handlers* as it is possible and viable to define multiple event handlers based on the event that has occurred. For instance, when an appointment gets created, we might have a handler that will update the event store with the new record, or have one that dispatches an email alert, separate from one that updates a **SignalR** hub, for example.

To facilitate updating an event store, we would need to first have a handler defined that would look something like this:

```

public class UpdateAppointmentEventStore :
    INotificationHandler<AppointmentCreated>
{
    private readonly AppointmentsEventStoreService
        _appointmentsEventStore;

    public UpdateAppointmentEventStore
        (AppointmentsEventStoreService
         appointmentsEventStore)
    {
        this._appointmentsEventStore =
            appointmentsEventStore;
    }

    public async Task Handle(AppointmentCreated
        notification, CancellationToken cancellationToken)
    {

```

```
        await _appointmentsEventStore.CreateAsync  
            (notification.Appointment);  
    }  
}
```

Our AppointmentCreated event type is used as the target type for our INotificationHandler. This is all it takes to add specific logic sequences to a raised event. This also helps us to separate concerns and better isolate bits of code associated with raised events. Our notification object contains the appointment record, and we can easily use the data we need.

This code will automatically get fired when the event occurs and handle the event-store-update operation accordingly.

Let us look at our event handler that will dispatch our email alert:

```
public class NotifyAppointmentCreated :  
    INotificationHandler<AppointmentCreated>  
    {  
        private readonly IEmailSender _emailSender;  
        private readonly IPatientsRepository  
            _patientsRepository;  
  
        public NotifyAppointmentCreated(IEmailSender  
            emailSender, IPatientsRepository  
            patientsRepository)  
        {  
            this._emailSender = emailSender;  
            this._patientsRepository = patientsRepository;  
        }  
  
        public async Task Handle(AppointmentCreated  
            notification, CancellationToken cancellationToken)  
        {  
            // Get patient record via Patients API call  
            var patient = await _patientsRepository.Get  
                (notification.Appointment.  
                    PatientId.ToString());  
            string emailAddress = patient.EmailAddress;  
  
            // Send Email Here
```

```
var email = new Email
{
    Body = $"Appointment Created for
           {notification.Appointment.Start}",
    From = "noreply@appointments.com",
    Subject = "Appointment Created",
    To = emailAddress
};

await _emailSender.SendEmail(email);
}
```

Notice as well that even though we did not have direct access to the patient's record, we had their ID. With that value, we could make a synchronous API call to retrieve additional details that can assist us in crafting and dispatching the notification email.

In the same way, if we wanted to define an event handler for SignalR operations, we could simply define a second handler for the same event type:

```
public class NotifySignalRHubAppointmentCreated :
    INotificationHandler<AppointmentCreated>
{
    public Task Handle(AppointmentCreated notification,
        CancellationToken cancellationToken)
    {
        // SignalR awesomeness here

        return Task.CompletedTask;
    }
}
```

Now that we can raise an event, we can refactor our application a bit to reflect this. We can refactor our `CreateAppointmentHandler` and `CreateAppointmentCommand` classes to return an object of `Appointment` instead of the previously defined string value:

```
public record CreateAppointmentCommand(int
    AppointmentTypeId, Guid DoctorId, Guid PatientId, Guid
    RoomId, DateTime Start, DateTime End, string Title) :
```



```
        IRequest<Appointment>;

public class CreateAppointmentHandler :
    IRequestHandler<CreateAppointmentCommand, Appointment>
public async Task<Appointment> Handle
    (CreateAppointmentCommand request, CancellationToken
    cancellationToken) { ... }
```

With this adjustment, we can now retrieve an object of the created appointment and publish an event from the original calling code, which was in the controller. Our POST method for our appointments API now looks like this:

```
// POST api/<AppointmentsController>
[HttpPost]
public async Task<ActionResult> Post([FromBody]
    CreateAppointmentCommand createAppointmentCommand)
{
    // Send appointment information to create
    handler
    var appointment = await
        _mediator.Send(createAppointmentCommand);
    //Publish AppointmentCreated event to all
    listeners
    await _mediator.Publish(new AppointmentCreated
        (appointment));
    // return success code to caller
    return StatusCode(201);
}
```

Now we use the Publish method from the MediatR library to raise an event, and all handlers that have been defined to watch for the specified event type will be called into action.

These refactors to our code will introduce even more code and files, but they do assist in helping us maintain a distributed and loosely coupled code base. With this activity, we have reviewed how we can cleanly introduce domain events to our application, and now we need to appreciate how we can store our events.

Creating an event store

Before we get to the scoping phase of creating an event store, it is important for us to fully understand what one is. A simple search on the topic might yield many results from various sources, with each citing varied definitions. For this book, we will conclude that an event store *is an ordered, easily queryable, and persistent source of long-term records that represents events that have happened against entities in a data store.*

In exploring the implementation of a data store, let us break out the key parts and how they connect to give the resulting event store. An event record will have an aggregate ID, a timestamp, an EventType flag, and data representing the state at that point in time. An application will persist event records in a data store. This data store has an API or some form of interface that allows for adding and retrieving events for an entity or aggregate. The event store might also behave like a message broker allowing for other services to subscribe to events as they are published by the source. It provides an API that enables services to subscribe to events. When a service saves an event in the event store, it is delivered to all interested subscribers. *Figure 6.1* shows a typical event store architecture.

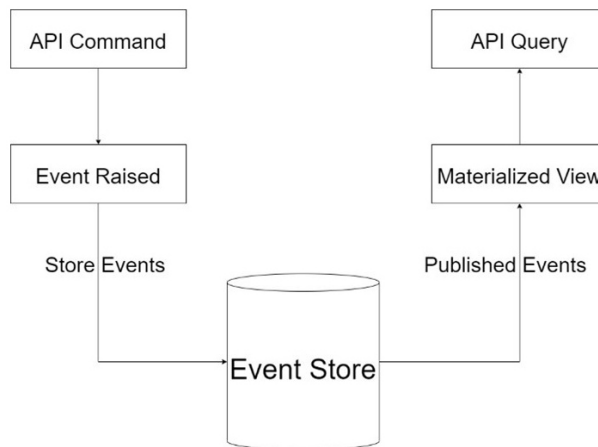


Figure 6.1 – An event store sits between the command handlers of an API and the query handler, which may require a different projection of the originally stored data

Let us review event storage strategies that we can employ.

How to store events

We have already established that events should be immutable, and the data store should be append-only. Armed with these two requirements, we can deduce that our dos and don'ts need to be at a minimum as we scope the data store.

Each change that takes place in the domain needs to be recorded in the event store. Since events need to contain certain details relevant to the event being recorded, we need to maintain some amount of flexibility with the structure of the data. An event might contain data from multiple sources in the domain. That means that we need to leverage possible relationships between tables in order to grab all the details required to log the event.

This makes the standard relational database model less feasible as a data store for events since we want to be as efficient as possible in retrieving our event records for auditing, replay, or analytics later. If we are using a relational data store, then we would do well to have denormalized tables modeled from the event data that we intend to store. An alternative and more efficient manner to handle event storage is the use of a non-relational or NoSQL database. This will allow us to store the relevant event data as documents in a far more dynamic manner.

Let us explore some options regarding storing events in a relational database.

Implementing event sourcing using a relational database

We have already reviewed some of the drawbacks of using a relational database as the event store. In truth, the way you design this storage, alongside the technology that is used, can have a major bearing on how future-proofed your implementation will be.

If we go the route of using denormalized table representations of the events, then we will end up going down a rabbit hole of modeling several tables based on several different events. This might not be sustainable in the long run, since we would need to introduce new tables with each newly scoped event and constantly change designs as the events evolve.

An alternative would be that we create a singular log table that has columns that match the data points we just outlined. For example, this table and the matching data types would be as follows:

Column Name	Data Type
Id	Int
AggregateId	Guid
Timestamp	DateTime
EventType	Varchar
Data	Varchar
VersionNumber	Int

- **Id**: Unique identifier for the event record.
- **AggregateId**: Unique identifier for the aggregate record to which the event is related.

- **Timestamp:** The date and time that this event was logged.
- **EventType:** This is a string representation of the name of the event that is being logged.
- **Data:** This is a serialized representation of the data associated with the event record. This serialization is best done in an easy-to-manipulate format such as **JSON**.
- **VersionNumber:** The version number helps us to know how to sort events. It represents the sequence in which each new event was logged in the stream and should be unique to each aggregate. We can add constraints to our records by introducing a **UNIQUE** index on both the **AggregateId** and **VersionNumber** columns. This will help us with speedier queries and ensure that we do not repeat any combination of these values.

The type of database technology that is employed does play a part in how flexibly and efficiently we can store and retrieve data. The use of **PostgreSQL** and later versions of **Microsoft SQL Server** will see us reap the advantages of being able to manipulate the serialized representation of the data more efficiently.

Now let us look at how we can model our NoSQL data stores.

Implementing event sourcing using a non-relational database

NoSQL databases are also called **document databases** and are characterized by their ability to effectively store unstructured data. This means the following:

- Records do not need to meet any minimum structure. Columns are not mandated in the design phase, so it is easy enough to extend and contract the data based on the immediate need.
- Data types are not strictly implemented, so the data structure can evolve at any point in time without having detrimental effects on previously stored records.
- Data can be nested and can contain sequences. This is significant since we do not need to spread related data across multiple documents. One document can represent a denormalized representation of data from several sources.

Popular examples of document stores are **MongoDB**, **Microsoft Azure Cosmos DB**, and **Amazon DynamoDB**, to name a few. Outside of the specific querying and integration requirements for each of these database options, the concepts of how documents are formed and stored are the same.

We can outline the properties of the document in a very similar manner to how a table would look. In a document data store, however, the data is stored in JSON format (unless specifically requested or implemented otherwise). An event entry would look something like this:

```
{  
  "type": "AppointmentCreated",  
  "aggregateId": "aggregateId-guid-value",
```

```
"data": {
  "doctorId": "doctorId-guid-value",
  "customerId": "customerId-guid-value",
  "dateTime": "recorded-date-time",
  ...
},
"timestamp": "2022-01-01T21:00:46Z"
}
```

Another advantage to using a document data store is that we can more easily represent a record with its event history in a materialized view. That could look something like this:

```
{
  "type": "AppointmentCreated",
  "aggregateId": "aggregateId-guid-value",
  "doctorId": "doctorId-guid-value",
  "customerId": "customerId-guid-value",
  "dateTime": "recorded-date-time",
  ...
  "history": [
    {
      "type": "AppointmentCreated",
      "data": {
        "doctorId": "doctorId-guid-value",
        "customerId": "customerId-guid-value",
        "dateTime": "recorded-date-time",
        ...
      },
      "timestamp": "2022-01-01T21:00:46Z"
    },
    {
      "type": "AppointmentUpdated",
      "data": {
        "doctorId": "different-doctorId-guid-value",
        "customerId": "customerId-guid-value",
        "comment": "Update comment here"
      },
    },
  ]
}
```

```
        "timestamp": "2022-01-01T21:00:46Z",  
        ...  
    },  
    ...  
],  
    "createdDate": "2022-01-01T21:00:46Z",  
    ...  
}
```

This type of data representation can be advantageous for retrieving a record with all its events, especially if we intend to display this data on a user interface. We have generated a view that acts as both a snapshot of the current state of the aggregate data and the stream of events that have affected it. This form of data aggregation helps us to reduce some complexity and keep the concept of event retrieval simple.

Now that we see how we can implement a read-only and denormalized data store, let us review how we can use the CQRS pattern to retrieve the latest state of the data.

Reading state with CQRS

We have reviewed the CQRS pattern, and we see where we can create handlers that will perform write operations. Earlier in this chapter, we enhanced our command handler functionality with the ability to trigger events, which are capable of performing triggering additional actions after a command has been completed.

In the context of the event sourcing pattern, this additional action involves updating our read-only data stores with the appropriate data per view. When creating our query handlers, we can rely on these tables for the latest version of the data that is available. This ties in perfectly with the ideal implementation of the CQRS pattern where separate data stores are to be used for read and write operations.

It also presents an excellent opportunity for us to provide more specific representations of the data we wish to present from our read operations. This approach, however, introduces the risk that our data stores might become out of sync in between operations, which is a risk that we must accept and mitigate as best as possible.

Now that we have explored events, event sourcing patterns, and event storage options, let us review these concepts.

Summary

Event sourcing and event-driven design patterns bring a whole new dimension to what is required in our software implementation. These patterns involve additional code but do assist in helping us to implement additional business logic for completed commands while maintaining a reliable log of all the changes happening in our data store.

In this chapter, we explored what events are, various factors of event sourcing patterns, how we can implement certain aspects in a real application, and the pros and cons of using relational or non-relational storage options.

In our next chapter, we will explore the Database Per Service pattern and look at best practices when implementing the data access layer in each microservice.

Part 2:

Database and Storage Design Patterns

Understanding and implementing data management patterns and techniques is vital when designing a microservices application. Each microservice might need its database, and we need to understand the intricacies surrounding managing each database and how we coordinate efforts across services. By the end of this part, you will come to appreciate the tough decisions that need to be made surrounding databases in a microservices application.

This part has the following chapters:

- *Chapter 7, Handling Data for Each Microservice with the Database per Service Pattern*
- *Chapter 8, Implement Transactions across Microservices Using the Saga Pattern*

Handling Data for Each Microservice with the Database per Service Pattern

In the previous chapter, we explored the concepts of event sourcing and event stores. Event sourcing patterns help us to reconcile changes made to our data stores across our microservices. An operation in one microservice might require that data be sent to other microservices. For efficiency reasons, we create an event store as an intermediary area to which microservices can subscribe for changes and will be able to get the latest version of the data as needed.

This concept revolves around the assumption that each microservice has its own database. This is the recommended approach in a microservice architecture, given the fundamental requirement that each microservice needs to be autonomous in its operations and data requirements.

Building on this notion, we will explore best practices and techniques for handling data for each microservice.

After reading this chapter, you will know how to do the following:

- How to make use of the Database-per-Service pattern
- How to develop a database
- How to implement the repository pattern

Technical requirements

The code references used in this chapter can be found in the project repository, which is hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch07>.

How to make use of the Database-Per-Service pattern

A core characteristic of microservices architecture is loose coupling between our services. We need to maintain the independence and individuality of each service and allow them to autonomously interact with the data they need when they need it. We want to ensure that one service's manipulation of data does not inhibit another service's ability to use its own data.

Each microservice will be left to define its own data access layers and parameters and, unless deliberately implemented, no two services will have direct access to the same data store. Data is not persisted across two services and the overall decoupling that comes with this pattern means that one database failure will not inhibit the operation of the other services.

We also need to bear in mind that microservices architecture allows us to select different development technologies that best meet the needs of the service being implemented. Different technologies tend to work better with or support specific databases. Therefore, this pattern may be more of a requirement than a suggestion given that we will want to use the best database technology to support the needs of a service.

As usual, where there is a pro, there is a con. We need to consider the costs involved with this kind of heterogenous architecture and how we might need to adapt our knowledge base and team to support several databases and, by extension, various database technologies. We now have to account for additional maintenance, backup, and tuning operations, which might lead to maintenance overheads.

There are several approaches that can be taken in implementing this pattern and some of the approaches reduce our infrastructure needs and help us to save costs. We discuss those options in the next section.

A single database technology per service

Ultimately, we need to establish clear boundaries between the source of truth for each service. This doesn't mean that you absolutely need to have different databases, but you can take advantage of features of relational data stores such as the following:

- **Tables-per-service:** We can define tables that are optimized for the data to be stored for each service. We will model these tables in the microservice code and ensure that we only include these tables. In this model, it is common that some tables are denormalized representations of data that can be found in other tables, as the service they are created for requires the data in this format.
- **Schema-per-service:** Relational databases allow us to specify schema values to categorize our tables. A schema is an organizational unit in a database that helps us to categorize tables. We can use these to organize tables per service. Like the **tables-per-service** pattern, the tables in each schema are optimized based on the needs of the matching microservice. We also have a better opportunity to tune access rights and restrictions per schema and reduce the security administration overhead.

- **Database-per-service:** Each microservice has its own database. This can be taken a step further by placing each database on its own server to reduce the infrastructural dependencies.

Figure 7.1 shows multiple services connected to one database.

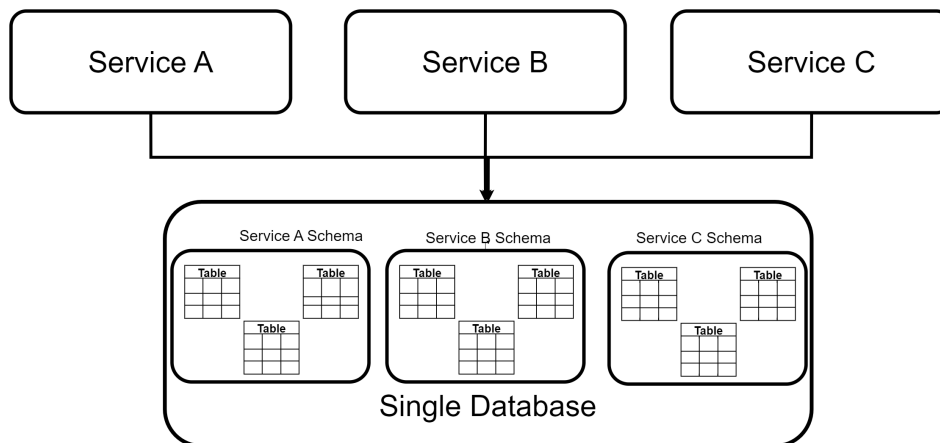


Figure 7.1 – Multiple services share one database, but schemas are created for each service to preserve segregation and data autonomy

Implementing tables and schemas per service has the lowest resource requirements since it would be the same thing as building one application on top of one database. The downside is that we retain a single point of failure and do not adequately scale the varied service demands.

In the **Database per Service** approach, we can still use one server but provision each microservice with its own database. This approach matches the name of the pattern more appropriately, but infrastructurally maintains a common choke point that has all the databases on the same server. The most appropriate implementation to maintain service autonomy and reduce infrastructural dependencies would be to have each database and its reliant microservice in its own fault domain.

Looking at this issue from another dimension, we can see that we have the flexibility to choose the best database technology for each service. Some services might favor a relational data store, while others might use a document data store more efficiently. We will discuss this further in the next section.

Using different database technologies per service

Databases are the foundation of any application. A good or poor database design can determine how efficient your application is, how easily it can be extended, and how efficiently you can write your code.

The use of the database-per-service pattern allows us to choose the best database for the operation that each microservice might complete. It would be ideal for a more homogenous technology stack that all

developers can identify with and easily maintain. Attempting to remain homogenous, however, has led to shortcuts and extensive integration projects in the past, where the need to use one technology overshadowed the opportunity to use the best technology for the problem being addressed.

Figure 7.2 shows multiple services connected to individual databases.

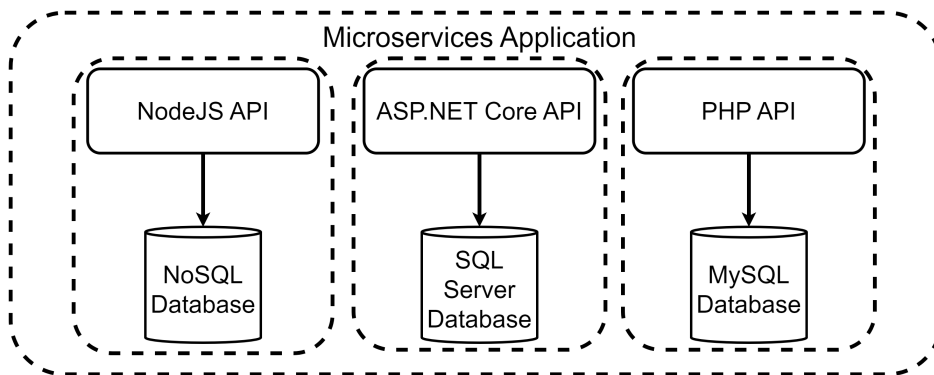


Figure 7.2 – Multiple services can be built using different technologies alongside the most appropriate development framework technology

Microservices allow us to have multiple teams that can choose the best technology stack to implement the solution and, by extension, they can use the best type of database technology to complement the technology and the problem. Some development frameworks tend to work best with certain database technologies, and this makes it easier to select the entire stack that is to be used for a particular microservice.

Now that we have reviewed our database-per-service options, let us review some disadvantages of using this development pattern for our microservices architecture.

Disadvantages of this pattern

We could spend all day singing the praises of this pattern and pointing out why it is the ideal path to take during the microservices development process. Despite all these advantages, however, we can point to potential pitfalls that we must overcome or learn to mitigate during runtime:

- **Additional costs:** When we talk about having fewer infrastructural dependencies between our services, we talk about introducing more robust networking software and hardware, more servers, and more software licenses for the supporting software. Using a cloud platform might more easily offset some of the infrastructural and software costs, but even this approach will have a modest price tag.
- **Heterogenous development stack:** This is an advantage from the perspective of attempting to meet the business needs of the microservice as appropriately as possible. The bigger picture, however, comes when we need to source talent to maintain certain technologies that might

have been used. Cross-training between teams is recommended but not always effective and a company can risk having a microservice built by past staff members that none of the current ones can maintain.

- **Data synchronization:** We have already discussed the issue of **Eventual Consistency**, which leads us down the path of implementing contingencies to deal with data being out of sync across multiple databases. This comes with additional code and infrastructure overhead to properly implement it.
- **Transactional handling:** We are unable to ensure ACID transactions across databases, which can lead to inconsistent data between data stores. This will lead us to need another coding pattern called the **Saga Pattern**, which we will discuss further in the next chapter.
- **Communication failure:** Because one microservice cannot directly access another's database, we may need to introduce synchronous microservice communication to complete an operation. This introduces more potential failure points in the operation at hand. These can be mitigated using the **Circuit Breaker Pattern**, which will be discussed in subsequent chapters.

It is always important to remember that with every pattern, we have pros and cons. We should never do an implementation solely because it is the *recommended way to go*. We should always properly assess the problems that need to be addressed and choose the most appropriate solutions and patterns to ensure full coverage and at the best price.

Now that we have discussed the dos and don'ts for our Database-per-Service pattern, let us move on to discuss best practices when designing a database.

Developing a database

The ability to develop a capable database is paramount to a developer's career. This role was once given to the database developer in a team, whose sole purpose was to do all things database. The application developer would simply write code to interact with the database based on the application's needs.

More recently, the role of a typical application developer evolved into what is now called a *full stack developer* role. This means that the modern developer needs to have as much application development knowledge as they do database development knowledge. It is now very common to see teams of two to three developers who work on a microservice team, and who can develop and maintain the user interface, application code, and database.

Developing a database transcends one's comfort level with the technology being used. In fact, that is the easy part. Many developers neglect to consult the business and fully understanding the business requirements before they start implementing the technology. This often leads to poor design and rework, and additional maintenance in the lifetime of the application.

Since we are in the realm of microservices, we have the unique opportunity to build much smaller, target data stores, for tranches of the entire application. This makes it easier for us to ingest and

analyze the storage needs of the service that we are focusing on and reduces the overall complexity of building a large database as a catch-all, thus reducing the margin for error during the design phase. As discussed previously, we can make better choices about the most appropriate database, which influences the design considerations that we make.

Some services need relational integrity based on the nature of the data they process; others need to produce results fast more than they need to be accurate; some only need a key-value store. In the next section, we will compare the pros and cons of relational and non-relational data stores and when it is best to use each one.

Databases are an integral part of how well an application performs and it is important to make the correct decision on which technology is used for which microservice. In the next section, start by assessing the pros and cons of using relational databases.

Relational databases

Relational databases have been a mainstay for years. They have dominated the database technology landscape for some time and for good reason. They are built on rigid principles that complement clean and efficient data storage while ensuring a degree of accuracy in what is stored.

Some of the most popular relational database management systems include the following:

- **SQL Server:** This is Microsoft's flagship relational database management system, which is widely used by individuals and enterprises alike for application development.
- **MySQL / MariaDB:** MySQL is a traditionally open source and free-to-use database management system that is mostly used for PHP development. MariaDB was forked from the original MySQL code base and is maintained by a community of developers to maintain the free-to-use policy.
- **PostgreSQL:** A free and open source database management system that is robust enough to handle a wide range of workloads, from individual projects to data warehouses.
- **Oracle Database:** This is Oracle Corporation's flagship database management system, which is designed to handle a wide range of operations, from real-time transaction processing to data warehousing and even mixed workloads.
- **IBM DB2:** Developed by IBM on top of one of the most reliable systems for high transaction and traffic enterprise settings. This database supports relational and object-relational structures.
- **SQLite:** A free and lightweight database storage option for a quick and easy database. Unlike most alternatives that require a server, oftentimes a dedicated machine, SQLite databases can live in the same filesystem as the app it is being integrated into, making it an excellent choice for mobile-first apps.

Through a process called *normalization*, data is efficiently shared across multiple tables, with references or *indexes* created between each table. Good design principles encourage you to have a *primary key* column present in each table, which will always have a unique value, to uniquely identify a record.

This unique value is then referenced by other tables in the form of a *foreign key*, which helps to reduce the number of times data repeats across tables.

These simple references go a long way to ensure that the data is accurate across tables. Once this association is created between the primary and foreign keys, we have created what we call a *relationship*, which introduces a *constraint* or restriction on what values are possible in the foreign key column. This is also referred to as *referential integrity*.

Relationships can be further defined by their *cardinality*. This refers to the nature by which the primary key values will be referenced in the other table. The most common cardinalities are as follows:

- **One-to-one:** This means that a primary key value is referenced exactly once in another table as a foreign key. For instance, if a customer can only have one address on record. The table storing the address cannot have multiple records that refer to the same customer.
- **One-to-many:** This means that the primary key value can be referenced multiple times in another table. For instance, if a customer has made multiple orders, then we have one customer's ID referenced multiple times in the orders table.
- **Many-to-many:** This means that a primary key can be referenced multiple times as a foreign key in another table, and that table's primary key can be referenced multiple times in the original table. This can get confusing and implementing it as it is described will directly violate the referential integrity that we are fighting hard to maintain. This is where a *linker table* is introduced as an intermediary to associate the different combinations of key values from either table.

Using the example of our health care management system, if we revisit how appointments are tracked relative to the customer who has made them, we will see that we implemented a reference point between the customer's ID, their primary key, and the appointment. This ensures that we do not repeat the customer's information every time an appointment is booked. An appointment also cannot be created for a customer who is not already in the system.

Figure 7.3 shows a typical relationship.

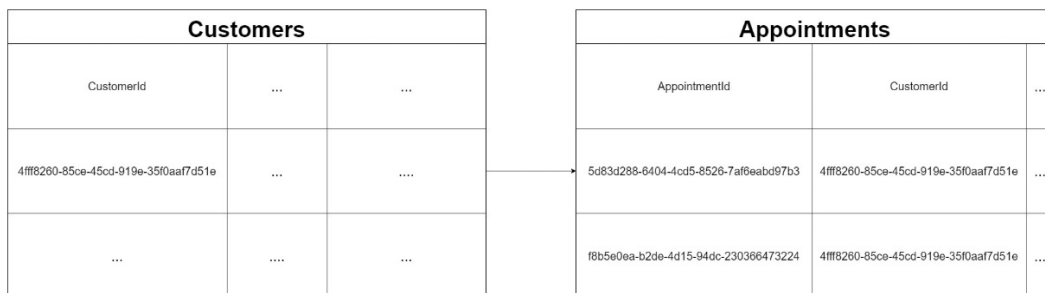


Figure 7.3 – An example of a one-to-many relationship where one customer is referenced many times in the appointments table

Many-to-many relationships do occur often and it is important to recognize them. Building on the example of customers and their appointments, we can expand and see that we also have a need to associate a customer with a room for an appointment. The same room is not always guaranteed, which leads us to realize that many customers will book appointments, and appointments might happen in many rooms. If we try a direct association, then details of either the customer or the room will need to be repeated to reflect the many possible combinations of customers, rooms, and appointments.

Ideally, we would have a table storing rooms and the associated details, a table storing customers, and an appointments table sitting between the two, which seeks to associate them on the day as needed.

Figure 7.4 shows a typical many-to-many relationship.



Figure 7.4 – An example of a many-to-many relationship where we associate many records from two different tables using an intermediary linker table

We have already discussed ACID principles and why they are important. Relational databases are designed to ensure that these principles can be observed as a default mode of operation. This makes changing the layout and references of tables relatively difficult, especially if the changes involve changing how tables are related to each other.

Another potential drawback of relational database storage comes in terms of performance with large datasets. Relational databases are typically efficient in data storage, retrieval, and overall speed. They are built to manage large workloads, so the technology itself is hardly at fault, but our design will either complement speed or inhibit it.

These are trade-offs of proper relational database design and upholding relational integrity. Design principles favor normalization, where bits of data are spread across multiple tables, and this is good until we need the data and need to traverse multiple tables with potentially thousands of records to get it. Considering this deficiency, NoSQL or document storage databases have become more and more popular, as they seek to store data in one place, making the retrieval process much faster.

We discuss the use of non-relational databases in the next section.

Non-relational databases

Non-relational databases took the database world by storm. That might be a naive assessment of the impact that they have had, but the fact remains that they introduced a dimension to data storage that was not very popular or widely used. They came along and proposed a far more flexible and scalable data storage technique that favored a more agile development style.

Agile development hinges on the ability to morph a project as we go along. This means that the excess amounts of scoping and planning that would be recommended when using a relational data store would not be necessary upfront. Instead, we could start with an idea of a system and begin using a non-relational data store to accommodate that tiny part, and as the system evolves, so could the data store, with minimal risk of data loss or attrition.

Non-relational databases are also referred to as **NoSQL databases**, or *not only SQL*, and they favor a non-tabular data storage model. The most popular NoSQL databases use a document style of storage, where each record is stored in one document, containing all the details needed for that record. This directly violates the principles of *normalization*, which would have us spread the data to reduce possible redundancy. The major advantage of this model, however, is the speed with which we can write and retrieve data since all the details are in one place.

Using the example of our health care appointment management system, if we were to store an appointment in the form of a document, it would look something like this:

```
{
  "AppointmentId": "5d83d288-6404-4cd5-8526-7af6eabd97b3",
  "Date": "2022-09-01 08:00",
  "Room": {
    "Id": "4fff8260-85ce-45cd-919e-35f0aaf7d51e",
    "Name": "Room 1"
  },
  "Customer": {
    "Id": "4fff8260-85ce-45cd-919e-35f0aaf7d51e",
    "FirstName": "John",
    "LastName": "Higgins"
  }
}
```

Unlike with the relational model, where these details were split across multiple tables and simply referenced, we have the opportunity to include all the details needed to fully assess what an appointment entails.

Types of NoSQL databases include the following:

- **Document database:** Stores data in the form of **JavaScript Object Notation (JSON)** objects. This JSON document outlines fields and values and can support a hierarchy of JSON objects and collections all in one document. Popular examples include *MongoDB*, *CosmosDB*, and *CouchDB*.
- **Key-value database:** Stores data in simple key-value pairs. Values are usually stored as string values, and they do not natively support storing complex data objects. They are usually used as quick lookup storage areas, like for application caching. A popular example of this is *Redis*.
- **Graph database:** Stores data in nodes and edges. A node stores information on the object, such as a person or place, and the edges represent relationships between the nodes. These relationships link how data points relate to each other, as opposed to how records relate to each other. A popular example of this is *Neo4j*.

The clear advantages of using NoSQL document databases come from the way that data is stored. Unlike with the relational model, we do not need to traverse many relationships and tables to get a fully human-readable record of data. As seen in the preceding example, we can store all the details in one document and simply scale from there. This comes in handy when we might need to do fast read operations and do not want to compromise system performance with complex queries.

A clear disadvantage, however, comes from how storage is handled. Document databases do very little, if anything at all, to reduce the risk of data redundancy. We will end up repeating details of what would have been related data, which means that the maintenance of this data, in the long run, could become a problem. Using our customer records as examples, if we needed to add a new data point for each customer that has booked an appointment, we would need to traverse the appointment records to make one change. This would be a much easier and more efficient operation using a relational model.

Now that we have discovered some of the advantages and disadvantages of relational and non-relational database stores, let us discuss scenarios where either would be a good choice.

Choosing a database technology

When choosing the best technology for anything, we are faced with several factors:

- **Maintainability:** How easy is this technology to maintain? Does it have excessive infrastructural and software requirements? How often are updates and security patches produced for the technology? Ultimately, we want to ensure that we do not regret our selection weeks into starting the project.
- **Extensibility:** To what extent can I use this technology to implement my software? What happens when the requirements change and the project needs to adapt? We want to ensure that our technology is not too rigid and can support the dynamics of the business needs.

- **Supporting technologies:** Is the technology stack that I am using the best fit? We are often forced to take shortcuts and implement methods, sometimes contrary to best practices, to facilitate matching technologies that are not the best fit for each other. With more libraries being produced to support integrations of heterogeneous technology stacks, this is becoming less of an issue, but is still something that we want to take into consideration from day one.
- **Comfort level:** How comfortable are you and the team with the technology? It is always nice to use new technology and expand your scope and experience, but it is important to recognize your limitations and knowledge gaps. These can just as easily be addressed with training, but we always want to ensure that we can handle the technology that we choose and are not prone to surprises in the long run.
- **Maturity:** How mature is the technology? In some development circles, technology lives for months at a time. We don't want to embrace new technology the day it is released without doing our due diligence. Seek to choose more mature technologies that have been tried and proven and have strong enterprise or community support and documentation.
- **Appropriateness:** Lastly, how appropriate is the technology for the application? We want to ensure that we use the best technology possible to do the job at hand. This sometimes gets compromised considering the bigger picture, which includes budget constraints, the type of project, team experience, and the business' risk appetite, but as much as possible, we want to ensure that the technology we choose can adequately address the needs of the project.

Now let us narrow this down to database development. We have gone through the pros and cons of the different types of database models and the different technology providers for each. While we are not limited to the options outlined, they serve as guidelines to help us make our assessments from the most unbiased point of view possible.

Within the context of microservices, we want to choose the best database for the service's needs. There are no hard and fast rules surrounding which storage mechanism should be used outright, but there are recommendations that can help to guide you during the system design.

You can consider using relational databases in the following situations:

- **Working with complex reports and queries:** Relational databases can run efficient queries across large datasets and are a much better storage option for generating reports.
- **Working with a high transaction application:** Relational databases are a good fit for heavy-duty and complex transaction operations. They do a good job of ensuring data integrity and stability.
- **You require ACID compliance:** Relational databases are based on ACID principles, which can go a long way in protecting your data and ensuring accuracy and completeness.
- **Your service will not evolve rapidly:** If your service doesn't have changing requirements, then a database should be easy to design and maintain in the long run.

You can consider using a non-relational database in the following situations:

- Your service is constantly evolving, and you need a flexible data store that can adapt to new requirements without too much disruption.
- You anticipate that data may not be clean or meet a certain standard all the time. Given the flexibility that we have with non-relational data stores, we accommodate data of varying levels of accuracy and completeness.
- You need to support rapid scaling: This point goes hand in hand with the need to evolve the data store based on the needs of the business, so we can ensure that the database can be changed with minimal code changes and low costs.

Now that we have explored some of the major considerations regarding choosing a database technology, let us review our options for interacting with a database with code.

Choosing an ORM

Choosing an **ORM** is an important part of the design process. This lays a foundation for how our application will communicate with our database. The abbreviation **ORM** is short for **Object Relational Mapping**.

Every language has support for some form of ORM. Sometimes this is built into the language, and sometimes developers and architects alike contract the use of an external package or library to support database-related operations. In the context of .NET, we have several options, and each has its pros and cons. The most popular options are as follows:

- **Entity Framework Core:** The most popular and obvious choice of .NET developers. It is a Microsoft-developed and maintained ORM that is packaged with .NET. It implements a C# query-like syntax called LINQ, which makes it easy to write C# code that will execute a query at runtime on the developer's behalf and has support for most relational and non-relational database technologies.
- **Dapper:** Dapper is considered a *MicroORM* as it is a fast, lightweight ORM for .NET. It provides a clean and extendable interface for constructing SQL queries and executing them in a secure and efficient manner. Its performance has always rivaled that of Entity Framework.
- **NHibernate:** NHibernate is an open source ORM that has been widely used as an alternative to Entity Framework. It has wide support for database technologies and offers alternative methods of handling object mapping and query construction that developers have come to prefer.

There are other ORMs, but these are arguably the most popular and widely used options.

Entity Framework Core has undergone constant improvement and is currently open source and provides excellent interfaces and abstractions that reduce the need for specific code to be written based on the database being accessed. This is a significant feature as it allows us to reuse code across

different database technologies and helps us to be flexible in terms of the database technology being used. We can easily change the database technology without affecting the main application and its operations. Barring the presence of any obvious bias on why we should use Entity Framework in our .NET projects, it does help that we can keep our technology stack homogenous.

To integrate Entity Framework Core, or *EF Core* for short, into our .NET application, we would need to add the package that is designed for our preferred database technology. For this example, we can use SQLite given its versatility. The commands to add the packages would look like this:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite  
dotnet add package Microsoft.EntityFrameworkCore.Design
```

The first command adds all the core and supporting EF Core libraries needed to communicate with a SQLite database and the `Microsoft.EntityFrameworkCore.Design` package adds support for design time logic for migrations and other operations. We will investigate migrations in the next section.

The next thing we need is a data context class, which serves as a code-level abstraction of the database and the tables therein. This context file will outline the database objects that we wish to access and refer to them. So, if our SQLite database should have a table that stores patient information, then we need a class that is modeled from how the patient table should look as well.

Our data context file looks like this:

```
public class ApplicationDatabaseContext : DbContext  
{  
    public DbSet<Patient> Patients { get; set; }  
    protected override void OnConfiguring  
        (DbContextOptionsBuilder optionsBuilder)  
    {  
        optionsBuilder.UseSqlite("Data Source=patients.db");  
    }  
}
```

This `ApplicationDatabaseContext` class inherits from `DbContext`, which is a class provided by EF Core that gives us access to the database operations and functions. We also have a property of type `DbSet`, which requires a reference type and a name. The reference type will be the class that models a table, and the name of the property will be the code-level reference point for the table. A `DbSet` represents the collection of records in the referenced table. Our `OnConfiguring` method sets up the connection string to our database. This connection string will vary based on the type of database we are connecting to.

Now, our patients table will have a few fields and we need a class called `Patient`, which has C# properties that represent the column names and data types as accurately as possible, relative to how they are presented in the database:

```
public class Patient
{
    // EF Core will add auto increment and Primary Key
    // constraints to the Id property automatically
    public int Id { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string TaxId { get; set; }

    // Using ? beside a data type indicates to the database
    // that the field is nullable
    public DateTime? DateOfBirth { get; set; }
}
```

Our `Patient` class is a typical class. It simply represents what we want a patient record to look like both in the database and in our application. This way, we abstract much of the database-specific code and use a standard C# class and standard C# code to interact with our tables and data.

A simple database query to retrieve and print all the records from the patients table would look like this:

```
// initialize a connection to the database
using var context = new ApplicationDatabaseContext();

// Go to the database and retrieve a list of patients
var patients = await context.Patients.ToListAsync();

// Iterate through the list and print to the screen
foreach (var patient in patients)
{
    Console.WriteLine($"{patient.FirstName}
        {patient.LastName}");
}
```

We can see how cleanly we can use the LINQ syntax in C# and execute a query. EF Core will attempt to generate the most efficient SQL syntax, carry out the operation, and return the data in the form of

objects fashioned by the class that models the table. We can then interact with the records as standard C# objects.

EF Core gives us a complete way to interact with our database and carry out our operations without needing to break much from our C# syntax. To go beyond the example shared above, EF Core has full support for dependency injection, which makes connection management and garbage collection almost a non-issue.

A fundamental step that was not addressed above comes in the form of database development techniques that govern how we can create our database in our project. We also have the issue of migrations. We will discuss those in the next section.

Choosing a database development technique

There are best practices and general guidelines that govern how databases are designed. In this section, we are not going to explore or contest those, but we will discuss the possible approaches that exist to materialize a database to support our application.

There are two popular types of development techniques:

- **Schema-first:** Also called **Database-First**, this technique sees us creating the database using the usual database management tools. We then *scaffold* the database into our application. Using Entity Framework, we will get a database context class that represents the database and all the objects within, as well as classes per table and view.
- **Code-first:** This technique allows us to model the database more fluently alongside our application code. We create the data models using standard C# classes (as we saw in the previous section) and manage the changes to the data models and eventual database using *migrations*.

Whether we intend to scaffold a database or use the code-first approach, we can use *dotnet CLI* commands that are made available via the `Microsoft.EntityFrameworkCore.Tools` package. You can add this package to your project using the `dotnet add package` command.

To scaffold a database, the commands look like this:

```
dotnet ef dbcontext scaffold "DataSource=PATH_TO_FILE"
Microsoft.EntityFrameworkCore.Sqlite -o Models
```

This command simply states that we wish to generate a `dbcontext` based on the database that is located at the connection string that is provided. Based on the technology of the target database (SQL Server, Oracle, and so on), this connection string will be different, but in this example, we're looking at scaffolding an SQLite database in our application. We then specify the database provider package that is most appropriate for the target database and output the generated files to a directory in our project called `Models`. This command is also agnostic to the IDE being used.

This command can also look like this, and this format is more popularly used when working in Visual Studio:

```
Scaffold-DbContext "DataSource=PATH_TO_FILE"
Microsoft.EntityFrameworkCore.Sqlite --output-dir Models
```

Either command needs to be executed within the directory of the project being worked on. Each time a change is made to the database, we need to rerun this command to ensure that the code reflects the latest version of the database. It will overwrite the existing files accordingly.

One limitation here is that we are not always able to track all the changes that are happening in the database and adequately track what changes each time we run this command. Therefore, code-first is a very popular option for database development and its use of migrations helps us to solve this problem.

The concept of migrations is not unique to EF Core, but it is present in several other frameworks in other languages as well. It is a simple process that tracks all the changes being made to a data model as we write our applications and the tables and objects evolve. A migration will evaluate the changes being applied to the database and generate commands (which eventually become SQL scripts) to effect those changes against the database.

Unlike with the database-first model, where we need to use source control with a dedicated database project, migrations are natively available in our code base and tell the story of every adjustment that is made to the database along the way.

Following the example of using EF Core to add a database to our application, we already created the database context class as well as a patient class. So, a direct translation of the code would be that our database context is the database, and `Patient.cs` is our table. To materialize the database with the table, we need to create a migration that will generate the assets as outlined. We need the same `Microsoft.EntityFrameworkCore.Tools` package to run our command-line commands and the command to create our first migration will look like this:

```
dotnet ef migrations add InitialCreate
```

Or, it will look like this:

```
Add-Migration InitialCreate
```

Here, we simply generate a migration file and give it the name `InitialCreate`. Naming our migrations helps us to visibly track what might have changed in that operation and helps our team as well. Each time we make a change to a data model and/or the database context class, we need to create a new migration, which will produce commands that outline the best possible interpretations of what has changed since the last known version of the database.

This command will generate a class with `Up` and `Down` methods that outline what changes are to be applied, and what will be undone if the migration is reversed. This makes it convenient for us to make

sure that the changes we intended are what will be carried out and we can remove a migration and make corrections before these changes are applied. Migrations can only be removed before they are applied to the database and the command for that looks like this:

```
dotnet ef migrations remove
```

Or, it looks like this:

```
Remove-Migration
```

When we are satisfied that a migration correctly outlines the changes that we intend to make, we can apply it using the simple `database update` command:

```
dotnet ef database update
```

Or, we can use the following:

```
Update-Database
```

If source control is being used with your project, then migrations will always be included in the code base for other team members to see and evaluate if necessary. It is important to collaborate with team members when making these adjustments as well. It is possible to have migration collisions if changes are not coordinated and this can lead to mismatched database versions if a central database is being used. Ultimately, if we need a fresh copy of the database as of the latest migrations, we can simply change our connection string to target a server and an intended database name, run the `Update-Database` command, and have EF Core generate a database that reflects the most recent version of our database.

Now that we understand how to use EF Core to either reverse engineer a database into our application or create a new database using migrations, let us investigate writing extensible database querying code on top of the interfaces provided by EF Core.

Implementing the repository pattern

By definition, a repository is a central storage area for data or items. In the context of database development, we use the word repository to label a widely used and convenient development pattern. This pattern helps us to extend the default interfaces and code given to us by our ORM, making it reusable and even more specific for certain operations.

One side effect of this pattern is that we end up with more code and files, but the general benefit is that we can centralize our core operations. As a result, we abstract our business logic and database operations away from our controllers and eliminate repeating database access logic throughout our application. It helps us to maintain the *single responsibility principle* in our code base as we seek to author clean code.

In any application, we need to carry out four main operations against any database table. We need to create, read, update, and delete data. In the context of an API, we need a controller per database table, and we would have our *CRUD* code repeating in every controller, which is good for getting started, but becomes dangerous as our application code base grows. This means that if something changes in the way that we retrieve records from every table, we have that many places that we need to change code in.

An API controller with a GET operation for our patients table would look like this without using the repository pattern:

```
[Route("api/[controller]")]
[ApiController]
public class PatientsController : ControllerBase
{
    private readonly ApplicationDatabaseContext
        _context;

    public PatientsController
        (ApplicationDatabaseContext context)
    {
        _context = context;
    }
    // GET: api/Patients
    [HttpGet]
    public async Task<ActionResult<IEnumerable
        <Patient>>> GetPatients()
    {
        if (_context.Patients == null)
        {
            return NotFound();
        }
        return await _context.Patients.ToListAsync();
    }
}
```

We simply inject the database context into the controller and use this instance to carry out our queries. This is simple and efficient as the query logic is not very complicated. Now imagine that you have several controllers with similar GET operations. All is well until we get feedback that we need to add paging to our GET operations for all endpoints. Now, we have that many places that need to be refactored with a far more complex query, which may also change again in the future.

For scenarios like this, we need to centralize the code as much as possible and reduce the rework that becomes necessary across the application. This is where the repository pattern can come to our rescue. We typically have two files that contain generic templating code, then, we derive more specific repositories for each table where we can add custom operations as needed.

Our generic repository includes an interface and a derived class. They look something like this:

```
public interface IGenericRepository<T> where T : class
{
    Task<T> GetAsync(int? id);
    Task<List<T>> GetAllAsync();
    Task<PagedResult<T>> GetAllAsync<T>(QueryParameters
        queryParameters);
}
```

Here, we only outline methods for read operations, but this interface can just as easily be extended to support all CRUD operations. We use generics as we are prepared to accept any type of class that represents a data model. We also outline a method for a GET operation that returns paged results in keeping with the recent requirement. Our derived class looks like this:

```
public class GenericRepository<T> : IGenericRepository<T>
    where T : class
{
    protected readonly ApplicationDbContext
        _context;

    public GenericRepository(ApplicationDbContext
        context)
    {
        this._context = context;
    }

    public async Task<List<T>> GetAllAsync()
    {
        return await _context.Set<T>().ToListAsync();
    }

    public async Task<PagedResult<T>> GetAllAsync<T>
        (QueryParameters queryParameters)
```

```

    {
        var totalSize = await _context.Set<T>()
            .CountAsync();
        var items = await _context.Set<T>()
            .Skip(queryParameters.StartIndex)
            .Take(queryParameters.PageSize)
            .ToListAsync();

        return new PagedResult<T>
        {
            Items = items,
            PageNumber = queryParameters.PageNumber,
            RecordNumber = queryParameters.PageSize,
            TotalCount = totalSize
        };
    }

    public async Task<T> GetAsync(int? id)
    {
        if (id is null)
        {
            return null;
        }

        return await _context.Set<T>().FindAsync(id);
    }
}

```

We see here that we are injecting the database context into the repository and then we can write our preset queries in one place. This generic repository can now be injected into the controllers that need to implement these operations:

```

[Route("api/[controller]")]
[ApiController]
public class PatientsController : ControllerBase
{
    private readonly IGenericRepository<Patient>

```

```

        _repository;

public PatientsController(IGenericRepository<Patient>
    repository)
{
    _repository = repository;
}
// GET: api/Patients
[HttpGet]
public async Task<ActionResult<IEnumerable
    <Patient>>> GetPatients()
{
    return await _repository.GetAllAsync();
}

// GET: api/Patients/?startIndex=0&pageSize=25
    &PageNumber=1
[HttpGet()]
public async Task<ActionResult< PagedResult
    <Patient>>> GetPatients([FromQuery]
    QueryParameters queryParameters)
{
    return await _repository.GetAllAsync
        (queryParameters);
}

```

Now we can simply call the appropriate method from the repository. The repository gets instantiated in our controller relative to the class that is used in its injection and the resulting query will be applied against the related table. This makes it much easier to standardize our queries across multiple tables and controllers. We need to ensure that we register our `GenericRepository` service in our `Program.cs` file like this:

```

builder.Services.AddScoped(typeof(IGenericRepository<>),
    typeof(GenericRepository<>));

```

Now we may need to implement operations that are specific to our patient table and are not needed for other tables. This means that the generic approach will not be best moving forward, as we would end

up cluttering it with custom logic for our tables. We can now extend it and create a specific interface for our table and write our custom logic:

```
public interface IPatientsRepository : IGenericRepository
    <Patient>
{
    Task<Patient> GetByTaxIdAsync(string id);
}

public class PatientsRepository : GenericRepository
    <Patient>, IPatientsRepository
{
    public PatientsRepository(ApplicationDatabaseContext
        context) : base(context)
    {}

    public async Task<Patient> GetByTaxIdAsync(string id)
    {
        return await _context.Patients.FirstOrDefaultAsync
            (q => q.TaxId == id);
    }
}
```

Now we can register this new service in the `Program.cs` file like this:

```
builder.Services.AddScoped<IPatientsRepository,
    PatientsRepository>();
```

And then we can inject it into our controller instead of `GenericRepository` and use it like this:

```
public class PatientsController : ControllerBase
{
    private readonly IPatientsRepository _repository;

    public PatientsController(IPatientsRepository
        repository)
    {
        _repository = repository;
    }
}
```

```
// GET: api/Patients
[HttpGet]
public async Task<ActionResult<IEnumerable<Patient>>>
    GetPatients()
{
    return await _repository.GetAllAsync();
}

// GET: api/Patients/taxid/1234
[HttpGet("taxid/{id}")]
public async Task<ActionResult<Patient>>
    GetPatients(string id)
{
    var patient = await _repository.GetByTaxIdAsync
        (id);
    if (patient is null) return NotFound();
    return patient;
}

// GET: api/Hotels/?startIndex=0&pageSize=25
// &PageNumber=1
[HttpGet()]
public async Task<ActionResult<PagedResult<Patient>>>
    GetPatients([FromQuery] QueryParameters
        queryParameters)
{
    return await _repository.GetAllAsync
        (queryParameters);
}
}
```

Here, we can cleanly call the custom code while maintaining access to the base functions that we implemented in the generic repository.

There are debates as to whether this pattern saves us time and effort or just makes our code base more complicated. There are pros and cons to this pattern but ensure that you do a fair assessment of the benefits and pitfalls before you choose to include it in your project.

Summary

Databases are a critical part of application development, and we need to ensure that we make the right decisions as early as possible. The technology, type of storage mechanism, and supporting application logic all play a large role in making our application as effective as possible in implementing business requirements. In this chapter, we explored the different considerations involved when developing supporting databases for microservices, the best type of database to use and when, and choosing a pattern of development that helps us to reduce redundant code.

In the next chapter, we will investigate implementing transactions across multiple services and databases using the *saga pattern*, since this is a big issue when we choose the database-per-service approach to microservice architecture.

Implement Transactions across Microservices Using the Saga Pattern

We have just looked at database development and what we need to consider when building an application developed using a microservices architecture. We discussed the pros and cons of creating individual databases per microservice. It does allow each microservice to have more autonomy, allowing us to choose the best technology needed for the service. While it is preferred and a recommended technique, it does have significant drawbacks when it comes to ensuring data consistency across the data stores.

Typically, we ensure consistency through transactions. Transactions, as discussed earlier in this book, ensure that all data is committed or none. That way, we can ensure that an operation will not partially write data and that what we see truly reflects the state of the data being tracked.

It is difficult to enforce transactions across microservices with different databases, but that is when we employ the *saga pattern*. This pattern helps us to orchestrate database operations and ensure that our operations are consistent.

After reading this chapter, we will understand how to do the following:

- Use the Saga pattern to implement transactions across microservices
- Choreograph data operations across microservices
- Implement orchestration

Exploring the Saga pattern

We have previously explored the *database-per-service* pattern, which encourages us to have individual data stores per service. With this in place, each microservice will handle its own database and transactions internally. This presents a new challenge where an operation that requires several services to take part and potentially modify their data runs the risk of partial failures and eventually leads to data inconsistency in our application. This is the major drawback of this pattern choice as we cannot guarantee that our databases will remain in sync at all times.

This is where we employ the saga pattern. You may think of a saga as a predefined set of steps that outline the order in which the services should be called. The saga pattern will also have the responsibility of providing oversight across all our services watching and listening, so to speak, for any signs of failure in any service along the way.

If a failure is reported by a service, the saga will also contain a rollback measure for each service. So, it will proceed, in a specific order, to prompt each service that might have been successful before the failure to undo the change it made. This comes in handy since our services are decoupled and ideally will not communicate directly with each other.

A saga is a mechanism that spans multiple services and can implement transactions across various data stores. We have distributed transaction options such as *two-phase commit*, which can require that all data stores commit or rollback. This would be perfect, except some NoSQL databases and message brokers are not entirely compatible with this model.

Imagine that a new patient registered with our healthcare center. This process will require that the patient provides their information and some essential documents, and books an initial appointment, which requires payment. These actions require four different microservices to get involved and thus, four different data stores will be affected.

We can refer to an operation that spans multiple services as a *saga*. Once again, a saga is a sequence of local transactions. Each transaction updates the data target database and produces a message or event that triggers the next transaction operation of the saga. If one of the local transactions fails along the chain, the saga will execute rollbacks across the databases that were affected by the preceding transactions.

Three types of transactions are generally implemented by a saga:

- **Compensable:** These are transactions that can be reversed by another transaction with the opposite effect.
- **Retryable:** These transactions are guaranteed to succeed and are implemented after pivot transactions.
- **Pivot:** As the name suggests, the success or failure of these transactions is pivotal to the continuation of the saga. If the transaction commits, then the saga runs until it is completed.

These transactions can be placed as a final compensable transaction or the first retryable transaction of the saga. They may also be implemented as neither.

Figure 8.1 shows the saga pattern:

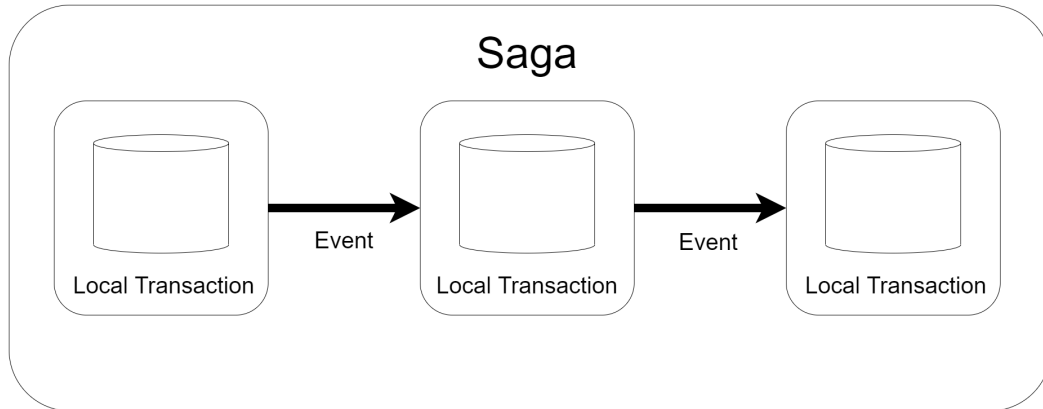


Figure 8.1 – Each local transaction sends a message to the next service in the saga until the saga is completed

As we know with every pattern, we have advantages and disadvantages, and it is important to consider all angles so that we can adequately plan an approach. Let us review some known issues and considerations that need to be taken when implementing this pattern.

Issues and considerations

Given the fact that, up until this chapter, we would have written off the possibility of implementing ACID transactions across our data stores in a microservices architecture, we can imagine that this pattern is not easy to implement. It requires absolute coordination and a good understanding of all the moving parts of our application.

This pattern is also difficult to debug. Given that we are implementing a singular function across autonomous services, we have now introduced a new touch point and potential point of failure for which special effort must be made to track and trace where the failure may have been. This complexity increases with each added step to the participating services of the saga.

We need to make sure that our saga can handle transient failures in the architecture. These are errors that happen during an operation that might not be permanent. Thus, it is prudent of us to include retry logic to ensure that a single failure in an attempt does not end the saga prematurely. In doing so, we also need to ensure that our data is consistent with each retry.

This pattern is certainly not without its challenges, and it will increase the complexity of our application code significantly. It is not foolproof as it will have its fallacies, but it will certainly assist us in ensuring that our data is more consistent across our loosely coupled services, by either rolling back or compensating for operational failures.

Sagas are usually coordinated using either *orchestration* or *choreography*. Both methods have their pros and cons. Let us begin with exploring choreography.

Understanding and implementing choreography

Choreography is a method of coordinating sagas where participating services used messages or events to notify each other of completion or failure. In this model, the event broker sits in between the services but does not control the flow of messages or the flow of the saga. This means that there is no central point of reference or control, and each service is simply watching for a message that acts as a confirmation trigger for it to start its operation.

Figure 8.2 shows the choreography flow:

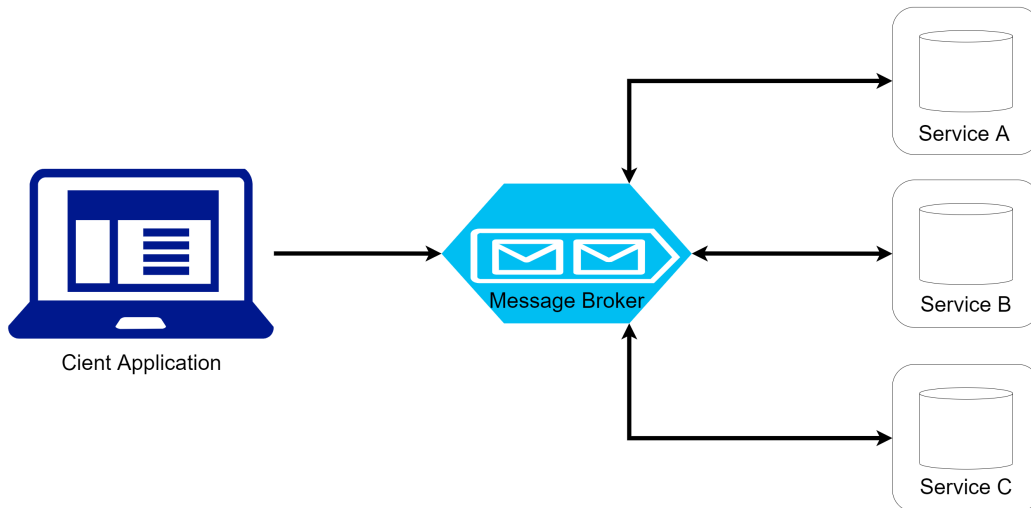


Figure 8.2 – An application request sends a message to the queue to inform the first service in the saga to begin, and messages flow between all participating services

The main takeaway from the choreography model is that there is no central point of control. Each service will listen to events and decide whether it is time to take an action. The contents of the message will inform the service it should act and if it acts, it will reply with a message stating the success or failure of its action. If the last service of the saga is successful, then no message is produced, and the saga will end.

If we were to visualize this process using our user registration and appointment booking example mentioned earlier in this chapter, we would have a flow looking like this:

1. The user submits a registration and appointment booking request (client request).
2. The *registration service* stores the new user's data and then publishes an event with relevant appointment and payment details. This event could be called, for example, `USER_CREATED`.
3. The *payment service* listens for `USER_CREATED` events and will attempt to process a payment as necessary. When successful, it will produce a `PAYMENT_SUCCESS` event.
4. The *appointment booking service* processes `PAYMENT_SUCCESS` events and proceeds to add the appointment information as expected. This service makes the booking arrangements and produces a `BOOKING_SUCCESS` event for the next service.
5. The *document upload service* receives the `BOOKING_SUCCESS` event and proceeds to upload the documents and add a record to the document service data store.

This example shows that we can track the processes along the chain. If we wanted to know each leg and the outcome, we can have the registration service listen to all events and make state updates or logs of the progress along the saga. It will also be able to communicate the success or failure of the saga back to the client.

What happens though when a service fails? How do we mitigate or reap the benefit of the the saga pattern's ability to reverse changes that have already gone? Let's review that next.

Rolling back on failure

Sagas are necessary because they allow us to roll back the changes that have already happened when something fails. If a local transaction fails, the service will publish an event stating that it was unsuccessful. We then need additional code in the preceding service that will react with the rollback procedures accordingly. For example, if our payment service operation failed, then the flow would look something like this:

1. The appointment booking service failed to confirm the appointment booking and publishes a `BOOKING_FAILED` event.
2. The payment service receives the `BOOKING_FAILED` event and proceeds to issue a refund to the client. This would be a remediation step.
3. The preceding registration service will see the `BOOKING_FAILED` event and notify the client that the booking was not successful.

In this situation, we are not completely reversing every step since we retain the user's registration information for future reference. What is important, though, is that the next service in the saga, which uploads the documents, is not configured to listen for the `BOOKING_FAILED` event. So, it will have nothing to do unless it sees a `BOOKING_SUCCESS` event.

We can also take note of the fact that our remediation steps are relative to the actual operation being carried out. Our *payment service* is likely a wrapper around a third-party payment engine that will also write a local database record of the payment operation. In its remediation steps, it will not remove the payment record, but simply mark it as a refunded payment or cancel the payment, given the lack of completion of the saga.

While this is not *ACID* in the true sense of what a local database would do, and undo a database the effects of a write operation, a rollback might look different for each service, based on the business rules or nature of the operation. We also see that our rollback did not span every single service, since our business rules suggest that we keep the user registration information for future reference.

Another thing that we need to consider is whether there is a necessity in our rollback operations. Given the event-based nature of our services, if we want to implement an order, then we will need more event types that services will listen for specifically.

Let us review the pros and cons of this choreography implementation.

Pros and cons

In the choreography model, we have a simple approach to implementing a saga. This method makes use of some of the previous patterns that we have discussed in *event sourcing* and *asynchronous service communication*. Each service retains its autonomy, and a rollback operation might look different per service. It is a clean way to implement a saga for a smaller operation with fewer participants and fewer potential outcomes based on success or failure.

We can also take the asynchronous approach to the saga as some form of advantage, as we can trigger multiple simultaneous operations stemming from each service's success. This is good for getting operations done quickly while the client is waiting on the outcome.

We also see that we need to always be expanding our code base to facilitate the varying operations and their outcomes, especially if we intend to implement an order for the rollback operations. Given the asynchronous model that is used to implement this type of saga, it might be dangerous to use one event type to trigger operations simultaneously.

As the number of participants grows, we run the risk of implementing a complex web of participants, events, and remediations. It grows increasingly difficult to properly monitor all the services and adequately trace the points of failure. If an operation is to be tested, all services must be running to properly troubleshoot our operations. The bigger the saga gets, the more difficult it is to monitor.

For this reason, we look to another saga pattern in the form of orchestration, which implements a central point of control. We will review it next.

Understanding and implementing orchestration

When we think of the word *orchestration*, we think of coordination. An orchestra is a coordinated combination of musicians all working towards producing the same kind of music. Each musician plays their part, but they are led by a conductor who guides each of them along the same path.

The orchestration method of implementing a saga is not very different in terms of how we need a central point of control (like a conductor), and all the services are monitored by the central point of control to ensure that they play their part well, or report failure accordingly. The central control is referred to as an *orchestrator* and it is a microservice that sits between the client and all other microservices. It handles all the transactions, telling participating services when to complete an operation based on feedback it receives during the saga. The orchestrator executes the request, tracks and interprets the request's state after each task, and handles the remediating operations, as necessary.

Figure 8.3 shows the orchestrator flow:

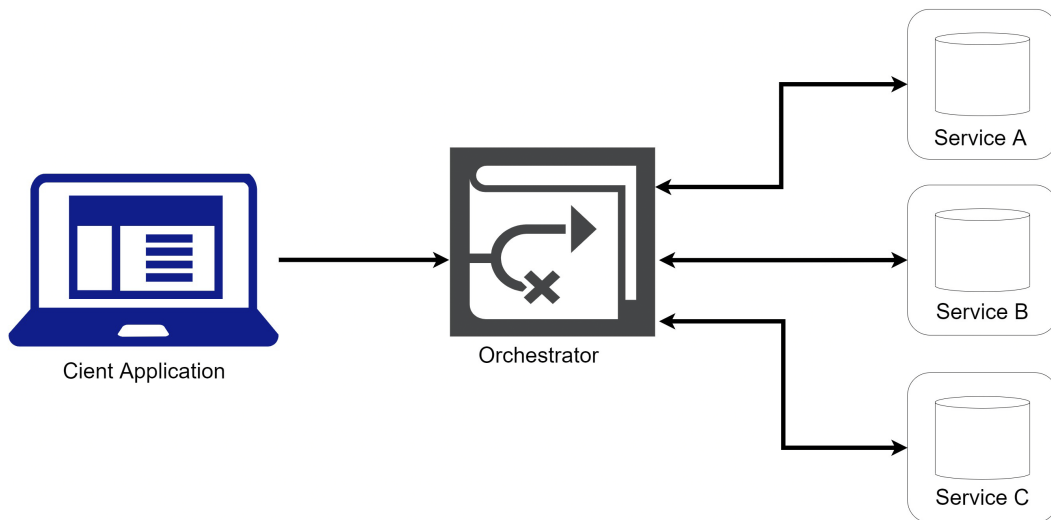


Figure 8.3 – An application request sends a message to the orchestrator, which begins to coordinate and monitor the subsequent calls to the participating services

Let us revisit our appointment booking operation from the perspective of the orchestration saga implementation:

1. The user submits a registration and appointment booking request (client request).
2. The client request is passed to the *orchestrator* service.
3. The *orchestrator* service centrally stores the data from the client request. This data will be used during the *User Registration* saga.

4. The *orchestrator* service begins the saga by passing the user's information to the *registration* service, which will add a new record to its database and respond with a `201Created` HTTP response. The *orchestrator* will store the user's ID, as it will be needed during the saga.
5. The *orchestrator* then sends the user's payment information to the *payment* service, which will respond with a `200OK` HTTP response. The *orchestrator* will store the payment response details, in the event that a rollback is needed and the payment should be canceled.
6. The *orchestrator* then sends a request to the *appointment booking* service, which processes the appointment booking accordingly and responds with a `201Created` HTTP response.
7. The *orchestrator* will finally trigger the *document upload* service, which uploads the documents and adds records to the document service database.
8. The *orchestrator* then confirms that the saga has ended and will update the state of the operation. It will then respond to the client with the overall result.

We can see that the orchestrator is at the helm of every step of the operation and remains informed of each service's outcome. It acts as the main authority on whether we should move to the next step or not. We can also see that a more *synchronous service communication* model is implemented in this saga pattern.

Let us review what a rollback operation might look like.

Rolling back on failure

Rolling back is the most important part of implementing a saga, and like the choreography pattern, we are governed by the business rules of the operation and individual service operations. The main takeaway here is that the services will respond with failure to a central point, which will then coordinate the rollback operations across the various services. Reusing the failure scenario previously discussed, our orchestration would look something like this:

1. The *appointment booking* service sends a `400BadRequest` HTTP response to the *orchestrator*.
2. The *orchestrator* proceeds to call the *payment* service to cancel the payment. It already stored the relevant information about the payment during the saga.
3. The *orchestrator* will trigger additional clean-up operations such as flagging the user's registration record as incomplete, as well as purging any additional data that may have been stored at the beginning of the operation.
4. The *orchestrator* will notify the client of the operation's failure.

A rollback here is arguably easier to implement – not because we are changing how and what the services do, but because we can be sure of the order in which the remediations will happen in case the order is important, and we can accomplish that without introducing too much more complexity to the flow.

Let us discuss the benefits of using this pattern in more detail.

Pros and cons

One obvious advantage to using this implementation of the saga pattern is the level of control that we can be sure to implement. We can orchestrate our service calls and receive real-time feedback, which can be used to decide and have a set path along the saga that we can track and monitor. This makes it easier to implement complex workflows and extend the number of participants over time.

This implementation is excellent for us if we need to control the exact flow of saga activities and be sure that we do not have services being triggered simultaneously and from information that they may think is relevant. Services only act when called upon, and misconfigurations are less likely. Services do not need to directly depend on each other for communication and are more autonomous, leading to simpler business logic. Troubleshooting also becomes easier since we can track what the singular code base is doing and more easily identify the point of failure.

Despite all these proposed benefits of orchestration, we need to remember that we are simply creating a central point of synchronous service calls. This can become a choke point along the saga if one of the services runs more slowly than desired. This can be managed, of course, through properly implemented *retry* and *circuit breaker* logic, but it remains a risk worthy of consideration.

We also run into a situation where we end up with yet another microservice to develop and maintain. We will introduce a new and more central point of failure since no other microservice gets called into action if the orchestrator is out of operation.

Let us review what we have learned in this chapter.

Summary

Until now, we have seen several patterns surrounding microservices architecture and development. Each pattern's purpose is to reduce the attrition that comes with this kind of architecture.

We saw a potential pain point and point of concern with our *database-per-service* pattern implementation and the difficulty that comes from having disparate data stores. We cannot always guarantee that all services will be successful in an operation and as such, we cannot guarantee that the data stores will reflect the same thing.

To address this, we look to the saga pattern, which can either be leveraged through an event-based *choreography* implementation or a more centralized *orchestration* method. We have reviewed the pros, cons, and considerations surrounding either implementation and how they help us to more effectively help microservices maintain data consistency.

In the next chapter, we will review the potential flaws involved in communication between microservices, and review how we can implement more fault-tolerant communication between services using the *circuit breaker pattern*.

Part 3:

Resiliency, Security, and Infrastructure Patterns

Reliability is one of the most critical aspects of API design. This part discusses the technique surrounding robust API design, security, and hosting. By the end of this part, you should be able to design advanced and secure APIs that can communicate with a lower failure rate and be hosted efficiently.

This part has the following chapters:

- *Chapter 9, Building Resilient Microservices*
- *Chapter 10, Performing Health Checks on Your Services*
- *Chapter 11, Implementing the API and BFF Gateway Patterns*
- *Chapter 12, Securing Microservices with Bearer Tokens*
- *Chapter 13, Microservice Container Hosting*
- *Chapter 14, Implementing Centralized Logging for Microservices*
- *Chapter 15, Wrapping It All Up*

Building Resilient Microservices

Coming off the heels of the Saga pattern, we can appreciate the value of having fail-safes built into our microservices application. We need to ensure that we adequately handle inevitable failures.

We can't assume that our distributed microservices will always be up and running. We also can't assume that our supporting infrastructure will be reliable. These considerations lead us down a path where we must anticipate the occurrence of failures, whether prolonged or transient.

A prolonged outage can be due to a downed server or service, some generally important part of the infrastructure. These tend to be easier to detect and mitigate since they have a more obvious impact on the runtime of the application. Transient failures are far more difficult to detect since they can last a few seconds to a few minutes at a time and aren't usually tied to any obvious issue in the infrastructure. Something as simple as a service taking 5 seconds extra to respond can be seen as a transient failure.

It is very important that we not only write code that doesn't break the application because of a transient issue but also know when to break that application when we have a more serious failure. This is an important part of gauging the user's experience with our application.

In this chapter, we will look at various scenarios and countermeasures that we can implement when navigating possible failures in our microservices architecture.

After reading this chapter, we will understand how to do the following:

- Build resilient microservice communication
- Implement a caching layer
- Implement retry and circuit breaker policies

Technical requirements

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch09>.

The importance of service resiliency

Before we get into technical explanations, let us try to understand what it means to be resilient. The word *resilient* is the base word for *resiliency*, and it refers to how impervious an entity is to negative factors. It refers to how well an entity reacts to an inevitable failure and how well an entity can resist future failures.

In the context of our microservices architecture, our entities are our services, and we know that failures will happen. A failure can be as simple as a timeout during an internal operation, a loss of communication, or an unexpected outage of an important resource for the service.

Possible failure scenarios and how to handle them

Using the example of our healthcare booking system, let us say that our appointments service needs to retrieve the details of the related patient. The *appointments service* will make a synchronous HTTP call to the *patients service*. The steps in this communication step may look like this:

1. The *appointments service* makes an HTTP request to the *patients service*, passing the patient ID (`AppointmentsController.cs`).
2. The *patients service* receives the HTTP request and executes a query to look up the patient's record (`PatientsController.cs`).
3. The *patients service* responds to the *appointment booking service* with the appropriate data.

So far, we have come to expect this from our *synchronous service* communication flow. This, however, is the ideal flow where everything works as expected, and based on your infrastructure, you can guarantee a certain degree of success each time. What we need to account for is the balance—the few times that the flow might get interrupted and not complete the chain of operations successfully. It could be because of a failure, or maybe we just need a little more time. It could also be that we need to cut the call short because it is taking too long.

Now, let us review the same kind of service call where something fails along the way:

1. The *patients service* makes an HTTP request to the *appointment booking service*, passing the patient ID.
2. The *appointment booking service* responds with a `BAD_GATEWAY (502)` error code.
3. The *patients service* throws an exception immediately when given the `BAD_GATEWAY` response.
4. The user receives an error message.

In this situation, we received a premature termination of the appointment booking service call. HTTP responses in the 5xx range indicate that there is an issue with the resource or server associated with the appointment booking service. These 5xx errors may be temporary, and an immediate follow-up request would work. A BAD GATEWAY error, specifically, can be due to poor server configuration, proxy server outage, or a simple response to one too many requests at that moment.

In addressing these issues, sometimes we can retry the requests, or have an alternative data source on standby. While we will be discussing retry logic later in the book, we explore using some form of caching layer that allows us to maintain a stable data layer from which we can pull the information we require.

Let us review how we can implement a caching layer to assist with this.

Implementing resiliency with caching and message brokers

We will be diving into how we can make our services resilient using **retry policies** and the **circuit breaker pattern**, but we are not limited to these methods in our microservices architecture. We can help to support service resiliency using **caching** and **message broker** mechanisms. Adding a caching layer allows us to create a temporary intermediary data store, which becomes useful when we are attempting to retrieve data from a service that is offline at the moment. Our message brokers help to ensure that messages will get delivered, which is mostly useful for write operations.

Let us discuss message brokers and how they help us with our resiliency.

Using a message broker

Message brokers have a higher guarantee of data delivery, which increases resiliency. This is built on the foundation that the message broker will not be unavailable for an extended period, but once a message is placed on the message bus, it will not matter if the listening service(s) is not online. As we discussed earlier, we can almost guarantee that data will be posted successfully through asynchronous communication, since message brokers are designed to retain the information until it is consumed.

Message brokers also support retry logic where if a message is not processed successfully for whatever reason, it is returned to the queue for processing later. We want to manage the number of message delivery retries, so we should configure our message broker to transfer a message to a *dead-letter* queue, where we store *poisoned* messages.

We also need to consider how we handle message duplications. This could happen if we send a message to the queue that does not get processed immediately for some reason, and then the message gets sent to the queue again from a retry. This would result in the same message in the queue twice and not necessarily in the correct order, or one behind the other. We must ensure that our messages contain enough information to allow us to adequately develop redundancy checks in our message consumers.

We explored integrating with message brokers in earlier chapters as we discussed asynchronous communication between services. Now, let us explore implementing a caching layer.

Using a caching layer

Caching can be a valuable part of the resiliency strategy. We can incorporate a caching strategy where we fall back on this cache if a service is offline. This means that we can use the caching layer as a fallback data source and create an illusion to the end user that all services are up and running. This cache would get periodically updated and maintained each time data is modified in the database of the source service. This will help with keeping the cached data fresh.

Of course, with this strategy, we need to accept the implications of having potentially stale data. If the source service is offline and the supporting database is being updated (possibly by other jobs), then the cache will eventually become a stale data source. The more measures we put in place to ensure its freshness, the more complexity we introduce to our application.

Notwithstanding the potential pros and cons of adding a caching layer, we can see where it will be a great addition to our microservices application and reduce the number of errors that a user might see, stemming from transient and even longer-term failures.

The most effective way to implement a caching layer is as a distributed cache. This means that all systems in the architecture will be able to access the central cache, which exists as an external and standalone service. This implementation can increase speed and support scaling. We can use Redis Cache as our distributed cache technology, and we will investigate how we can integrate this into an ASP.NET Core application.

Using Redis Cache

Redis Cache is a popular caching database technology. It is an open source in-memory data store that can also be used as a message broker. It can be implemented on a local machine for local development efforts but is also at times deployed on a central server for more distributed systems. Redis Cache is a key-value store that uses a unique key to index a value, and no two values can have the same key. This makes it very easy to store and retrieve data from this type of data store. In addition to that, values may be stored in very simple data types such as strings, numbers, and lists, but JSON format is popularly used for more complex object types. This way, we can serialize and deserialize this string in our code and proceed as needed.

There is also extensive support for Redis Cache on cloud providers such as Microsoft Azure and **Amazon Web Services** (AWS). For this exercise, you may install the Redis Cache locally, or use a **Docker** container. To start using Redis Cache in our project, we need to run the following command:

```
dotnet add package Microsoft.Extensions.Caching
    .StackExchangeRedis
```

We then need to register our cache in our `Program.cs` file, like this:

```
// Register the RedisCache service
services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = "Configuration.GetSection
        ("Redis") ["ConnectionString"]
});
```

You may configure your connection string in either the `appsettings.json` file or in application secrets. It will look something like this:

```
"Redis": {
    "ConnectionString": "CONNECTION_STRING_HERE"
}
```

These steps add caching support to our application. Now, we can read from and write to the cache as needed in our application. Generally, we want to write to the cache when data is augmented. New data should be written to the cache—we can remove the old version and create a new version for modified data; for deleted data, we also delete the data from the cache.

We can create a singular `CacheProvider` interface and implementation as a wrapper around our desired cache operations. Our interface will look like this:

```
public interface ICacheProvider
{
    Task ClearCache(string key);
    Task<T> GetFromCache<T>(string key) where T : class;
    Task SetCache<T>(string key, T value,
        DistributedCacheEntryOptions options) where T :
        class;
}
```

Our implementation looks like this:

```
public class CacheProvider : ICacheProvider
{
    private readonly IDistributedCache _cache;

    public CacheProvider(IDistributedCache cache)
    {
```

```
        _cache = cache;
    }

    public async Task<T> GetFromCache<T>(string key) where
        T : class
    {
        var cachedResponse = await
            _cache.GetStringAsync(key);
        return cachedResponse == null ? null :
            JsonSerializer.Deserialize<T>(cachedResponse);
    }

    public async Task SetCache<T>(string key, T value,
        DistributedCacheEntryOptions options) where T :
        class
    {
        var response = JsonSerializer.Serialize(value);
        await _cache.SetStringAsync(key, response,
            options);
    }

    public async Task ClearCache(string key)
    {
        await _cache.RemoveAsync(key);
    }
}
```

This code allows us to interact with our distributed caching service and retrieve, set, or remove values based on their associated key. This service can be registered in our **inversion of control (IoC)** container and injected into our controllers and repositories as needed.

The idea here is that we can use the `GetFromCache` method when we need to read the values from the cache. The key allows us to narrow down to the entry we are interested in, and the `T` generic parameter allows us to define the desired data type. If we need to update the data in the cache, we can clear the cache record associated with the appropriate key and then use `SetCache` to place new data with an associated key. We will parse the new data to JSON to ensure that we do not violate the supported data types while maintaining the ability to store complex data. When we are adding new data, we simply need to call `SetCache` and add the new data.

We also want to ensure that we maintain the freshness of the data as much as possible. A popular pattern involves clearing the cache and making a fresh entry each time data is entered or updated.

We can use these bits of code in our application and implement a caching layer to improve not only performance but resiliency and stability. We still have the issue of retrying operations when they fail the initial call. In the next section, we will look at how we can implement our retry logic.

Implementing retry and circuit breaker policies

Services fail for various reasons. A typical response to a service failure is an HTTP response in the 5xx range. These typically highlight an issue with the hosting server or a temporary outage in the network hosting the service. Without trying to pinpoint the exact cause of the failure at the time it happens, we need to add some fail-safes to ensure the continuity of the application when these types of errors occur.

For this reason, we should use retry logic in our service calls. These will automatically resubmit the initial request if an error code is returned, which might be enough time for a transient error to resolve itself and reduce the effects that the initial error might have on the entire system and operation. In this policy, we generally allow for some time to pass between each request attempt. This sums up our retry policy.

What we don't want to do with our retries is to continue to execute them without some form of exit condition. This would be like implementing an infinite loop if the target service remains unresponsive and inadvertently executing a **denial-of-service (DoS)** attack on our own service. For this reason, we implement the circuit breaker pattern, which acts as an orchestrator for our service calls.

We will need to implement a **retry policy** to at least make the call several times before concluding a definite failure. This will make our service more resilient to a potentially fleeting error and allow the application to ensure that the user's experience isn't directly affected by such an issue. Now, retries are not always the answer. A retry here makes sense since the service is responding with a clear-cut failure, and we are deciding to try again. We need to decide how many retries are too many and stop accordingly.

We can use the **circuit breaker pattern** to control the number of retries and set parameters that will govern how long a connection should stay open and listen for a response. This simple technique helps to reduce the number of retries and provides better control over how retries occur.

A *circuit breaker* sits in between the client and the server. In our microservices application, the service making the call is the client, and the target service is the server receiving the request. Initially, it allows all calls to go through. We call this the *closed state*. If an error is detected, which can be in the form of an error response or a delayed response, the *circuit breaker opens*. Once the circuit breaker is open, subsequent calls will fail faster. This will shorten the time spent waiting for a response. It will wait for a configured timeout period and then allow the calls again, in case the target service has recovered. If there is no improvement, then the circuit breaker will break the transmission.

Using these two techniques, we can both counter transient failures and ensure that longer-term failures do not surface in the form of a poor user experience. In .NET Core, we have the benefit of **Polly**, which is a package that allows us to almost support natively both retry and circuit break policies and implement resilient web service calls. We will explore integrating Polly into our app next.

Retry policy with Polly

Polly is a framework that allows us to add a new layer of resilience to our applications. It acts as a layer between two services that stores the details of an initiated request and monitors the response time and/or the response code. It can be configured with parameters that it uses to determine what a failure looks like, and we can further configure the type of action we would like to take. This action can be in the form of a retry or a cancellation of the request.

Polly is conveniently available in .NET Core and is widely used and trusted around the world. Let us review the steps needed to implement this framework in our application and monitor the calls the *Patients API* will make to the *Documents API*.

To add it to our .NET Core application and allow us to write extension code for our `HttpClient` objects, we start by adding these packages via NuGet:

```
Install-Package Polly
Install-Package Microsoft.Extensions.Http.Polly
```

Now, in our `Program.cs` file, we can configure our typed HTTP client for our *Documents API* to use our extension code for its Polly-defined policies. In the `Program.cs` file, we can define the registration of our typed client like this:

```
builder.Services.AddHttpClient<IDocumentService,
    DocumentService>()
    .AddPolicyHandler(GetRetryPolicy());
```

We have added a policy handler to our HTTP client, so it will automatically be invoked for all calls made using this client. We now need to define a method called `GetRetryPolicy()` that will build our policy:

```
static IAsyncPolicy<HttpResponseMessage> GetRetryPolicy()
{
    return HttpPolicyExtensions
        .HandleTransientHttpError()
        .OrResult(r => !r.IsSuccessStatusCode)
        .Or<HttpRequestException>()
        .WaitAndRetryAsync(5, retryAttempt =>
```

```
        TimeSpan.FromSeconds(Math.Pow(2,
            retryAttempt)), (exception, TimeSpan,
            context) => {
            // Add logic to be executed before each
            retry, such as logging or
            reauthentication
        });
    }
```

It may seem complicated because of the use of the builder pattern, but it is simple to understand and flexible enough to customize to your needs. Firstly, we define the return type or the method to be `IAsyncPolicy<HttpResponseMessage>`, which corresponds with the return type of our calls from our HTTP client. We then allow the policy to observe for transient HTTP errors, which the framework can determine by default, and we can extend that logic with more conditions such as observing the value of `IsSuccessStatusCode`, which returns `true` or `false` for the success of an operation, or even if `HttpRequestException` has been returned.

These few parameters cover the general worst-case scenarios of an HTTP response. We then set the parameters for our retries. We want to retry at most 5 more times, and each retry should be done at a rolling interval starting at 2 seconds from the previous call. This way, we allow a little time between each retry. This is the concept of a *backoff*.

Finally, we can define what action would like to take between each retry. This could include some error handling or reauthentication logic.

Retry policies can have negative effects on your system where we might have high concurrency and high contention for resources. We need to ensure that we have a solid policy and define our delays and retries efficiently. Recall that a carelessly configured retry policy may well result in a DoS attack on your own service, opening the application to significant performance issues.

Given the possibility of implementing something that could have negative effects in this manner, we now need a defense barrier that will mitigate this risk and break the retry cycle if the errors never stop. The best defense strategy comes in the form of the circuit breaker, which we will configure using Polly next.

Circuit breaker policy with Polly

As we have discussed for this chapter, we should handle faults that take a bit longer to resolve and define a policy that abandons retry calls to a service when we have concluded that it is unresponsive for a longer term than hoped for.

We can continue from our code that added the retry policy using Polly by defining a circuit breaker policy and adding it as an HTTP handler to our client. We modify the client's registration in the `Program.cs` file, like this:

```
builder.Services.AddHttpClient<IDocumentService,  
    DocumentService>().AddPolicyHandler(GetRetryPolicy())  
    .AddPolicyHandler(GetCircuitBreakerPolicy());
```

Now, we can add the `GetCircuitBreakerPolicy()` method as this:

```
static IAsyncPolicy<HttpResponseMessage>  
    GetCircuitBreakerPolicy()  
{  
    return HttpPolicyExtensions  
        .HandleTransientHttpError()  
        .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));  
}
```

This policy defines a circuit breaker that opens the circuit when there have been 5 consecutive retry failures. Once that threshold is reached, the circuit will break for 30 seconds and automatically fail all subsequent calls, which will be interpreted as HTTP failure responses.

With these two policies in place, you can orchestrate your service retries and significantly reduce the effects that unplanned outages might have on your application and the end user's experience. The circuit breaker policy also adds a layer of protection from any potential adverse effects of the retry policy.

Now, let us summarize what we have learned about implementing resilient web services.

Summary

The contents of this chapter help us to be more mindful of the potential for failures in our microservices application. These concepts help us not only construct powerful and stable web services but also supercharge the communication mechanisms that exist between them.

We see that service outages are not always due to faulty code or the database and server of the initial web service, but we are facilitating inter-service communication as well, which leads to greater dependence on the network, third-party service, and general infrastructure uptime. This leads down a path where we implement contingencies that assist in ensuring that our application gives our users as good an experience as possible.

We looked at several techniques for increasing service reliability, such as using a caching layer with technology such as Redis Cache for our GET operations, a message broker for our write operations, and writing more foolproof code using frameworks such as Polly. With Polly, we looked at how we

can automatically retry service calls and use a circuit breaker to prevent these retries from being too liberal and causing other problems.

Since services fail and we need a retry method, we also need a way to monitor the health of the services so that we can be aware of why the retries are not effective. This means that we need to introduce *health checks* that alert us to outages in a service's infrastructure. We will explore this in the next chapter.

Performing Health Checks on Your Services

Maintaining maximum uptime is an important aspect of any system. In the previous chapter, we saw where we can write code in a fault-tolerant manner that will reduce the prevalence of outages in our infrastructure and network. This, however, is not a long-term solution, and things fail regardless of these measures. It then leads to the notion that we need to know when there are failures.

This is where we start thinking about health checks. Health checks exist as a mechanism to inform us of outages in our services and supporting databases and connections in our application. Generally, this can be accomplished with a simple ping request to a resource. The resource is available and operating as expected if we get a response. In the absence of a response, we assume that the resource is down and trigger an alert.

There are statuses between the service's up and down status, and we will discover those options in this chapter. We will also explore some of the functionality that .NET Core provides to implement these checks.

In this chapter, we will look at various scenarios and countermeasures that we can implement when navigating possible failures in our microservices architecture.

After reading this chapter, we will be able to do the following:

- Understand why health checks are necessary
- Understand how to implement health checks in ASP.NET Core
- Understand how orchestrators monitor and respond to failures

Technical requirements

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch10>.

Health checks and microservices

A health check allows us to monitor the health of our services. Frankly, another service or resource that exposes an HTTP endpoint becomes a capable candidate for health checks. We can simply make a request to this endpoint and hope for a response that indicates a successful response. The simplest form of a health check can come from implementing a simple GET request, which returns a 200OK HTTP response. We can add further intelligence to such an endpoint, check connectivity to other key services, and use those to influence the response code returned.

Health checks are useful mechanisms for both monolithic and microservices applications. In the context of microservices, however, we have an increased challenge of monitoring and maintaining several services. Even more so if they are configured to scale on individual levels. Health checks can be used to monitor the health and uptime of interdependent services and carry out some form of corrective action when a service is down.

Using .NET Core, we can return a successful response and include additional information that gives some details on the health of the service. In this case, we cannot simply go by the 200OK response, but we need to interrogate the actual response body to check whether the service is healthy, degraded, or unhealthy.

Figure 10.1 shows a typical health check:

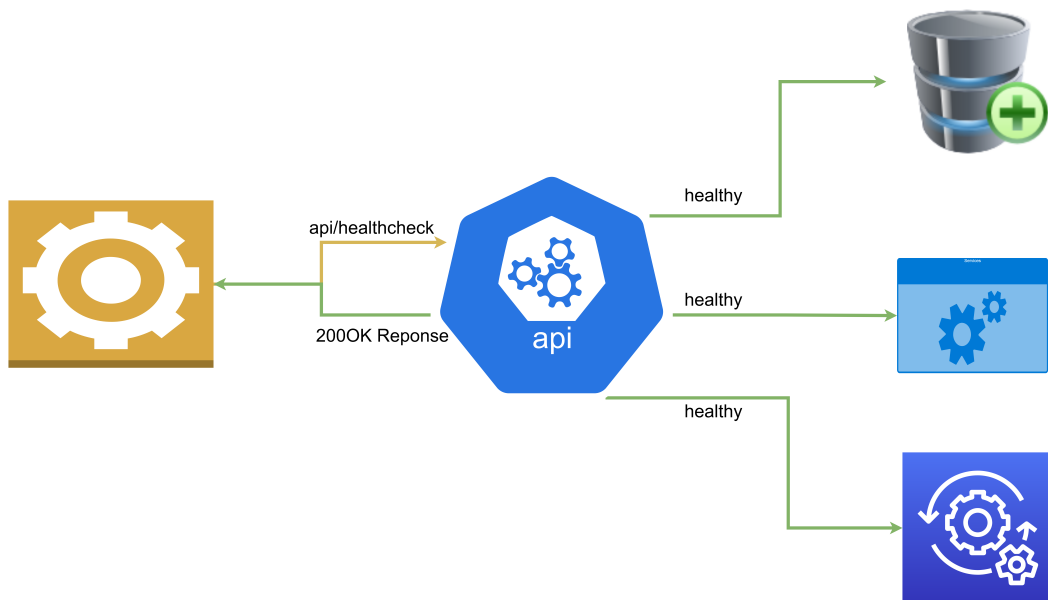


Figure 10.1 – Shows a health check request and healthy response after verifying that all services are available

Let's break down what each state entails:

- **Healthy:** This indicates that the service is healthy and the application is operating as expected
- **Degraded:** This indicates that the service is live, but some functionality may be unavailable
- **Unhealthy:** This indicates that the service is failing and is not operating as expected

Figure 10.2 shows a failed health check:

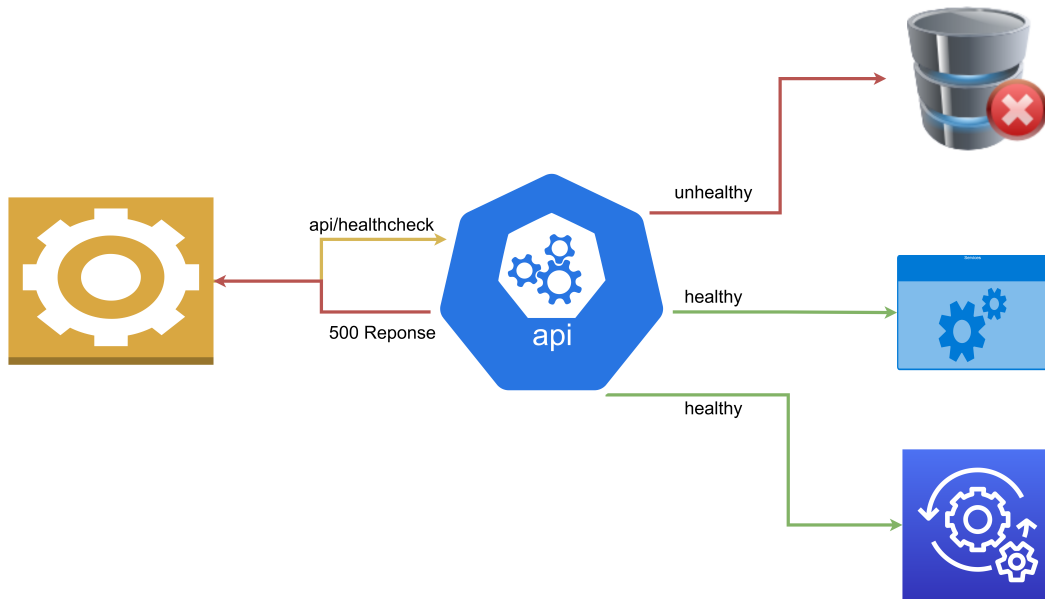


Figure 10.2 – Shows a health check that sent a failure response since one of the services was not available

The health of a service depends on several things, including correct configuration, access to keys and dependencies, the status of the hosting platform and infrastructure, and the connection to the database. They may also be used for external application monitoring and overall application health.

A common deployment model for microservices is using container orchestrators, such as *Kubernetes*, to deploy and run our services in production. Most orchestrators perform periodical *liveness health checks* on their pods during runtime and *readiness health checks* during deployments. Health checks help the orchestrator determine which pods are in a ready state and are capable of handling traffic. It is important to understand the differences between the liveness health check and the readiness health check and when which one is most suitable. The easier one to implement is the liveness health check; we will discuss this next.

The liveness health check

The liveness health check endpoint is a specific endpoint that is implemented for the sole purpose of the health check. In this health probe, the service is considered healthy when it responds to the *liveness health check*. A failure to respond to this endpoint indicates a serious issue with the application. This issue could be caused by a range of reasons, such as a crash or unplanned application restart. For this reason, restarting an application that has failed this check is a common course of action.

Applications that monitor the infrastructure, such as *Kubernetes* monitoring Docker containers, use *liveness health checks* to determine the health of pods and trigger restarts as needed. Cloud providers also offer health probe functionality with load balancers, which can check the availability of the deployed application by periodically sending a request to the liveness check endpoint. This approach is generally sufficient for web applications and services as we do not need a complicated liveness check endpoint. If the service can accept the request and return a response, then we consider it healthy.

Checking whether the application or services is alive is simple enough, but we may also need to mitigate false positives after application deployments and/or upgrades. This can occur when the application might not be completely ready for usage, yet we are getting positive responses from the liveness checks. At this point, we need to consider implementing *readiness health checks*.

Readiness health checks

Readiness health checks are used in situations where we need to verify more than just an HTTP response. An application with several third-party dependencies might take longer to be ready for use. So, while it is operational and able to respond to a simple HTTP request, the databases or message bus services, for instance, might not yet be ready. We want to ensure that we have a full picture of the status of the application from a startup perspective before proceeding to use it or continuing with the deployment activity.

A *readiness health check* will generally only return a healthy status once the startup task has been completed. These checks will then take a bit longer to return a healthy status than a *liveness health check* will. With readiness health checks in place, an orchestrator will not attempt to restart the application, but it will not route request traffic. *Kubernetes* can perform readiness probes periodically during the application's runtime, but it can also be configured to only perform this probe during the startup of the application. Once the application reports that it is healthy, then this probe will not be executed again for the lifetime of the application.

This *readiness health check* is best used for applications where there are long-running tasks that must finish before the application can be considered ready and operational. Recall that with microservices, we introduce several additional infrastructure dependencies, and we need to monitor and confirm the overall health of the system to ensure that only the healthiest pods get traffic directed to them. Therefore, properly configuring health checks is essential to ensuring that we have the best representation of our application's health.

Now that we have explored how health checks work and how orchestrators and monitoring systems use them, we can explore implementing health checks in our ASP.NET Core API.

Implementing ASP.NET Core health checks

ASP.NET Core has a built-in health check middleware that allows us to natively implement very robust health checks. This middleware is not limited to API projects, and it comes in handy to help us to monitor the health of our application. Both readiness and liveness health checks can be created natively, and there is support for a UI dashboard. Using the liveness health check, which is relatively simple to implement, we can implement a simple API endpoint that returns a simple response, as expected. We can also check on the health of the dependencies of the app using a more comprehensive readiness health check.

For this example, we will be adding liveness and readiness health checks to our appointment booking service. This service has several dependencies and is integral to several operations in our application. We need to ensure that it is always healthy and react quickly if it degrades.

Let us start off by exploring how we can outfit an ASP.NET Core API with a liveness health check.

Adding liveness health checks

As discussed, a liveness check is the most basic health check that can be implemented. The basic configuration needed for this in our ASP.NET Core application is to register the `AddHealthChecks` service and the addition of the health check middleware, where we define a URL.

We make the following changes to the `Program.cs` file:

```
var builder = WebApplication.CreateBuilder(args);  
// code omitted for brevity  
builder.Services.AddHealthChecks();  
  
var app = builder.Build();  
// code omitted for brevity  
app.MapHealthChecks("/healthcheck");  
  
app.Run();
```

Any attempt to navigate to the `/healthcheck` endpoint will yield a simple plain text response as `HealthStatus`. The possible `HealthStatus` values are `HealthStatus.Healthy`, `HealthStatus.Degraded`, or `HealthStatus.Unhealthy`.

Health checks are created using the `IHealthCheck` interface. This interface allows us to extend the default health checks and add more logic to our health check and further customize the possible response values. We can create a health check extension using the following code block:

```
public class HealthCheck : IHealthCheck
{
    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context, CancellationToken =
            default)
    {
        var healthy = true;
        if (healthy)
        {
            // additional custom logic when the health is
            // confirmed.
            return Task.FromResult(
                HealthCheckResult.Healthy("Service is
                    healthy"));
        }

        // additional custom logic when the api is not
        // healthy
        return Task.FromResult(
            new HealthCheckResult(
                context.Registration.FailureStatus,
                "Service is unhealthy"));
    }
}
```

This inheriting from `IHealthCheck` forces us to implement the `CheckHealthAsync` method. This method gets called when a health check is triggered, and we can include additional code to check other factors and determine whether we deem our application to be healthy or not. Based on the value of `healthy`, we can return a custom message.

Now to add `HealthCheck` to our services, we modify the `AddHealthChecks` service registration like this:

```
builder.Services.AddHealthChecks()  
    .AddCheck<HealthCheck>("ApiHealth");
```

Here, we add our new health check logic and give it a name for a specific reference in other parts of the application. This `AddCheck` method allows us to define a name for the health check, a default failure status value, tags to map to custom health check endpoints, and a default timeout value.

Now building on the notion that our orchestrators and load balancers that are performing health checks will also prefer to see appropriate responses relative to the health status, we can extend the `app.MapHealthChecks` middleware code to return a specific HTTP response relative to the health status. While we are at it, we can also disable cached responses:

```
app.MapHealthChecks("/healthcheck", new HealthCheckOptions  
{  
    AllowCachingResponses = false,  
    ResultStatusCodes =  
    {  
        [HealthStatus.Unhealthy] =  
            StatusCodes.Status503ServiceUnavailable,  
        [HealthStatus.Healthy] = StatusCodes.Status200OK,  
        [HealthStatus.Degraded] = StatusCodes.Status200OK,  
    }  
});
```

The next thing we may want to investigate is returning details in our response. As it stands, we are only returning the plain text response with the status. We can use methods found in the `System.Text.Json` library to create a custom delegate method that can be implemented as follows.

We first need to indicate to the middleware that we have a custom `ResponseWriter` called `WriteJsonResponse`. We need to add this to the list of `HealthCheckOptions`, using the following:

```
app.MapHealthChecks("/healthcheck", new HealthCheckOptions  
{  
    // code omitted for brevity  
    ResponseWriter = WriteJsonResponse  
});
```


We then define the `WriteJsonResponse` writer with the following:

```
private static Task JsonResponse(HttpContext context,
    HealthReport healthReport)
{
    context.Response.ContentType = "application/json;
        charset=utf-8";
    var options = new JsonSerializerOptions { Indented = true };

    using var memoryStream = new MemoryStream();
    using (var jsonWriter = new Utf8JsonWriter
        (memoryStream, options))
    {
        jsonWriter.WriteStartObject();
        jsonWriter.WriteString("status",
            healthReport.Status.ToString());
        jsonWriter.WriteStartObject("results");

        foreach (var healthReportEntry in
            healthReport.Entries)
        {
            jsonWriter.WriteStartObject
                (healthReportEntry.Key);
            jsonWriter.WriteString("status",
                healthReportEntry.Value.Status.ToString());
            jsonWriter.WriteString("description",
                healthReportEntry.Value.Description);
            jsonWriter.WriteStartObject("data");

            foreach (var item in
                healthReportEntry.Value.Data)
            {
                jsonWriter.WritePropertyName(item.Key);

                JsonSerializer.Serialize(jsonWriter,
                    item.Value,
                    item.Value?.GetType() ??
```

```
        typeof(object));
    }

    jsonWriter.WriteEndObject();
    jsonWriter.WriteEndObject();
}

jsonWriter.WriteEndObject();
jsonWriter.WriteEndObject();
}

return context.Response.WriteAsync(
    Encoding.UTF8.GetString(memoryStream.ToArray()));
}
```

Figure 10.3 shows the results of a health check:

```
{
  "status": "Healthy",
  "results": {
    "ApiHealth": {
      "status": "Healthy",
      "description": "Service is healthy",
      "data": {}
    },
    "DatabaseHealth": {
      "status": "Healthy",
      "description": null,
      "data": {}
    }
  }
}
```

Figure 10.3 – Shows a health check response where both the service and database are available and in good health

Now we can include details about the health status if the API reports an unhealthy or degraded status. Furthermore, when we add more health checks, the content of this JSON response will be populated with each check's details.

Figure 10.4 shows the results of an unhealthy check:

```
{
  "status": "Unhealthy",
  "results": {
    "ApiHealth": {
      "status": "Healthy",
      "description": "Service is healthy",
      "data": {}
    },
    "DatabaseHealth": {
      "status": "Unhealthy",
      "description": null,
      "data": {}
    }
  }
}
```

Figure 10.4 – Shows a health check response where the database is not available

Now that we have more detailed responses, we can add more detailed checks, such as a database probe. This will serve as a check to verify that the API can communicate with the database through the configured database. By extension, since we are using Entity Framework for this connection, we can implement a `DbContext` check. We start with the `Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore` NuGet package. We then modify the `AddHealthChecks` method registration using the following piece of code:

```
builder.Services.AddHealthChecks()
    .AddCheck<HealthCheck>("ApiHealth")
    .AddDbContextCheck<ApplicationDbContext>
        ("DatabaseHealth");
```

This context health calls Entity Framework Core's built-in `CanConnectAsync` method and uses that response to infer the database connectivity health.

Now that we can check on the health of our service and its connectivity to our database let us configure it for readiness checks.

Adding readiness health checks

As we have discussed, the readiness check indicates when the application and its dependencies have started successfully and are ready to begin receiving requests. We can define a separate endpoint for the readiness check and further customize the checks that should be performed based on the URL used.

To implement liveness and readiness checks on different URLs, we can add a `tags` parameter to the extensions to the `AddHealthChecks` method. This allows us to pass in an array of tag names. We can tag our health checks like this:

```
builder.Services.AddHealthChecks()
    .AddCheck<HealthCheck>("ApiHealth", tags: new[] {
        "live"})
    .AddDbContextCheck<ApplicationDbContext>("DatabaseHealth",
        tags: new[] { "ready" });
```

Now that we have tagged our health checks, we can proceed to create our specific check endpoints and associate them with the tags:

```
app.MapHealthChecks("/healthcheck/ready", new
    HealthCheckOptions
    {
        Predicate = healthCheck =>
            healthCheck.Tags.Contains("ready"),
        // code omitted for brevity
    });

app.MapHealthChecks("/healthcheck/live", new
    HealthCheckOptions
    {
        Predicate = healthCheck => false;
        // code omitted for brevity
    });
```

With this new code, the `/healthcheck/ready` endpoint will filter only health checks that are tagged as `ready`. In the `/health/live` endpoint, we set the predicate value to `false` to ignore all tags and conduct all health checks.

While we will not be exploring Kubernetes or other orchestrators in detail, we want to look at how orchestrators interact with our health check endpoints.

Configuring health probes in orchestrators

Monitoring is not unique to orchestrators, as we have already established. There are services that offer monitoring services for our applications and allow us to configure probes into our applications. These services generally allow us to add alerts and configure response time thresholds. These alerts can come in handy in helping us to respond to failures or situations of concern based on our configured thresholds.

In a microservices application, we need a way of monitoring many services as efficiently as possible. The fewer unique configurations we need to do, the better. We have several deployment models that can be used, and most predominantly, containers managed by orchestrators. Microsoft Azure has several web application deployment models, including **Web App for Containers (WAC)**, **Azure Container Instances (ACI)**, and **Azure Kubernetes Service (AKS)**.

WAC is a part of App Service, so the health check works the same way as it would for an Azure web app. It allows you to specify a health check endpoint that will return a response within the 2xx and 3xx HTTP response range. It should also return this health check response within a minute for the service to be considered healthy.

The next option is the ACI, where health checks are called health probes. These probes are configured with a check period, which determines the frequency with which checks are made. When the health check is completed successfully, then the container is considered healthy, and if not, then it is unhealthy or just unavailable. With the ACI, we can configure both liveness and readiness health checks. Our probes can either execute a command on the container or perform an HTTP GET request. When we perform a liveness probe, we verify that our container is healthy, and if not, the ACI might proceed to shut down the container and spin up a new instance. The readiness probe is designed to confirm whether a container is available for request processing, which, as we discussed, is more important during the application startup process.

In the Azure Kubernetes Service of AKS, we have a very similar approach to health checks and probes, as we saw in the ACI. Out of the box, Kubernetes supports both liveness and readiness probes; as seen before, the major difference is that Kubernetes suggests that you have a separate probe for checking the application's health at startup, separate from the readiness probe that is continuous during the application runtime. We can also implement HTTP GET request probes as well as TCP probes to check on our containers.

Kubernetes is configured using a markup language called **YAML Ain't Markup Language (YAML)**, which is a human-friendly scripting language. Kubernetes administrators use YAML to define configurations called a *manifest*. This manifest is then used to infer Kubernetes objects. A Kubernetes deployment specifies the configuration for a *deployment object*, which then creates *Pods*. Each governs the running of specific containers as outlined in the `spec.template` field of the YAML configuration.

The following is an example of a YAML configuration that creates a deployment object that performs startup, liveness, and readiness health checks on a container:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness-api
  name: liveness-http

spec:
  ports:
    - name: api-port
      containerPort: 8080
      hostPort: 8080
  containers:

    - name: liveness-api
      image: registry.k8s.io/liveness
      args:
        - /server

      livenessProbe:
        httpGet:
          path: /healthcheck/live
          port: api-port
          initialDelaySeconds: 3
          failureThreshold: 1
          periodSeconds: 3
      startupProbe:
        httpGet:
          path: /healthcheck/ready
          port: api-port
          failureThreshold: 30
          periodSeconds: 10
      readinessProbe:
        httpGet:
```

```
path: /healthcheck/ready
port: api-port
failureThreshold: 30
periodSeconds: 10
```

The sections of the YAML file that outline the health checks are `livenessProbe`, `startupProbe`, and `readinessProbe`. The probe definition with the major difference is the readiness probe, which executes a command as opposed to making a call to an endpoint.

With this, we have gained some fundamental knowledge of health checks, how they work, and why we need them.

Summary

Health checks are simple yet powerful constructs that aid us in ensuring that our applications are running at maximum efficiency. We see where it is important for us to not only monitor and report on the uptime of the service but also the dependencies, such as the database and other services that may be needed for the application to operate properly.

Using ASP.NET Core, we have access to a built-in health check mechanism that can be customized and extended to implement specific checks and associate them with different endpoints. This is especially useful when we need to separate the types of tests that are conducted relative to the endpoint being called.

We have also explored how orchestrators can be configured to poll our health check endpoints. Orchestrators make monitoring and responding to failures easier since they will handle the routing of traffic to healthy instances and restart instances as needed.

Health checks help us not only monitor the target web service, but we can also configure health checks to report on downstream services as well. This will come in handy, especially when we implement dependencies between our microservices through patterns like the API gateway pattern. We will investigate implementing this pattern in the next chapter.

Implementing the API and BFF Gateway Patterns

When building an application using the microservices architectural approach, we have come to realize that we will need to keep track of several API endpoints. We have effectively gone from one endpoint, which would have been made available through a monolith, to a series of endpoints. Some of these endpoints will be called by other APIs and some will integrate directly into the client applications that interact with the microservices.

This becomes a challenge because we end up conflating the client application with custom logic to cater to integrating with the various services and possibly orchestrating inter-service communications. We want to keep the client application code as simple and extensible as possible, but integrating with each service does not support that notion.

This is where we will consider implementing the API gateway pattern, which introduces a central point of contact between the client and the services. Our API gateway will keep a record of all the endpoints and expose a single API address where the endpoints will map to the various endpoints of the other microservices.

In this chapter, we will look at various scenarios that will make an API gateway a good selection for our application and methods of implementation.

After reading this chapter, we will be able to do the following:

- Understand API gateways and why they are important
- Implement an API gateway using industry-leading technology and methods
- Properly implement the **backend for frontend (BFF)** pattern

Technical requirements

The code references used in this chapter can be found in this project's repository, which is hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch11>.

What is the API gateway pattern?

To understand the API gateway pattern and why we need it, we need to review the fundamentals of the service-oriented architecture and how we architect such solutions.

The service-oriented architecture includes three main layers to an application:

- **Client:** Also referred to as the frontend. This client app is what the user sees and is designed to consume its data from an API. Its functionality is generally limited to functions that the API makes available, and a frontend developer can leverage several techniques to expose functionality to the end user.
- **Server:** Also referred to as the backend. This section of the architecture houses the API and the business logic. The client app is only as intelligent as the backend. The backend can be made up of one or more services, as would be the case with microservices.
- **Database:** The database is the anchor of this entire application since it stores all the data being used by the API backend and is displayed on the frontend.

This application layout is popular in monolithic applications, where all the functionality that is needed on the frontend can be found in one API. This is an effective development method that has been at the helm of many successful and powerful applications. We have, however, explored the downsides to a monolithic approach, where the API might become bloated and difficult to maintain in the long run. The main advantage that we would seem to forsake in the pursuit of a microservices approach would be where we have a single point of entry for the client application, as opposed to several services each with requirements.

While the microservices architecture leads us down the path of having an application that is implemented with the service-oriented architecture, we will need to account for the fact that our client will need to keep track of several backends or APIs and be intelligent enough to orchestrate calls for each user request. This is a lot of responsibility for the portion of the application that should be the least intelligent, based on the description provided.

Figure 11.1 shows the client and microservices:

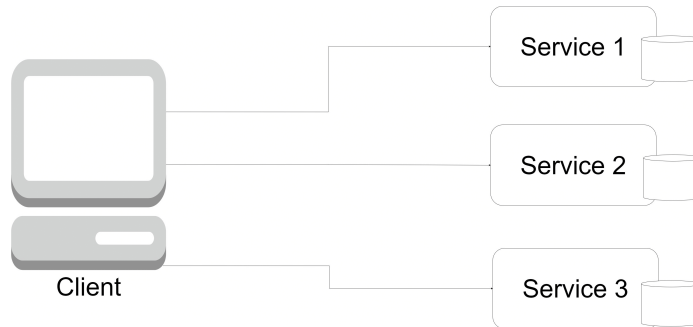


Figure 11.1 – The client app needs to be aware of all the endpoints of all the microservices and retain the knowledge of how each one works

This is where we introduce an API gateway. This gateway will sit between our services and the client app and simplify the communication between the two. For the client, it will expose a singular base URL, which the client will gladly interact with and see as one API service; to the microservices, it will act as a conduit, where it will forward requests coming in from the client to the appropriate microservice.

Let us review the advantages of introducing an API gateway.

Advantages of an API gateway

When a request comes in from a client, it is received by the gateway, which interprets the request, transforms the data if necessary, and then forwards it to the appropriate microservice. In fact, for situations where multiple microservices may need to be called, we can implement orchestration and result aggregation and return more accurate representations of the data to the client as needed.

Our API gateway also allows us to centralize the following tasks for our microservices:

- **Centralized Logging:** From the API gateway, we can centrally log all the traffic to our various endpoints and keep track of the success and error responses from the downstream services. This is advantageous because it spares us the need to implement logging in each service and potentially have a very chatty log. Using an API gateway allows us to centralize our implementation and prioritize what gets written to the log and can help us to better catalog the outcomes of synchronous operations. We can also use the gateway to track and log statistics and response times of calls to the downstream services.
- **Caching:** Caching acts as a temporary data store that comes in handy when the main data source might be offline or when we need to reduce the number of times the services are called. We can use a caching layer in the gateway to stabilize our application and potentially increase the application's performance. With proper coordination and customization, we can use this caching for high-speed read operations on endpoints that have high volumes of traffic and even use it to handle partial failure, where we use the cached data for a response when a service is unavailable.

- **Security:** Securing microservices can be a tedious and technical task. Each service might have unique security requirements and may lead to development overhead when coordinating security measures and implementations. Using an API gateway, we can centralize security measures at the gateway level. This can remove the burden from the microservice to authenticate and authorize access to resources since the gateway will manage most of those requirements. We can also implement IP whitelisting at this level and limit access to an approved list of IP addresses.
- **Service Monitoring:** We can configure our API gateway to conduct health probes on the downstream services. As previously discussed, health checks or probes help us to ascertain the status of our services. Since the gateway will need to forward requests, it is important to be able to determine the health of a downstream service before attempting an operation. Since the gateway can determine the health of a service, it can be configured to gracefully handle failures and partial failures.
- **Service Discovery:** Our gateway needs to know the addresses of all services and how to transform and forward requests as needed. For this, the gateway needs a register of all the downstream services. The gateway will simply act as a wrapper around the services endpoints and expose a singular address to the client application.
- **Rate Limiting:** Sometimes, we want to limit the number of requests that can be sent in quick succession, from the same source, on the suspicion that such activity might be a **distributed denial-of-service (DDoS)** attack on a service endpoint. Using the API gateway, we can implement generic rules that govern how often endpoints can be accessed.

Once again, the most important aspect of the gateway's implementation is that it takes much of the responsibility away from our client, making scaling and diversifying client code much easier.

Figure 11.2 shows the client, a gateway, and microservices:

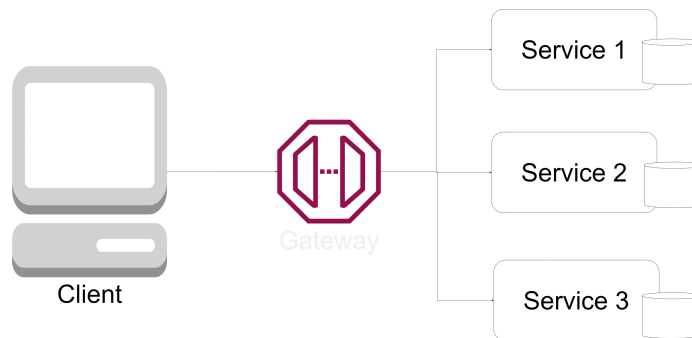


Figure 11.2 – With the gateway introduced, the client app now has one endpoint and doesn't need to know about the underlying services

Now that we have seen where the API gateway helps us to centralize access to several API endpoints and make it easier for the client application to integrate API operations, let us review some of the disadvantages of using this pattern.

Disadvantages of an API gateway

While the advantages are clear and irrefutable, we must also be aware of the downsides of introducing an API gateway. An API gateway may come in the form of another microservice and ironically so. The remedy for dealing with too many services is to build one to rule them all. This then introduces a single point of failure since when this is offline, our client app will have no way to send requests to the various services. Now, additional maintenance is required as our gateway service needs to morph alongside each service it interacts with to ensure the requests and responses are accurately interpreted. We also run the risk of increasing the roundtrip time for requests, since this new layer will need to be performant enough to receive the original request, forward it, and then retrieve and forward the response from the microservice.

While we have obvious advantages that we can reference, we need to ensure that we know, accept, and mitigate the risks involved with implementing a gateway service for our microservices application.

As we have seen, there are several cross-cutting and generic concerns that all APIs share. Implementing these generic requirements in each service can lead to bloat and attempting to build a singular service to implement them can lead to a monolithic application being created. It is easier to use a third-party application that is fortified with the main features that we require of an API gateway.

Now, let us review the ways an API gateway could be implemented.

Implementing the API gateway pattern

Certain guidelines should be followed when implementing an API gateway. Given its description, we might be inclined to develop a new microservice, label it the gateway, and develop and maintain the API integrations ourselves.

Surely, this is a viable approach, and it does give you full control over the implementation, rules, and features that you deem necessary for your application and downstream services. We can also implement specific business logic to govern certain operations by orchestrating requests and responses to the downstream services and aggregating and transforming data accordingly. However, this can lead to having a *thick API gateway*. We will discuss this further in the next section.

Thick API gateways

The expression *thick API gateway* is coined when we realize we are placing too much business operation logic into our API gateway. Our gateway should act more as an abstraction layer between the client and the microservices, not the main hub for business logic. We should avoid placing business logic in the gateway, which will increase the complexity of the implementation and increase the maintenance effort required for the gateway.

We can also call this an *overambitious gateway* and generally should try to avoid making the API gateway a central point for how our application behaves. We also risk implementing a monolith and ending up at square one with our microservices application. At the same time, we should not avoid such a gateway implementation entirely, since there are additional patterns that can be leveraged by having a gateway with some business logic.

Earlier in this book, we reviewed the *Saga pattern* and, more specifically, the *orchestration pattern*. Recall that the orchestration pattern hinges on the presence of a central service that has oversight of the downstream services, monitors the service responses, and decides to continue or terminate the saga accordingly. In this situation, a *thick API gateway* would be an asset in implementing this kind of behavior.

Ultimately, we all have different needs in our applications, and these are, once again, guidelines that we should abide by in doing our implementations. We should always make the best decision for our application based on our needs.

In a situation where all these factors might not be applicable and we need to minimize the amount of business logic that the gateway implements, we may look to existing tools and services that can help us to accomplish these with much less maintenance and development effort. At this point, we can begin thinking about *Amazon API Gateway*, *Microsoft Azure API Management*, and open source solutions such as *Ocelot*.

In the next section, we will review implementing API gateway functionality using *Microsoft Azure API Management*.

Implementing an API gateway using Azure API Management

Microsoft Azure API Management is a cloud-based solution that can be found in the Microsoft Azure suite of development tools. It is designed to abstract, protect, accelerate, and observe backend APIs. While doing this, it securely exposes APIs through service discovery, to internal and external clients, inside and outside of the Azure ecosystem.

It serves several purposes, including the following:

- **API Gateway:** Allows controlled access to backend services and allows us to enforce throttling and access control policies. The gateway acts as a façade to the backend services, allowing API providers to reduce the attrition involved in making changes to the ever-evolving suite of

services in the backend. The gateway provides consistent and powerful configuration options for security, throttling, monitoring, and even caching.

- While this is a cloud-based service, the API gateway can also be deployed in a local environment for customers who wish to self-host their APIs for performance and compliance reasons. This *self-hosted gateway* is packaged as a Docker container and is commonly deployed to Kubernetes.
- **Developer Portal:** An automatically generated and fully customizable website. Third-party developers can use the developer portal to review API documentation and learn how to integrate it into their applications.
- **Management Plane:** This section of Azure API Management allows us to provision and configure the service's settings. We can define API schemas from several sources and configure support for different protocols and standards such as *OpenAPI specifications*, *WebSockets*, or *GraphQL*.

Now, let us explore some of the steps required to set up our first Azure API Management service. For these exercises, you will need an *Azure subscription* and if you don't already have one, you may create a free *Microsoft Azure account* before you begin.

Our first action is to sign in to the Azure portal. You can then use the search feature and type in *API Management services* and select the matching option in the search results. The resulting page will list all the instances of the *API Management services* that you currently have. For this exercise, you may proceed by clicking **Create**.

Figure 11.3 shows the Azure API Management services search results:

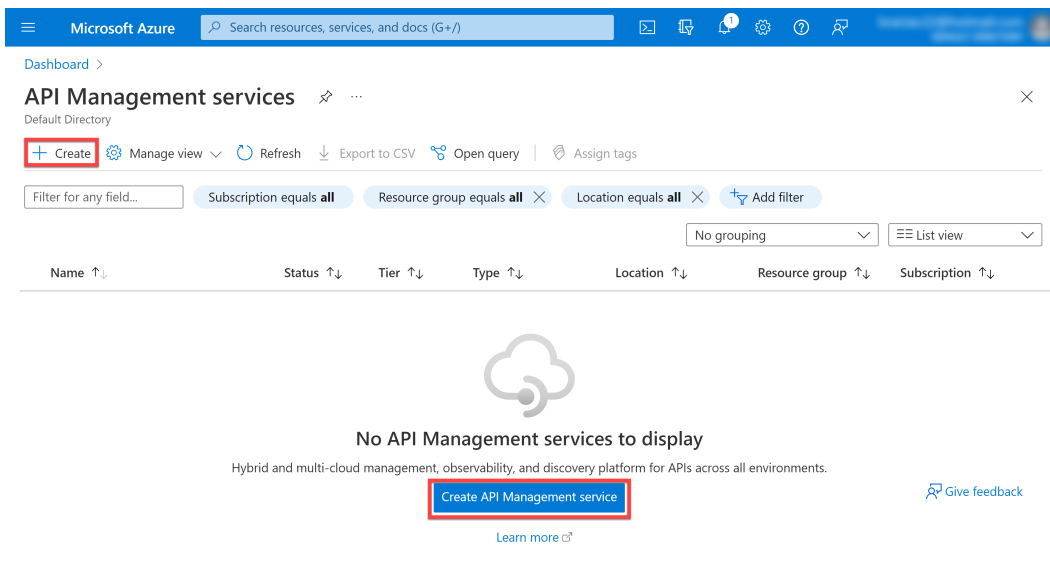


Figure 11.3 – Proceed to create a new API Management service for this exercise

On the next screen, we can proceed to fill in the details of our service and select the following options:

- **Subscription:** The subscription this new service will be provisioned under.
- **Resource group:** The logical group of resources associated with the service being provisioned. A new one can be created for this exercise.
- **Region:** The best geographical representation of where the bulk of the users of the services will be located.
- **Resource name:** A unique name for the instance that you will be provisioning. You will need to modify the name displayed in *Figure 11.4* to proceed.
- **Organization name:** The name of your organization. This will be the name associated with ownership of the API service.
- **Administrator email address:** The email address to be used for all communication and notifications from API Management.
- **Pricing tier:** This determines the level of service uptime that we prefer. For this instance, we will use the *Developer* tier, which isn't for production use.

Figure 11.4 shows the various Azure API Management options:

The screenshot shows the 'Install API Management gateway' page in the Azure portal. The 'Basics' tab is selected, and the 'Project details' section is visible. Below it, the 'Instance details' section contains several form fields. The 'Subscription' field is set to 'Pay-As-You-Go Dev/Test'. The 'Resource group' field is set to '(New) api-mng-rg' with a 'Create new' link below it. The 'Region' field is set to '(US) East US 2'. The 'Resource name' field is set to 'healthcare-api-mng' with a green checkmark. The 'Organization name' field is set to 'Healthcare Org' with a green checkmark. The 'Administrator email' field is set to 'admin@healthcare.com' with a green checkmark. The 'Pricing tier' field is set to 'Developer (no SLA)'. At the bottom, there are three buttons: 'Review + create' (highlighted with a red box), '< Previous', and 'Next: Monitoring >'.

Install API Management gateway

API Management service

Basics Monitoring Scale Managed identity Virtual network Protocol settings Tags Review + install

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ Pay-As-You-Go Dev/Test

Resource group * ⓘ (New) api-mng-rg
[Create new](#)

Instance details

Region * ⓘ (US) East US 2

Resource name * healthcare-api-mng ✓

Organization name * ⓘ Healthcare Org ✓

Administrator email * ⓘ admin@healthcare.com ✓

Pricing tier ⓘ Developer (no SLA)

[Review + create](#) < Previous Next: Monitoring >

Figure 11.4 – Minimum values needed to create the API Management service

After creating the API Management service, we can begin importing our microservices into the management portal. Now, our API Management service will act as a façade in front of our services, allowing us to control access and transform data as needed. We can import APIs from any source if their API is discoverable across the internet or network.

The API Management service will handle all communications between a client and the target service that maps to the requested endpoint, regardless of the technology used to implement the API.

Figure 11.5 shows the APIs added to the Azure API Management service:

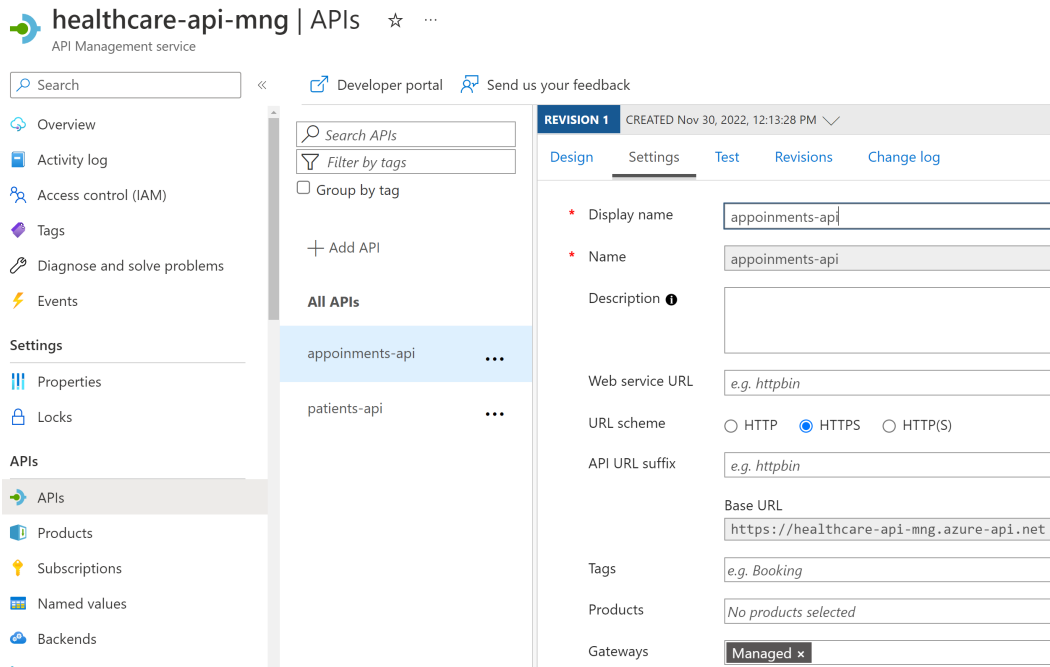


Figure 11.5 – The API Management service allows you to add APIs and map custom routes, that when called, will reroute the request to the mapped resource

In Figure 11.5, we can see where we have mapped our appointments and customer APIs to the API Management service and have defined a base URL based on the primary endpoint now available through the service.

In Figure 11.6, we can see where we can manage the request types that are allowed, as well as define our policies and transformations for each request type.

Figure 11.6 also shows the various request processing options in the Azure API Management service:

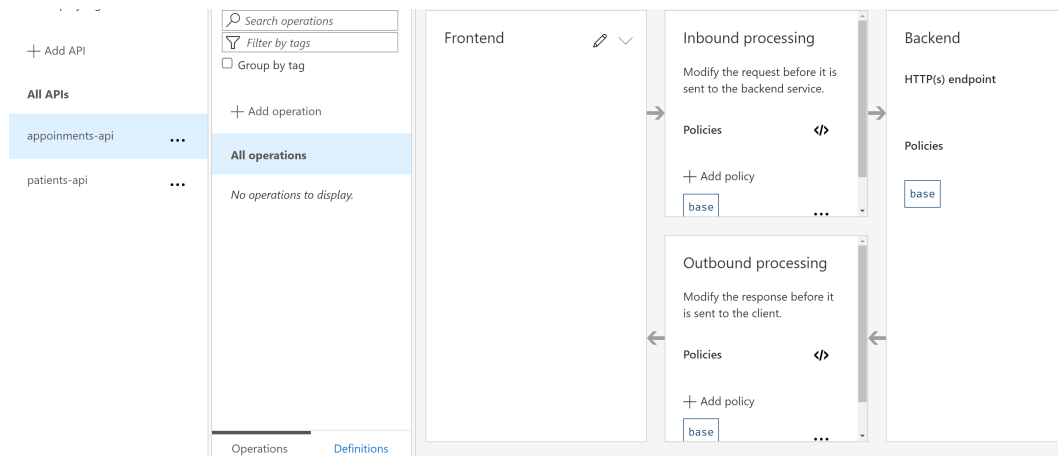


Figure 11.6 – The API Management service allows you to easily manage the request types that are allowed for each API and define transformation policies for requests and responses

Using Azure API Management, we gain many standard API gateway features out of the box and the further benefit of availability and service uptime guarantees when we have production-grade pricing tiers. If we choose not to self-host this application, we can take advantage of its **Software-as-a-Service (SaaS)** model, where we have a greatly reduced responsibility to do any infrastructure work related to getting it up and running.

We may end up in a situation where we need to self-host our gateway and API Management is not an option. In this situation, we can look to provide our own API gateway application. A great candidate for this implementation is *Ocelot*, which is a lightweight API gateway package that can be installed directly into a standard ASP.NET Core project. We will discuss this further in the next section.

Implementing an API gateway using Ocelot

Ocelot is an open source API gateway developed on the .NET Core platform. It is a simple implementation of a gateway that unifies the communication to microservices through abstraction, as we have come to expect from a gateway. The Ocelot API gateway transforms incoming HTTP requests and forwards them to the appropriate microservice address based on preset configurations.

It is a popular and widely used API gateway technology and can easily be installed into a .NET Core application using the NuGet package manager. Its configurations can be outlined in JSON format; here, we define *upstream* and *downstream* routes. *Upstream* routes refer to the service address that is exposed to the client, while *downstream* routes are the real addresses of the mapped microservices. We can also define the allowed protocols for each upstream service route, allowing robust control over the kind of traffic that we are willing to accept on a route.

Let us set up an Ocelot API gateway application together. We will use a simple ASP.NET Web API project template and start by adding the Ocelot package via the NuGet package manager:

Install-Package Ocelot

Now that we have our package installed, we need to begin outlining our routing configurations. We can create a new configuration file and call it `ocelot.json`. In this JSON file, we will outline all *upstream* and *downstream* routes. This configuration will look something like this:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/Patients",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5232
        }
      ],
      "UpstreamPathTemplate": "/Patients",
      "UpstreamHttpMethod": [
        "GET",
        "POST"
      ]
    },
    {
      "DownstreamPathTemplate": "/api/Patients/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
```

```
        "Port": 5232
      }
    ],
    "UpstreamPathTemplate": "/Patients/{id}",
    "UpstreamHttpMethod": [
      "GET",
      "PUT"
    ]
  },
  {
    "DownstreamPathTemplate": "/api/Appointments",
    "DownstreamScheme": "http",
    "DownstreamHostAndPorts": [
      {
        "Host": "localhost",
        "Port": 5274
      }
    ],
    "UpstreamPathTemplate": "/Appointments",
    "UpstreamHttpMethod": [
      "POST",
      "PUT",
      "GET"
    ]
  }
],
"GlobalConfiguration": {
  "BaseUrl": "http://localhost:5245"
}
```

This configuration file is straightforward and once we pick up on the pattern, we can extend it as needed for the rest of our services. The sections are explained here:

- **Routes:** This is the parent section of our JSON configuration, where we begin to define the upstream and downstream configurations.

- **DownstreamPathTemplate:** This section outlines the address at which the microservice can be found.
- **DownstreamScheme:** This outlines the protocols that we will use to communicate to the microservice being defined.
- **DownstreamHostAndPorts:** The host address and port are defined in this section.
- **UpstreamPathTemplate:** We outline the path that we expose to the client apps. By calling this defined route, Ocelot will automatically reroute the request to the service defined in the **DownstreamPathTemplate**. Notice that in the preceding example, we can rename the route if we need to. The **Customers** API endpoint originally found in the downstream API can only be reached via a **Patients** endpoint address.
- **UpstreamHttpMethod:** Here, we define the methods that we will accept as legitimate requests from a client.
- **GlobalConfiguration:** We outline the **BaseUrl** in the configuration, where all request traffic should be sent through.

Now, let us configure our application to use these configurations and use the Ocelot package. We will start by adding the following lines to the `Program.cs` file:

```
builder.Configuration.AddJsonFile("ocelot.json", optional:
    false, reloadOnChange: true);
builder.Services.AddOcelot(builder.Configuration);
```

These lines add the `ocelot.json` file to our global configuration at application startup and then register Ocelot as a service. Then, we need to add the Ocelot middleware, like this:

```
await app.UseOcelot();
```

With these few configurations, we can now use the gateway URL as the API URL in our client apps.

Ocelot is well-documented and extendable. It supports other features, such as the following:

- Built-in cache management
- A rate limiter
- Support for native .NET Core logging integrations
- Support for **JSON Web Token (JWT)** authentication
- Retry and circuit breaker policies (using *Polly*)
- Aggregating
- Pre- and post-downstream request transformations

Now that we have learned how to set up a simple gateway with Ocelot, let us look into extending this functionality. We will begin by adding cache management.

Adding cache management

Caches act as temporary data stores in between requests to a more reliable data store. This means that a cache will temporarily store data based on the last set of data it was given. Good cache management would suggest that we flush our cache based on an interval and refresh it with a newer version of the data.

Caching comes in handy when we need to reduce the number of trips that are made to the main database, reducing latency and read/write costs that come with database calls. Ocelot has some support for caching, which is good for solving small caching concerns natively in the gateway application.

This can be added with a fair amount of ease. We will begin by using NuGet Package Manager to execute the following command:

```
Install-Package Ocelot.Cache.CacheManager
```

This package gives us the caching extensions that we need to then introduce an extension method in the `Program.cs` file. This extension method looks like this:

```
builder.Services.AddOcelot()
    .AddCacheManager(x =>
    {
        x.WithDictionaryHandle();
    });
```

Finally, we add the following line to our `ocelot.json` configuration file:

```
"FileCacheOptions": {
  "TtlSeconds": 20,
  "Region": "SomeRegionName"
}
```

Now that we have introduced a configuration to govern how caching should occur in our gateway, we must outline that values should be cached for a maximum of 20 seconds. This will add native caching support for the downstream services that have been defined. Once that cache period has expired, requests will be forwarded as expected and then the new response values will be cached once again, for the defined period.

Caching helps to reduce the amount of pressure that we place on a service, but it reasonably only imposes that limit for a short period. If we extend that period, then we run the risk of returning stale data for too long. Another layer of protection that we will want to implement is rate limiting. Let us explore this next.

Adding rate limiting

Rate limiting helps us to defend our application from the effects of DDoS attacks. Essentially, we impose rules on how frequently our service endpoints can be accessed by the same resource. When the request frequency violates our rules, we reject other incoming requests. This helps to prevent probable service performance degradation. Our service will not be attempting to fulfill all requests, especially those that may look like attacks.

Rate limiting works by recording the IP address of the originating request. For all other requests from the same IP address, we evaluate if it is legal and within the set constraints that govern how often a request should come from the same sender. When a rule violation is detected, we send a failure response and do not forward the request in the service.

Ocelot allows us to configure rate limiting for the configured downstream services. This is good because it allows us to globally manage these rules and we do not need to implement these rules in each service.

First, let us modify our code to implement rate limiting for a particular downstream service. We can add the following code to the service's configuration file:

```
{
  "DownstreamPathTemplate": "/api/Patients",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [
    {
      "Host": "localhost",
      "Port": 5232
    }
  ],
  "UpstreamPathTemplate": "/Patients",
  "UpstreamHttpMethod": [
    "GET",
    "POST"
  ],
  "RateLimitOptions": {
    "ClientWhitelist": [],
    "EnableRateLimiting": true,
    "Period": "5s",
    "PeriodTimespan": 1,
    "Limit": 1
  }
}
```

```
    }  
  },
```

We have introduced a new section called `RateLimitingOptions` to the `ocelot.json` file. More specifically, we have added this new configuration to our patient's downstream service configuration. This will now impose the following restrictions on how this downstream service can be accessed:

- **ClientWhiteList:** List of allowed clients that are not subjected to the rate limiting restrictions.
- **EnableRateLimiting:** A flag that indicates whether the rate-limiting restrictions should be enforced or not.
- **Period:** This value specifies the amount of time that we use to determine if a client is making a request that violates the limiting options. We can use the following:
 - s for seconds
 - m for minutes
 - h for hours
 - d for days

The pattern is fairly easy to follow. In our example, we have a 5-second limit on requests.

- **PeriodTimeSpan:** This is like a cooldown period. For this period, subsequent requests from the client that violated the limiting restrictions will be rejected and the clock will restart. Once this period has elapsed, the client can continue making requests.
- **Limit:** The number of requests that a client is allowed to make during the period. Here, we are defining that only one request should come in from the client every 5 seconds.

Then, we can define global values that govern how the gateway will handle rate limiting. We can add a similar `RateLimitingOptions` section to our `GlobalConfiguration` section:

```
"GlobalConfiguration": {  
  "BaseUrl": "http://localhost:5245",  
  "RateLimitOptions": {  
    "DisableRateLimitHeaders": false,  
    "QuotaExceededMessage": "Too many requests!!!",  
    "HttpStatusCode": 429,  
    "ClientIdHeader": "ClientId"  
  }  
}
```

Now, we have some new options, which are as follows:

- **DisableRateLimitHeaders:** A flag that determines whether we disable or enable rate-limiting headers. These header values are generally as follows:
 - **X-Rate-Limit:** Maximum number of requests available within the timespan
 - **Retry-After:** Indicates how long the client should wait before making a follow-up request
- **QuotaExceededMessage:** Allows us to define a custom message to send to the client that has violated the limiting rules.
- **HttpStatusCode:** This outlines the response code to be sent when the rules are violated. **429TooManyRequests** is the standard response for this situation.
- **ClientIdHeader:** Specifies the header that should be used to identify the client making the request.

With these minor changes, we have enforced rate limiting on all requests coming into the `/patients` endpoint. We will respond with a **429TooManyRequests** HTTP response if two or more requests come in within 5 seconds, from the same client address.

Another consideration we might have when using Ocelot is to aggregate our responses. This allows us to string multiple calls along and reduce the client's need to orchestrate these calls. We'll learn how to add this next.

Adding response aggregation

Response aggregation is a method used for merging responses from multiple downstream services and sending one response accordingly. Essentially, an API gateway can achieve this by accepting a single request from a client and then making distributed parallel requests to several downstream services. Once all the responses are in from the downstream services, it will merge the data into a single object and return it to the client.

Several benefits come with this approach. The most prevalent one is that we can reduce the number of requests that the client needs to make to get data from several services. The API gateway will handle that orchestration automatically. The client also only needs to know one schema. So, several potentially complex requests can be merged into a single request body, which will reduce the number of schemas that the client needs to track. This approach will also speed up the response times involved with calling several services. Since the calls will be made in parallel, we do not have to wait the entire period that would be required when making service calls one after the other.

Ocelot allows us to configure aggregate calls with a fair amount of ease. We will decorate our downstream service configurations with keys that act as a point of reference for our aggregate configuration. If we want to aggregate a call that should return a patient and all the appointments that they have made, we would need to make the following modifications:

```
{
  "DownstreamPathTemplate": "/api/Patients/{id}",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [
    {
      "Host": "localhost",
      "Port": 5232
    }
  ],
  "UpstreamPathTemplate": "/Patients/{id}",
  "UpstreamHttpMethod": [
    "GET",
    "PUT"
  ],
  "Key": "get-patient"
}
```

We start by adding a new key to the `api/patients/{id}` downstream service configuration. This key acts as an alias, which we will use later. We will also add a new downstream service configuration for appointments and a new endpoint. The configuration looks like this:

```
{
  "DownstreamPathTemplate":
    "/api/user/Appointments/{id}",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [
    {
      "Host": "localhost",
      "Port": 5274
    }
  ],
  "UpstreamPathTemplate": "/Appointments/user/{id}",
  "UpstreamHttpMethod": [
```

```

    "GET"
  ],
  "Key": "get-patient-appointments"
}

```

The matching endpoint that will be implemented in the appointments services looks like this:

```

// GET: api/Appointments/user/{id}
[HttpGet("user/{id}")]
public async Task<ActionResult<List<Appointment>>>
    GetAppointmentsByUser(Guid id)
{
    var appointments = await _context.Appointments
        .Where(q => q.PatientId == id)
        .ToListAsync();

    return appointments;
}

```

Now that we have configured the new endpoints and modified the downstream service configurations, we need to add a new configuration for our aggregate orchestration:

```

"Aggregates": [
  {
    "RouteKeys": [
      "get-patient",
      "get-patient-appointments"
    ],
    "UpstreamPathTemplate": "/get-patient-details/{id}"
  }
],

```

Now, we can use the endpoint as defined by the aggregate configuration and execute a single call that will return a patient's record alongside all the appointments that they have made. This information comes from multiple services almost simultaneously. Our client no longer needs to make multiple calls to get this information.

This simple and powerful technique helps us to better orchestrate API calls and present exactly the information that a client app needs. It promotes a more behavior-driven workflow when retrieving data and reduces the development overhead that each client application will need.

Now that we have seen how we can implement API gateways using either our API project or Azure API Management, we have overcome a major hurdle in our microservices application. We no longer need to build client apps that need to keep track of all the addresses of our microservices.

This now raises another cause for concern. Unfortunately, different devices might have different requirements for how they interact with our services. Mobile clients might need special security and caching considerations that web applications do not. This adds more complication to how we keep track of configurations in the central gateway, relative to the devices hosting the client apps.

These considerations lead us down the path of implementing a gateway per type of service client. This method of implementation is called the *Backend for Frontend pattern*, which we will discuss next.

Backend for Frontend pattern

While API gateways solve several problems, it is not a one size fits all solution. We still end up contending with the possibility of catering to multiple device types and, by extension, client applications. For example, we may need to use additional compression and caching rules with data being consumed by a mobile client, whereas a website might not need many special considerations. The more devices become capable of interacting with APIs, the more we need to ensure that we can support integrations.

Figure 11.7 shows multiple clients with one gateway:

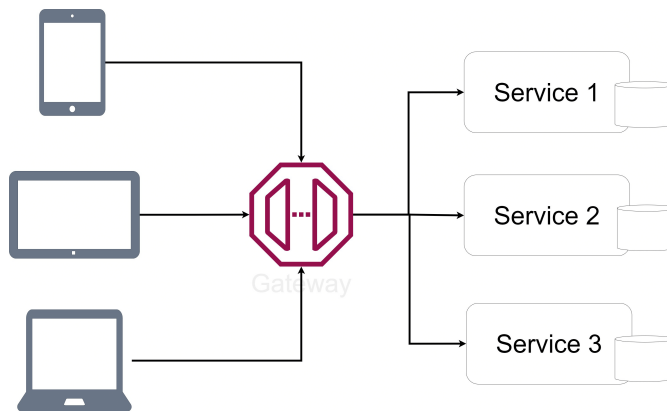


Figure 11.7 – All client devices access the same gateway, leading to inefficient behavior for some devices

All these considerations make a good case for the **Backend for Frontend (BFF)** pattern. This pattern allows us to supply a service-per-device API approach. The BFF pattern allows us to acutely define our API functionality based on the experience that we hope for a user to have on a particular user interface. This makes it easier for us to develop and maintain and adjust our API based on the client's requirements and simplifies the process of delivering functionality across multiple clients.

Figure 11.8 shows a BFF setup:

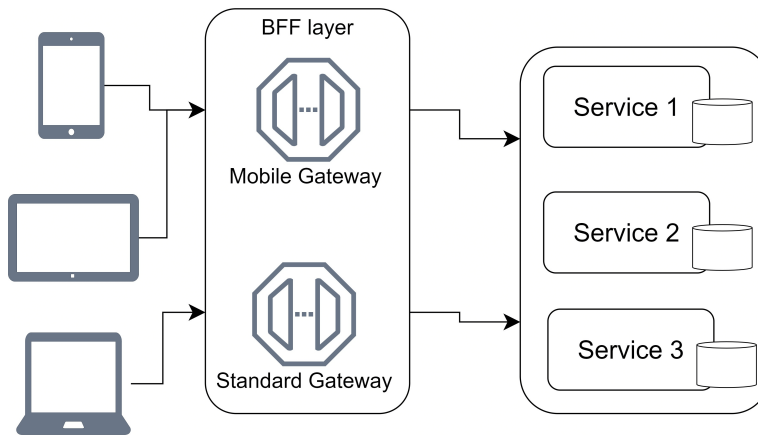


Figure 11.8 – Each client app has an endpoint to a gateway that is specially configured to optimize API traffic for the target device type

Now, we can optimize each gateway instance to handle traffic for specific devices in the most efficient way possible. For instance, our mobile applications might require additional caching or compression settings and we may need to rewrite request headers. We might even define additional header information to be provided from our mobile devices as we may need to track the device type and location. In a nutshell, we need to ensure that we are catering to each possible device as much as possible.

Azure API Management has features that allow us to interrogate the incoming request and redirect or modify the request before forwarding it or modifying the response before it is sent to the requesting client. By defining these policies, we can implement a BFF-like mechanism where policies are defined to look for the type of device or, generally, the source of the request and modify it as optimally as possible for forwarding or returning.

Ocelot might require a bit more potentially confusing logic to support policies of this nature. The more recommended way to implement this pattern using Ocelot is to use multiple implementations of Ocelot. In this implementation style, we would create multiple Ocelot projects, each with its specific purpose, such as mobile, web, and public gateways, and add each configuration for the allowed up and downstream services. We would also be able to specify the rate-limiting and caching options per implementation.

Let us review how this pattern can be implemented using Ocelot.

BFF pattern using Ocelot

We have already seen that we can configure Ocelot to be our API gateway. A simple enough extension to what we have done is to create additional projects and configure them similarly. We can retain

the gateway that we have already and use it exclusively for third-party application access. With the up and downstream services we have defined, we can restrict third parties to only be able to access those endpoints.

We can then create a new Ocelot project and use it specifically for our web client. Let us say that we do not want rate limiting on the web client and can decrease the cache time to 10 seconds instead of 20. Given that this is our web application, we can lift most of these restrictions and allow for less strict interactions.

This configuration file will simply look like this:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/Patients",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5232
        }
      ],
      "UpstreamPathTemplate": "/web/Patients",
      "UpstreamHttpMethod": [
        "GET",
        "POST"
      ]
    },
    // omitted for brevity  ],
    "FileCacheOptions": {
      "TtlSeconds": 10,
      "Region": "SomeRegionName"
    },
    "GlobalConfiguration": {
      "BaseUrl": "http://localhost:5245"
    }
  ]
}
```

This looks similar to what we have already done with the previous gateway, but note that now, we have the unique opportunity to define custom paths that match with the web entry point that we are implementing while adding/removing configurations as we deem necessary for the web client. Also, notice that it will broadcast from a separate address, which will prevent any reference overlaps between the clients.

We may also want to implement a mobile client that has fewer restrictions similar to what we have outlined in the web gateway, but we may also want to customize the aggregation operation. So, for our mobile client gateway, we can add the following aggregator definition to the Ocelot configuration:

```
"Aggregates": [
  {
    "RouteKeys": [
      "get-patient",
      "get-patient-appointments"
    ],
    "UpstreamPathTemplate": "/get-patient-details/{id}",
    "Aggregator": "PatientAppointmentAggregator"
  }
],
```

In the `Program.cs` file, we add the following line to register the aggregator:

```
builder.Services.AddOcelot().AddSingletonDefinedAggregator<
    PatientAppointmentAggregator>()
```

Now, we need to define a class called `PatientAppointmentAggregator`, which will implement our custom aggregation logic. This custom aggregator will intercept the responses from the downstream server and allow us to interrogate and modify what is returned:

```
public class PatientAppointmentAggregator :
    IDefinedAggregator
{
    public async Task<DownstreamResponse>
        Aggregate(List<HttpContext> responses)
    {
        var patient = await responses[0].Items.Downstream
            Response().Content.ReadAsStringAsync();
        var appointments = await responses[1]
            .Items.DownstreamResponse()
```

```
        .Content.ReadAsStringAsync();

var contentBuilder = new StringBuilder();
contentBuilder.Append(patient);
contentBuilder.Append(appointments);

var response = new StringContent
    (contentBuilder.ToString())
{
    Headers = { ContentType = new
        MediaTypeHeaderValue("application/json") }
};

return new DownstreamResponse(response,
    HttpStatusCode.OK, new List<KeyValuePair<string,
        IEnumerable<string>>>(), "OK");
}
}
```

This aggregator code receives a list of responses, where each entry represents the response from the downstream services in the order they were defined in the configuration. We then extract the response as a string and append it in one string value. We also add a `ContentType` header to the ultimate response, which is sent with a 200OK HTTP response. This is a simple example, but it shows how easy it is for us to customize the default aggregation behavior and, by extension, for a specific BFF gateway.

The BFF pattern allows us to further diversify our development teams and their efforts in maintaining the various microservices. Teams can now manage their gateways and implement gateway methods and features that are unique to the devices they are catering to.

Now that we understand API gateways, the BFF pattern, and how we can implement either one of these using industry-standard software, let us review what we have learned in this chapter.

Summary

This chapter has reviewed the need for an API gateway. When building a monolith, we have a single point of entry to our application's supporting API and this single point of entry can be used for any type of client.

The downside to this is that we might end up with an API that becomes increasingly difficult to improve on and scale as the demands change. We also need to consider the fact that different devices have different needs from the API in terms of caching, compression, and authentication to name a few.

We then attempt to diversify our application's capabilities into multiple services or microservices and then implement only what is needed per service. This approach simplifies each service's code base while complicating the code base of the client applications. Where there was one service endpoint, we now have several to keep track of.

API gateways will sit on top of all the microservices and expose a single point of entry and allow us to implement several instances, which can cater to the direct needs of the client applications that will use them. This adjustment is called BFF, and it allows us to curate backend services specifically for the client applications that need them.

The major downside here is that we have reintroduced a single point of failure by providing the gateway layer, which can introduce potential performance issues. The goal, however, is to reduce the need for our client apps to have intimate knowledge of the complex web of services that they need to interact with, and this layer of abstraction also helps us to maintain our services with less effect on the client applications.

We also learned that when attempting to add the BFF pattern, we introduce the need for more services and more code to maintain. Ideally, we would like to have a single implementation that can be provisioned multiple times, all with their specific configurations. This is where technology such as Docker will help, but we will review that later in this book.

Now that we have seen the pros and cons of the API gateway pattern, we need to explore security for our APIs. In the next chapter, we will explore API security using bearer tokens.

Securing Microservices with Bearer Tokens

Security is one of the most important and tedious aspects of any application. We need to ensure that our application is built using secure code and always pursue the most effective ways to reduce intrusions and loopholes in our systems. Despite this, however, security also comes at the cost of usability, and we should always seek to find a balance between the two.

Basic application security begins with a login system. We should be able to allow a user to register themselves in a system and store some identifying information accordingly. When the user returns and wishes to access certain parts of the application, we will query the database and verify the identity of the user through their identifying information and decide to grant or restrict access accordingly.

In modern applications, we find it increasingly difficult to maintain a data store as an authority on all our users, while accounting for all the possible channels through which they may access our application. We have been exploring using microservices architecture, which takes our security considerations to a new level, where we now have multiple parts of an application that we need to secure for different users who are accessing from several devices.

In this chapter, we will explore the major considerations to be made in securing our microservices application and the best configurations and technologies to use.

After reading this chapter, we will have done the following:

- Understand bearer token security
- Learn how to implement bearer token security in an ASP.NET Core API
- Learn how to use an identity provider to secure our microservices

Technical requirements

The code references used in this chapter can be found in the project repository that is hosted on GitHub here: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch12>.

Bearer tokens for securing communications

Bearer tokens are a fairly recent solution to a number of security, authentication, and authorization challenges that we have faced when developing modern applications. We have gone from working with standard desktop and web applications to catering to various internet-capable devices that have similar security needs. Before we start exploring what these modern security needs are, let us review some of the challenges that we have faced with web applications over the years.

When securing web applications, we face several challenges:

- We need a way to collect user information.
- We need a way to store user information.
- We need a way to validate user information. This is called *authentication*.
- We need a way to track the user's authenticated state in between requests.
- We need a way to track what the user is allowed to do in our system. This is called *authorization*.
- We need to cater to various channels or device types through which a user might access the web application.

In a typical web application, most of these factors can be implemented through form authentication, where we ask for uniquely identifying information and check our database for a match.

When a match is found, we instantiate a temporary storage mechanism that will identify the user as authenticated in our system. This temporary storage construct can come in the form of the following:

- **Sessions:** A way to store information in a variable that can be used across a website. Unlike typical variables that lose their value with each request, a session retains its value for a certain period until it either expires or is destroyed. Session variables are typically stored on the server, and one or many session variables are created each time a user authenticates successfully. Session variables can store information such as a username, role, and so on. With too many users logging in simultaneously, using session variables can lead to memory issues on a less powerful server.
- **Cookies:** An alternative to sessions, where a small file is created and stored on the user's device. It serves a similar purpose for storing information between requests, as well as tracking a user's authenticated state. Each time a request is sent from the user's device, this cookie is sent, and the server web application uses this information to be informed of whether actions can be taken

and if so, which ones. Cookies are sometimes preferred to sessions given that they reduce the load on the server and place more responsibility on the user's device.

Both options work fantastically when we are sure that we will be dealing with a web application that maintains a *state*. A state means that we retain user information in between requests and remember who is logged in and their basic information for the period that they are using the website – but what happens when you need to authenticate against APIs? An API, by nature, does not maintain a state. It does not attempt to retain the knowledge of the users accessing it since APIs are designed for sporadic access from any channel at any point. For this reason, we implement bearer tokens.

A bearer token is an encoded string that contains information about a user who is attempting to communicate with our API. It helps us facilitate stateless communication and facilitate general user authentication and authorization scenarios.

Understanding bearer tokens

A *bearer token* or **JSON Web Token (JWT)** is a construct that is widely used in *authentication* and *authorization* scenarios for *stateless* APIs. Bearer tokens are based on an open industry standard of authentication that has made it easy for us to share authenticated user information between a server and a client. When an API is accessed, a temporary state is created for the duration of the request-response cycle. This means that when the request is received, we can determine the originating source of the request and can decode additional header information as needed. Once a response is returned, we no longer have a record of the request, where it came from, or who made it.

Bearer tokens are issued after a successful authentication request. We receive a request to our authentication API endpoint and use the information to check our databases, as previously described. Once a user is verified, we compile several data points, such as the following:

- **Subject:** Usually a unique identifier for the user, such as the user ID from the originating database.
- **Issuer:** Usually a name that is associated with the service that has generated the token for issuance.
- **Audience:** Usually a name that is associated with the client application that will be consuming the token.
- **Username:** The user's unique system name, usually used for login.
- **Email address:** The user's email address.
- **Role:** The user's system role that determines what they are authorized to do.
- **Claims:** Various bits of information about the user that can be used to aid in authorization or information display in the client application. This can include the user's name, gender, and even the path to their profile picture.
- **Expiry Date:** Tokens should always have a moderate expiry date relative to their generation. When this expires, the user will need to reauthenticate, so we don't want it to only be valid for a short period, but it should also not last forever.

Ultimately, a login flow between a client application and an API is as follows:

1. A user will use a client application to log in
2. The client application forwards the information collected from the login form to the login API endpoint for verification
3. The API returns an encoded string, or token, that contains the most relevant bits of information about the user
4. The client application stores this encoded string and uses it for subsequent API communications

Based on this kind of flow, the client application will use information from the token to display information about the user on the UI, such as the username or other information that may have been included such as the first name and last name. While there are recommended bits of information that you should include in a token, there is no set standard on what should be included. We do, however, avoid including sensitive information, such as a password.

Bearer tokens are encoded but not encrypted. This means that they are self-contained blocks of information that contain all the information that we have mentioned earlier but are not human-readable at first sight. The encoding compresses the strings, usually as a *base64* representation, and this is the format used for transportation between the client and the server, as well as for storage. Token strings are not meant to be secure since it is easy to decode the string and see the information therein, and once again, that is why we do not include sensitive and incriminating data in the token. This token string comprises three sections. Each section is separated by a full stop (.) and the general format is *aaaa.bbbb.cccc*. Each section represents the following:

- **Header:** The *a* section of the token, which contains information about the type of token and the signing algorithm that was used for the encoding, such as HMAC SHA-256 and RSA.
- **Payload:** The *b* section of the token string, which contains user information in the form of claims. We will discuss claims in a bit more detail later in this chapter.
- **Signature:** The *c* section of the token, which contains a string representation of the encoded header, the encoded payload, and the secret key that was used for the encoding. This signature is used to verify that the token has not been tampered with since its generation.

Most development frameworks include tools and libraries that can decrypt bearer tokens during the runtime of the application. Since bearer tokens are based on an open standard, support for decoding tokens is widely available. This allows us to write generic and consistent code to handle tokens being issued by an API. Each API implementation can include different tokens relative to the exact needs of the application, but there are certain standards that we can always count on.

During development, however, we might want to test a token to see the contents that we can expect to be present in a more human-readable form. For this reason, we turn to third-party tools that decode and show us the contents of a token and allow us to reference different bits of information as needed.

Tools such as jwt.io provide us with the ability to simply paste in a token and view the information in a more human-readable format. As stated, there are three sections in each token string and we can view each of the sections in plaintext using this website or a similar tool. The payload section of the token, when decoded, will yield the information displayed in *Figure 12.1*. It shows a sample bearer token and its contents on www.jwt.io.

The screenshot shows the JWT.io website interface. On the left, under the 'Encoded' tab, a long base64-encoded token is pasted: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4uRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c`. Below the token, a tooltip indicates 'Issued at (seconds since Unix epoch)'. On the right, under the 'Decoded' tab, the token's structure is shown. The header is `{ "alg": "HS256", "typ": "JWT" }`. The payload is `{ "sub": "1234567890", "name": "John Doe", "iat": 1516239022 }`.

Figure 12.1 – We see the encoded string and the plaintext translation of its contents to the right

All the information that is placed into a bearer token represents a key-value pair. Each key-value pair represents a unit of information about the user or the token itself, and the keys are really short names for the previously mentioned claims that are usually present in a token:

- **iss**: Represents the **issuer** value.
- **sub**: Represents the **subject** value.
- **aud**: Represents the **audience** value.
- **nonce**: Represents a unique value that should always change with each token to prevent replay attacks. This value is always new, and this ensures that no two tokens that are issued to the same user are the same. This can sometimes be called a **jti** claim.
- **exp**: Represents the expiration date of the token. The value is in the form of a UNIX epoch, which is a numerical representation of a moment in time.
- **iat**: Represents the date and time of issuance.

Now that we have explored why we need bearer tokens and how they are used, let us review how we can implement token security in our ASP.NET Core API application.

Implementing bearer token security

ASP.NET Core offers native authentication and authorization support through its `Identity Core` library. This library has direct integration with Entity Framework and allows us to create standard user management tables in the target database. We can also further specify the authentication methods that we prefer and define policies that define authorization rules throughout that application.

This robust library has built-in support for the following:

- **User registration:** The user manager library has functions that make user creation and management easy. It has functions that cover most of the common user management operations.
- **Login, session, and cookie management:** The sign-in manager library has functions that can manage user authentication and session management scenarios.
- **Two-factor authentication:** Identity Core allows us to implement multi-factor authentication natively with email or SMS. This can be easily extended.
- **Third-party identity providers:** Social logins are important for any modern web application and Identity Core makes it easy to integrate this feature into your application.

Securing an API using bearer tokens ensures that each API call is required to have a valid token in the header section of the request. An HTTP header allows for additional information to be provided with an HTTP request or response.

In our case of securing an API, we enforce that each request must have an authorization header that contains the bearer token. Our API will assess the incoming request headers, retrieve the token, and validate it against the predefined configurations. If the token doesn't meet the standards or is expired, an HTTP 401 Unauthorized response will be returned. If the token meets the requirements, then the request will be fulfilled. This built-in mechanism makes it easy and maintainable to support wide-scale and robust authentication and authorization rules in our application.

Now that we have an idea of the `Identity Core` library and how it is natively supported in ASP.NET Core applications, we can explore the necessary package and configurations needed to secure an API using bearer tokens.

Securing API with bearer tokens

We can begin by installing the following packages using the NuGet package manager:

```
Install-Package Microsoft.AspNetCore.Authentication  
                  .JwtBearer
```

The first package supports direct integration between Entity Framework and `Identity Core`. The second package contains extended methods that allow us to implement token generation and validation rules in our API configuration.

Next, we need to define constant values that will inform the token generation and validation activities in the API. We can place these constants in `appsettings.json` and they will look as follows:

```
"Jwt": {  
  "Issuer": "HealthCare.Appointments.API",  
  "Audience": " HealthCare.Appointments.Client",  
  "DurationInHours": 8  
  "Key": "ASecretEncodedStringHere-Minimum16Charatcters"  
}
```

We have already discussed what the issuer and audience values help to enforce. We can also state a value for the proposed lifetime of the token that is generated. This value should always be relative to the API's capabilities and operations, as well as your risk tolerance. The longer a token remains valid, the longer we provide a potential attacker with a window into our system. At the same time, if the period is too short, then the client will need to reauthenticate too often. We should always seek to strike a balance.

Our key value here is demonstrative in its value, but we use this signing key as an encryption key when generating the token. The key should always be kept secret, so we may use application secrets or a more secure key store to store this value.

Now that we have the application constants, we can proceed to specify the global authentication settings in our `Program.cs` file:

```
builder.Services.AddAuthentication(options =>  
{  
    options.DefaultAuthenticateScheme =  
        JwtBearerDefaults.AuthenticationScheme;  
    options.DefaultChallengeScheme =  
        JwtBearerDefaults.AuthenticationScheme;  
}).AddJwtBearer(o =>  
{  
    o.TokenValidationParameters = new  
        TokenValidationParameters  
    {  
        ValidateIssuer = true,  
        ValidateAudience = true,  
        ValidateLifetime = true,  
        ValidateIssuerSigningKey = true,  
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
```



```
ValidAudience = builder.Configuration
    ["Jwt:Audience"],
IssuerSigningKey = new SymmetricSecurityKey
    (Encoding.UTF8.GetBytes(builder.Configuration["Jwt:
        Key"]))

    };
});
```

Here, we are adding configurations to the application that will declare to the API application that it should enforce a particular type of authentication scheme. Given that Identity Core has support for several authentication schemes, we need to specify the ones that we intend to enforce and by extension, the type of challenge scheme that we require. The challenge scheme refers to the authentication requirements that the application will need. Here, we specify `JwtBearerDefaults.AuthenticationScheme` for both the challenge and authentication schemes. This `JwtBearerDefaults` class contains generally available and used JWT constants. In this case, `AuthenticationScheme` will render the value `bearer`, which is a keyword.

After we are done defining the authentication scheme, we go on to set configurations that will enforce certain rules that will govern how a bearer token is validated. By using `true` for `ValidateIssuer`, `ValidateAudience`, and `ValidateLifetime`, we are enforcing that the matching values in an incoming token must match the values that we set in the `appsettings.json` configuration constants. You can be flexible with the validation rules based on how strictly you want to check the bearer token contents against your system. The fewer validations in place, the higher the chances of someone using fake tokens to gain access to the system.

We will also need to ensure that our API knows that we intend to support authorization, so we need to add this line as well:

```
builder.Services.AddAuthorization();
```

Then, we also need to include our middleware with the following two lines, in this order:

```
app.UseAuthentication();
app.UseAuthorization();
```

Now that we have taken care of the preliminary configurations, we need to include our default identity user tables in our database. We first change the inheritance of our database context from `DbContext` to `IdentityDbContext`:

```
public class AppointmentsDbContext : IdentityDbContext
```

We will also add code to generate a sample user in the database context. When we perform the next migration, then this user will be added to the table and we can use it to test authentication:

```
protected override void OnModelCreating(ModelBuilder
    builder)
    {
        base.OnModelCreating(builder);
        var hasher = new PasswordHasher<ApiUser>();
        builder.Entity<ApiUser>().HasData(new ApiUser
        {
            Id = "408aa945-3d84-4421-8342-
                7269ec64d949",
            Email = "admin@localhost.com",
            NormalizedEmail = "ADMIN@LOCALHOST.COM",
            NormalizedUserName = "ADMIN@LOCALHOST.COM",
            UserName = "admin@localhost.com",
            PasswordHash = hasher.HashPassword(null,
                "P@ssword1"),
            EmailConfirmed = true
        });
    }
```

After these changes, the next migration that we perform will generate user tables that will be created when the update-database command is executed. These new tables will, by default, be prefixed with `AspNet`.

We also need to register the `Identity Core` services in our application and connect it to the database context as follows:

```
builder.Services.AddIdentityCore<IdentityUser>()
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<AppointmentsDbContext>();
```

Here, we register our identity-related services in our applications, specify that we are using the default user type called `IdentityUser`, the default role type called `IdentityRole`, and the data store associated with `AppointmentsDbContext`.

Now that we have specified what is required for the integration of `Identity Core` and JWT authentication, we can look to implement a login endpoint that will verify the user's credentials and generate a token with the minimum identifying information accordingly. We will investigate this in the next section.

Generating and issuing bearer tokens

ASP.NET Core has support for generating, issuing, and validating bearer tokens. To do this, we need to implement logic in our authentication flow that will generate a token with the authenticated user's information, and then return it to the requesting client in the body of the response. Let us first define a **data transfer object (DTO)**, that will store the user's Id value and the token and wrap them both in their own `AuthResponseDto`:

```
public class AuthResponseDto
{
    public string UserId { get; set; }
    public string Token { get; set; }
}
```

We will also have a DTO that will accept login information. We can call this `LoginDto`:

```
public class LoginDto
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [StringLength(15, ErrorMessage = "Your Password is
        limited to {2} to {1} characters",
        MinimumLength = 6)]
    public string Password { get; set; }
}
```

Our DTO will enforce validation rules on the data being submitted. Here, our users can authenticate using their email address and a password, and invalid attempts that violate the validation rules will be rejected with a `400BadRequest` HTTP code.

Our authentication controller will implement a login action that will accept this DTO as a parameter:

```
[Route("api/[controller]")]
[ApiController]
public class AccountController : ControllerBase
{
    private readonly IAuthManager _authManager;
```

```
public AccountController(IAuthManager authManager)
{
    _authManager = authManager;
}

// Other Actions here

[HttpPost]
[Route("login")]
public async Task<IActionResult> Login([FromBody]
    LoginDTO loginDto)
{
    var authResponse = _authManager.Login(loginDto);
    if (authResponse == null)
    {
        return Unauthorized();
    }
    return Ok(authResponse);
}
}
```

We inject an `IAuthmanager` service into the controller, where we have abstracted the bulk of the user validation and token generation logic. This service contract is as follows:

```
public interface IAuthManager
{
    // Other methods
    Task<AuthResponseDto> Login(LoginDTO loginDto);
}
```

In the implementation of `AuthManager`, we use the `UserManager` service, which is provided by `Identity Core`, to verify the username and password combination that is submitted. Upon verification, we will generate and return an `AuthResponseDto` object containing the token and user's ID. Our implementations will look like the following code block:

```
public class AuthManager : IAuthManager
{
    private readonly UserManager<IdentityUser>
```

```
        _userManager;
private readonly IConfiguration _configuration;
private IdentityUser _user;
public AuthManager(UserManager<IdentityUser>
    userManager, IConfiguration configuration)
{
    this._userManager = userManager;
    this._configuration = configuration;
}

// Other Methods
public async Task<AuthResponseDto> Login(LoginDto
    loginDto)
{
    var user = await _userManager.FindByEmailAsync
        (loginDto.Email);
    var isValidUser = await _userManager
        .CheckPasswordAsync(_user, loginDto.Password);

    if(user == null || isValidUser == false)
    {
        return null;
    }
    var token = await GenerateToken();
    return new AuthResponseDto
    {
        Token = token,
        UserId = _user.Id
    };
}
```

We inject both `UserManager` and `IConfiguration` into our `AuthManager`. In our login method, we attempt to retrieve the user based on the email address that was provided in `LoginDto`. If we then attempt to validate that the correct password was provided. If there is no user, or the password was incorrect, we return a null value, which the login action will use to indicate that no user was found and will return a 401 Unauthorized HTTP response.

If we can validate the user, then we generate a token and then return our `AuthResponseDto` object with the token and the user's Id value. The method to generate the token is also in `AuthManager` and it looks like this:

```
private async Task<string> GenerateToken()
{
    var securitykey = new SymmetricSecurityKey
        (Encoding.UTF8.GetBytes(_configuration["
            Jwt:Key"]));
    var credentials = new SigningCredentials
        (securitykey, SecurityAlgorithms.HmacSha256);
    var roles = await _userManager.GetRolesAsync
        (_user);
    var roleClaims = roles.Select(x => new
        Claim(ClaimTypes.Role, x)).ToList();
    var userClaims = await _userManager.GetClaimsAsync
        (_user);
    var claims = new List<Claim>
    {
        new Claim(JwtRegisteredClaimNames.Sub,
            _user.Email),
        new Claim(JwtRegisteredClaimNames.Jti,
            Guid.NewGuid().ToString()),
        new Claim(JwtRegisteredClaimNames.Email,
            _user.Email),
        new Claim("uid", _user.Id),
    }
    .Union(userClaims).Union(roleClaims);

    var token = new JwtSecurityToken(
        issuer: _configuration[" Jwt:Issuer"],
        audience: _configuration[" Jwt:Audience"],
        claims: claims,
        expires: DateTime.Now.AddMinutes
            (Convert.ToInt32(_configuration["
                Jwt:DurationInMinutes"])),
        signingCredentials: credentials
```



```
pc3MiOiJib3RlbExpc3RpbmdBUEkiLCJhdWQiOiJib3RlbExpc3Rpbm
dBUElDbGllbnQifQ.yuYUIFKPTyKKUpsVQhbs4NinGLSF5_XXPEbtAEf
j05E"
}
```

Implementing token authentication in an API is easy enough, but we are not only accounting for one API in our application. We have several APIs that need to be secured and preferably, one token should be accepted across all the services. If we continue down this path, we may end up making these configurations per service and then needing extra code to have all the other services acknowledge a token that might be issued by any other service.

We need a more global solution and more suitably, a central authority of security and token generation and management for all the services in our microservices application. This is where we begin to explore separating the token management responsibilities from each API and placing them in one that implements **IdentityServer**, which is **OpenID Connect** and the **OAuth 2.0** framework for ASP.NET Core. We will investigate implementing **IdentityServer** in the next section.

Using IdentityServer4 to secure microservices

A key feature in any modern application or suite of applications is the concept of **single-sign-on (SSO)**. This feature allows us to provide our credentials once and retain an authenticated user state across several applications within the suite. This is a feature that can be observed in Google or Microsoft Online products, to name a few.

This concept will come in handy when securing a microservices application. As we can see, it is not feasible to implement token-issuing logic in many APIs across an application and then attempt to coordinate access to all the APIs when it was granted to one. We also run the risk of requiring a user to reauthenticate each time they attempt to access a feature that requires another API to complete, and this will not make for a good user experience.

With these considerations in mind, we need to use a central authority that can allow us to implement more global token issuing and validation rules given all the security considerations of our services. In ASP.NET Core, the best candidate for such services is **IdentityServer**.

IdentityServer is an open source framework built on top of ASP.NET Core that extends the capabilities of **Identity Core** and allows developers to support **OpenID Connect** and **OAuth2.0** standards in their web application security implementation. It is compliant with industry standards and contains out-of-the-box support for token-based authentication, SSO, and API access control in your applications. While it is a commercial product, a community edition is available for use by small organizations or personal projects.

The recommended implementation style of **IdentityServer** would have us do the following:

1. Create a new microservice for authentication
2. Create a new database just for our authentication-related tables (optional)
3. Configure scopes to be included in the token information
4. Configure our services to know which scopes are allowed to access them

Figure 12.2 shows the IdentityServer authentication flow:

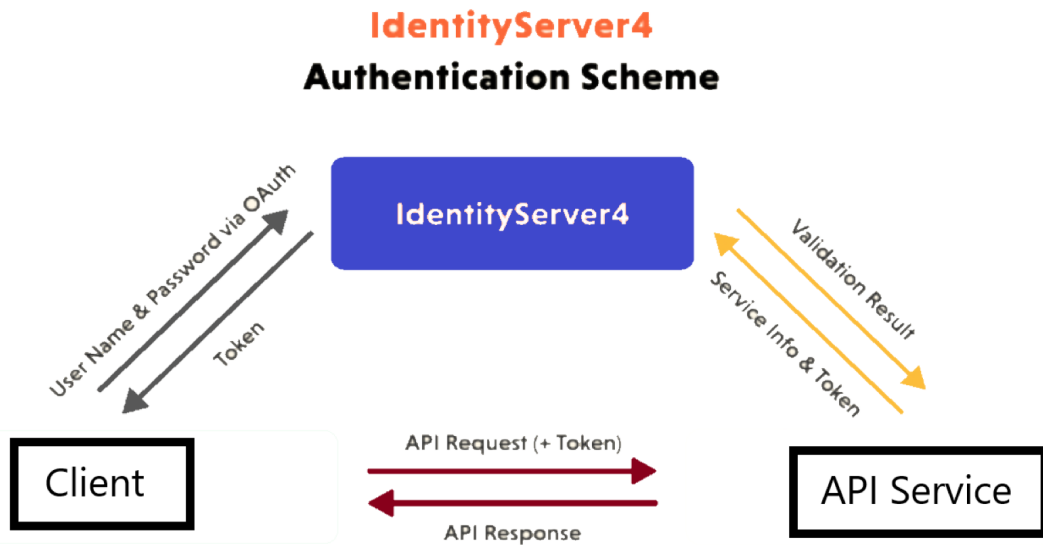


Figure 12.2 – This depicts how IdentityServer sits between a client and service and handles the flow of authentication and token exchange

Now, let us explore creating a new service and configuring it to be our central authority for authentication and authorization in our microservices application.

Configuring IdentityServer

Duende offers us some quick-start ASP.NET Core project templates that are easy to create in our solution. These quick-start templates bootstrap the minimum requirements needed to bootstrap IdentityServer functionality in an ASP.NET Core project. The general steps involved in setting up an IdentityServer service are as follows:

- Add Duende IdentityServer support to a standard ASP.NET Core project
- Add data storage support, preferably using Entity Framework configurations

- Add support for ASP.NET Identity Core
- Configure token issuing for client applications
- Secure client applications with IdentityServer

To get started, we need to use the .NET CLI and run the following command:

```
dotnet new --install Duende.IdentityServer.Templates
```

That command will now give us access to new project templates prefixed with **Duende . IdentityServer**. *Figure 12.3* depicts what we can expect to see in Visual Studio once these templates are installed.

Figure 12.3 shows the Duende IdentityServer project templates:

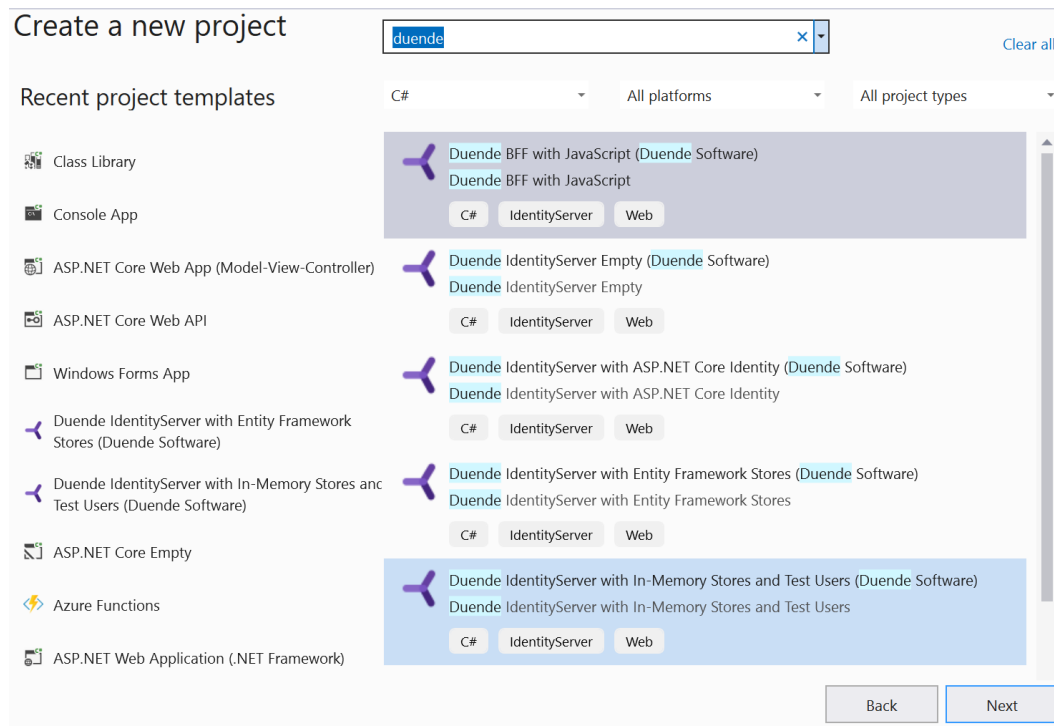


Figure 12.3 – We get a variety of project templates that help us to speed up the IdentityServer implementation process

Using our healthcare microservices application, let us start by adding a new **Duende IdentityServer with Entity Framework Stores** project to handle our authentication. We will call it **HealthCare . Auth**. Now, we have a preconfigured IdentityServer project with several moving parts. We need to

understand what the major components are and have an appreciation of how we can manipulate them for our needs. Let us conduct a high-level review of the file and folder structure that we get out of the box:

- `Wwwroot`: A standard folder that is shipped with ASP.NET Core web application templates. It stores static assets such as JavaScript and CSS files that are used in the website.
- `Migrations`: Stores preset migrations that will be used to populate the data store with supporting tables. This is handy, as it removes the need for us to create the databases.
- `Pages`: Stores default Razor pages that are used to support the UI requirements of user authentication operations. Out of the box, we get login, register, grant, and user data management pages.
- `appsettings.json`: The standard file that contains logging and database connection configurations. We can change this connection string to reflect our requirements better.
- `buildschema.bat`: Contains Entity Framework commands using .NET command-line commands (`dotnet ef`) that will run migration scripts that have been included in the `Migrations` folder. We will use these commands to create our databases.
- `Config.cs`: This static class serves as a configuration authority. It is used to outline `IdentityResources`, `Scopes`, and `Clients`:
 - `IdentityResources`: Map to scopes that grant access to identity-related information. The `OpenId` method supports the expected *subject* (or *sub-claim*) value and the `Profile` method supports additional claim information such as `given_name` and `family_name`. We can also extend the default offerings and include additional details such as the user's *roles*.
 - `Scopes`: Can be used to outline permissions that can be included in the token when it is issued.
 - `Clients`: Third-party clients that we expect to use `IdentityServer` as a token-issuing authority.
- `HostingExtension.cs`: Contains service and middleware registration extension methods. These methods are then called in the `Program.cs` file during startup.
- `Program.cs`: Primary program execution file in an ASP.NET Core application.
- `SeedData.cs`: Contains default methods that will ensure that data migrations and seeding operations are carried out at application startup.

`IdentityServer` uses two database contexts, a configuration store context and an operational store context. As a result, two database contexts are created in the `HostingExtension.cs` file. The `IdentityServer` libraries are registered using the following code:

```
var isBuilder = builder.Services
    .AddIdentityServer(options =>
```

```
{
    options.Events.RaiseErrorEvents = true;
    options.Events.RaiseInformationEvents =
        true;
    options.Events.RaiseFailureEvents =
        true;
    options.Events.RaiseSuccessEvents =
        true;
    options.EmitStaticAudienceClaim =
        true;
})
.AddTestUsers (TestUsers.Users)
.AddConfigurationStore (options =>
{
    options.ConfigureDbContext = b =>
        b.UseSqlite (connectionString,
            dbOpts => dbOpts.MigrationsAssembly
                (typeof (Program) .Assembly
                    .FullName));
})
.AddOperationalStore (options =>
{
    options.ConfigureDbContext = b =>
        b.UseSqlite (connectionString,
            dbOpts => dbOpts.MigrationsAssembly
                (typeof (Program) .Assembly.FullName
                    ));
    options.EnableTokenCleanup = true;
    options.RemoveConsumedTokens = true;
});
```

We are adding `TestUsers` to the configuration and then adding `ConfigurationStoreDbContext` and `OperationalStoreDbContext`. Other settings also govern how alerts and tokens are handled. The defaults are generally solid, but you may modify them based on your specific needs.

The default connection string and Entity Framework Core libraries give us support for an SQLite database. This can be changed to whatever the desired data store may be, but we will continue with SQLite for the purpose of this exercise. Let us proceed to generate the database and the tables, and we need the following commands:

```
Update-Database -context PersistedGrantDbContext
Update-Database -context ConfigurationDbContext
```

With these two commands, we will see our database scaffolded with all the supporting tables. At this point, they are all empty and we may want to populate them with some default values based on our application. Let us start by configuring the `IdentityResources` that we intend to support in our tokens. We can modify the `IdentityResources` method as follows:

```
public static IEnumerable<IdentityResource>
    IdentityResources =>
        new IdentityResource[]
        {
            new IdentityResources.OpenId(),
            new IdentityResources.Profile(),
            new IdentityResource("roles", "User role(s)",
                new List<string> { "role" })
        };
```

We have added the list of roles to the resources list. Based on the claims that are being accounted for, we need to ensure that our users will contain all their expected data, as well as the list of claims that they are expected to have. Bear in mind that claims are the information that a client application will have via the token since it is the only way a client can track which user is online and what they can do.

Now, we can refine the list of scopes that are supported by modifying the `ApiScopes` method as follows:

```
public static IEnumerable<ApiScope> ApiScopes =>
    new ApiScope[]
    {
        new ApiScope("healthcareApiUser",
            "HealthCare API User"),
        new ApiScope("healthcareApiClient",
            "HealthCare API Client ")
    };
```

Here, we are supporting two types of authentication scopes. These scopes will be used to support authentication for two different scenarios: client and user. Client authentication represents an unsupervised attempt to gain access to a resource, usually by another program or API. Client authentication means that a user will authenticate using credentials.

This brings us to the next configuration, which is for the clients. The term client is used a bit loosely since any entity that attempts to gain authorization from IdentityServer is seen as a client. The word client can also refer to a program that is attempting to gain authorization, such as a daemon or background service. Another scenario is when a user attempts to carry out an operation that requires them to authenticate against IdentityServer. We add support for our clients as follows:

```
public static IEnumerable<Client> Clients =>
    new Client[]
    {
        // m2m client credentials flow client
        new Client
        {
            ClientId = "m2m.client",
            ClientName = "Client Credentials Client",
            AllowedGrantTypes = GrantTypes
                .ClientCredentials,
            ClientSecrets = { new Secret("511536EF-
                F270-4058-80CA-1C89C192F69A ".Sha256())
            },
            AllowedScopes = { "healthcareApiClient" }
        },
        // interactive client using code flow + pkce
        new Client
        {
            ClientId = "interactive",
            ClientSecrets = { new Secret("49C1A7E1-
                0C79-4A89-A3D6-A37998FB86B0".Sha256()) },
            AllowedGrantTypes = GrantTypes.Code,
            RedirectUri = {
                "https://localhost:5001/signin-oidc" },
            FrontChannelLogoutUri =
                "https://localhost:5001/signout-oidc",
            PostLogoutRedirectUri = {
```

```
        "https://localhost:5001/signout-  
        callback-oidc" },  
        AllowOfflineAccess = true,  
        AllowedScopes = { "openid", "profile",  
        "healthcareApiUser", "roles" }  
    },  
};
```

Now, we have defined the `ClientId` and `ClientSecret` values for our clients. By defining several clients, we can support the applications that we expect to support at a more granular level, and we can define specific `AllowedScopes` and `AllowedGrantTypes` values. In this example, we have defined a client for an API, which can represent a microservice in our application that might need to authenticate with the authentication service. This type of authentication generally occurs without user interaction. We also define a web client, which could be a user-facing application. This presents the unique challenge where we configure sign-in and sign-out URLs to redirect our users during the authentication or logout flow. We also go on to state which scopes will be accessible via the generated token. We have added the `roles` value to the list of `AllowedScopes` since we want that information to be included when a user authenticates.

Now that we have our configuration values outlined, let us add a command-line argument for seeding to the `launchSettings.json` file in the `Properties` folder. The file's contents will now look as follows:

```
"profiles": {  
  "SelfHost": {  
    "commandName": "Project",  
    "launchBrowser": true,  
    "environmentVariables": {  
      "ASPNETCORE_ENVIRONMENT": "Development"  
    },  
    "applicationUrl": "https://localhost:5001",  
    "commandLineArgs": "/seed"  
  }  
}
```

If we run this application after making this adjustment, the `if (args.Contains("/seed"))` statement will evaluate to `true` in `Program.cs` and this will trigger the database seeding activity as outlined in the `SeedData.cs` file. After the first run, you may remove the `"commandLineArgs": "/seed"` section from the `launchSettings.json` file. Running it again will launch a browser application with a page similar to what is depicted in *Figure 12.4*. This is the home landing page and shows that our `IdentityServer` is up and running.

Figure 12.4 shows the Duende IdentityServer landing page:

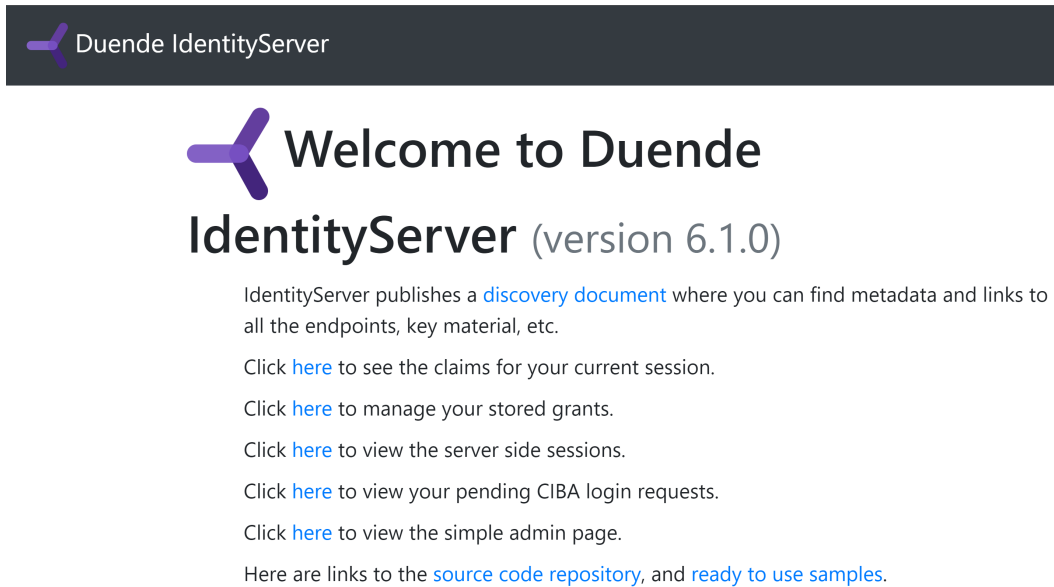


Figure 12.4 – This landing page shows us that our IdentityServer application is in a running state

You can find the `TestUsers.cs` file in the `Pages` folder. We will use `alice` as both the username and password for a quick test. You may proceed to use the credentials that have been provided in your instance of that file. We can then proceed to test a login operation using one of the test users that was added to the context by default, and we will be required to authenticate when attempting to access most of these links.

The most important link to discuss is the one that leads to the discovery document. Most OAuth2.0 and OpenID Connect service providers have a concept of a discovery document, which outlines the built-in routes in the API, supported claims and token types, and other key bits of information that make it easier for us to know and access these intricate bits of information from IdentityServer. Some of the key information available is as follows:

```
{
  "issuer": "https://localhost:5001",
  "jwks_uri": "
    https://localhost:5001/.well-known/openid-
    configuration/jwks",
  "authorization_endpoint": "
    https://localhost:5001/
```



```
        connect/authorize",
    "token_endpoint": "
        https://localhost:5001/connect/
        token",
    "userinfo_endpoint": "
        https://localhost:5001/connect/
        userinfo",
    "end_session_endpoint": "
        https://localhost:5001/connect
        /endsession",
    "check_session_iframe": "
        https://localhost:5001/connect
        /checksession",
    "revocation_endpoint": "
        https://localhost:5001/connect
        /revocation",
    "introspection_endpoint": "
        https://localhost:5001/
        connect/introspect",
    "device_authorization_endpoint": "
        https://localhost:5001/connect/deviceauthorization",
    "backchannel_authentication_endpoint":
        "https://localhost:5001/connect/ciba",
    ...
    "scopes_supported": [
        "openid",
        "profile",
        "roles",
        "healthcareApiUser",
        "healthcareApiClient",
        "offline_access"
    ],
    ...
}
```

We have a clear outline of the various endpoints that are now available to us for the different commonly access operations.

Next, we can test our `HealthCare.Auth` application and validate that we can retrieve a valid token. Let us attempt to retrieve a token using our machine client credentials. We will use an API testing tool called *Postman* to send the request. *Figure 12.5* shows the user interface in Postman and the information that needs to be added accordingly.

https://localhost:5001/connect/token

POST https://localhost:5001/connect/token

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Type OAuth 2.0

The authorization data will be automatically generated when you send the request. Learn more about [authorization](#)

Add authorization data to Request Headers

Current Token
This access token is only available to you. Sync the token to let collaborators on this request use it.

Access Token

Available Tokens

eyJhbGciOiJIUzU1NiIsImtpZCI6IjN0...

Header Prefix Bearer

Configure New Token

Configuration Options Advanced Options

Token Name Token

Grant Type Client Credentials

Access Token URL https://localhost:5001/connect/token

Client ID m2m.client

Client Secret 511536EF-F270-4058-80CA-1C89C192f...

Scope e.g. read:org

Client Authentication Send client credentials in body

Clear cookies

Get New Access Token

Figure 12.5 – Here, we add the client ID, client secret, and token URL values in Postman in order to retrieve a bearer token

Once we have added the required values, we proceed to click on the **Get New Access Token** button. This will send a request to our IdentityServer, which will validate the request and return a token if the information is found in the database.

Our token response automatically includes some additional information such as the type of token, the expiry timestamp, and the scope that is included. Our token is generated with several data points by default. Since IdentityServer follows the *OAuth* and *OpenID Connect* standards, we can be sure that we do not need to include basic claims such as **sub**, **exp**, **jti**, and **iss**, to name a few.

The values that get included are the scope and client ID. These are determined by the configurations that we have per client and the information that is presented by the authenticating user. In this example, we are accommodating APIs that only authenticated users should be able to access.

Figure 12.6 shows the payload of the token:

PAYLOAD: DATA

```
{
  "iss": "https://localhost:5001",
  "nbf": 1668216814,
  "iat": 1668216814,
  "exp": 1668220414,
  "aud": "https://localhost:5001/resources",
  "scope": [
    "healthcareApiClient"
  ],
  "client_id": "m2m.client",
  "jti": "FF86E33B7756615B609A61109BE0D4C8"
}
```

Figure 12.6 – Our token automatically contains some claims that we would have entered manually if it was generated without IdentityServer

Let us save our bearer token value that was returned, as we will use it in our next section. Now let us review the changes that are necessary to protect an API using IdentityServer.

Securing an API using IdentityServer

We now have the peculiar challenge of implementing the best possible security solution across our microservices application. We have several services that need to be secured and based on the architecture pattern you have implemented, you might also have a gateway that is routing traffic:

- **Securing each service:** Securing each service seems simple enough, but we must bear in mind that each service has different requirements and might need to be seen as a different client for each request. This can lead to a maintenance nightmare when trying to maintain all the scopes and clients, relative to each service. We then need to navigate how services will communicate as well since a token will be needed for service-to-service calls. One service's claims and scopes might not be sufficient for this communication. This might lead to a user having to authenticate several times when accessing different features that rely on different services.

- **Secured API gateway:** Securing our API gateway makes the most sense. If we implement a gateway that all apps will communicate with, we allow the gateway to orchestrate the authentication flow for the client and then manage the token to be shared between service calls. This support can be implemented in a custom-written API gateway and is supported by most if not all third-party gateway service providers. This is especially useful when combined with the *Backend For Frontend* pattern.

We have already seen how we can add JWT bearer protection to our API using functionality from the `Identity Core` library. We can leverage some of these configurations and override the native functionality with support for IdentityServer. Let us explore how we can secure our `Patients` API using IdentityServer. We start by adding the `Microsoft.AspNetCore.Authentication.JwtBearer` library using the NuGet package manager:

```
Install-Package Microsoft.AspNetCore.Authentication
.JwtBearer
```

We then modify the `Program.cs` file and add the following configuration:

```
builder.Services.AddAuthentication(JwtBearerDefaults
    .AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        // base-address of your identityserver
        options.Authority =
            "https://localhost:5001/";

        // audience is optional, make sure you read
        // the following paragraphs
        // to understand your options
        options.TokenValidationParameters
            .ValidateAudience = false;

        // it's recommended to check the type
        // header to avoid "JWT confusion" attacks
        options.TokenValidationParameters
            .ValidTypes = new[] { "at+jwt" };
    });
```

We will also need to register the authentication middleware in our application with the following line. We should ensure that we place this registration above the authorization middleware:

```
app.UseAuthentication();  
app.UseAuthorization();
```

This configuration will dictate to our service that we are now to refer them to the URL in the *Authority* option, for authentication instructions. We can now protect our API by implementing a global authorization policy. This will ensure that no endpoint can be accessed without a valid bearer token that has been issued by our IdentityServer:

```
builder.Services.AddAuthorization(options =>  
{  
    options.AddPolicy("RequireAuth", policy =>  
    {  
        policy.RequireAuthenticatedUser();  
    });  
});
```

We modify the controller's middleware as follows:

```
app.MapControllers().RequireAuthorization("RequireAuth");
```

Now, any attempt to interact with our `Patients` API endpoint will return a `401Unauthorized` HTTP response. The API is now expecting us to provide the bearer token in the authorization header value. In *Figure 12.7*, we see how we can make authorized API calls to our `Patients` API endpoints using the bearer token that was retrieved in the previous section from our machine client credentials authentication.

Figure 12.7 shows the authorized API request:

The screenshot shows a web browser's developer tools interface. The top bar indicates the URL is `https://localhost:7232/api/Patients`. The 'GET' method is selected, and the 'Send' button is visible. The 'Authorization' tab is active, showing a 'Bearer Token' type. The token value is a long alphanumeric string. The 'Body' tab is also active, displaying a JSON response:

```

{
  "firstName": "Patient",
  "lastName": "One",
  "dateOfBirth": "2022-11-12T02:17:13.858",
  "documents": [],
  "id": "3fa85f64-5717-4562-b3fc-2c963f66fa6"
}

```

The status bar at the bottom shows 'Status: 200 OK', 'Time: 3.02 s', and 'Size: 289 B'. The 'Save Response' button is also visible.

Figure 12.7 – Our bearer token is included in the request to our protected service and we can comfortably access endpoints

Now, we need to configure our API to force authentication and rely on the *HealthCare.Auth* service accordingly. If we reuse our appointments API, we can make a few modifications to the `Program.cs` file and introduce reliance on our authentication service.

We begin by modifying the `builder.Services.AddAuthentication()` registration as follows:

```

builder.Services.AddAuthentication(JwtBearerDefaults
    .AuthenticationScheme)
    .AddJwtBearer("Bearer", opt =>
    {
        opt.RequireHttpsMetadata = false;
    });

```

Now that we have secured our API directly, we can explore how we can manage this new security requirement in our API gateway. Recall that we have implemented aggregation methods and we will expect client applications to access the endpoints through the gateway.

Securing the Ocelot API gateway with IdentityServer

Now, when we access an API endpoint that is protected via IdentityServer, we need to retrofit our gateway service to support authentication and forwarding of the credentials to the target API. We start by adding the `Microsoft.AspNetCore.Authentication.JwtBearer` library using the NuGet package manager:

```
Install-Package Microsoft.AspNetCore.Authentication
.JwtBearer
```

We then modify the `ocelot.json` file with an `AuthenticationOptions` section. Now, our GET method for the `Patients` API is as follows:

```
{
  "DownstreamPathTemplate": "/api/Patients",
  ...
  "AuthenticationOptions": {
    "AuthenticationProviderKey":
      "IdentityServerApiKey",
    "AllowedScopes": []
  },
  ...
},
```

Now, we modify our `Program.cs` file and register our authentication service to use JWT bearer authentication, similar to what we did on the service itself:

```
builder.Services
    .AddAuthentication()
    .AddJwtBearer(authenticationProviderKey, x =>
    {
        x.Authority = "https://localhost:5001";
        x.TokenValidationParameters = new
            TokenValidationParameters
            {
                ValidateAudience = false
            };
    });
```

Now, we have secured our gateway using IdentityServer. This, once again, might be a better security solution for our suite of microservices that will be accessed through the gateway, and it can help us to centralize access to our services.

Now that we have explored API security at length, let us summarize the concepts that we have explored.

With this simple change, we no longer need to concern our appointments API with the inclusion of authentication tables in its database, or complex JWT bearer compilation logic. We simply point the service to our *Authority*, which is the authentication service, and include the *Audience* value so that it can identify itself to the authentication service.

With this configuration, a user will need to provide a token such as the one we retrieved to make any calls to our API. Any other token or lack thereof will be met with a *401 Unauthorized* HTTP response.

Configuring IdentityServer is not the most difficult task, but it can become complex when attempting to account for several scenarios, configurations, and clients. Several considerations can be made along the way, and we will discuss them next.

Additional API security considerations

We have configured an authentication service to secure our microservices application. Several scenarios can govern how each service is protected by this central authority and they all have their pros and cons.

What we also need to consider is that we want the entire responsibility of hosting and maintaining our own *OAuth* service. There are third-party services such as **Auth0**, **Azure Active Directory**, and **Okta**, to name a few. They all provide a hosted service that will abstract our need to stage and maintain our services, and we can simply subscribe to their services and secure our application with a few configurations.

This option takes advantage of **Software-as-a-Service (SaaS)** offerings that greatly reduce our infrastructure needs and increase the reliability, stability, and future-proofing of our application's security.

Summary

In this chapter, we have reviewed the current industry standard for API security. Using bearer tokens, we can support authorized API access attempts without maintaining state or sessions.

In a service-oriented architecture, a client app can come in several forms, whether a web application, a mobile application, or even a smart television. We cannot account for the type of device in use and our API does not keep track of the applications connecting to it. For this reason, when a user logs in and is verified against our user information data stores, we select the most important bit of information and compile them into a token.

This token is called a bearer token and is an encoded string that should contain enough information about a user that our API can determine the user with whom the token is associated and their privileges in our system.

Ultimately, attempting to secure each API using this method can lead to a lot of disconnection and complexity, so we introduce a centralized authentication management platform such as IdentityServer. This central authority will secure all the APIs using common configurations, and issue tokens based on those global configurations. Now, we can use these tokens once and access several services without needing to re-authenticate.

Security should never be neglected in any application and when it is well implemented, we can strike a balance between security and usability in our application.

Now that we have explored security for our microservices application, we will review how we can leverage *containers* to deploy our microservices application in the next chapter.

Microservice Container Hosting

Once we have completed a fair amount of development, our next major concern is **hosting**. Hosting comes with its own set of problems because there are many options, and the pros and cons of these options are relative to the application's architecture and overall needs.

Typical hosting options for a web application would be a simple server and a singular point of entry to that server via an IP address or domain name. Now, we are building a microservices application where we pride ourselves on the fact that we can promote loose coupling and have all the parts of our application act autonomously and without direct dependency on each other. The challenge now becomes how we cater to a potentially heterogeneous application. Each service is autonomous and might have varied hosting and database requirements. We would then need to consider creating specific hosting environments for each technology, which can lead to massive cost implications.

This is where we can take advantage of container hosting technologies and minimize the cost overheads of having several server machines. We can use containers as scaled-down imitations of the minimum hosting requirements for each technology and database that is being used in the microservices architecture, and we can configure endpoints by which each container can be accessed.

In this chapter, we will review how containers work, how they can benefit us, and why they are an essential tool for efficiently hosting a microservices application.

We will cover the following topics:

- Understand the use of Docker and containers
- Learn how to use Dockerfiles and commands
- Learn how to use `docker-compose` and orchestrate Docker images
- Learn how to deploy a microservices application in containers and to a container registry

Technical requirements

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch13>.

Using containers in microservices development

Containers are all the rage in the development space. They present us with a lightweight application hosting option that allows us to deploy our applications in a clean and repeatable manner. They are not new, but their use in more commercial and accessible spaces has been made far more popular in recent years. What, however, is a container, and why should we care about how it works? Let us review that next.

What can containers do for me?

Traditionally speaking, whenever we have applications that have specific requirements for environments and software, we would resort to using servers to facilitate the requirements. The problem with servers and servers per application is that they come with costs. A server machine is generally not cheap, and then we must also factor in licensing and energy costs when new machines are introduced. Also, consider that if a machine goes down, we will need to reconfigure that machine to the original environmental specifications and reprovision several aspects of the original deployment.

At this point, we begin to think about virtualization. This means that we now use **virtual machines (VMs)** for new servers and reuse the old infrastructure. This will go a long way in reducing the physical infrastructure requirements and costs and allow us to scale a lot more easily. We can also use snapshots of the machines to keep a quick recovery plan up our sleeves in times of failure. There are several virtualization solutions on the market, including popular ones such as **VMware, VirtualBox, and Microsoft Hyper-V**.

Figure 13.1 shows a server with several VMs:

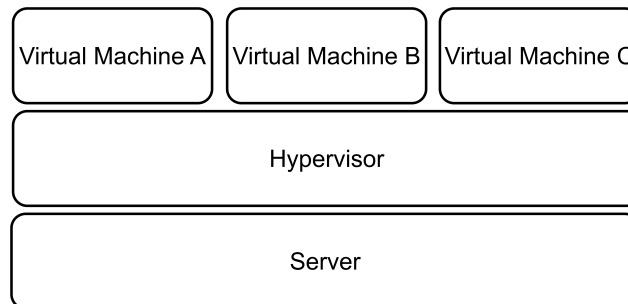


Figure 13.1 – One machine is required to support multiple VMs on top of a hypervisor

This visualization approach seems like the silver bullet we need, except we have found more problems with this solution, which are as follows:

- We will still need to factor in that we need very powerful machines to be able to handle multiple VMs
- We will still have the manual maintenance tasks required to keep our environments and operating systems up to date and secure

- We must remember that there are several situations where we attempt to provision the same environment on different installations and encounter unforeseen differences each time
- We cannot always rely on the machine environment to remain consistent with each instance

Now, this brings us to the most recent solution to this problem, which is containers. Containers build on the concept given to us VMs and allow us to downsize the overall space and resource requirements that an application might need. The use of containers to host our applications is called **containerization**—this is an approach in modern software development that allows us to package an application and all its dependencies and create a repeatable, testable, and reliable package called an **image**. This image can then be deployed directly to several places in a consistent way.

This consistency is very important in how we distinguish the benefits of containers from VMs. We just reviewed that we cannot always be sure that each **operating system (OS)** instance of a server or VM will be the same. Containers strip away many of the variables that we contend with during deployments and provide an environment that is specifically tuned for the application that needs to be deployed.

Figure 13.2 shows a server with several containers and apps:

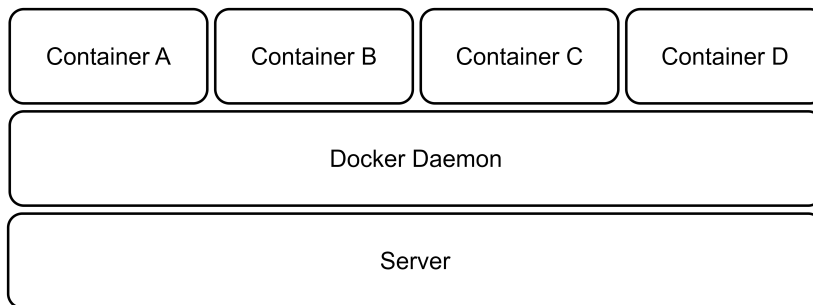


Figure 13.2 – One machine can host several container apps and make better use of its resources

Another benefit, as seen in Figure 13.2, is that we can now maximize the resource usage of a physical server since we no longer need to provide a set amount of RAM, CPU, and storage for entire instances of operating systems.

Containers also provide isolation from each other on a shared operating system, so we do not need to worry about heterogeneous requirements across containers. In simpler terms, it will be acceptable to run one container in a Linux environment alongside one that needs a Windows-based environment. Using containers will make our applications more scalable and reliable. We can easily provision new instances of containers as needed and run multiple identical instances.

There is a major security risk in using VMs that need to communicate with each other. They generate metadata files that contain very sensitive information about the data being exchanged and how it is exchanged. With this information, an attacker could seek to replay operations and insert malicious information into the mix. Containers do not transfer this risk and will support inter-container communication in a far more secure manner.

Containers are not new, but they are becoming more and more popular for hosting isolated operations in consumer applications and improving efficiency for applications deployed to less powerful machines. To use containers, we need a container hosting solution such as Docker. We will review how we can get started with Docker and containers next.

Understanding Docker

Before we dive into Docker and how it works, it is noteworthy that Docker is not the only application that handles containerization. There are several alternatives that follow the same **Open Container Initiative (OCI)** standards and allow us to containerize our applications. Docker, however, revolutionized and propelled containerization into mainstream access and exposure. It is a free (for development and open source) application that is available cross-platform and allows us to version control our containers and generally manage several container instances through either a **user interface (UI)** or a **command-line interface (CLI)**.

Docker's engine has a client-server implementation style, where both client and server run on the same hardware. The client is a CLI, and it interacts with the server via a REST API to send instructions and execute functions. The Docker server is a background job, or daemon, called `dockerd`. It is responsible for tracking the life cycle of our containers. We also have objects that need to be created and configured to support deployments. These objects can be networks, storage volumes, and plugins, to name a few. We create these objects on a case-by-case basis and deploy them as needed.

So, let's recap a bit. A container is a self-contained space that is prepared to host one application at a time. The definition of a container's environment and dependencies is called an image. This image is a consistent blueprint for what the environment needs to look like. Images exist for several third-party applications as well, and this makes it easy for us to spin up an instance. It is good to have a central repository for images, and Docker provides *Docker Hub* for this reason.

Docker Hub is a container registry that stores and distributes container images. We can use it to host our own images, and there is a public space for generally available and shared images for industry-leading applications. Note—Docker Hub is not the only registry. There are alternatives such as Microsoft **Azure Container Registry (ACR)**, which allows us to integrate with other Azure services more seamlessly.

We have mentioned container images a few times now. Let us discuss them a bit more in the next section.

Understanding container images

A container image, as mentioned before, is a blueprint for the contents of a container. It is a portable package that materializes as an in-memory instance of a container. A key feature is that images are immutable. Once we have defined the image, it cannot be changed. Each container that is based on the image, or a specific version of that image, will be the same. This helps us to guarantee that the environment that worked in development and staging will be present in production. No longer will we need the *it worked on my machine* excuse during deployments.

A base image is an image that acts as a foundation for other images. It starts off using Docker's scratch image, which is an empty container image that does not create a filesystem layer. This means that the image assumes that the application we will be running will run directly from the operating system's kernel.

A parent image is a container image that is a foundational image for other images. It is usually based on an operating system and will host an application that is designed to run on that operating system. For instance, we might need a Redis cache instance on our machine. This Redis cache image will be based on Linux. That is the parent image.

In both cases, the images are reusable, but base images allow us to have more control over the final image. We can always add to the image, but we cannot subtract, since images are immutable.

Docker Hub is a reliable and secure registry of popular and in-demand container images that can easily be pulled onto your machine. The Docker CLI provides a direct connection to Docker Hub, and with a few commands, we can pull images to the host machine from which it was executed.

If we were to set up a Redis cache container, we could do so with a few simple steps. First, we should install Docker on our device, which you can get from www.docker.com. Once it is installed, we can proceed to run the following command in our CLI:

```
docker pull redis
```

This will pull the latest Redis cache application image from the Docker Hub registry and create a container on your machine. Now that we have the image, we can create a container based on the image with a `docker run` command:

```
docker run --name my-redis-container -d redis
```

Now, we have an instance of Redis cache running on the host machine, and we can connect using any Redis cache management tool. When we no longer need this container to be running, we can simply use the `docker stop` command, like this:

```
docker stop my-redis-container
```

Now, this is an example of how we can quickly create an optimized environment for a third-party application, but a major part of why we use Docker is the fact that we can provide containers for our own applications. Before we explore how this is done, we should seek to appreciate the pros and cons of using containers. We will look at these next.

Pros and cons of containers

The benefits of using containerization in our applications are clear. We can take advantage of managing our hosting environments, consistency in how we deliver software, more efficient use of system resources, and software portability.

Recall that containers will only require the exact resources needed for the hosted application to run. This means that we can rest assured that we are not overextending or over-provisioning resources to accommodate a container. We can also benefit from how easy it is to spin up new containers as needed. If we were using VMs, then each application might require an entire machine instance. Containers have a much smaller footprint and require far less to host a new application.

Amidst all these advantages, we need to be aware of the possible drawbacks of using this hosting and deployment method. Containers will share a single operating system and this shared reliance means that we now have a single point of failure or attack. This can be concerning for security teams. Monitoring our applications also becomes a bit more difficult since we have less to work with. Containers generally provide log information to give us insight into the health of the application, but we are not always aware of the additional resources and plugins that are at work, and this makes a total monitoring operation a bit more difficult.

Should we proceed to use containerization in our applications, we need to be familiar with how we author images for the sole purpose of hosting our application. In this case, we need to know and understand how to use a base image and deploy our own application to a new container. For this, we need a Dockerfile, and we will review how we can create one next.

Authoring a Dockerfile

A **Dockerfile** is a text file that outlines how a Docker image should be built. The language used is **Yet Another Markup Language (YAML)**, which is a popular markup language used for configuration files. This file usually contains the following elements:

- A base or parent image to base the new image on
- Commands to update the operating system and additional software and plugins as needed
- Compiled application assets to be included in the image
- Additional container image assets for storage and networking needs
- A command to run the application when the container launches

In our case, we are building a microservices-based application with several web services. This means that we need to author Dockerfiles per web service. Since all our services are ASP.NET Core based, we can use the following example of a Dockerfile as a base example for our appointment web service and others.

To add a Dockerfile to our project, we can use **Visual Studio** and simply right-click our project in **Solution Explorer**, go to **Add**, and then click **Docker Support....**

Figure 13.3 shows the **Docker Support...** option:

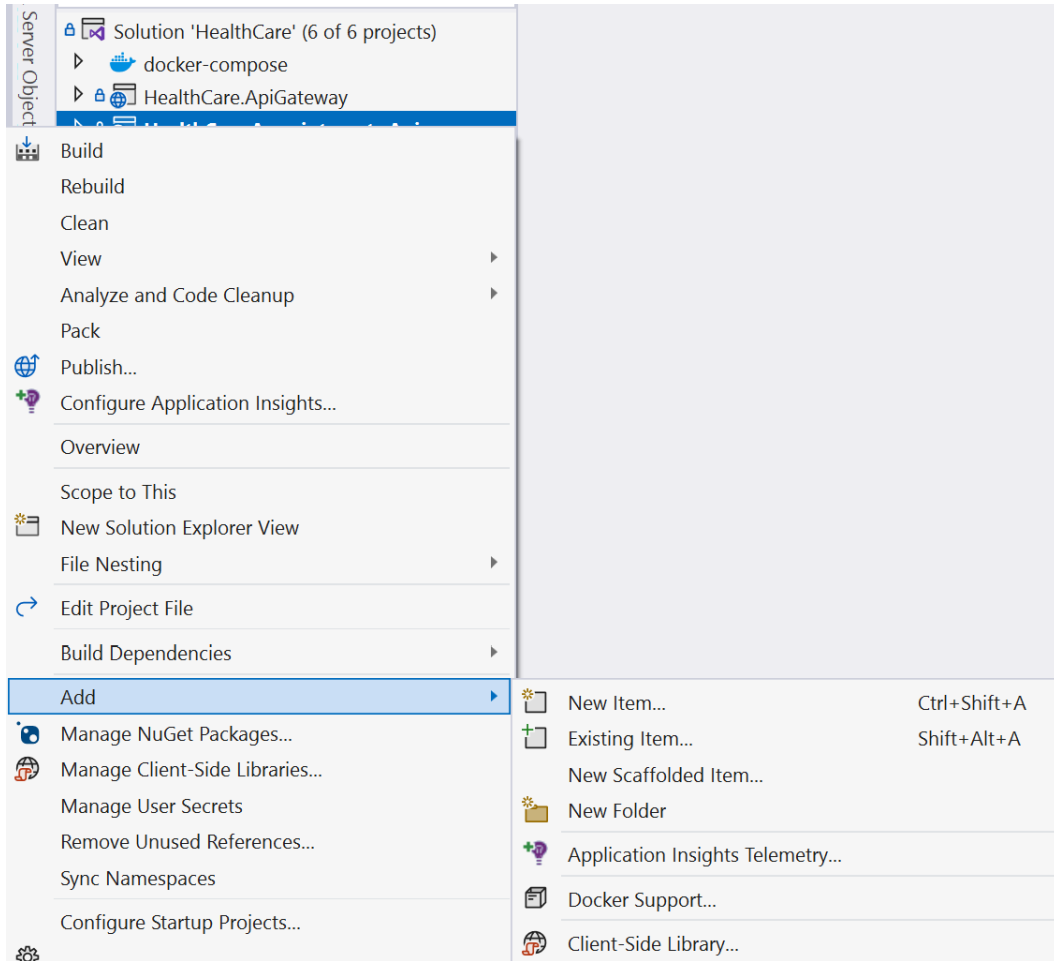


Figure 13.3 – Adding Docker Support in Visual Studio 2022

In **Visual Studio Code**, you can install the official Docker extension provided by Microsoft. Once this extension is installed, you can then add Dockerfiles to your workspace by selecting the **Docker: Add Docker Files to Workspace** command from the command palette. It will then seek to confirm the type of application runtime that we wish to support with our container and confirm if we need `docker-compose` support. Once we complete selecting our options, it will proceed to generate our files.

Both paths will produce two new files in the target project. We get a Dockerfile and a `.dockerignore` file. In the case of the appointments booking service project, if we complete the preceding steps, we end up with a Dockerfile that looks like this:

```
FROM mcr.microsoft.com/dotnet/aspnet:6.0 AS base
WORKDIR /app
EXPOSE 80
EXPOSE 443

FROM mcr.microsoft.com/dotnet/sdk:6.0 AS build
WORKDIR /src
COPY ["HealthCare.Appointments.Api/HealthCare.Appointments.
Api.csproj", "HealthCare.Appointments.Api/"]
COPY ["HealthCare.SharedAssets/HealthCare.SharedAssets.csproj",
"HealthCare.SharedAssets/"]
RUN dotnet restore "HealthCare.Appointments.Api
/HealthCare.Appointments.Api.csproj"
COPY . .
WORKDIR "/src/HealthCare.Appointments.Api"
RUN dotnet build "HealthCare.Appointments.Api.csproj" -c
Release -o /app/build

FROM build AS publish
RUN dotnet publish "HealthCare.Appointments.Api.csproj" -c
Release -o /app/publish /p:UseAppHost=false

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "HealthCare.Appointments.Api.dll"]
```

This Dockerfile has instructions that will create an image for the target service and build the container. The instructions execute the following actions:

1. Use the `mcr.microsoft.com/dotnet/aspnet:6.0` image as the base from which we will derive the rest of our new image.
2. Define which ports we wish to expose from our container, which are the standard web traffic ports for HTTP (80) and HTTPS (443).

3. Define our own content in the image by copying the contents of key files and directories that we need for the app.
4. Perform *build*, *restore*, and *publish* operations to generate binaries, and after the publish operation, run and copy the files from the `publish` directory to the container space.
5. Define *ENTRYPOINT* as the executing binary for the project that will launch our application.

We also get a `.dockerignore` file that outlines files and directories that should not be included in the container when it is created. Its contents look like this:

```
*/.classpath
*/.dockerignore
*/.env
*/.git
*/.gitignore
*/.project
*/.settings
*/.toolstarget
*/.vs
*/.vscode
*/.*proj.user
*/.*.dbmdl
*/.*.jfm
*/azds.yaml
*/bin
*/charts
*/docker-compose*
*/Dockerfile*
*/node_modules
*/npm-debug.log
*/obj
*/secrets.dev.yaml
*/values.dev.yaml
LICENSE
README.md
```

This file is simple to understand, and modifying it is not generally required. It is simply outlining the different areas of the project file composition that it doesn't deem necessary to transport to the container once the application is built and deployed.

Our Dockerfile is a starting point for our own container that will house the target web service. Just to recap on how containers are built, we start off with a *registry*, and this registry contains *images*. In this set of images, we have base images, which are a starting point for all subsequent images. In this case, our first line refers to a base image from which we wish to derive our web service container. Note that the origin and base images of the base image that we ended up using are not visible through this process. The truth is, we don't know the hierarchy behind the `mcr.microsoft.com/dotnet/aspnet:6.0` base image, nor the hierarchy of those images. This approach helps us to make use of the various images that have contributed to our current base image, without needing to make direct references to them or bloat our file with references. We are simply making our own derivative henceforth. This ties in with the goal of keeping our container images small.

Now, let us explore how we can use this Dockerfile going forward.

Launching a containerized application

Now that we have done this process for one of our services, we can easily repeat it for the others. By doing this, we will have completely and successfully containerized each of our web services and, by extension, our microservices application. It is noteworthy as well that Visual Studio and Visual Studio Code will always generate the best Dockerfile to suit the project type that you are working with. We can now also enjoy the new launch feature that gets introduced where we can launch our new containerized web service in a Docker container and still retain real-time analytics and debugging features as if it were running in a normal debugging setting.

In **Solution Explorer**, we can look under the project folder, inspect the folder called `Properties`, and open the file called `launchSettings.json`. This is a JSON configuration file that comes as standard in every ASP.NET Core project, and unless you have a specific reason to, you wouldn't normally open or modify this file. It was, however, modified and given a new launch profile, which informs *Visual Studio* that there is a new way to launch this application for debugging. The file now looks like this:

```
"profiles": {
  "HealthCare.Appointments.Api": {
    // Unchanged Content
  },
  "IIS Express": {
    // Unchanged Content
  }
},
"Docker": {
  "commandName": "Docker",
```

```

    "launchBrowser": true,
    "launchUrl": "{Scheme}://{ServiceHost}:{ServicePort}
      /swagger",
    "publishAllPorts": true,
    "useSSL": true
  }
},

```

This new Docker section was created during our introduction of the Dockerfile, and it now allows us to select **Docker** as a launch option from Visual Studio. This will proceed to execute the instructions outlined in the Dockerfile where it will create a new image based on the base Microsoft image, build, restore, and publish our web project, and then move the files to a freshly created container and execute the application.

The only major difference that we will see in this experience is that the UIs in Visual Studio or Visual Studio Code will show more information regarding the containers, such as their health, version, ports, environment variables, logs, and even the filesystem that the container is using.

Figure 13.4 shows the Visual Studio UI while using Docker for debugging:

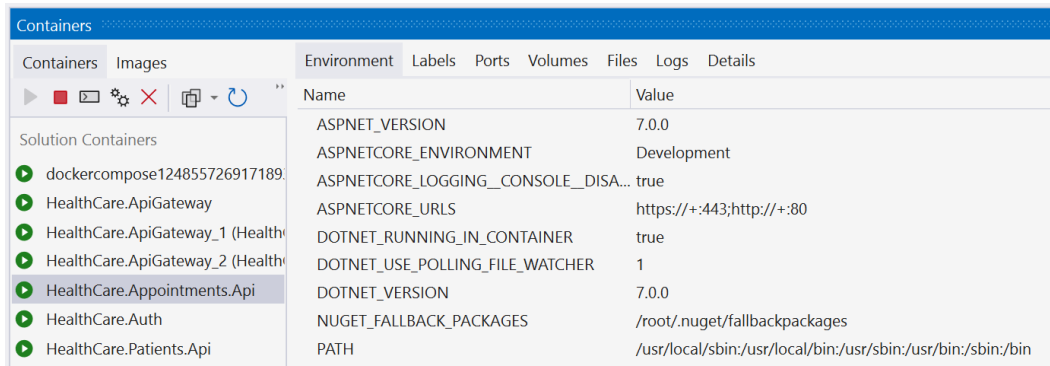


Figure 13.4 – We see Visual Studio in debug mode while containers are in use; it shows us information about the container during runtime

While we are debugging with our Docker containers, you may notice the containers in Docker's **graphical UI (GUI)**, and they show which ports are available for HTTP traffic. We will use these ports to handle the configured port mapping accordingly. If you want to see the running containers using the CLI, you can run this command:

```
docker ps -a
```

This will produce a list of containers, showing their names, ports, and status. Docker has several commands that help us to automate our container operations, such as the following:

- `docker run` starts or creates a container. It takes a `-d` parameter, which is the name of the container to be started.
- `docker pause` will pause a running container and suspend all services and activities.
- `docker unpause` does the opposite of `docker pause`.
- `docker restart` encapsulates the stop and start commands and will reboot a container.
- `docker stop` sends a termination signal to a container and the processes running in the container.
- `docker rm` removes a container and all data associated with the container.

The amazing thing to note here is that using containers has made our application extremely portable and deployable. Now, we do not need to make special configurations on a server and risk one server behaving differently from the other. We can more easily deploy the same environment through the container on any machine, and we can always expect the same outcome.

As of .NET 7, we can containerize our applications without needing a Dockerfile. We will review the steps that can be completed next.

Using native .NET container support

With .NET 7, we have native support for containerization. This means that we can use .NET packages to add container support to our application and then publish our app directly to a container, all without needing Docker.

Container support is made available through the `Microsoft.NET.Build.Containers` package. We can add this package using the following command:

```
Install-package Microsoft.NET.Build.Containers
```

Now that we have our package added, we can publish our application to a container and then use Docker to run the published application. Using the CLI, we can run the following commands:

```
dotnet publish --os linux --arch x64 -c Release -  
p:PublishProfile=DefaultContainer  
docker run -it --rm -p 5010:80 healthcare-patients-  
api:1.0.0
```

Now, our self-hosted container will run and listen for traffic on the configured 5010 port.

Now we see that we have several ways that can lead to us containerizing our applications, we do this for all our other services. This, however, brings a new challenge where we might need to launch our containerized web services in a specific order, or with default values and settings. For this, we need an orchestrator. We have mentioned orchestrators before in the form of Kubernetes, which is an industry-leading metaphorical glove for Docker images (the metaphorical hand) to fit in.

Before we get to Kubernetes, though, Docker provides an orchestration engine that is defined by instructions outlined in a `docker-compose` file. We will explore how this works in the next section.

Understanding docker-compose and images

Before we explore `docker-compose` and how it compiles our images, we need to understand the concept of orchestration and why we need it. Container orchestration is an automated approach to launching containers and related services. In context, when we have a containerized application, we might end up with several containers between our application that has been containerized and third-party applications that we are using as containers.

In the context of our microservices application where we have several individual services, each one is containerized. We might also end up using a containerized cache server, and other supporting services such as email and logging applications. We now need a way to organize our list of containers and launch them in a particular order, such that the supporting applications are available before the dependent service containers are launched.

This requirement introduces a considerable amount of complexity and can make a manual effort very difficult. Using container orchestration, we can make this operation manageable for development and operations. We now have a declarative way of defining the work that needs to be done and the order in which our containers should be launched and with which dependencies. We will now have the following:

- **Simplified operations:** To reiterate, container orchestration vastly simplifies the recurring effort of launching containers in a specific order and with specific configurations.
- **Resiliency:** Can be used to carry out our specific operations based on container health, system load, or scaling needs. Orchestration will manage our instances as needed and automate actions that should be taken in the best interest of the application's stability.
- **Security:** This automated approach helps us to reduce the chance of human error and ensure security in our application.

`docker-compose` is the simplest form of orchestration that we can employ. Alternatives include Kubernetes and Docker Swarm, which are industry-leading options in the container orchestration space.

`docker-compose` is a tool that helps us to define multi-container applications. We can define a YAML file and define the containers that need to be launched, and their dependencies, and then we can launch the application with a single command. The major advantage of `docker-compose` is

that we can define everything about our application's stack in a file and have this defined at the root of our application's folder.

Another advantage here is that if our project is version controlled, we can easily allow for the evolution of this file through outside contributions, or we can share our containers and the launch steps for our application easily with others.

Now, let us review the steps to add `docker-compose` support to our microservices application.

Adding docker-compose to a project

Adding `docker-compose` support to our ASP.NET Core microservices application is easy. In Visual Studio, we can simply right-click one of our service projects, go to **Add**, and select **Container Orchestrator Support...**. This launches a window where we can confirm that we prefer **Docker Compose** and select **OK**. We can select either **Windows** or **Linux** as the target OS. Either OS option works since ASP.NET Core is cross-platform.

Figure 13.5 shows the **Container Orchestrator Support...** option:

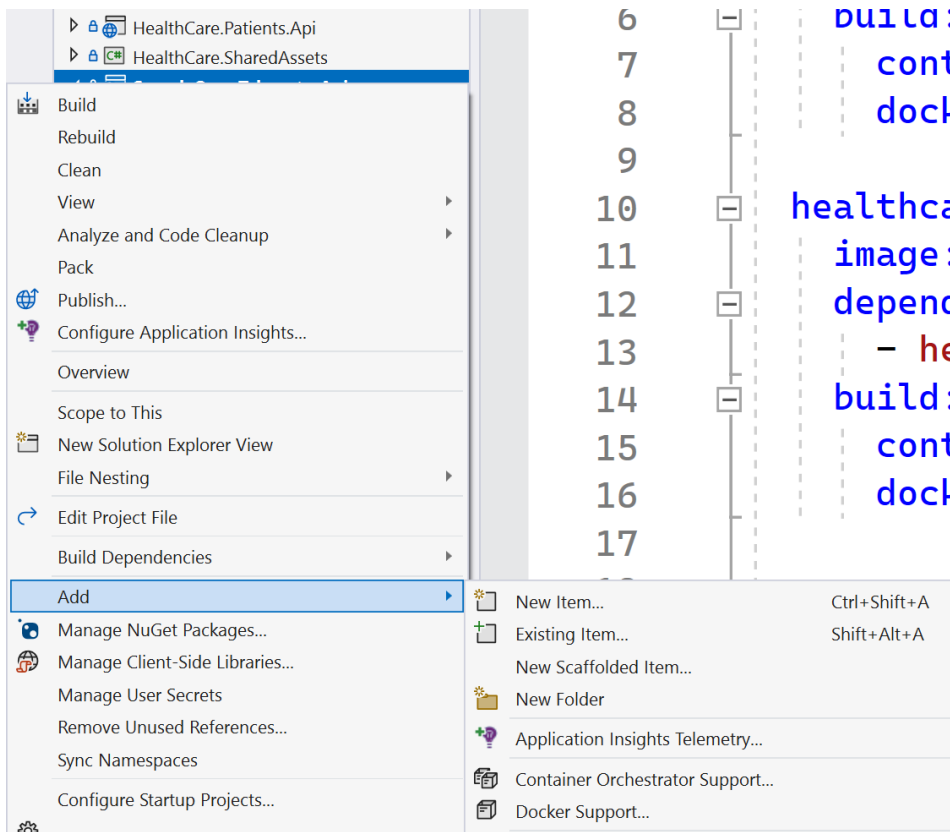


Figure 13.5 – Adding container orchestrator support using Visual Studio 2022

This introduces a new project to our solution where we have a `.dockerignore` file and a `docker-compose.yml` file. If we inspect the `docker-compose.yml` file, we will see that we have a version number and at least one service defined:

```
version: '3.4'
services:
  healthcare.patients.api:
    image: ${DOCKER_REGISTRY-}healthcarepatientsapi
    build:
      context: .
      dockerfile: HealthCare.Patients.Api/Dockerfile
```

Under the definition of our service, we define the containers that we wish to launch and state the name of the image and the Dockerfile that will be a reference point for the image definition. Following similar steps for the other services, Visual Studio will automatically append additional services to the file accordingly. If we proceed to do this with the additional services, we will end up with a `docker-compose` file like this:

```
version: '3.4'
services:
  healthcare.patients.api:
    image: ${DOCKER_REGISTRY-}healthcarepatientsapi
    build:
      context: .
      dockerfile: HealthCare.Patients.Api/Dockerfile

  healthcare.auth:
    image: ${DOCKER_REGISTRY-}healthcareauth
    build:
      context: .
      dockerfile: HealthCare.Auth/Dockerfile

  healthcare.appointments.api:
    image: ${DOCKER_REGISTRY-}healthcareappointmentsapi
    build:
      context: .
      dockerfile: HealthCare.Appointments.Api/Dockerfile
```



```
healthcare.apigateway:
  image: ${DOCKER_REGISTRY-}healthcareapigateway
  build:
    context: .
    dockerfile: HealthCare.ApiGateway/Dockerfile
```

Now, we have a `docker-compose` file that has a record of each container that needs to be launched in our microservices application. This file can now be extended to include additional containers as required by the entire application. If we need a Redis cache instance, we can add a command to this `docker-compose` file to launch a container for Redis. This is what it would look like:

```
services:
  redis:
    image: "redis:alpine"
  ... Other services ...
```

This addition to the file will simply spin up a Redis cache container using the defined base image. Note that the Redis cache image composition can be extended to use storage volumes, and we can pass specific configuration files to our image and indicate that we wish to persist the container's information in a data volume. When we restart this image, the data from the last run will still be available.

We might also want to indicate that some containers should be available before others are started. This could come in handy if a container has a dependency on the Redis cache container or even on another service. For this, we can add another node called `depends_on`, which will allow us to indicate the name of the container that should be launched before we attempt to launch the other. For example, our appointments service communicates with our patients service from time to time. It would be prudent of us to make sure that the patients service is launched before we attempt to launch the appointments service. We can modify the appointment container orchestration to look like this:

```
healthcare.appointments.api:
  image: ${DOCKER_REGISTRY-}healthcareappointmentsapi
  depends_on:
    - healthcare.patients.api
  build:
    context: .
    dockerfile: HealthCare.Appointments.Api/Dockerfile
```

It is possible for us to provide more specific configurations for each image and even provide more specific configurations for our image, without needing to directly repeat the Dockerfile execution. This

is where the `docker-compose.override.yml` file comes into play. It is a nested child item that can be found under the `docker-compose.yml` file, and its contents look like this:

```
version: '3.4'

services:
  healthcare.patients.api:
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=https://+:443;http://+:80
    ports:
      - "80"
      - "443"
    volumes:
      - ${APPDATA}/Microsoft/UserSecrets:/root/.microsoft/
        usersecrets:ro
      - ${APPDATA}/ASP.NET/Https:/root/.aspnet/https:ro
    ... Other overrides...
```

Here, we see that our API's container will be launched with specific environment variables and will have preset ports and volumes. This file can be removed if you prefer to have more explicit control over your containers, but you need to make sure that the relevant values are configured in the Dockerfile.

To revisit our Backend for Frontend pattern from a previous chapter, we outlined that we could provision multiple gateway projects and provide specific configurations for each instance. Now that we have containerization, we can remove the need for additional code projects and reuse the same project while using different configurations for each one. Let us add the following lines to the `docker-compose.yml` file and create two separate containers based on the same gateway image:

```
mobileshoppingapigw:
  image: ${DOCKER_REGISTRY-}healthcareapigateway
  build:
    context: .
    dockerfile: HealthCare.ApiGateway/Dockerfile

webshoppingapigw:
  image: ${DOCKER_REGISTRY-}healthcareapigateway
  build:
```

```
context: .  
dockerfile: HealthCare.ApiGateway/Dockerfile
```

Now that we have defined two new containers based on the same `ApiGateway` image definition, we can add to the `docker-override.yml` file and specify the sources of the specific configuration files that should be used in either case:

```
mobilegatewaygw:  
  environment:  
    - ASPNETCORE_ENVIRONMENT=Development  
    - IdentityUrl=IDENTITY_URL  
  volumes:  
    - ./HealthCare.ApiGateway/mobile:/app/configuration  
  
webhealthcaregw:  
  environment:  
    - ASPNETCORE_ENVIRONMENT=Development  
    - IdentityUrl=IDENTITY_URL  
  volumes:  
    - ./HealthCare.ApiGateway/web:/app/configuration
```

We can specify the volume sources as the paths to the configuration files, but we also took the opportunity to define environment variable values relative to configurations that may vary or need to be set when the container is being created.

In general, we will also want to consider the need for other dependencies such as a database server, message bus provider (such as RabbitMQ), supporting web services and utilities, and so on. All the components of your microservices application can be containerized, and the containers can make direct reference to each other as needed. The `docker-compose.yml` file allows us to outline all these containers and their variables as needed, and we can easily plug in the values accordingly. With a few lines, we can get started, but there are many more avenues to explore and master.

Now, we can comfortably launch our entire microservices application and all the components with one click or command. If using Visual Studio, the button to start with debugging would traditionally have cited the name of one service at a time, or we would need to enable multiple projects to be run at debugging. Now, the presence of the `docker-compose.yml` file overrides the need to be that specific and gives us the ability to run the `compose` command and initialize all our containers in one go. Using the CLI, we can simply run `docker-compose up`.

Now, we have a fully containerized application, and we can simply adjust the parameters used to launch our containers and remain agile and scalable in how we handle our application hosting and

deployments. It also makes it easier to share our applications with our development team and have the members run our applications with only Docker installed as a dependency.

We have also seen that there are base images that allow us to extend and create our own images. There will come a time when we have a particular application or our own version of an application that we wish to preserve and reshare beyond just our project. We can publish our container images to a central repository. We will review this concept next.

Publishing containers to a container registry

A **container registry** is a centralized repository that stores several containers. It allows for remote access to these containers and comes in handy for general development and deployment needs when we need a consistent source for a container image for some applications. Container registries are generally connected directly to platforms such as Docker and Kubernetes.

Registries can save developers time and effort in creating and delivering cloud-native solutions. Recall that a container image contains files and components that make up an application. By maintaining a registry, we can maximize our agile development efforts and deliver incremental image updates efficiently, and with the registry, we can store them in a central area for the team.

A container registry can be private or public. We will discuss our options next.

Public versus private container registries

A registry can be public or private. Public repositories are generally used by developers or teams of developers who either want to provision a registry as quickly as possible or want to publicly share containers that they have developed. These images are then used as base images by others and are sometimes tweaked. This is a great way to contribute to the open source collection of container images. Docker Hub is the largest public repository and community for containers and is the default source of our container images when we do a `docker pull` command.

Private repositories provide a more secure and private way to host and maintain enterprise containers. These types of registries can be remote through established registries such as **Google Container Registry (GCR)**, Microsoft ACR, or Amazon **Elastic Container Registry (ECR)**.

When using a private container registry, we have more control over security and configuration, but we also take on more management responsibilities for access control and compliance within your organization. We need to maintain the following:

- Support for several authentication options within our organization
- **Role-based access control (RBAC)** for images
- Image versioning and maintenance against vulnerabilities
- User activity auditing and logging

We can properly control who is able to upload images through more strict controls and measures and prevent unauthorized access and contributions to the registry.

Recall that a container registry can be hosted locally. This means that we can stage a local instance of Docker Hub on a server and add configurations that are specific to our organization's needs and policies. We can also leverage cloud-hosted container registry services. The advantage of using cloud-hosted solutions is that we can reduce the infrastructural consideration that comes with setting up a local server as well as taking advantage of a fully managed and geo-replicated solution.

Let us review how we can create a customized version of an image and upload it to a registry.

Creating and uploading custom images

We have already created several images to support our application. There are times when we might need a particular application to be deployed in a consistent and repeatable manner, and we don't want to risk recreating the `Dockerfile` or `docker-compose.yml` file.

In this case, we can pull a copy of the base image, add our own configurations and variables to it, and then push the image back to the registry with a new and unique name. Now, other team members—or even you in a future project—can pull this new image at will and leverage the preset environment as needed. For this exercise, we need to have an account on Docker Hub.

Let us use an example of a SQL Server image. If we needed to create an image that has a default database as a starting point, then we could do this in a few steps. First, we pull the base image from Docker Hub with this command:

```
docker pull mcr.microsoft.com/mssql/server
```

SQL Server images tend to be large, so this may take a while to download. Once downloaded, however, we can execute this command to run the container:

```
docker run -e "ACCEPT_EULA=Y" -e  
  "MSSQL_SA_PASSWORD=AStrongP@ssword1" -p 1434:1433 -d  
  mcr.microsoft.com/mssql/server
```

The first command is like what we have seen before, where we pull the image to our local machine. We then run the image and create an instance of SQL Server based on the latest image version and make it available through port 1434. 1433 is the default port for SQL Server, so we can use a different port to avoid conflicts with other SQL instances that might be present. We also set default *sa* user credentials and accept the terms of use agreements.

Now that we have the instance of SQL Server up and running from our image, we can connect using **SQL Server Management Studio (SSMS)** or Microsoft Azure Studio and then run a script. We will keep it simple and create a database and a table:

```
CREATE DATABASE PatientsDb
GO
USE PatientsDb

CREATE TABLE Patients(
    Id int primary key identity,
    FirstName varchar(50),
    LastName varchar(50),
    DateOfBirth datetime
)
```

Now that we have the updated database, let us commit the updated image to a registry. We can use the Docker UI to see the name of the running container for SQL Server or use the `docker ps` command, which will list all the running containers. We then run this `docker commit` command using the name of the running SQL Server container:

```
docker commit -m "Added Patients Database" -a "Your Name"
adoring_boyd Username/NewImageName:latest
```

This creates a local copy of the image based on the current state of the container. We add a `commit` message so that we can keep track of the change that was made and also add an author tag, state the name of the container on which we need to base the image, and then add our Docker Hub username and the new image's name. Now, we can use the Docker UI and click the **Images** option and see our new image created and ready for usage for future projects. We can also use the `docker images` command to list all the images that are currently on the system. You will now see the original SQL Server image as well as our recently committed image.

Now, we have successfully published our own image to our own local registry. If we want to make this image accessible on Docker Hub, we need to use the **Push to Hub** option that is available to us in the Docker UI. Alternatively, we can run the following command:

```
docker push Username/NewImageName
```

Going forward, if we needed to add this database image to our orchestration, we could modify our `docker-compose.yml` file like this:

```
patients_sql_db:
  image: Username/NewImageName
```

```
restart: always
ports:
  1434:1433
```

Now, we will always launch a SQL Server instance with this base database.

We have not only containerized our application, but we have configured orchestration and created our very own custom image. Let us review all that we have learned in this chapter.

Summary

In this chapter, we have reviewed the pros and cons of containerization. We saw where containers help us to downsize some of the resource requirements that our applications have and created a portable version of these applications for development and deployment use.

Docker is an industry-leading containerization software that has a growing community of contributors. Docker can be installed on a machine and then used to manage images and containers as needed. We will also have access to Docker Hub, which is a popular repository for publicly accessible images.

When we integrate Docker into our ASP.NET Core application, we open a new dimension to what it means to build and host our own applications. We can now guarantee that our services will behave in a more consistent manner regardless of the machine they are deployed on. This is because we will have created containers to host our services, and these containers will be optimized to the needs of the service and will never change unless we adjust their definitions.

We also looked at container orchestration, which is where we can outline the containers that we need in one setting and launch them at once, or in a particular order according to the dependencies. This is perfect for our microservices application, which comprises several services and dependencies and would be tedious to launch individually.

Finally, we reviewed how to create our own image and host it on a local container registry. We could then publish the custom image to a public registry, Docker Hub, and have it accessible to all. Now, we can create specific containers with versions of our applications that we need to share with our teams, and we can better control the container versions that are distributed and used.

In the next chapter, we will review a major cross-cutting concern in application development, which is log aggregation in microservices.

Implementing Centralized Logging for Microservices

One of the biggest challenges with APIs is the fact that we rarely get actual feedback on what is happening in our service. We do our best to design our services in a way that our HTTP responses indicate the success or failure of each operation, but this is not always enough. The most concerning types of responses are those in the 5xx range, without any useful information behind them.

For this reason, we need to employ logging in our microservices application. Logging produces real-time information on the operations and events occurring in the service. Each log message helps us to understand the behavior of the application and aids with our investigations when things go wrong. So, the logs are the first line of defense against the ambiguous 5xx HTTP responses.

During development, logs help us to contextualize some of the issues that we face and give us, when implemented well, a play-by-play sequence of the functions being called and their output. This will help us to more easily discern where our code might be breaking or why a function's output is less than desired.

In the case of a distributed application, we need to implement special measures that help us to centralize the logs being produced by the individual services. Yes, we are promoting autonomy in this kind of architecture, but all the components still combine to produce one application. This makes the point of failure more difficult to find if we need to sift through several log files and sources.

After reading this chapter, we will be able to do the following:

- Understand the use of log aggregation
- Know how to implement performance monitoring
- Know how to implement distributed tracing

Technical requirements

The code references used in this chapter can be found in this project's repository, which is hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET/tree/master/Ch14>.

Logging and its importance

Logs are blocks of text that most applications produce during runtime. They are designed to be human-readable mini-reports about what is happening in the application, and they should allow us to be able to track and trace errors that occur in our applications.

In a monolithic application, we typically write logs to a log file or a database. In fact, in .NET Core, we have access to powerful logging providers and third-party libraries that allow us to integrate with several log output destinations. There is no real best destination and while some work better than ours, it is a matter of project preference and overall comfort. Our monolithic logs contain information about everything happening in one application.

In a distributed system, this becomes more complex since we have activities happening across several applications. The first inclination is to create logs per service, which might result in several log files, each containing bits of the overall information that we need to see for the application. Imagine needing to visit several log files to investigate a failure that occurred at 5:00 P.M. To understand this failure, we will need to review several sources to piece together anything sensible, which can be a difficult task.

This investigation gets even more difficult if our logs are too verbose. A verbose log reports everything that occurs in the application. Even if we don't report everything, we need to be pointed with what we log to reduce the noise and better highlight the key events that need to be captured.

We will then need a clean way to centralize the logs that are generated across the services. A centralized database may seem like a good idea, but it may lead to resource and table locking if several services are attempting to write logs frequently. We will review the best centralization techniques later. For now, let us focus on deciding what are the best bits of information to log and how that can be achieved in .NET Core.

Choosing what to log

An important decision while implementing logging is what we want to log. Do we want to log a play-by-play sequence of everything that happens in our application, or do we only want to make note of the errors? Different systems have different requirements, and the correct choices depend on how crucial the service is to the overall running of the application.

Now that we have determined the most essential services that need to be monitored more through logs, we need to determine what information will be logged. Recall that we don't want our logs to be too chatty, but we don't want to neglect to place pertinent information in the logs. Too much information can lead to large useless logs and high storage costs, and too little information will give us useless files.

Useful information would include, but is not limited to the following:

- The ID of a resource that is being accessed through an API request
- The different functions being invoked during a request cycle

What we want to avoid is logging sensitive information. We do not want to log the following, for example:

- User credentials during an authentication flow
- Payment information
- Personally identifiable information

Despite our best security efforts, logs remain a source of truth about our system, and a security breach on log files that contain sensitive information could prove to be a detrimental event. There are legislations and data protection laws that govern how we should store and secure our logs. Therefore, it is better to log just IDs that can be looked up on demand, without giving any information away upfront.

We also have the concept of *log levels*, which are a categorization of the severity of a log message. These levels are split into *information*, *debug*, *warning*, *error*, and *critical*. There might be other categories in between, but these are the most used ones. This is what they stand for:

- **Information:** A standard log level that is used when something has happened as expected. They are generally used to share information about an operation and can be useful in helping to trace the sequence of operations that might have led to an error.
- **Debug:** A very informational log level that is more than we might need for everyday use. It is mostly useful for development and helps us to track more intricate operations in the code. Production systems generally do not produce debug logs.
- **Warning:** This log level indicates that something has happened that isn't an error but isn't normal. Think of it as an amber light that suggests that some attention should be given to a situation, but it might not be mission critical.
- **Error:** An error is an error. This type of log entry is usually created when an exception is encountered. This can be paired with the exception and stack trace and proves to be a critical type of log to have when debugging issues.
- **Critical:** This indicates that we encountered an error that we cannot recover from. This kind of log entry can be used when the application fails to start or fails to connect to a critical data source or dependency.

Log levels are a universal language, and we should ensure that we accurately represent the situation being logged with the appropriate log level. We also want to avoid misclassifying our events and logging misleading information about what has happened in our system.

Once again, the ultimate decision on what is logged is relative to the application, developer's, and organization's needs and we need to ensure that we are thorough enough to capture the essential bits about the application's runtime. Now, let us review how we implement logging in to a .NET Core application.

Using the .NET logging API

.NET has a built-in logging mechanism that is baked into our application startup operation. We get an out-of-the-box logging library that produces logs on all the happenings of our application as soon as it is started. This mechanism works with several third-party logging providers, making it extensible and powerful just the same. With our providers, we can determine the target destinations for our logs.

We will start with the *ILogger* interface. This interface is an abstraction of the logging API that ships with .NET. It is made available to us through the `Microsoft.Extensions.Logging` NuGet package. This library provides us with the necessary classes, interfaces, and functionality for application logging and has providers for logging to *Console*, *Debug*, *Azure Log Stream*, *EventSource*, and *Windows Event Log*:

- **Console:** This provider outputs logs to the console. A console window appears when debugging and most IDEs (contextually, Visual Studio and Visual Studio Code) provide a debugging console window for runtime logs.
- **Debug:** This provider writes log entries using the `System.Diagnostics.Debug` class.
- **EventSource:** A cross-platform provider that can act as an event source with the name *Microsoft-Extensions-Logging*.
- **Windows Event Log:** A Windows-only provider that sends log output to the Windows Event Log. It will only log *Warning* level messages by default but can be configured as needed.
- **Azure Log Stream:** Azure Log Stream supports viewing logs from the Azure portal during application runtime. We can easily write logs to this provider.

To get our .NET application to begin writing logs, we can simply inject `ILogger<T>` into the class from which logs should originate. `T` represents the name of the class that we are injecting the service into. This helps with log categorization and filtering later because the logs will automatically indicate the class name when they are generated. `ILogger<T>` is usually used by application code, which may exist in several places. Because the class name is being used as a *category*, it makes it easy for us to link the log entries back to the class that produced them. In the following code, we are injecting `ILogger<T>` into our appointments service controller:

```
public class AppointmentsController : ControllerBase
{
    /* Other fields */
    private readonly ILogger<AppointmentsController>
```

```
        logger;

        public AppointmentsController(/* Other Services */,
            ILogger<AppointmentsController> logger)
        {
            this.logger = logger;
        }
    }
```

Injecting `ILogger<T>` is standard compared to how we inject other services. The benefit of now having this logger present is that we can write logs to inform of the activities and errors in our API. If we need to log each time a list of appointments is retrieved through an API call, we can modify our GET method like this:

```
// GET: api/Appointments
[HttpGet]
public async Task<ActionResult<IEnumerable
    <Appointment>>> GetAppointments()
{
    Logger.LogInformation("Returning
        Appointments");
    return await _context.Appointments
        .ToListAsync();
}
```

Now, when we make requests to the GET method for this service, we will see a message that looks like this appear in our console. Here, “console” refers to the console window that launches and shows us startup messages about the running .NET application, as well as the debug output in the IDE we are using:

```
HealthCare.Appointments.Api.Controllers.AppointmentsController:
Information: Retrieving appointments
```

Note that we can see not only the source calls but its namespace, which also plays a big role in helping us to determine which exact class is producing the log. We also get a log-level flag so that we can tell the severity at first glance. You will also notice that there are many other default log entries that we didn’t orchestrate. We can control the global levels and sources of logs that we wish to have in our application through the `appsettings.json` file. By default, it will have the following configuration:

```
"Logging": {
  "LogLevel": {
```

```
        "Default": "Information",  
        "Microsoft.AspNetCore": "Warning"  
    }  
},
```

This specifies that the minimum default level that should be output to the destinations is `Information` for the default logging source. Anything related to the inner workings of our application should only surface when it is a `Warning`. If we modified and placed them both at `Information`, then our logs would become far more chatty from the get-go.

Our `LogLevel` methods have a standard layout that allows us to easily include additional information as needed. The possible parameters are as follows:

- **eventId**: A numeric value that you would associate with an action in your application. There is no predefined standard for this, and you can assign your own values based on your organization and needs. This is an optional parameter but can be useful when we need to associate logs with a particular action.
- **Exception**: This is most useful when we are logging an error or critical message and wish to include the details of the exception in the log. It is optional, but it is strongly suggested that you include it whenever an exception object is available, such as after catching an exception in a `try/catch` block.
- **Message**: Message is a straightforward text field. It is arguably the most important one as it allows us to express our custom thoughts on the event being logged. It is a parameterized string that allows us to use placeholders and provide variable values, without needing to concatenate or interpolate the string. An example of such a text looks like *"Logging request from {username}"*. The value of `{username}` should then be provided in `messageArgs`.
- **messageArgs**: This is an array of objects that will bind to placeholders that are outlined in the message string. The binding will occur in the order that the parameters appear, so the values should be provided in that order as well. If a value is not provided, the placeholder will be printed out in the string.

An example that uses all the parameters could look something like this:

```
public async Task<ActionResult<AppointmentDetailsDto>>  
    GetAppointment(Guid id)  
    {  
        try  
        {  
            var appointment = await
```

```
        _context.Appointments.FindAsync(id);

        if (appointment == null)
        {
            return NotFound();
        }
        // Other service calls
        var patient = await _patientsApiRepository
            .GetPatient(appointment
                .PatientId.ToString());
        var appointmentDto = _mapper.Map
            <AppointmentDetailsDto>(appointment);
        appointmentDto.Patient =
            _mapper.Map<PatientDto>(patient);
        return appointmentDto;
    }
    catch (Exception ex)
    {
        logger.LogError(100, ex, "Failure
            retrieving appointment with Id: {id}",
                id);
        throw;
    }
}
```

Here, we added exception handling to our endpoint, which gets the appointment record by ID. If there is an exception in carrying out any of the operations, we catch it and log it. We are using 100 as the `eventId` property for this operation, and we are logging the exception and including a custom message with some more information to help us contextualize the nature of the exception. We also included the ID of the record that caused the failure; notice the `{id}` placeholder that will map to the `id` argument. Giving your parameters the same name is not necessary, but it does help to reduce any confusion with the value bindings.

If we want to extend the number of providers that should be used for each log message, we can configure the logging settings in the `Program.cs` file of our application. In a standard boilerplate ASP.NET Core project, we can add code that looks like this:

```
builder.Logging.ClearProviders();
builder.Logging.AddConsole();
```

```
.AddEventLog(new EventLogSettings { SourceName =  
    "Appointments Service" })  
.AddDebug()  
.AddEventSourceLogger();
```

First, we must clear any preconfigured providers and then add all the providers that we wish to support. Now, one log message will be written to several destinations. This can be a convenient way to fan out our log messages and have different monitoring methods attached to each destination. Remember that you should always be aware of the information security rules that govern your country and company and try not to expose too much information in too many places. We can also provide specific configurations per provider by modifying the logging configuration in the `appsettings.json` file, like this:

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Error",  
      "Microsoft": "Warning",  
      "Microsoft.Hosting.Lifetime": "Warning"  
    },  
    "Debug": {  
      "LogLevel": {  
        "Default": "Trace"  
      }  
    },  
    "Console": {  
      "LogLevel": {  
        "Default": "Information"  
      }  
    },  
    "EventSource": {  
      "LogLevel": {  
        "Microsoft": "Information"  
      }  
    },  
    "EventLog": {  
      "LogLevel": {  
        "Microsoft": "Information"  
      }  
    }  
  }  
}
```

```
    }  
  },  
}  
}
```

We retain our default `LogLevel` for each log source, but then provide overrides per provider. If the log source is not defined underneath the provider's configuration, then it will retain the default behaviors, but we do reserve the right to control what type of log each provider should prioritize.

If we need to extend support to Azure systems, we could make use of the Azure application's file storage and/or blob storage. We can start by including `Microsoft.Extensions.Logging.AzureAppServices` via the NuGet package manager and then we can configure the logging services with code like this:

```
builder.Logging.AddAzureWebAppDiagnostics();  
builder.Services.Configure<AzureFileLoggerOptions>  
    (options =>  
    {  
        options.FileName = "azure-log-filename";  
        options.FileSizeLimit = 5 * 2048;  
        options.RetainedFileCountLimit = 15;  
    });  
builder.Services.Configure<AzureBlobLoggerOptions>  
    (options =>  
    {  
        options.BlobName = "appLog.log";  
    });
```

This will configure the app to use the filesystem as well as Blob storage in Azure. Based on App Services logs settings, there are some defaults that we can look for in terms of log output locations. Similarly, we can override the default logs that are being outputted by this provider by adding sections to the `appsettings.json` file with the aliases `AzureAppServicesBlob` and `AzureAppServicesFile`. We can also define `ApplicationInsights` if we intend to use that service for our application. To support `ApplicationInsights`, we need the `Microsoft.Extensions.Logging.ApplicationInsights` NuGet package. Azure Application Insights is a powerful log aggregation platform provided by Microsoft Azure and is an excellent choice for an Azure-hosted solution.

Several third-party frameworks extend the capabilities of the built-in logging API of .NET. We will explore how to integrate a popular one called **Serilog** in the next section.

Adding Serilog

Several third-party frameworks exist that extend the logging capabilities and options available to us in our .NET applications. Popular options include **Serilog**, **Loggr**, **Elmah.io**, and **NLog** to name a few. Each one has its pros and cons, but in this section, we will be exploring Serilog, how we can integrate it into our application, and what options it introduces to us.

Serilog extends `Microsoft.Extensions.Logging` and provides quick and fairly easy ways to override the default settings while retaining the full power and flexibility of the original framework. To get started, we need to install the `Serilog.AspNetCore` package. For non-web .NET Core projects, we need to use `Serilog.Extensions.Hosting`:

```
Install-package Serilog.AspNetCore
```

Serilog has the concept of using sinks. A sink is like a logging provider in concept and represents an output channel for the logs being written by the framework. We need to add additional packages per sink that we wish to support. Popularly used sinks include *Console*, *File*, *Seq*, *SQL Server*, and *Azure Application Insights*, just to name a few. You can get a complete list from their GitHub wiki page: <https://github.com/serilog/serilog/wiki/Provided-Sinks>.

For this exercise, we will be configuring Serilog to use the file and console sinks. We will also be adding parameters to our `appsettings.json` file. We will need the *expressions* extension to the base library to support parsing the JSON text into the required settings:

```
Install-package Serilog.Expressions
```

Now, we can remove the `Logging` section from our `appsettings.json` file and replace it with the following JSON text:

```
"Serilog": {
  "MinimumLevel": {
    "Default": "Information",
    "Override": {
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "WriteTo": [
    {
      "Name": "File",
```

```
        "Args": { "path":  "./logs/log-.txt",  
                  "rollingInterval": "Day" }  
      }  
    ]  
  },  
}
```

Now, we have a similar structure where we can define logging defaults for the application, but we also have a `WriteTo` section that allows us to list the different channels that we want to support. We have only included the settings for `File` writes, and we have specified the target location to be a local folder called `logs`. Files will be created daily and automatically be given a name that is a combination of the `log-` expression and a date. This will make it easy for us to source the relevant files by day and each log will indicate a timestamp, making it easier to review the events.

Now, we can remove the `builder.Logging(...)` configuration and replace it with this:

```
builder.Host.UseSerilog((ctx, lc) => lc  
    .WriteTo.Console()  
    .ReadFrom.Configuration(ctx.Configuration));
```

This will initialize our logger to use the `Console` sink and read the configuration object for the Serilog section defined previously. This will initialize the `Console` and `File` sinks. We can now look forward to seeing text files getting created and populated daily for each microservice that has implemented the file logging configuration for Serilog. The code that is needed to write the logs remains the same. We only need to repeat the injection steps outlined for `ILogger<T>`; Serilog will do the rest.

Now, we have solved one issue where we are no longer blind to what is happening in our application. We can easily integrate logging into our services and review a more persistent output in the form of log files. However, we still have the challenge of needing to review several disparate logs across different systems to properly track what may have led to a failure at some point.

This is where we need to explore methods of aggregating the logs and having them visible and searchable from one interface. We will explore how this can be done in the next section.

Log aggregation and its uses

Log aggregation is the concept of capturing and consolidating logs from different sources into a centralized platform. This comes in handy in a distributed system where logs are generated from several sources, and we need a comprehensive view of all the messages and need to correlate and analyze the logs efficiently.

This log aggregation acts as a single source of truth when we need to troubleshoot application performance and errors or identify bottlenecks and points of failure or vulnerability. Several platforms allow us to aggregate our logs and they range from free to paid to cloud-hosted solutions. Some popular ones

are *Azure Application Insights*, *Seq*, *DataDog*, and *ELK* (*Elasticsearch*, *Logstash*, and *Kibana*), to name a few. In selecting a platform, we must consider the following:

- **Efficiency:** We all love and want efficient systems. The platform that we choose needs to feed into this narrative and make it as easy as possible to integrate logging, export log information in various formats, and sift and sort through log information as quickly as possible. Most log aggregators allow us to author queries that can intelligently sift through the logging noise and give us more pointed data.
- **Processing power:** The platform needs to allow for comfortable throughput from several sources and be able to index, compress, and efficiently store these logs. We may not necessarily know their techniques to achieve this, but we can assess the accuracy of the indexing functions through our queries and the overall presentation of the data.
- **Real-time features:** Real-time monitoring is very important since we generally need log aggregation to monitor what is happening in our application. The more quickly the information is made available to us, the more quickly we can respond to a failure.
- **Scalability:** The platform needs to be capable of handling varied periods of traffic and not breaking when there is a sudden shift in volume. We need to ensure that the performance of the system does not degrade under increased load.
- **Alerting mechanisms:** Some platforms have the built-in functionality to alert us to certain types of log events. Even if this is not built-in, we should have integration options through *APIs* and *WebHooks* that allow us to integrate with our third-party applications, which is where we spend most of our time.
- **Security:** Security is very important for our logging information, as we have mentioned previously. An ideal platform will encrypt data while it is at rest and while it is in transit. This is often a given, but we need to make sure. We may also need to be able to control user access.
- **Cost:** We all love free and cheap solutions. We cannot always have the best of both worlds but we can be sure that the platform offers a good return on the investment, relative to the features that we are being offered. Ensure that you do a proper cost-benefit analysis.

The easier way to integrate with a log aggregation platform is through tools and packages that are optimized for that kind of integration. We need to employ the services of libraries that are tuned to efficiently integrate with these platforms. In the next section, we will see how we can leverage *Serilog* and integrate with *Seq*.

Integrating with Seq

Seq, pronounced seek, is a sleek (see what I did there?) log aggregation developed and maintained by *Datalust*. This platform has been developed to support log messaging templates output by *ASP.NET Core*, *Serilog*, and *Nlog*. It can also be extended to support other development frameworks as needed.

It gives us access to a powerful dashboard with leading data presentation and querying features. *Seq* can be installed on a local machine for free for individual development but will attract some costs as we look to use it in a more enterprise setting. It also offers a hosted solution, which removes the need for users to set it up locally.

For this activity, we will use it locally and for free on our machine. We now have two options; we can use a *Docker* image and spin up a container for the application or install it on our local machine. It is available for *Windows* and *Linux* operating systems, so we will use the *Docker* option to cater to all scenarios.

We will start by downloading the Docker image with this command:

```
docker pull datalust/seq
```

Now that we have the latest Seq image, we will create our container with this command:

```
docker run --name seq -d --restart unless-stopped -e  
ACCEPT_EULA=Y -p 5341:80 datalust/seq:latest
```

Now, we have a container that hosts an instance of Seq and can be accessed through port 5341, which is Seq's default port. We can now navigate to `http://localhost:5341/#/events` to see our user interface for the aggregator. It will be empty, so now, we need to integrate our API with this new logging channel.

Now that we have Seq up and running, we can modify our service to begin sending logs to this platform. We already have Serilog installed, so we can add the Seq sink to our project by adding this package:

```
Install-Package Serilog.Sinks.Seq
```

With this new package, we can modify our `appsettings.json` Serilog section and add a new object block to the `WriteTo` section of the configuration. It will now look like this:

```
"WriteTo": [  
  {  
    // File Configuration  
  },  
  {  
    "Name": "Seq",  
    "Args": { "serverUrl": "http://localhost:5341" }  
  }  
]
```

We already have the configuration section being read at application startup, so the next time that the application starts up, all the default logs will be written to our local file as expected, but now also the Seq platform.

Figure 14.1 shows the Seq interface:

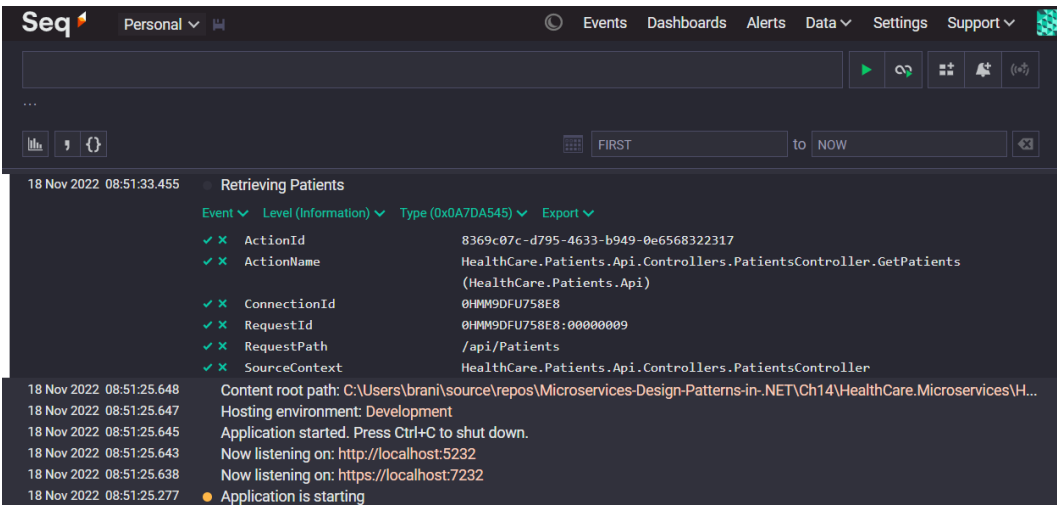


Figure 14.1 – The Seq interface receiving logs from a microservice

Here, we can see the user interface outlining the default logs that get produced at application startup. What appears in this interface is relative to the logging configurations that we have added, as well as the log entries that we make as we go along. You will also notice some color-coded dots, which indicate the log level for the log entry. We can click on a line and expand it to see the details of the log message.

Now, these code modifications can be made to all services that we wish to add to the log aggregation initiative, and we can use this unified platform to interrogate logs as needed. With this in place, we need to understand the concept of log tracing in a distributed setting. We will discuss this next.

Distributed log tracing

Distributed tracing is the method of monitoring logs and tracing issues in a microservices application. Developers and DevOps engineers both rely on logs to follow the path of a request as it travels through the various systems and checkpoints and then attempts to ascertain the point of failure. The more robust the logging, the easier it will be for them to pinpoint weak points, bugs, and bottlenecks in the application.

Because microservices are designed to be autonomous and scale independently, it is common to have multiple instances of the service running simultaneously, which further complicates the request tracing process. We now need to backtrack which instance of the service handled the request, leading to even more complex tracing situations.

Distributed tracing is a technique that is geared toward solving these problems. It refers to a diagnostic methodology behind observing requests as they make their way through distributed systems. Each trace shows the activity of an individual user in the application. In an aggregated logging system, we will end up with a collection of traces that highlight the backend service and dependencies that have the biggest impact on performance. In distributed tracing, we have three main factors that help us to find our way around:

- **Trace:** Represents an end-to-end request from user activity.
- **Span:** Represents work done by a single service in a specific period. Spans combine to form a trace.
- **Tags:** Metadata about the span that helps us to properly categorize and contextualize the logs.

Each span is a step in the entire journey of the request and is encoded with important data relating to the process that is carried out in the operation. This information can include things such as the following:

- The service's name and address
- Tags that can be used in queries and filters, such as the HTTP method, database host, and session ID, to name a few
- Stack traces and details error messages

.NET has evolved over the years to provide top-notch support for producing logs with these details as seamlessly as possible, through integrations with *OpenTelemetry*. Microsoft Azure also provides an excellent distributed tracing platform in *Azure Application Insights*, which is a platform that we mentioned previously. There are many other paid and open source solutions that can support our distributed tracing needs. For this chapter, we will use a free and simple platform called Jaeger. Let us explore how we can add telemetry enhancements to our services and visualize them with Jaeger.

Enhanced logging for distributed tracing

OpenTelemetry is a popular open source project that is at the helm of standardizing logging standards for distributed and cloud-native applications. It helps us to generate and collect detailed logs, also called telemetry data, that contain traces and metrics. Given that it is an open standard, we are at liberty to choose a suitable visualization and analysis tool.

To install *OpenTelemetry* in our ASP.NET Core application, we need to execute the following commands in `dotnet cli`:

```
dotnet add package --prerelease
    OpenTelemetry.Instrumentation.AspNetCore
dotnet add package OpenTelemetry.Exporter.Jaeger
dotnet add package --prerelease
    OpenTelemetry.Extensions.Hosting
```

Between these three packages, we are installing ASP.NET Core OpenTelemetry support and support for exporting our telemetry data to a distributed tracing analysis platform called Jaeger. Jaeger is free and can be downloaded in ZIP format or set up as a Docker container. You can learn more here: <https://www.jaegertracing.io/download/>.

Now that we have the packages, we can make the following adjustments to our `Program.cs` file:

```
builder.Services.AddOpenTelemetryTracing((builder) =>
    builder
        .AddAspNetCoreInstrumentation(o =>
            {
                o.EnrichWithHttpRequest = (activity,
                    httpRequest) =>
                {
                    activity.SetTag("requestProtocol",
                        httpRequest.Protocol);
                };
                o.EnrichWithHttpResponse = (activity,
                    httpResponse) =>
                {
                    activity.SetTag("responseLength",
                        httpResponse.ContentLength);
                };
                o.EnrichWithException = (activity,
                    exception) =>
                {
                    activity.SetTag("exceptionType",
                        exception.GetType().ToString());
                };
            })
        .AddJaegerExporter()
    );
```

With this configuration, we are adding OpenTelemetry support to our application's startup and then outlining various enrichments that we wish to include with each message that is sent to Jaeger. Note that OpenTelemetry has support for a few platforms, and you are at liberty to choose the one that best suits your needs. With this configuration, all traffic to our API endpoints will be logged with as many enrichment data points as we specified.

Figure 14.2 shows the Jaeger interface:

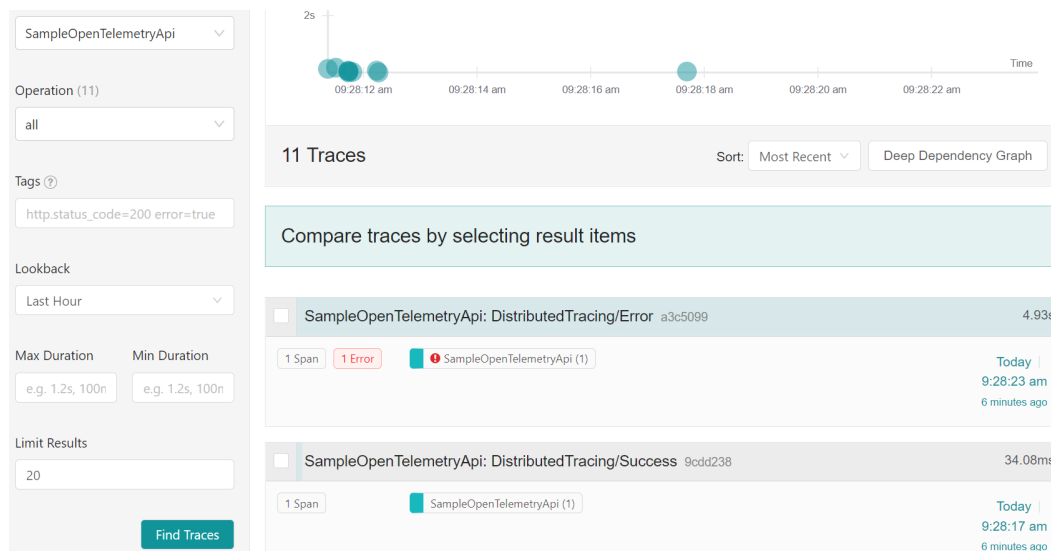


Figure 14.2 – Telemetry data that has been generated and deposited in the Jaeger aggregation platform

Jaeger is simple enough for us to get started and, as depicted in *Figure 14.2*, we can view all the services that send telemetry data, filter based on the operations we need to see, and review data across a specified timeline. These are general features of distributed tracing platforms and, once again, we need to ensure that we choose one that gives us the best value for our needs.

Now that we have explored logging and distributed tracing, let us wrap up this chapter.

Summary

Logging is a simple concept that can save us a lot of time and trouble when reviewing our applications. The ability to write logs ships with .NET Core and we can easily leverage the native capabilities to begin producing log information about the operations of our applications.

We need to ensure that we do not log sensitive information and we must be aware of company and security policies when authoring our logs. We also need to ensure that our logs are stored securely both in transit and at rest. We also can log to multiple channels, but we should be careful when choosing these channels, relative to our security guidelines.

Several .NET Core frameworks enhance the natural capabilities of the built-in API and introduce even more integrations. A popular choice is Serilog, which has many extensions called sinks, which offer us a wide range of simultaneous log channel options. With it, we can easily create and manage log files on a rolling interval that we specify.

Ideally, we will have multiple services writing logs and it will be tedious having each one log to its own file. This will force us to review multiple files to trace one request that might have spanned many touch points in our distributed application. For this reason, we employ the services of an aggregator, which will give our services a central area to deposit logs and give us and our team one area to focus on when sifting through logs.

Then, we ran into another issue where our logs need to contain certain details that allow us to properly associate them with a request. After, we looked at enriching our logs with unique IDs that help us to associate them to a point of origin and each other. This is called distributed tracing. We also reviewed how to include `OpenTelemetry` in our service and the use of a visualization tool to assist with the querying activities.

Now we have finished exploring logging activities and best practices in a distributed system, in the next chapter, we will conclude what we have learned so far.

Wrapping It All Up

At this point, we have discovered several patterns and nuances surrounding microservices design. Now, let us explore our patterns at a high level and tie all the concepts together. It is essential to scope which pattern fits best into each situation.

The microservices architectural approach to software development promotes loose coupling of autonomous processes and creating standalone software components that handle these processes. An excellent approach to scoping these processes is to employ the **domain-driven design (DDD)** pattern. In DDD, we categorize the system's functionality into sub-sections called domains and then use these domains to govern what services or standalone apps are needed to support each domain. We then use the aggregator pattern to attempt to scope the domain objects needed per service.

Aggregator pattern

We scope the data needed in each domain and what data needs to be shared between domains. At this point, we do risk duplicating data points across domains. Still, it is a condition we accept, given the need to promote autonomy across the services and their respective databases.

In scoping the data requirements, we use the aggregator pattern, which allows us to define the various data requirements and relationships the different entities will have. An aggregate represents a cluster of domain objects that can be seen as a single unit. In this scoping exercise, we seek to find a root element in this cluster, and all other entities are seen as domain objects with associations with the root.

The general idea in scoping our domain objects per service is to capture the minimum amount of data needed for each service to operate. This means we will try to avoid storing entire domain records in several services and instead allow our services to communicate to retrieve details that might be domain-specific and reside in another service. This is where we need our services to communicate.

Synchronous and asynchronous communication

Our microservices need to communicate from time to time. The type of communication that we employ is based on the type of operation that we need to complete in the end. Synchronous communication means that one service will directly call another and wait for a response. It will then use this response to inform the process it sought to complete. This approach is ideal for situations where one service might have some data and needs the rest from another. For instance, the appointment booking service knows the patient's ID number but has no other information. It will then need to make a synchronous API call to the patients' service and GET the patient's details. It can then carry one to process those details as necessary.

Synchronous communication is great when we need instant feedback from another service. Still, it can introduce issues and increase response time when several other services must be consulted. We also run the risk of failures with each API call attempt, and one failure might lead to a total failure. We need to handle partial or complete failures gracefully and relative to the rules governing the business process. To mitigate this risk, we must employ asynchronous communication strategies to hand it off to a more stable and always-on intermediary that will transport data to the other services as needed.

Asynchronous communication is better used in processes that need another service's participation but not necessarily immediate feedback. The process of booking an appointment, for instance, will need to complete several operations that involve other microservices and third-party services. The process, for instance, will take and save the appointment information, make a calendar entry, and send several emails and notifications. We can then use an asynchronous messaging system (such as RabbitMQ or Azure Service Bus) as the intermediary system that will receive the information from the microservice. The other services that need participation are configured to monitor the messaging system and process any data that appears. Each service can then individually complete its operation in its own time and independently. The appointment service can also confirm success based on its needs without worrying about whether everything has been done.

As we separate our business process and scope and figure out which operations require synchronous communication and which ones require asynchronous communication, we find that we need better ways to format our code and properly separate the moving parts of our application's code. This is where we begin looking at more complex design patterns such as **Command and Query Responsibility Separation (CQRS)**.

CQRS

CQRS is a popular pattern employed to allow developers to better organize application logic. It is an improvement to the originally drafted pattern called **Command Query Separation (CQS)**, which sought to give developers a clean way to separate logic that augments data in the database (commands) from the logic that retrieves data (query).

By introducing this level of separation, we can introduce additional abstractions and adhere to our SOLID principles more easily. Here, we introduce the concept of handlers, which represent individual

units of work to be done. These handlers are implemented to specifically complete an operation using the minimum needed and fewest number of dependencies. This allows the code to become more scalable and easier to maintain.

One downside to introducing this level of separation and abstraction is a major increase in the number of files and folders. To fully implement CQRS based on the recommended approach, we might also have several databases to support a single application. This is because the database used for the query operations needs to be optimized, which usually means we need a denormalized and high-speed lookup database structure. Our command operations might use a different database since storing data generally has stricter guidelines than reading it.

Using the **MediatR** pattern with CQRS helps us more easily refer to the specific handlers needed without introducing too many lines of code to make a simple function call. We have access to NuGet packages that help us to easily implement this pattern and reduce our overall development overhead.

Ultimately, this pattern should be leveraged for applications that have more complex business logic needs. It is not a recommended approach for standard applications that do the basic **Create, Read, Update, and Delete (CRUD)** operations, given the complexity level and project bloat it brings with it from an application code and supporting infrastructure perspective. It also introduces a new problem: keeping our read and write databases in sync.

Let us take the approach where we use separate databases for query and command operations. We run the risk of having out-of-date data available for read operations in between operations. The best solution for the disconnect between the databases is called event sourcing.

Event sourcing patterns

Event sourcing patterns bridge the gap between databases that need to be in sync. They help us track the changes across the system and act as a behind-the-scenes transport or lookup system to ensure that we always have the best data representation at any time.

First, an event represents a moment in time. The data contained in the event will indicate the type of action taken and the resulting data. This information can then be used for several reasons within the system:

- Complete tasks for third-party services that need the resulting data for their operations
- Update the database for query operations with the latest copy of the augmented record
- Add to an event store as a versioning mechanism

Event sourcing can play several roles in a system and can aid us in completing several routines and unique tasks. Routine tasks within the context could include updating our read-only query database and acting as a source of truth for services that need to be executed based on the latest data after an operation.

Less routine operations would depend on implementing an event store, another database provisioned to keep track of each event and its copy of the data. This acts as a versioning mechanism that allows us to easily facilitate auditing activities, point-in-time lookups, and even business intelligence and analytics operations. By keeping track of each data version over time, we can see the precise evolution of the records and use it to inform business decisions.

Not surprisingly, this pattern works naturally with CQRS, as we can easily and naturally trigger our events from our handlers. We can even use the event store as our query database lookup location, easing the tension associated with reading stale data. We can then extend our query capabilities and leverage the version and point-in-time lookups we now have access to.

Through the previously mentioned NuGet packages that allow us to implement the MediatR pattern, we can raise events at the end of an operation. We can also implement handlers that subscribe to specific events and carry out their operations once an event is raised. This allows us to easily scale the number of subscribers per event and individually and uniquely implement operations that execute per event.

These patterns are implemented per service and in no way unify code spread across several individual applications. Ensure that the patterns you choose are warranted for the microservice. Between event sourcing and CQRS, we have increased the number of scoped databases from one to potentially three. This can introduce hefty infrastructural requirements and costs.

Now, let us review how we should handle database requirements in our microservices application.

Database per service pattern

The microservices architecture promotes autonomy and loose coupling of services. This concept of loose coupling should ideally be implemented throughout the entire code base and infrastructure. Sadly, this is only sometimes possible for cost reasons, especially at the database level.

Databases can be expensive to license, implement, host, and maintain. The costs also vary based on the needs of the service that the database supports and the type of storage that is needed. One compelling reason to have individual databases is that we always want to choose the best technology stack for each microservice. Each one needs to retain its individuality, and the database choice is integral to the implementation process.

We have different types of databases, and it is important to appreciate the nuances between each and use that knowledge to scope the best database solution for the data we can expect to store for each microservice. Let us look at some of the more popular options.

Relational databases

Relational databases store data in a tabular format and have strict measures to ensure that stored data is of the highest possible quality by its standards. They are best for systems that need to ensure data accuracy and might have several entities they need to store data for. They generally rely on a language called SQL to interact with data and, through a normalization process, will force us to spread data

across several tables. This way, we can avoid repeating data and establish references to a record found in one table in other tables.

The downside is that the strict rules make it difficult to scale on demand, which leads to slower read times for data related to several entities.

Non-relational databases

Non-relational databases are also referred to as NoSQL databases, given their structural differences from traditional relational databases. They are not as strict regarding data storage and allow for greater scalability. They are best used for systems that require flexible data storage options, given rapidly changing requirements and functionality. They are also popularly used as read-only databases, given that they support the data being structured acutely to the system's needs. The most popular implementations of these kinds of databases include document databases (such as **MongoDB** or **Azure Cosmos DB**), key-value databases (such as **Redis**), and graph databases (such as **Neo4j**).

Each type has its strengths and weaknesses. The document databases option is most popularly used as an alternative to a relational database, given that it offers a more flexible way to store all the data points but keeps them in one place. This, however, can lead to data duplications and a reduction in overall quality if not managed properly.

When considering the best database option for services, we must consider maintainability, technology maturity and ease of use, and general appropriateness for the task. One size certainly does not fit all, but we must also consider costs and feasibility. We have several approaches to implementing supporting databases for our services; each has pros and cons.

One database for all services

This is the ideal solution from a cost analysis and maintenance perspective. Database engines are powerful and designed to perform under heavy workloads, so having several services using the same database is not the most difficult to implement. The team also doesn't need a diverse skill set to maintain the database and work with the technology.

This approach, however, gives us one point of failure for all services. If this database goes offline, then all services will be affected. We also forfeit the flexibility of choosing the best database technology to support the technology stack that best implements each service. While most technology stacks have drivers to support most databases, the fact remains that some languages work best with certain databases. Be very careful when choosing this approach.

One database per service

This solution provides maximum flexibility in our per-service implementations. Here, we can use the database technology that best serves the programming language and framework used and the microservice's data storage needs. Services requiring a tabular data storage structure can rely on a

relational database. By extension, microservices developed using PHP technology may favor a MySQL database, and using ASP.NET Core may favor Microsoft SQL Server. This will ease the attrition in supporting a database because a language might have less than adequate tooling. On the other hand, a NodeJS-based microservice might favor MongoDB since its data doesn't need to be as structured and might evolve faster than the other services.

The obvious drawbacks here are that we need to be able to support multiple database technologies, and the skill sets must be present for routine maintenance and upkeep activities. We also incur additional costs for licensing and hosting options since the databases may (ideally, will) require separate server hosting arrangements.

Individually, each service needs to ensure its data is as accurate and reliable as possible. Therefore, we use a concept called transactions to ensure that data either gets augmented successfully or not. This is especially useful for relational databases where the data might be spread across several tables. By enforcing this all-or-nothing mechanism, we mitigate partial successes and ensure that the data is consistent across all tables.

Always choose the best technologies for the microservice you are constructing to address the business problem or domain. This flexibility is one of the more publicized benefits of having a loosely coupled application where the different parts do not need to share assets or functionality.

Conversely, having separate databases supporting autonomous services can lead to serious data quality issues. Recall that some operations need the participation of several services, and sometimes, if one service fails to augment its data store, there is no real way to track what has failed and take corrective measures. Each service will handle its transaction, but an operation involving several independent databases will run the risk of partial completion, which is bad. This is where we can look to the Saga pattern to help us manage this risk.

Using the saga pattern across services

The saga pattern is generally implemented to assist with the concept of all-or-nothing in our microservices application. Each service will do this for itself, but we need mechanisms to allow the services to communicate their success or failure to others and, by extension, act when necessary.

Take, for instance, if we have an operation that requires the participation of four services, and each one will store bits of data along the way; we need a way to allow the services to report on whether their database operations were successful. If not, we trigger rollback operations. Two ways we can implement our saga patterns are through choreography or orchestration.

Using choreography, we implement a messaging system (such as RabbitMQ or Azure Services Bus) where services notify each other of the completion or failure of their operations. There is no central control of the flow of messages. Still, each service is configured to act on the receipt of certain messages and publish messages based on the outcome of its internal operation. This is a good model where we want to retain each service's autonomy, and no service needs any knowledge of the other.

Choreography seems straightforward in theory but can be complex to implement and extend when new services need to be added to the saga. In the long run, each time the saga needs modification, several touchpoints will need attention. These factors promote the orchestration approach as a viable alternative.

Using the orchestration method, we can establish a central observer that will coordinate the activities related to the saga. It will orchestrate each call to each service and decide on the next step based on the service's success or failure response. The saga in the orchestrator is implemented to follow specific service calls in a specific order along a success track and, separately, along a failure track. If a failure occurs in the middle of the saga, the orchestrator will begin calling the rollback operations for each service that previously reported success.

Comparably, the orchestrator approach allows for better control and oversight of what is happening at each step of the saga but might be more challenging to implement and maintain in the long run. We will have just as many touchpoints to maintain as the saga evolves.

Your chosen approach should match your system's needs and your desired operational behavior. Choreography promotes service autonomy but can lead to a spaghetti-like implementation for a large saga where we need to track which service consumes which message. This also makes it very difficult to debug. The orchestrator method forces us to introduce a central point of failure since if the orchestrator fails, nothing else can happen.

Both approaches, however, hinge on the overall availability of the services and dependencies involved in completing the operation. We need to ensure that we do not take the first failure as the final response and implement logic that will try an operation several times before giving up.

Resilient microservices

Building resilient services is very important. This acts as a safety net against transient failures that otherwise break our system and lead to poor user experiences. No infrastructure is bulletproof. Every network has failure points, and services that rely on an imperfect network are inherently also imperfect. Beyond the imperfections of the infrastructure, we also need to consider the general application load and the act that our request now might be one too many. This doesn't mean that the service is offline; it just means that it is stressed out.

Not all failure reasons are under our control, but how our services react can be. By implementing retry logic, we can force a synchronous call to another service to make the call again until a successful call has been made. This helps us reduce the number of failures in the application and gives us more positive and accurate outcomes in our operations. Typical retry logic involves us making an initial call and observing the response. We try the call again when the response is something other than the expected outcome. We continue this until we receive a response that we can work with. This very simplified take on retry logic has some flaws, however.

We should only retry for a while since we are unsure if the service is experiencing an outage. In that case, we need to implement a policy that will stop making the retry calls after a certain number of attempts. We call this a circuit-breaker policy. We also want to consider that we want to add some time between the retry attempts.

Policies this complex can be implemented using simple code through a NuGet package called Polly. This package allows us to declare global policies that can be used to govern how our `HttpClient` services make API calls. We can also define specific policies for each API call.

Retries go a long way in helping us maintain the appearance of a healthy application. Still, prevention is better than a cure, and we prefer to track and mitigate failures before they become serious. For this, we need to implement health checks.

Importance of health checks

A health check, as the name suggests, allows us to track and report on the health of a service. Each service is a potential point of failure in an application, and each service has dependencies that can influence its health. We need a mechanism that allows us to probe the overall status of our services to be more proactive in solving issues.

ASP.NET Core has a built-in mechanism for reporting on the health of a service, and it can very simply tell us if the service is healthy, degraded, or unhealthy. We can extend this functionality to report the health of not only the service but to also account for the health of the connections to dependent services such as databases and caches.

We can also establish various endpoints that can be used to check different outcomes, such as general runtime versus startup health. This categorization comes in handy when we want to categorize monitoring operations based on the tools we are using, monitoring teams in place, or general application startup operations.

We can establish liveness checks, which can be probed at regular intervals to report on the overall health of an application that is expected to be running. We act whenever there is an unhealthy result, which will be a part of our daily maintenance and upkeep activities. When a distributed application is starting up, however, and several services depend on each other, we want to accurately determine which dependent service is healthy and available before we launch the service that depends on it. These kinds of checks are called readiness checks.

Given the complexity and often overwhelming number of services to keep track of in a distributed application, we tend to automate our hosting, deployment, and monitoring duties as much as possible. Containerization, which we will discuss shortly, is a standard way of hosting our applications in a lightweight and stable manner, and orchestration tools such as Kubernetes make it easy for us to perform health probes on the services and the container, which will inform us of the infrastructure's health. Ultimately, we can leverage several automated tools to monitor and report on our services and dependencies.

We have spent some time exploring nuances surrounding our microservices and how they relate to each other. However, we have yet to discuss the nuances surrounding having one client or more client applications that need to relate to several services.

API Gateways and backend for frontend

An application based on the microservices architecture will have a user interface that will interact with several web services. Recall that our services have been designed to rule over a business domain, and many operations that users complete span several domains. Because of this, the client application will need to have knowledge of the services and how to interact with them to complete one operation. By extension, we can have several clients in web and mobile applications.

The problem is that we will need to implement too much logic in the client application to facilitate all the service calls, which can lead to a chatty client app. Then, maintenance becomes more painful with each new client that we introduce. The solution here is to consolidate a point of entry to our microservices. This is called an API gateway, and it will sit between the services and the client app.

An API gateway allows us to centralize all our services behind a single endpoint address, making it easier to implement API logic. After a request is sent to the central endpoint, it is routed to the appropriate microservice, which exists at a different endpoint. The API gateway allows us to create a central register for all endpoint addresses in our application and add intermediary operations to massage request and response data in between requests as needed. Several technologies exist to facilitate this operation, including a lightweight ASP.NET Core application called **Ocelot**. As far as cloud options go, we can turn to Azure API Management.

Now that we have a gateway, we have another issue where we have multiple clients, and each client has different API interaction needs. For instance, mobile devices will need different caching and security allowances than the web and smart device client apps. In this case, we can implement the backend for frontend pattern. This is much simpler than it sounds, but it needs to be properly implemented to be effective and can lead to additional hosting and maintenance costs.

This pattern behooves us to provide a specially configured gateway to cater to the targeted client app's needs. If our healthcare application needs to be accessed by web and mobile clients, we will implement two gateways. Each gateway will expose a specific API endpoint that the relevant client will consume.

Now that we are catering to various client applications and devices, we need to consider security options that facilitate any client application.

Bearer token security

Security is one of the fundamental parts of application development that we need to get right. Releasing software that does not control user access and permissions can have adverse side effects in the long run and allow for the exploitation of our application.

Using ASP.NET Core, we have access to an authentication library called `Identity Core`, which supports several authentication methods and allows us to easily integrate authentication into our application and supporting database. It has optimized implementations for the various authentication methods and authorization rules we implement in web applications and allows us to easily protect certain parts of our application.

Typically, we use authentication to identify the user attempting to gain access to our system. This usually requires the user to input a username and a password. If their information can be validated, we can check what they are authorized to do and then create a session using their basic information. All of this is done to streamline an experience for the user where they can freely use different parts of the application as needed without reauthenticating each step. This session is also referred to as a state.

In API development, we do not have the luxury of creating a session or maintaining a state. Therefore, we require that a user authenticates each request to secured API endpoints. This means we need an efficient way to allow the user to pass their information with each request, evaluate it, and then send an appropriate response.

Bearer tokens are the current industry standard method of supporting this form of stateless authentication needs. A bearer token gets generated after the initial authentication attempt, where the user shares their username and password. Once the information is validated, we retrieve bits of information about the user, which we call claims, and combine them into an encoded string value, which we call a token. This token is then returned in the API response.

The application that triggered the authentication call initially will need to store this token for future use.

Now that the user has been issued a token, any follow-up API calls will need to include this token. When the API receives subsequent requests to secure endpoints, it will check for the presence of the token in the header section of the API request and then seek to validate the token for the following:

- **Audience:** This is a value that depicts the expected receiving application of the token
- **Issuer:** This states the application that was issued to the token
- **Expiration Date and Time:** Tokens have a lifespan, so we ensure that the token is still usable
- **User claims:** This information usually includes the user's roles and what they are authorized to do

We can gauge all the points we wish to validate each time a request comes in with a token; the stricter the validation rules, the more difficult it is for someone to fake or reuse a token on an API.

Securing one API is simple enough, but this becomes very tedious and difficult to manage when this effort is spread across several APIs, as in a microservice-based application. It is not a good experience to have a user required to authenticate several times when accessing different parts of an application that might be using different services to complete a task. We need a central authority for token issuance and validation that all services can leverage. Essentially, we need to be able to use one token and validate a user across several services.

Considering this new challenge, we need to use an OAuth provider to secure our services centrally and handle our user information and validation. An OAuth provider application can take some time to configure and launch, so several companies offer OAuth services as SaaS applications. Several options exist to set up and host your OAuth provider instance, but this will require more maintenance and configuration efforts. The benefit of self-hosting is that you have more control over the system and the security measures you implement.

Duende IdentityServer is the more famed self-hosted option for an OAuth provider. It is based on ASP.NET Core and leverages Identity Core capabilities to deliver industry-standard security measures. It is free for small organizations and can be deployed as a simple web service and the central security authority for our microservices. They do also have a hosted model and can be compared with other hosted options, such as Microsoft Azure AD, and Auth0, to name a few.

Now that we have explored securing our microservices, we need to figure out the best way to host them alongside their various dependencies. Do we use a team of web servers, or do more efficient options exist?

Containers and microservices

We can typically host a web application or API and its supporting database on one server. This makes sense because everything is in one place and is easy to get to and maintain. But this server will also need to be very powerful and be outfitted to several applications and processes to support the different moving parts of the application.

Therefore, we should consider splitting the host considerations and placing the API and the database on separate machines. This costs more, but we get to maintain or host better and ensure that we do not burden either the machine or the environment with applications that are not needed.

When dealing with microservices, we will run into a challenging situation when attempting to replicate these hosting considerations for several services. We want each microservice to be autonomous functionally and from a hosting standpoint. Our services should share as little infrastructure as possible, so we don't want to risk placing more than one service on the same machine. We also don't want to burden a single device with supporting several hosting environment requirements since each microservice might have different needs.

We turn to container hosting as a lightweight alternative to provisioning several machines. Each container represents a slice of machine resources with optimized storage and performance resources needed for an application to run. Translating this concept into our hosting needs, we can create slices of these optimized environments for each microservice, database, and another third-party service as needed.

The advantage here is that we can still create optimal hosting environments for each service and supporting database, requiring far fewer machines to support this endeavor. Another benefit here is that each container is based on an image, representing the exact needs of the environment for the container. This image is reusable and repeatable, so we have less to worry about when transitioning

between environments and trying to provision an environment per service. The image will always produce the same container, and there be no surprises during deployments.

Containers are widely used and supported in the development community. The premiere container hosting option is Docker, an industry-leading container technology provider. Docker provides an extensive repository of container images that we can leverage for safe and maintained images from popular third-party applications that we typically leverage during development. It is also an open community, so we can create containers for our own needs and add them to the community repository for later access, whether for public or private use.

When using .NET, we can generate a `Dockerfile`, a file containing declarations about the image that should be used to create a container for the service we wish to host. This `dockerfile`, written using a language called **Yet Another Markup Language (YAML)**, outlines a base image, and then special build and deploy instructions. The base image states that we are borrowing information from an existing image and then we state that we wish to deploy our application to a container after combining the existing image and this application.

When we use container hosting, we generate a `dockerfile` for each service, and we need to orchestrate the order in which they are started and their dependencies. For instance, we probably don't want to start a service before its supporting database's container starts. For this, we must use an orchestrator. Industry-leading options include `docker-compose` and Kubernetes.

`docker-compose` is a simple and easy-to-understand option for container orchestration operations. `docker-compose` will refer to each `dockerfile` and allow us to outline any unique parameters we wish to include when executing this `dockerfile`. We can also outline dependencies and provide specific configuration values for the execution of that `dockerfile` and the resulting container. Now, we can orchestrate the provisioning of the containers to support our web services, databases, and other applications with one command. We can even reuse `dockerfiles` to create more than one container and have several containers with the same service on a different port and possibly with different configurations. We can see where this can come in handy when implementing the backend for frontend pattern.

Container hosting is platform-agnostic – we can leverage several hosting options, including cloud hosting options. Major cloud hosting providers such as Microsoft Azure and Amazon Web Services provide container hosting and orchestration support.

Now that we have our hosting sorted out, we need to be able to track what is happening across the application. Each service should provide logs of its activities, and more importantly, we need to be able to trace the logs across the various services.

Centralized logging

Logging is an essential part of post-deployment and maintenance operations. Once our application has been deployed, we need to be able to track and trace errors and bottlenecks in our application. This is easy enough to accomplish when we have one application and one logging source. We can always go to one space and retrieve the logs of what has happened.

.NET has native support for simple to advanced logging options. We can leverage the native logging operations and support powerful integrations for several logging destinations, such as the following:

- **Console:** Shows the log outputs in a native console window. Usually used during development.
- **Windows Event Log:** Also known as Event Viewer, this is a convenient way to view logs of several applications on a Windows-based machine.
- **Azure Log Stream:** Azure has a central logging service that supports logging for the application.
- **Azure Application Insights:** A power log aggregation service provided by Microsoft Azure.

When writing logs, we need to decide on the type of information we are logging. We want to avoid logging sensitive information such as compromising user or system information since we want to protect the integrity of our system and user secrets as much as possible. This will be relative to the context under which the application operates. Still, responsibility, wisdom, and maturity must be exercised during this scoping exercise. We also want to consider that we do not want to include too much clutter in the logs. Having chatty logs can be as bad as having no logs at all.

We also want to ensure we choose the correct classification for each log message. We can log messages as being any of the following levels:

- **Information:** General information about an operation.
- **Debug:** Usually used for development purposes. Should not be visible in a live environment.
- **Warning:** Depicts that something might not have gone as expected but is not a system error.
- **Error:** This occurs when an operation fails. They are usually used when an exception is caught and/or handled.
- **Critical/Fatal:** Used to highlight that an operation has failed and led to a system failure.

Choosing the correct classification for the log messages goes a long way in helping the operations team to monitor and track messages that need to be prioritized.

We can also add unique configurations for each logging destination and fine-tune the types of messages each will receive. This ability becomes relevant if we only wish to log messages that are informational to the Windows event log and all warnings, errors, and critical messages should be visible in Azure Log Stream and Application Insights. .NET Core allows us to make these granular adjustments.

We can further extend the capabilities of the native logging libraries by using extension packages such as **Serilog**. **Serilog** is the most popular logging extension library used in .NET applications. It supports more logging destinations such as rolling text files, databases (SQL Server, MySQL, PostgreSQL, and more), and cloud providers (Microsoft Azure, Amazon Web Services, and Google Cloud Platform), to name a few. We can write to multiple destinations with each log message by including this extension package in our application.

Individual application logging can be set up relatively quickly, but this concept becomes complex when we attempt to correlate the logs. When a user has trouble accessing one feature, we need to check several possible points of failure, considering that our microservice application will trigger several actions across several services. We need an efficient way to collate the logs produced by each service and, by extension, be able to trace and relate calls associated with a single operation.

Now, we turn to log aggregation platforms. Simply put, they act as log destinations and are designed to store all logs that are written to them. They also provide a user interface with advanced querying support. This is needed for a distributed application since we can now configure the aggregator as a central logging destination for several applications, and we can more easily query the logs to find logs that might be related but from different sources. We can also configure them to monitor and alert when logs of specific categorizations are received.

Popular options for log aggregation include **Seq**, the **Elasticsearch, Logstash, and Kibana (ELK)** stack, and hosted options such as **Azure Application Insights** and **DataDog**. Each platform has its strengths and weaknesses and can be leveraged for small to large applications. **Seq** is a popular option for small to medium-sized applications, and it has easy-to-use tools and supports robust querying operations. Still, aggregators have some limitations, and those come up when we need to properly trace logs from several sources.

Tracing logs from several sources is referred to as distributed logging. It involves us using common information in our log messages and tracing related tags and trace IDs to correlate logs to a single event. This requires us to write more enriched logs containing more details and headers that a log tracing tool can use and give us the best possible information about. An emerging technology to support this concept is **OpenTelemetry**, which will produce logs with greater detail and correlation from our various applications.

We can now use more specialized tools, such as **Jaeger**, to sift through the enriched logs and perform even more complex queries across the logs. **Jaeger** is a free, lightweight, and open source tool that can get us started with this concept, but we can once again use Microsoft Azure Insights for production workloads.

Summary

In this chapter, we explored the various moving parts of microservices and how we can leverage different development patterns to ensure that we deliver a stable and extendable solution. We saw where the microservices architecture has a problem for every solution it introduces, and we need to ensure that we are aware of all the caveats of each decision we make.

Ultimately, we need to ensure that we properly assess and scope the needs of our application and refrain from introducing a microservices architecture where it might not be required. If we end up using one, we must ensure that we make the best use of the various technologies and techniques that support our application. Always seek to do the minimum necessary to address an issue before introducing complexity in the name of advanced architecture.

I hope you enjoyed this journey and have enough information to inform the decision-making and development processes that will be involved when you start developing microservices with ASP.NET.

Index

Symbols

.NET

advanced logging options 261

.NET Core

using, as development stack for
microservices 12, 13

.NET logging API

using 234-239

A

aggregate ID 73

aggregate pattern

purpose 19

usage 19

aggregate roots 19, 20

aggregates 19, 20

relationships 20

relationships, handling that span 21

versus entities 21

aggregator pattern 15, 249

Amazon Web Services (AWS) 128

anemic domain model

versus rich domain model 23-26

Apache Kafka 47

API gateway pattern 152, 153

advantages 153-155

cache management, adding 164

client (fronted) 152

database 152

disadvantages 155

implementing 155

implementing, with Azure API

Management 156

rate limiting, adding 165-167

response aggregation, adding 167-169

server (backend) 152

thick API gateway 156

API Gateways 257

append-only data store 74

ASP.NET Core API

Azure Service Bus, implementing 51-55

ASP.NET Core, for microservices

benefits 12

ASP.NET Core health checks

implementing 141

liveness health checks, adding 141-146

readiness health checks, adding 147

ASP.NET Core web API

RabbitMQ, implementing 48-50

asynchronous communication 31, 42, 250

functioning with 41-43

- asynchronous communication,**
 - between microservices**
 - disadvantages 55, 56
- atomicity, consistency, isolation, and durability (ACID) 19, 73**
- authentication 178**
- authorization 178**
- availability 46**
- Azure API Management**
 - API gateway pattern, implementing with 156-160
- Azure Application Insights 262**
- Azure Container Instances (ACI) 148**
- Azure Container Registry (ACR) 212**
- Azure Cosmos DB 10, 253**
- Azure Kubernetes Service (AKS) 148**
- Azure Service Bus 10, 47**
 - implementing, in ASP.NET Core API 51-55

B

- backend for frontend (BFF)**
 - pattern 170, 171, 257**
 - implementing, with Ocelot 171-174
- bearer tokens 179-181**
 - data points, compiling 179
 - generating 186-190
 - issuing 186-190
 - security, implementing 182
 - used, for securing API 182-185
 - used, for securing communications 178, 179
- bearer token security 257-259**
- bounded context 16**
- Bus Factory 50**
- business approach, for microservices**
 - availability 9
 - deployment 11
 - development speed 10

- improved data storage 10
- monitoring 10
- scalability 9

C

- C# 33**
- caching layer**
 - used, for implementing resiliency 128
- CAP theorem 46**
- cardinalities 95**
- cardinalities, relational database**
 - many-to-many 95
 - one-to-many 95
 - one-to-one 95
- centralized logging 261, 262**
- choreography model 116, 117**
 - cons 118
 - pros 118
- Circuit Breaker Pattern 93, 127, 131**
- circuit breaker policies**
 - implementing 131, 132
- circuit breaker policy**
 - with Polly 133, 134
- Code-First technique 103**
- command**
 - implementing 62
 - invoking 64-66
- Command and Query Responsibility Separation (CQRS) patterns 250, 251**
 - benefits 59, 60
 - disadvantages 60
 - need for, microservices development 58
 - state, reading with 85
- command handler**
 - creating 63, 64
- command-line interface (CLI) 212**
- command messages 45**

- command model**
 - creating 62, 63
- Command Query Separation (CQS) 250**
- concurrency management 46**
- consistency 46**
- constraint 95**
- container images 212, 213**
- containerization 211**
- containerized application**
 - launching 218, 219
- container registry**
 - containers, publishing to 227
 - custom images, creating 228-230
 - custom images, uploading 228-230
- containers 259, 260**
 - cons 213, 214
 - pros 213, 214
 - publishing, to container registry 227
 - usage 210-212
 - using, in microservices development 210
- cookies 178**
- Cosmos DB 59**
- CQRS, in .NET**
 - Mediator pattern, using with 60-62
- Create, Read, Update, and Delete (CRUD) 21, 251**

D

- Dapper 100**
- database**
 - developing 93
 - non-relational databases 97
 - ORM, selecting 100
 - relational databases 94
 - technology, selecting 98-100
- database design patterns 12**

- database development technique**
 - Code-First 103
 - Schema-First 103
 - selecting 103-105
- Database-First technique 103**
- database-per-service 91**
- database-per-service approach 10**
- database-per-service pattern 73, 252**
 - disadvantages 92, 93
 - non-relational databases 253
 - one database per service 253, 254
 - relational databases 252
 - single database technology per service 90, 91
 - single database, using for
 - several services 253
 - technologies, using 91, 92
 - using 90
- DataDog 262**
- data transfer object (DTO) 65, 186**
- dead-letter queue 127**
- denial-of-service (DoS) 131**
- distributed denial-of-service (DDoS) 154**
- distributed tracing 244, 245**
 - enhanced logging 245-247
- Docker 11, 212**
- docker-compose 221**
 - adding, to project 222-227
- Dockerfile**
 - authoring 214-217
 - containerized application,
 - launching 218-220
 - native .NET container support,
 - using 220, 221
- Docker Swarm 11**
- document database 83, 98**
- document stores**
 - examples 83

domain-driven design (DDD) 15, 18, 19, 67

- exploring 16, 17
- keywords, exploring 16
- pros and cons, exploring 18
- significance 16, 17

domain events 75

- actions 75
- and event sourcing 75
- exploring, in application 75-80

downstream 161

E

Elastic Container Registry (ECR) 227**Elasticsearch, Logstash, and
Kibana (ELK)** 262**Elmah.io** 240**enhanced logging**

- for distributed tracing 245-247

entities 21

- need for 21
- practical uses, in code 22
- versus aggregates 21
- versus value objects 26

Entity Framework Core 100**event messages** 45**events** 72

- audit logs 73
- entity state identification 73
- historical 72
- immutability 72
- key attributes 72
- persistent events 73
- single occurrence 72
- storing 81, 82

event sourcing 73

- cons 74

implementing, with non-relational
database 83-85

implementing, with relational
database 82, 83

pros 73, 74

event sourcing patterns 73, 251, 252**event store**

creating 81

eventual consistency 10, 46, 47, 93**exchanges** 48

F

first-in-first-out (FIFO) 43**foreign key** 95**full-stack developer role** 93

G

gateway pattern 9**Google Container Registry (GCR)** 227**Google Remote Procedure Call (gRPC)** 4**Grafana** 11**graph database** 98**graphical UI (GUI)** 219**gRPC communication**

versus HTTP communication 37, 38

gRPC synchronous communication

implementing 35-37

GUID 22

H

health check 138, 139

degraded state 139

healthy state 139

liveness health check 140

need for 256, 257

readiness health checks 140, 141
unhealthy state 139

health probes

configuring, in orchestrators 148, 150

HTTP asynchronous communication 43

HttpClient 33

HTTP communication

versus gRPC communication 37, 38

HTTP Status Code

categories 33

HTTP synchronous communication

implementing 32-35

I

IBM DB2 94

IdentityServer 191

API security considerations 207
configuring 192-202
used, for securing API 202-205
used, for securing Ocelot API
gateway 206, 207

IdentityServer4

using, to secure microservices 191, 192

image 211, 221

indexes 94

infrastructure patterns 12

integration patterns 12

inversion of control (IoC) 130

J

Jaeger 262

JavaScript Object Notation (JSON) 98

JSON Web Token (JWT) 163, 179

jwt.io 181

K

Kafka 10

key-value database 98

Kibana 11

kubernetes 11

L

LINQ 100

liveness health check 140

adding 141-146

log aggregation 241, 242

integrating, with Seq 242-244
platform, selecting 242
usage 241, 242

logging 232

log, selecting 232, 234
need for 232
Serilog, adding 240, 241

Loggr 240

LogLevel methods

parameters 236

LogStash 11

M

manifest 148

many-to-many relationship 95, 96

Marathon 11

Mediator pattern

using, with CQRS in .NET 60-62

MediatR 75

MediatR pattern 251

message broker

used, for implementing resiliency 127

message bus

configuring 47

- message bus systems** 44-46
- message queues** 43, 44
- microservice communication**
 - challenges 30, 31
- microservice design patterns** 18, 19
- microservices** 3, 4, 138, 139, 259, 260
 - building 7, 8
 - business need, assessing 8
 - design pattern, exploring 12
 - elements 3, 4
 - feasibility, determining of implementing 12
 - .NET Core, using as development stack 12, 13
 - securing, with IdentityServer4 191, 192
 - synchronous communication, disadvantages between 38
- microservices development**
 - containers, using 210
 - CQRS pattern 58
- Microsoft SQL Server** 10
- models** 103
- MongoDB** 59, 253
- monolith**
 - architecture, challenges 5
 - building 4-6
- MySQL / MariaDB** 94

N

- native .NET container support**
 - using 220, 221
- Neo4j** 253
- NHibernate** 100
- NLog** 240
- non-relational database** 97, 253
 - event sourcing, implementing with 83-85
 - using, scenarios 100

- normalization** 94, 97
- NoSQL databases** 83, 97
 - advantages 98
 - disadvantage 98
- NoSQL databases, types**
 - document database 98
 - graph database 98
 - key-value database 98

O

- OAuth, Azure Active Directory** 207
- object-relational mapping (ORM)** 59
 - selecting 100-103
- Ocelot** 257
 - API gateway pattern, implementing with 160-164
 - BFF pattern, implementing with 171-174
 - DownstreamHostAndPort 163
 - DownstreamPathTemplate 163
 - DownstreamScheme 163
 - GlobalConfiguration 163
 - routes 162
 - UpstreamHttpMethod 163
 - UpstreamPathTemplate 163
- Ocelot API gateway**
 - securing, with IdentityServer 206, 207
- Okta** 207
- one-to-many relationship** 95
- one-to-one relationship** 95
- Open Container Initiative (OCI)** 212
- OpenID Connect** 191
- OpenTelemetry** 262
- operating system (OS)** 211
- Oracle Database** 94
- orchestra** 119
- orchestration** 119
- orchestration pattern** 156

orchestration saga implementation 119, 120

- cons 121
- pros 121
- rolling back, on failure 120

orchestrator 119

- health probes, configuring 148-150

overambitious gateway 156**P****PaperTrail 11****partition tolerance 46****poisoned messages 127****Polly 132**

- circuit breaker policy with 133, 134
- retry policy with 132, 133

PostgreSQL 94**Postman 190****primary key 94****proto class 35, 36****public container registries**

- versus private container registries 227, 228

Pub-Sub communication 43

- eventual consistency 46, 47
- message bus systems 44-46
- message queues 43, 44

Q**query**

- implementing 66
- invoking 68

query handler

- creating 67, 68

query model

- creating 66

queues 48**R****RabbitMQ 10, 47**

- implementing, in ASP.NET
Core web API 48-50

readiness health checks 140, 141

- adding 147

read-only data store 74**Redis 253****Redis Cache 47**

- used, for implementing resiliency 128, 129

referential integrity 95**relational databases 94-96, 252**

- event sourcing, implementing with 82, 83
- IBM DB2 94
- MySQL / MariaDB 94
- Oracle Database 94
- PostgreSQL 94
- SQLite 94
- SQL Server 94
- using, scenarios 99

relationship 95**Remote Procedure Call (RPC) 35****replaying 73****repository pattern**

- implementing 105-111

Representational State Transfer (REST)

- using, benefits 37

**Representational State Transfer
(REST) API 32**

- methods 32

resiliency

- implementing, with caching layer 128
- implementing, with message broker 127
- implementing, with Redis Cache 128-131

resiliency patterns 12**resilient microservices 255, 256**

REST communication

- merits 37

retry policies 127

- implementing 131, 132
- with Polly 132, 133

rich domain model

- versus anemic domain model 23-26

Role-based access control (RBAC) 227

S

saga pattern

- compensable transactions 114
- considerations 115
- exploring 114
- issues 115
- pivotal transactions 114
- retryable transactions 114
- rolling back, on failure 117, 118
- using, across services 254, 255

Saga pattern 93, 156**schema-per-service 90****security patterns 12****separation of concerns (SoC) 60****Seq 262**

- used, for integrating log aggregation 242-244

Serilog

- adding 240, 241

service-level agreements (SLAs) 60**service resiliency**

- failure scenarios, handling 126, 127
- need for 126

session variables 178**SignalR hub 77****single-sign-on (SSO) 191****Software-as-a-Service (SaaS) 160****Splunk 11****SQLite 94****SQL Server 94****SQL Server Management Studio (SSMS) 229****storage design patterns 12****Swagger UI 190****synchronous communication 31, 42, 250**

- disadvantages, between microservices 38
- implementing 32
- use cases 30

T

tables-per-service pattern 90**thick API gateway 156****ti claim 181****timestamp 73**

U

ubiquitous language 16**UNIQUE index 83****upstream 161****user interface (UI) 212**

V

value objects 26-28

- using 26-28
- versus entities 26

virtual machines (VMs) 210**Visual Studio Code 12**

W

Web App for Containers (WAC) 148**web applications**

- securing, challenges 178

Y

YAML Ain't Markup Language (YAML) 148

**Yet Another Markup Language
(YAML)** 214, 260



Packt . com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packt . com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub . com for more details.

At www . packt . com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



Microservices with Go

Alexander Shuiskov

ISBN: 978-1-80461-700-7

- Get familiar with the industry's best practices and solutions in microservice development
- Understand service discovery in the microservices environment
- Explore reliability and observability principles
- Discover best practices for asynchronous communication
- Focus on how to write high-quality unit and integration tests in Go applications
- Understand how to profile Go microservices



Event-Driven Architecture in Golang

Michael Stack

ISBN: 978-1-80323-801-2

- Understand different event-driven patterns and best practices
- Plan and design your software architecture with ease
- Track changes and updates effectively using event sourcing
- Test and deploy your sample software application with ease
- Monitor and improve the performance of your software architecture

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *Microservices Design Patterns in .NET*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/9781804610305>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly